

Analysis of Inventory Systems

Manuel D. Rossetti

2026-09-16

Table of contents

Preface	1
Approach	1
How this book is organized	2
Software	2
A note on notation	3
About the Author	4
1. Introduction to Inventory Systems	5
1.1. Why Hold Inventory?	6
1.1.1. Pipeline Inventory	6
1.1.2. Cycle Inventory	7
1.1.3. Safety Inventory	7
1.1.4. Anticipation Inventory	8
1.1.5. Speculative Inventory	8
1.1.6. Different Causes, Different Levers	9
1.1.7. The Cost of Not Holding It	10
1.2. Classifying Inventory Systems	12
1.2.1. Where the Demand Comes From	13
1.2.2. Known or Uncertain Demand	13
1.2.3. Constant or Time-Varying Demand	14
1.2.4. How the System Is Reviewed	14
1.2.5. The Lead Time	15
1.2.6. What Happens to Unmet Demand	16
1.2.7. The Planning Horizon	16
1.2.8. One Item or Many	17

Table of contents

1.2.9.	One Location or Several	17
1.2.10.	The Dimensions Together	18
1.2.11.	Which Costs the Classification Implies	19
1.2.12.	Choosing a Classification	20
1.3.	The State of an Inventory System	21
1.3.1.	On Hand, On Order, and Backordered	21
1.3.2.	Lost Sales	23
1.3.3.	Why Ordering Decisions Use the Inventory Position	24
1.3.4.	Little's Law and the Averages It Connects	27
1.3.5.	Stockout Indicators	29
1.3.6.	The Demand Process	29
1.4.	Cost Parameters	30
1.4.1.	What an Inventory Model Charges	30
1.4.2.	The Marginality Test	33
1.4.3.	Unit Cost, c	34
1.4.4.	Holding Cost, h	34
1.4.5.	Fixed Order Cost, k	37
1.4.6.	Stockout and Backorder Costs, π and b	38
1.4.7.	Stocking Position Cost, f	40
1.4.8.	How Accurate Do These Need to Be?	41
1.5.	Estimating Cost Parameters from Standard Data	42
1.5.1.	Where the Numbers Come From	43
1.5.2.	Unit Cost	43
1.5.3.	The Carrying Charge	44
1.5.4.	The Fixed Order Cost	47
1.5.5.	Stockout and Backorder Costs, and What to Do Instead	49
1.5.6.	Avoiding Double Counts	50
1.5.7.	When a Point Estimate Is Not Defensible	51
1.6.	Performance Measures	52
1.6.1.	Cost as a Performance Measure	53
1.6.2.	Service Measures	54
1.6.3.	The Problem the Rest of the Book Solves	60
1.7.	Exercises	62

Table of contents

2. Managing Inventory	66
2.1. The Manager's Problem	66
2.1.1. What Scale Changes	67
2.1.2. What the Manager Decides	68
2.1.3. What This Chapter Does Not Cover	70
2.2. Grouping and Segmentation	70
2.2.1. Two Reasons to Group	71
2.2.2. Pareto Analysis and ABC Classification	72
2.2.3. What One Criterion Misses	74
2.2.4. Deriving the Criterion Instead of Choosing Weights	77
2.2.5. Which Attributes, and Which Forbid Grouping	78
2.2.6. Demand Pattern Segmentation	79
2.2.7. Clustering for Policy	82
2.2.8. Evaluating a Grouping	83
2.2.9. Combining Schemes	85
2.2.10. Using a Classification	86
2.3. Inventory Record Analysis	87
2.3.1. What a Record Claims, and Why It Drifts	87
2.3.2. Why Accuracy Comes First	89
2.3.3. What Error Actually Costs	91
2.3.4. Measuring Record Accuracy	92
2.3.5. Physical Inventory and Cycle Counting	95
2.3.6. Cycle Counting Methods	96
2.3.7. Setting Count Frequency	98
2.3.8. Monitoring Accuracy Over Time	103
2.3.9. Accuracy Is Not a Local Property	106
2.3.10. Correction Is Not the Point	107
2.4. Other Responsibilities	109
2.4.1. Excess and Obsolete Stock	109
2.4.2. Performance Reporting	110
2.5. Summary	112
2.6. Exercises	113

Table of contents

I. Deterministic Models	118
3. Deterministic Lot Sizing	119
3.1. The General Cycle	120
3.1.1. Assumptions	120
3.1.2. One Cycle, Four Segments	121
3.2. Average Inventory and Backorders	124
3.3. The Total Cost Rate	126
3.4. Optimizing the General Model	127
3.5. The Classical Models as Special Cases	129
3.5.1. No Charge Per Unit Short	129
3.5.2. No Backorders	130
3.5.3. Instantaneous Replenishment	131
3.5.4. The Classical Economic Order Quantity	131
3.6. Lead Time and the Reorder Point	135
3.7. Sensitivity of the Order Quantity	136
3.8. Quantity Discounts	137
3.8.1. All-Units Discounting	138
3.8.2. Incremental Discounting	142
3.9. Performance Measures for the Cycle	145
3.10. Building These Models in a Spreadsheet	147
3.10.1. The Four-Block Layout	148
3.10.2. The Reorder Point Cell	152
3.10.3. Reducing the Production Model to the Order Model	154
3.10.4. Discount Schedules as Columns	157
3.10.5. What the Worksheets Do Not Do	164
3.11. Designing the Software	165
3.11.1. What Changes, and What Must Not	165
3.11.2. One Pipeline, Six Models	167
3.11.3. Sealed Types for Closed Sets	170
3.11.4. An Answer That Carries Its Reasoning	172
3.12. The Same Models in Code	174
3.12.1. The Four Blocks as Objects	175
3.12.2. The Reorder Point	177

Table of contents

3.12.3. The Same Substitution	178
3.12.4. Discount Schedules as Candidates	180
3.12.5. Doing the Three Things a Worksheet Does Not	182
3.13. Summary	183
3.14. Exercises	185
4. Multi-Item and Constrained Systems	191
4.1. When Items Stop Being Independent	192
4.2. A Constrained Lot Sizing Problem	193
4.2.1. First Check Whether the Constraint Binds	195
4.2.2. Moving the Constraint into the Objective	195
4.2.3. Finding the Multiplier	197
4.2.4. When the Limit Is on Ordering, Not on Stock	199
4.3. What the Multiplier Means	202
4.4. The Exchange Curve	204
4.4.1. One Item First	204
4.4.2. The Aggregate Curve	206
4.4.3. Reading a Cost Ratio from Practice	207
4.4.4. When Ordering Costs Differ Across Items	210
4.5. Joint Replenishment	216
4.5.1. The Coordinated Problem	216
4.5.2. Power-of-Two Policies	218
4.5.3. What a Power-of-Two Policy Costs	219
4.6. Multi-Echelon Systems	225
4.6.1. Echelon Inventory and Echelon Holding Cost	226
4.6.2. Serial Systems	231
4.6.3. Distribution Systems	237
4.7. Building These Models in a Worksheet	241
4.7.1. The Constrained Problem	241
4.7.2. The Exchange Curve	248
4.7.3. Intervals and Rounding	251
4.7.4. The Two Echelon Structures	256
4.7.5. What a Worksheet Can and Cannot Do	259

Table of contents

4.8.	Designing the Software	261
4.8.1.	What the Software Must Do	261
4.8.2.	Finding the Nouns	265
4.8.3.	What Each Thing Knows and Does	268
4.8.4.	The Classes	271
4.8.5.	Measuring, and Limiting a Measure	274
4.8.6.	Alternatives Are Types, Settings Are Values	279
4.8.7.	Three Couplings and No Common Base	282
4.8.8.	How the Objects Solve It	284
4.8.9.	Solving the Chapter's Examples	288
4.8.10.	Checking the Design Against the Mathematics	293
4.9.	Summary	295
4.10.	Exercises	298
5.	Dynamic Lot Sizing	303
5.1.	When the Demand Rate Changes	304
5.1.1.	The Requirements Schedule	305
5.1.2.	The Assumptions	306
5.1.3.	Where This Chapter Sits	307
5.2.	What a Plan Costs	307
5.2.1.	The Purchase Term	308
5.2.2.	Three Plans Without Any Theory	308
5.3.	Two Properties That Make the Problem Small	311
5.3.1.	Zero-Inventory Ordering	311
5.3.2.	The Window Cost	312
5.3.3.	Property 2: A Requirement That Forces an Order	314
5.4.	Choosing the Order Periods	315
5.4.1.	The Problem Is a Shortest Path	315
5.4.2.	The Recursion	315
5.4.3.	The Same Problem for Solver	321
5.4.4.	How Much Work It Is	322
5.5.	Three Heuristics and One Shape	323
5.5.1.	One Procedure	323

Table of contents

5.6.	What a Heuristic Costs	327
5.6.1.	When None of This Is Worth Doing	328
5.6.2.	Sensitivity to the Cost Ratio	329
5.6.3.	What the Aggregate Studies Say	330
5.7.	The Rolling Horizon	331
5.7.1.	What the Truncation Costs	331
5.7.2.	Does the Heuristic Ever Win?	332
5.7.3.	Nervousness	333
5.8.	Building These Models in a Worksheet	334
5.8.1.	The Data Sheet	335
5.8.2.	Costing a Plan	338
5.8.3.	The Recursion and the Traceback	338
5.8.4.	The Three Rules	343
5.8.5.	The Mixed Integer Program	344
5.8.6.	What the Worksheet Cannot Do	350
5.9.	Designing the Software	350
5.9.1.	What the Software Must Do	352
5.9.2.	Finding the Nouns	352
5.9.3.	What Each Thing Knows and Does	354
5.9.4.	The Schedule	355
5.9.5.	Three Rules Are One Rule	358
5.9.6.	The Algorithm Is Not a Heuristic	363
5.9.7.	How the Objects Solve It	365
5.9.8.	Rolling the Horizon	365
5.9.9.	Solving the Chapter's Examples	367
5.10.	Summary	369
5.11.	Exercises	371
5.11.1.	Terminology and Concepts	371
5.11.2.	Working the Methods by Hand	374
5.11.3.	Modeling Problems	376
5.11.4.	The Rolling Horizon and the Solver	378
5.11.5.	Using the Workbook	378
5.11.6.	Using the Software	379
5.11.7.	From the Literature	381

Table of contents

6. Material and Distribution Requirements Planning	382
6.1. Demand You Should Not Forecast	383
6.1.1. Why Forecasting It Is a Mistake	383
6.1.2. Where This Chapter Sits	384
6.2. The Bill of Material	384
6.2.1. Low-Level Codes	386
6.2.2. The Explosion Calculation	387
6.3. The MRP Record	387
6.3.1. The Maximum Is the Whole Idea	389
6.3.2. Safety Stock and Allocations	390
6.3.3. How Long the Horizon Has to Be	390
6.4. Running the Explosion	391
6.5. What Lot Sizing Does to the Level Below	396
6.5.1. One Rule Changes, and Everything Moves	396
6.5.2. What the Variability Coefficient Does Not Tell You	398
6.5.3. What Is Actually Done	399
6.6. Distribution Requirements Planning	399
6.6.1. It Is the Same Calculation	400
6.6.2. What the Regions Do to the Centre	402
6.7. Nervousness	403
6.8. What Requirements Planning Assumes	404
6.9. Building These Models in a Worksheet	406
6.9.1. The Record, Computed Twice	406
6.9.2. The Explosion	406
6.9.3. The Lumpiness Sheet	410
6.9.4. The Network	410
6.10. Designing the Software	410
6.10.1. What the Software Must Do	414
6.10.2. Finding the Nouns	414
6.10.3. The Join with Chapter 5	415
6.10.4. One Interface for Two Kinds of Rule	417
6.10.5. Two Readings of “Net Requirement”	419
6.10.6. A Network Is a Bill of Material	420
6.10.7. Running the Chapter’s Examples	422

Table of contents

6.11. Summary	426
6.12. Exercises	427
6.12.1. Terminology and Concepts	428
6.12.2. Working the Records by Hand	429
6.12.3. Modeling Problems	431
6.12.4. Using the Workbook	432
6.12.5. Using the Software	433
6.12.6. From the Literature	434
 II. Stochastic Models	 435
 7. Single-Period Stochastic Inventory	 436
7.1. One Order, One Chance	436
7.1.1. Demand Is Not Sales	438
7.2. Working the Decision by Hand	439
7.3. Getting a Demand Distribution	442
7.3.1. Look Before You Model	443
7.3.2. Summarize It	445
7.3.3. Hypothesize, Estimate, Test	447
7.3.4. What the Test Does Not Settle	450
7.4. Intermittent and Slow-Moving Demand	451
7.5. The Newsvendor Model	453
7.5.1. The Same Model as a Profit	454
7.6. The Critical Ratio	455
7.6.1. Reading the Critical Ratio	456
7.7. Continuous Demand	457
7.8. Discrete Demand	461
7.9. Extensions	464
7.9.1. A Penalty Beyond the Lost Margin	464
7.9.2. Stock Already on Hand	465
7.9.3. When the Model Refuses	465
7.9.4. Is It Really a Single Period?	466

Table of contents

7.10. Estimating the Answer by Simulation	467
7.10.1. When It Is Worth It	469
7.11. Building These Models in a Worksheet	470
7.12. Designing the Software	471
7.12.1. Finding the Nouns	472
7.12.2. The Two Routes Must Agree	472
7.13. Summary	473
7.14. Exercises	475
7.14.1. Terminology and Concepts	475
7.14.2. Working the Model by Hand	476
7.14.3. Modeling Problems	478
7.14.4. Using the Software	480
7.14.5. Using the Workbook	480
7.14.6. From the Literature	481
8. Continuous Review Systems	482
8.1. The (r, Q) Policy	482
8.2. Modeling Lead-Time Demand	482
8.3. Safety Stock and the Safety Factor	483
8.4. Type 1 Service: Cycle Service Level	483
8.5. Type 2 Service: Fill Rate	483
8.6. Setting Service Targets	483
8.7. An Approximate Cost Model	483
8.8. Exact Analysis under Poisson Demand	483
8.9. Stochastic Lead Times	484
8.10. The (s, S) Policy	484
8.11. Computing Policy Parameters	484
8.12. Exercises	484
9. Periodic Review Systems	486
9.1. The Base-Stock Policy	486
9.2. The (R, S) Policy	486
9.3. The (R, s, S) Policy	487
9.4. Choosing the Review Interval	487

Table of contents

9.5. Comparing Continuous and Periodic Review	487
9.6. Exercises	487
10. Multi-Echelon Inventory Systems	488
10.1. Stochastic Demand in a Multi-Echelon System	488
10.2. Serial Systems	489
10.3. Distribution Systems	489
10.4. The METRIC Model	489
10.5. The Bullwhip Effect	489
10.6. Exercises	489
11. Simulating Inventory Systems	490
11.1. Why Simulate an Inventory System?	490
11.2. Modeling the Inventory Position Process	490
11.3. Building an (r, Q) Model with the KSL	491
11.4. Output Analysis for Inventory Simulations	491
11.5. Comparing Policies by Simulation	491
11.6. Exercises	491
Appendices	492
A. Notation	492
A.1. Conventions	492
A.2. State Variables	493
A.3. Demand and Time	494
A.4. Cost Parameters	496
A.5. Performance Measures	497
A.6. Policy Parameters	498
A.7. Multi-Item Quantities	498
A.8. Multi-Echelon Quantities	502
A.9. Dynamic Lot Sizing Quantities	505
A.10. Requirements Planning Quantities	508
A.11. Distribution and Loss Function Quantities	512

Table of contents

A.12.A Note on Other Sources	515
B. Probability and Stochastic Processes Review	516
B.1. Random Variables and Expectation	516
B.2. Sums of Random Variables	516
B.3. The Poisson Process	517
B.4. Renewal Processes	517
B.5. Random Sums	517
B.5.1. The Mean	518
B.5.2. The Variance	518
B.5.3. Lead Time Demand	519
B.5.4. Batches, and the Compound Poisson	520
B.5.5. What This Does and Does Not Give You	521
C. Common Probability Distributions	522
C.1. Discrete Distributions	522
C.1.1. The Poisson Distribution	523
C.1.2. The Geometric and Negative Binomial Distributions	523
C.1.3. The Binomial Distribution	524
C.1.4. The Discrete Empirical Distribution	525
C.2. Continuous Distributions	525
C.2.1. The Exponential Distribution	526
C.2.2. The Gamma and Erlang Distributions	526
C.2.3. The Normal Distribution	527
C.2.4. The Lognormal Distribution	528
C.3. Loss Functions	529
C.3.1. Discrete Random Variables	530
C.3.2. The Discrete Empirical Distribution	531
C.3.3. Continuous Random Variables	532
C.3.4. The Standard Normal Case	533
C.3.5. Loss Functions for Common Distributions	535
C.3.6. Mixtures	537
C.3.7. Computing Them	539

Table of contents

C.4. Choosing a Demand Distribution	540
C.4.1. Fitting to a Sample	540
C.4.2. Matching the First Two Moments	542
C.4.3. When the Demand Is Intermittent	546
D. Statistical Tables	547
D.1. Standard Normal Distribution	547
D.2. Standard Normal Loss Function	547
D.3. Cumulative Poisson Probabilities	547
E. Kotlin and KSL Setup	548
E.1. What You Need	548
E.2. The Course Code Project	549
E.3. Running Your First Model	549
E.4. Adding Your Own Model	550
E.5. The KSL Itself	550
References	552

Preface

This book is intended as a textbook for an upper-level undergraduate or introductory graduate course on the modeling and analysis of inventory systems. It is written for engineering students, primarily industrial engineering, but is also suitable for students in operations management, supply chain management, and data science who have the necessary background in probability and statistics.

The material assumes that you have had a calculus sequence and a first course in probability and statistics. Familiarity with basic optimization concepts is helpful but not required; the optimization needed for the models developed here is introduced as it is used.

Approach

Inventory theory sits at the intersection of stochastic modeling and optimization. The organizing question throughout this book is simple to state and difficult to answer well:

How much should we order, and when should we order it?

Every model developed here is an answer to that question under a particular set of assumptions about demand, lead times, cost structure, and the review process. The emphasis is on understanding **which assumptions produce which policy**, and on being honest about what happens when the assumptions fail. Analytical models are developed first; simulation is

How this book is organized

then used to study systems that resist closed-form analysis and to check the robustness of the analytical results.

How this book is organized

The development moves from deterministic to stochastic, and from single-location to multi-echelon systems:

- **Foundations:** the structure of inventory systems, the costs and service measures used to evaluate them, and the demand models that drive them.
- **Deterministic models:** lot sizing when demand is known, both for constant and for time-varying demand.
- **Stochastic models:** single-period decisions, then continuous- and periodic-review policies for repeated ordering under uncertainty.
- **Systems:** multiple items competing for shared resources, and multiple stocking locations arranged in echelons.
- **Simulation:** modeling inventory systems computationally when analysis reaches its limits.

Each chapter closes with exercises. Some are analytical, some require computation, and some ask you to build and study a simulation model.

Software

Two tools carry the computational work in this book, and each is used where it is the right instrument.

Spreadsheets handle the numerical models. Most of what this book asks you to compute, such as a lot sizing schedule, an MRP record, or a policy's expected cost, is naturally tabular and time-phased, and a spreadsheet shows the whole calculation at once in a way that a printed formula cannot.

A note on notation

Worked spreadsheets accompany the chapters, and several exercises ask you to extend one. You are expected to be comfortable building and auditing a spreadsheet model.

Kotlin and the Kotlin Simulation Library (KSL) handle the simulation work, where a system's behavior has to be generated rather than solved for. Appendix E covers installing the tools and running your first model, and Chapter 11 develops the modeling itself. The KSL is developed and documented separately; see *Simulation Modeling using the KSL* for a full treatment of the library.

Some figures and tables in the text are generated programmatically so that they stay consistent with the numbers around them. That tooling is an authoring convenience and is not shown, taught, or required of you.

A note on notation

Inventory theory has an unfortunate amount of notational variation across the literature. This book fixes a single notation and uses it consistently; where a symbol differs from the common usage in a widely cited source, the difference is noted so that you can move between this text and the literature without confusion.

About the Author

Dr. Manuel Rossetti, P.E. Dr. Rossetti is a University Professor of Industrial Engineering at the University of Arkansas. He served as the inaugural Director for the University of Arkansas Data Science program from 2019-2024. In addition, Dr. Rossetti served as the Director for the NSF I/UCRC Center for Excellence in Logistics and Distribution (CELDi) from 2013-2020. Dr. Rossetti has published over 125 journal and conference articles in the areas of simulation, logistics/inventory, and healthcare and has been the PI or Co-PI on funded research projects totaling over 6.4 million dollars. He was selected as a Lilly Teaching Fellow in 1997/98 and was voted Best IE Teacher by IE students in 2007, 2009, and 2017. He won the IE Department Outstanding Teacher Award in 2001-02, 2007-08, and 2010-11. He received the College of Engineering Imhoff Teaching Award in 2012 and was elected an IIE Fellow. In 2013, the UA Alumni Association awarded Dr. Rossetti the Charles and Nadine Baum Faculty Teaching Award, the highest award for teaching at the university. In 2015, Dr. Rossetti served as Program Chair for the Winter Simulation Conference. In 2024, Dr. Rossetti served as the General Chair for the Winter Simulation Conference. He is also the author of the book, *Simulation Modeling and Arena*, published by John Wiley & Sons.

Dr. Rossetti grew up in Canton, Ohio and is an ardent Cleveland sports fan. He received his Ph.D. and MSIE degrees in Industrial and Systems Engineering from The Ohio State University and his BSIE degree from the University of Cincinnati. He is a registered professional engineer in the State of Arkansas.

1. Introduction to Inventory Systems

i Learning objectives

After reading this chapter you should be able to:

- explain why organizations hold inventory, and what it costs them
- classify an inventory system by its demand, review, and lead-time characteristics
- define the state variables of an inventory system (on hand, on order, backordered, inventory position) and the time averages built from them
- define the cost parameters used by inventory models, and state what each one multiplies
- decide which costs belong in a given parameter and reject those that do not
- estimate each parameter from standard accounting and transaction data
- distinguish the cost and service measures used to evaluate a policy
- state the general problem that the rest of the book solves

1. Introduction to Inventory Systems

1.1. Why Hold Inventory?

Inventory is capital converted into a form that cannot be spent. An organization holding forty million dollars of stock has forty million dollars it cannot use for anything else, sitting in a building it must heat and insure, exposed to theft, damage, and the possibility that nobody will ever want it. Held long enough, some of it will be written off. Thus, on any ordinary accounting of costs, inventory looks like a mistake.

It is not a mistake. Inventory exists to **decouple**, i.e. to break a linkage between two processes that would otherwise have to operate in lockstep. Supply and demand are separated in space, in time, in rate, and in predictability, and each separation forces stock into existence somewhere. Take the inventory away without removing the separation and the two processes must synchronize, which usually means one of them stops.

Each kind of separation creates a distinct kind of inventory, with a distinct cause and therefore a distinct lever for reducing it. Confusing them is the most expensive error in inventory management. In the sections that follow we take the five kinds in turn.

1.1.1. Pipeline Inventory

Supply and demand are separated in **space and time**. Units ordered from a supplier do not arrive at the loading dock the instant they are ordered. They spend the lead time in transit, in production, or awaiting inspection, and during that time they are owned, paid for, and unavailable. Those units are *pipeline* or *in-transit* inventory.

Notice that its size is not a decision. Equation 1.5 gives $\overline{IO} = \lambda \overline{L}$, i.e. the demand rate times the average lead time and nothing else. For example, an organization selling a hundred units a week from a supplier six weeks away owns six hundred units in transit at all times, whatever its reorder

1. Introduction to Inventory Systems

point. The only ways to change that number are to sell less or to source closer.

1.1.2. Cycle Inventory

Supply and demand are separated in **rate**. Consumption is more or less continuous, while replenishment arrives in discrete lots. Between one delivery and the next, the stock drawn down is inventory that exists only because the arrival was a lot and not a trickle. That is *cycle* inventory, and it averages half the lot size.

It exists because replenishing is not free. Something is incurred once per order regardless of size, e.g. the administrative cost k of Section 1.4.5, a production setup, or a truck that costs the same whether half full or full. That fixed charge makes ordering in batches cheaper than ordering continuously. Cycle inventory is the price paid to avoid it.

The lever is the fixed cost, not the ordering rule. Electronic ordering, blanket contracts, and setup time reduction all attack cycle inventory at its cause. Instructing the buyer to order less at a time does not, because it only moves cost from holding into ordering.

1.1.3. Safety Inventory

Supply and demand are separated in **predictability**. Demand over a lead time is not known in advance, and neither, often, is the lead time. Stock held against that uncertainty, beyond what average demand over the average lead time requires, is *safety* inventory.

Safety stock is created by **variability, not by level**. If demand were a steady hundred units a week and the lead time exactly six weeks every time, then no safety stock would be needed at any service target, however demanding, because the reorder point could be set exactly. That is, it is the variance, not the mean, that has to be covered.

1. Introduction to Inventory Systems

It is also the only kind of inventory that is a direct purchase of service. The others buy transportation or production economies, while safety stock buys the probability of being able to fill a demand, which Section 1.6 measures.

1.1.4. Anticipation Inventory

Supply and demand are separated in **rate over a horizon**, in a way that is known in advance. For example, a plant with fixed capacity facing a seasonal peak must build ahead of the peak or fail to meet it, and a distributor expecting a promotion, a price increase, a plant shutdown, or a holiday closure stocks up beforehand.

What separates anticipation from safety inventory is that the variation is *foreseen*. That is, it is covered not because it might happen but because it will. Thus, the lever is capacity or flexibility, not the replenishment policy, and the decision is usually made outside the inventory system.

1.1.5. Speculative Inventory

Speculative inventory is held because the item is expected to cost more later, or to become unavailable. For example, a hedge against a commodity price, a buy-ahead against an announced increase, or a strategic reserve against a supply interruption.

This is a financial position denominated in goods, governed by a view about future prices, not by any mechanics in this book. It appears here because it turns up on real balance sheets, where it can dominate the operational categories and distort any attempt to benchmark them.

1. Introduction to Inventory Systems

1.1.6. Different Causes, Different Levers

Table 1.1 collects the distinctions. Reading across a row gives the cause of that kind of inventory and the intervention that follows from it. Notice that no two rows share an entry in the last column.

Table 1.1.: The kinds of inventory, distinguished by what creates them.
Because the causes differ, so do the interventions that work.

Type	What it decouples	Created by	Reduced by
Pipeline	Supply and demand in space and time	Lead time	Shorter lead time; closer sourcing
Cycle	Continuous consumption from discrete replenishment	A fixed cost per order or per setup	Cheaper ordering; faster setups
Safety	The system from uncertainty	Variability in demand and lead time	Less variability; shorter lead time; less service
Anticipation	Fixed capacity from foreseen variation in demand	A capacity constraint	More or more flexible capacity
Speculative	The purchase decision from the price	An expected price or supply movement	A change of view about the future

A directive to reduce inventory by twenty percent is not actionable, because it does not say which kind. Each of the five responds to a different intervention, most of those interventions take months, and exactly one of them, reducing safety stock, can be executed by decree.

1. Introduction to Inventory Systems

That asymmetry makes the unspecified mandate dangerous. Faced with a target and a deadline, the only lever available on the deadline is the safety stock. So that is the lever that gets pulled, and inventory falls on schedule. The consequence arrives one lead time later as stockouts, by which time it is attributed to demand, to suppliers, or to bad luck. The mandate was met, and the service was spent to meet it.

Figure 1.1 shows the same point for a representative item, using results derived in Chapter 3 and Chapter 8. Each bar is the total stock remaining after one lever has been pulled, with the components stacked so that the pipeline, cycle, and safety contributions can be read separately. Notice that halving the fixed order cost moves only the cycle component, and that halving the variability of demand moves only the safety component. Halving the lead time moves two of the three at once.

That is why lead time wins. It is the only one of the three levers that appears in two components, directly in the pipeline and through the exposure window in the safety stock. Thus, lead time reduction is the most reliably underrated lever in inventory management.

1.1.7. The Cost of Not Holding It

All of this reads as an argument that inventory is a symptom of problems elsewhere, e.g. long lead times, expensive setups, or unpredictable demand. That much is correct. However, the conclusion sometimes drawn from it, that the right target is therefore zero, does not follow.

Inventory is not waste. It is a **purchase**, and what it buys is the ability to serve a demand from stock committed before anyone knew the demand would come. Thus, removing it without removing what created it converts a holding cost into a shortage cost, and shortage costs are the ones that appear on no ledger (Section 1.4.6).

The question is never whether to hold inventory. It is how much, and when to replenish, given what holding costs and what not holding costs.

1. Introduction to Inventory Systems

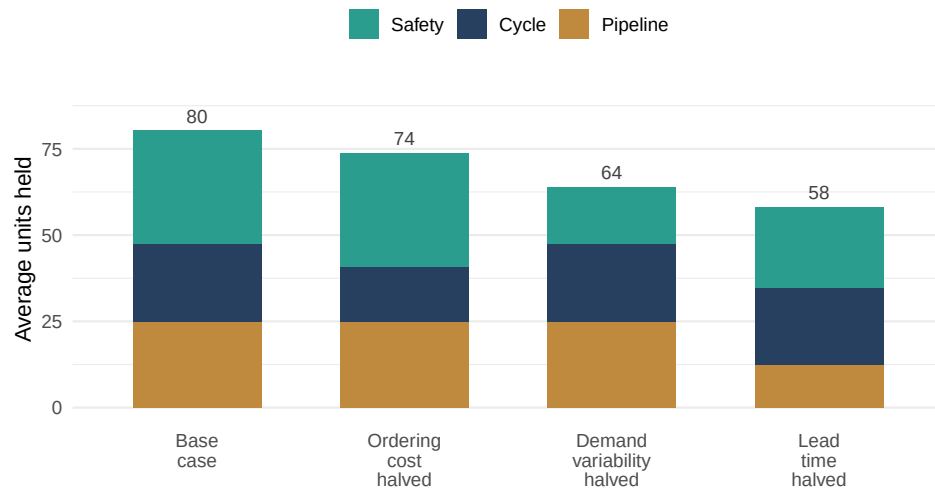


Figure 1.1.: Average inventory for a representative item, decomposed by cause, under three interventions. Cheaper ordering reduces only cycle stock; less variable demand reduces only safety stock; a shorter lead time reduces pipeline stock and safety stock at once, and so achieves the largest total reduction without any loss of service.

1. Introduction to Inventory Systems

Every model in this book answers that pair of questions under a particular set of assumptions about demand, lead times, costs, and review. The assumptions change from chapter to chapter. The question does not.

1.2. Classifying Inventory Systems

There is no single inventory model, and there could not be. For example, a hospital stocking a drug that expires, a distributor replenishing a fast moving consumable from a supplier three weeks away, and a manufacturer scheduling components against next quarter's build plan are all managing inventory, and almost nothing carries from one to the next. What they share is the pair of questions from Section 1.1, how much and when. They do not share the machinery that answers them.

Classification is how you find the machinery. Each dimension below is a question whose answer eliminates some models and selects others, and the chapters of this book are organized around the answers. Notice that most modeling errors in practice are not arithmetic errors. They are a model applied to a system it does not fit.

i A property of the system, not of the model

These are characteristics of the real system, in the sense of Section 1.3. For example, a supplier's lead time has a distribution whether or not your model represents it as a constant. Choosing to treat it as constant is an approximation you make deliberately, having judged that the decision at hand is insensitive to the difference. It is not a discovery that the lead time is constant.

1. Introduction to Inventory Systems

1.2.1. Where the Demand Comes From

The first question, because it governs whether anything else in this book applies.

Independent demand arises outside the system. End customers want the item for its own sake, and their wanting it is not deducible from anything the organization has scheduled. It must be observed, characterized, and, where it is uncertain, described by a probability distribution.

Dependent demand is implied. Demand for a component follows from the production schedule for the assembly that consumes it, and that schedule is already known. Such demand should be *calculated*, not forecast; forecasting it independently discards information the organization already has. Chapter 6 is about the calculation.

Every other dimension below concerns independent demand, so this question comes first. Thus, if you answer it “dependent” the correct treatment is requirements planning, regardless of how the remaining dimensions fall.

1.2.2. Known or Uncertain Demand

This is the dividing line of the book, and the reason it has two parts.

Deterministic demand is known in advance. Known does not mean constant: demand may vary from period to period and still be deterministic, provided the variation is known before the decisions are made. It means no probability distribution is needed.

Stochastic demand is uncertain and must be described by a distribution. The question then changes character, from “what will demand be” to “how much protection against what it might be,” and every result acquires a service dimension it did not have before.

No real demand is deterministic. We use deterministic models anyway, and legitimately, in two situations: when variability is small relative to

1. Introduction to Inventory Systems

the decision, so ignoring it changes nothing that matters, and when the model sits inside a larger deterministic plan, as lot sizing does inside a material requirements plan. Thus, the justification is the quality of the approximation, never a claim that demand is actually known.

1.2.3. Constant or Time-Varying Demand

Within either case, demand may proceed at a steady rate or change over the horizon.

A **constant** rate permits a single order quantity, repeated indefinitely. That is the setting of Chapter 3. A **time-varying** rate destroys that: with demand of 10 units this month and 400 next, no single lot size is sensible, and the problem becomes choosing a schedule of order sizes and timings. Chapter 5 treats the deterministic time-varying case.

The stochastic analogue separates **stationary** demand, whose distribution does not change over time, from **non-stationary** demand, whose does. Seasonality, product life cycles, and promotions all produce non-stationarity, and it is substantially harder. The policies of Chapter 8 and Chapter 9 assume stationarity, and the usual treatment of a seasonal item is to resolve the stationary problem periodically with updated parameters, and not to pretend the assumption holds.

1.2.4. How the System Is Reviewed

Continuous review means the inventory position is known at all times and an order may be placed at any instant. The policies of Chapter 8 trigger on the position crossing a threshold, which requires knowing the moment it crosses.

Periodic review means the position is observed at fixed intervals, and orders may be placed only at those moments. Chapter 9 develops the corresponding policies, in which the review interval becomes a parameter

1. Introduction to Inventory Systems

and adds its own exposure to risk: between reviews, the system cannot respond to anything.

Historically this was a technology constraint, since continuous review required a perpetual inventory record that manual systems could not maintain. That constraint is largely gone, and the distinction survives for a better reason. Review is often *logistically* periodic even when it is informationally continuous: if a truck calls weekly, knowing the position continuously confers no ability to order continuously. Classify by when an order can be placed, not by when the position can be seen.

1.2.5. The Lead Time

The interval between placing a replenishment order and having it available to satisfy demand.

A **zero** lead time is a simplification, not an approximation. With replenishment instantaneous there is no exposure between ordering and receiving, so no reorder point is needed and no safety stock. Several classical results assume it.

A **constant** lead time is the usual working assumption, and it makes the lead-time demand distribution of Chapter 8 tractable.

A **random** lead time compounds with demand uncertainty, since what matters is demand over an interval that is itself uncertain. That is, both the rate and the length of the exposure are unknown when the order is placed. It also raises a possibility usually assumed away: **order crossing**, where an order placed later arrives earlier. Most models assume orders arrive in the sequence placed. That is harmless when lead-time variability is modest relative to the time between orders, and not harmless for a system ordering frequently from a source with erratic transit.

1.2.6. What Happens to Unmet Demand

Whether a demand arriving to an empty shelf waits or departs, discussed as a state distinction in Section 1.3.2.

Under **backordering** the demand waits and is filled later, so the system carries an obligation, $B(t)$, and pays for the waiting through b . Under **lost sales** the demand departs, nothing is owed, and the failure is charged once through π . **Partial backordering**, where some customers wait and others do not, is the common reality and is usually modeled as one of the two extremes with the choice stated.

This is not only a costing convention. It changes what the system can do: under lost sales net inventory can never go negative, and the state description is smaller.

1.2.7. The Planning Horizon

A **single-period** problem is decided once. Stock is committed before demand is observed, demand occurs, and whatever is left has no further use. This is the newsvendor setting of Chapter 7, and it is not the multi-period problem with the number of periods set to one. Leftover stock has no future value, so the trade-off is purely one of overage against underage.

A **finite-horizon** problem has an end date that matters: a service part for equipment being retired, a component for a product with an announced end of life. The last order is a newsvendor problem in disguise.

An **infinite-horizon** problem assumes the system continues indefinitely, which licenses long-run averages as the objective. It is the setting of Chapter 8 and Chapter 9, and the assumption is usually harmless: what it requires is a horizon long relative to the replenishment cycle.

1. Introduction to Inventory Systems

1.2.8. One Item or Many

If items are genuinely independent, a multi-item problem is many single-item problems and nothing is gained by treating them together.

Items become **coupled** in two ways. A **shared constraint** (a budget, a storage capacity, a supplier's minimum) means one item's order quantity restricts another's, so the items must be solved jointly. A **shared setup** (several items on one purchase order or one truck) means ordering them together costs less than ordering them separately, which is the joint replenishment problem. Chapter 4 treats both.

Most organizations manage tens of thousands of items and cannot make a considered decision about each. Classification by value and volume, with policies applied by class instead of by item, is how that is handled, and Section 2.2 treats it.

1.2.9. One Location or Several

A **single stocking point** faces external demand and an external supplier.

A **multi-echelon** system has stocking points that supply each other: a central warehouse replenishing branches, a depot supplying bases. This is not several single-location problems, and treating it as such is a standard and expensive error. The demand a warehouse sees is not customer demand. It is the *replenishment orders* of the locations it serves, which are lumpier than the demand underneath them and are shaped by those locations' own ordering policies. Stock held at one level substitutes for stock at another, so the levels cannot be optimized independently. Chapter 10 takes this up.

1.2.10. The Dimensions Together

Table 1.2 collects the questions and where each answer is treated.

Table 1.2.: The dimensions along which inventory systems differ, and where each is treated.

Dimension	Alternatives	Where treated
Origin of demand	Independent / dependent	Chapter 6 for dependent
Certainty of demand	Deterministic / stochastic	Part I / Part II
Behavior over time	Constant / time-varying; stationary / non-stationary	Chapter 3, Chapter 5
Review	Continuous / periodic	Chapter 8, Chapter 9
Lead time	Zero / constant / random	Chapter 8
Unmet demand	Backordered / lost / partial	Section 1.3.2
Horizon	Single period / finite / infinite	Chapter 7; Chapter 8
Items	One / many independent / many coupled	Chapter 4
Locations	One / multi-echelon	Chapter 10

Read Table 1.2 as a questionnaire, not as a taxonomy. The left column is the question you ask of the system in front of you, the middle column is the set of answers the question admits, and the right column is where this book takes that answer up. Notice that the second row, the certainty of demand, is the only one that splits the book itself. Every row above it and

1. Introduction to Inventory Systems

below it recurs on both sides of that split, so review, lead time, and the treatment of unmet demand each appear twice in the book under different assumptions about demand.

Nine questions describe a large number of combinations, and this book does not address all of them. It addresses the ones that occur, and the ones whose answers illuminate the rest.

1.2.11. Which Costs the Classification Implies

Classifying a system also narrows which of the cost parameters of Section 1.4 matter. Estimation effort is expensive and should not be spread evenly.

Table 1.3.: What the classification implies about which cost parameters matter.

If the system is...	The costs that decide the answer are...
Deterministic, no shortages permitted	k and h only. Shortage costs never arise; unit cost enters only through h .
Deterministic, shortages permitted	k , h , and b . The backorder cost sets how deep the system is allowed to run negative.
Single period	The overage and underage costs: c , the salvage value, and π . Ordering cost is irrelevant because there is one order.
Stochastic, backordering	k , h , and b , or k , h and a service target standing in for b .
Stochastic, lost sales	k , h , and π .

1. Introduction to Inventory Systems

If the system is...	The costs that decide the answer are...
Multi-location	All of the above, plus f , which is what penalizes stocking an item in more places.

Notice that the first row of Table 1.3 names only two parameters. A deterministic system in which shortages are not permitted never realizes a shortage, so no amount of care spent estimating one changes the answer, and Chapter 3 derives its order quantity from k and h alone. Notice also that π and b never appear in the same row. A demand that is lost is not also backordered, and a model that charges both for the same event has double counted, which Section 1.5.6 takes up. Thus, you should settle the classification before any estimation begins, because it tells you which two or three numbers deserve the effort.

1.2.12. Choosing a Classification

Real systems do not present themselves neatly. Demand is neither purely independent nor purely dependent; some customers wait and some do not; lead times are usually stable and occasionally not. Every classification is therefore a decision, and it should be made on a single criterion:

! The criterion

Choose the simplest classification that preserves what the decision is sensitive to.

Simplicity repays real effort, because a simpler classification yields a model whose behavior can be understood and whose parameters can be estimated. But simplicity that discards what the answer depends on is not simplification. It is an error with a smaller model attached.

1. Introduction to Inventory Systems

The criterion is testable. When it is unclear whether a distinction matters, whether lead-time variability needs representing or the occasional lost sale needs its own treatment, model it both ways and compare the recommendations. If they agree, the comparison is the justification for the simpler classification. If they disagree, the question has answered itself. This is one of the things simulation is for, and Chapter 11 returns to it.

1.3. The State of an Inventory System

Before anything can be optimized it has to be described. This section defines the quantities that describe an inventory system at a point in time, and the averages built from them. They are properties of the physical system. That is, an observer with a clipboard, who knows nothing about how replenishment is controlled and nothing about probability, could walk into the warehouse and measure every one of them.

You should insist on that, because it locates what a model is for. The deterministic models of Chapter 3 and the stochastic models of Chapter 8 are not describing different systems, and they do not have different state variables. They are two ways of *computing* the same quantities under different assumptions about what is known. Everything in Section 1.4 and Section 1.6 is expressed in the variables defined here.

1.3.1. On Hand, On Order, and Backordered

Three counts describe the system at time t . Note that a policy does not read these counts from the shelf; it reads them from a record, and the two can differ. Section 2.3 is about that gap and how it is measured and closed.

Let $I(t)$ represent the **inventory on hand** at time t , meaning the units physically present and available to satisfy a demand. Let $IO(t)$ represent the **inventory on order**, meaning the units ordered from the supplier

1. Introduction to Inventory Systems

but not yet arrived, the replenishment pipeline of Section 1.1.1. Let $B(t)$ represent the **backorders**, meaning the units demanded but not yet supplied, which the system is still committed to supplying.

All three are counts of units and none of them can be negative. From them we build two composites. The **net inventory**

$$IN(t) = I(t) - B(t) \quad (1.1)$$

is positive when the system holds stock and negative when it owes units, and it is the single number that summarizes whether the system is ahead or behind. The **inventory position**

$$IP(t) = I(t) + IO(t) - B(t) \quad (1.2)$$

adds what is already coming.

One structural fact does a great deal of work throughout the book:

! On hand and backordered are never both positive

If $I(t) > 0$ then $B(t) = 0$, and if $B(t) > 0$ then $I(t) = 0$.

A system holding stock while owing units would be one that had received inventory and declined to ship it to a waiting customer. Under the ordinary assumption that arriving replenishments fill outstanding backorders first, that situation never arises. Thus, at every instant at most one of the two is positive, and $IN(t)$ determines both: $I(t) = [IN(t)]^+$ and $B(t) = [IN(t)]^-$.

This is also what makes the cost model of Section 1.4 coherent. The holding cost charges $I(t)$ and the backorder cost charges $B(t)$; because the two are never simultaneously positive, no unit is ever charged both ways, and the total is unambiguous.

1. Introduction to Inventory Systems

1.3.2. Lost Sales

Everything above assumes an unsatisfied demand *waits*. That is one of two possibilities, and the other changes the state description and not merely the arithmetic.

Under **lost sales**, a demand that cannot be filled from stock departs and does not return. Nothing is owed, so

$$B(t) \equiv 0, \quad IN(t) = I(t) \geq 0, \quad IP(t) = I(t) + IO(t)$$

for all t . The state collapses to two counts, net inventory can never go negative, and the backorder cost b has nothing to multiply.

What takes its place is the **rate of lost demand**. Let λ_ℓ represent the rate, in units per unit time, at which demand arrives and departs unfilled, and let λ represent the overall demand rate in the same units. Since a demand is lost exactly when it arrives to an empty shelf,

$$\lambda_\ell = \lambda (1 - \overline{FR}_u) \tag{1.3}$$

where $\overline{FR}_u$ is the fraction of demanded units filled from stock, defined in Section 1.6. This is the quantity the stockout cost π multiplies in Table 1.4, and writing it out shows why that term is a genuine rate while the backorder term is a level. That is, lost demand is a flow of units past the system, whereas backorders are a population sitting inside it.

The same quantity exists under backordering, since demand arrives to an empty shelf there too, but it is not what the cost model charges. There the harm is the waiting, not the loss.

Which regime applies is a fact about the system, not a modeling convenience. For example, a distributor whose customers will wait a week for a spare part is backordering, while a retail shelf whose customers walk to a competitor is losing sales. Many real systems are partially both, and the usual treatment

1. Introduction to Inventory Systems

is to model the dominant case and say so. This book develops the backorder case in detail, because it is the harder of the two and because the lost-sales results follow from setting $B(t) \equiv 0$.

1.3.3. Why Ordering Decisions Use the Inventory Position

Every replenishment policy in this book triggers on $IP(t)$, never on $I(t)$ alone, and you should understand the reason before the policies arrive.

Suppose a system reorders whenever on-hand inventory falls to some level. Stock runs down, the level is reached, an order is placed. The lead time has not yet elapsed, so on-hand inventory keeps falling, and the trigger level is still breached, so the rule places another order, and another, each for a shortfall the first order is already on its way to cover. Thus, by the time the shipments arrive the system is grossly overstocked.

Including $IO(t)$ is what prevents this. Because outstanding orders are counted in $IP(t)$, placing an order raises the position immediately, and the system orders again only when what is already outstanding is no longer enough. The position is what the system will *eventually* have; the on-hand inventory is only what it has now.

Figure 1.2 shows both for a system operating an (r, Q) policy, which orders Q units whenever the position falls to r . Let r represent the reorder point, in units, and let Q represent the fixed order quantity, also in units. The position sawtooths between r and $r + Q$. Net inventory follows the same pattern delayed by the lead time, so net inventory, and not the position, runs short.

Walk across Figure 1.2 from left to right. The dashed line is the inventory position and the solid line is net inventory. Both start high, just after a receipt, and both decline at the demand rate. Notice that the two lines lie on top of each other over the early part of each cycle. That is the stretch during which no order is outstanding, so $IO(t) = 0$ and Equation 1.2 reduces to Equation 1.1. The dashed line then reaches the dotted line

1. Introduction to Inventory Systems

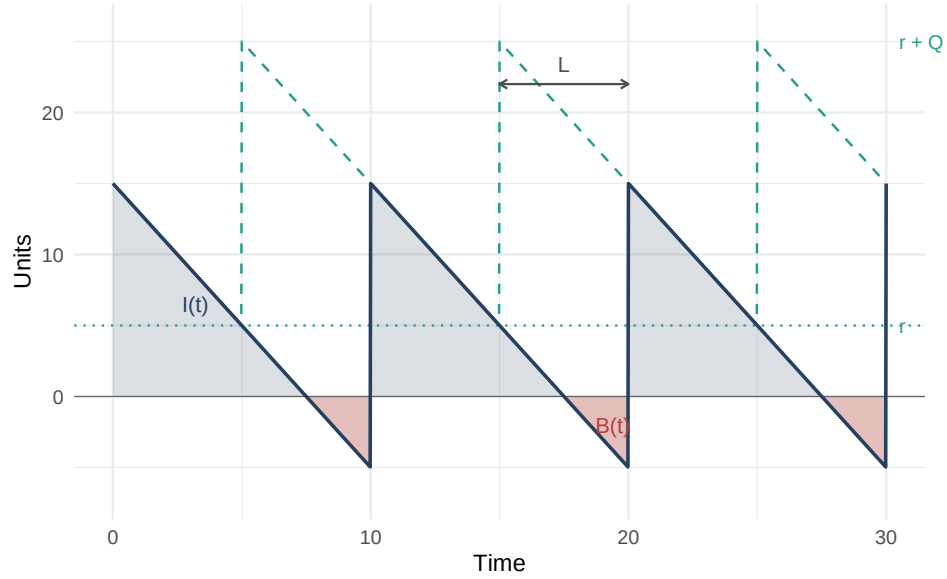


Figure 1.2.: Inventory position and net inventory for a system operating an (r, Q) policy with constant demand. The position (dashed) falls to r , an order is placed, and it jumps by Q ; net inventory (solid) repeats the same pattern delayed by the lead time L , which is why it, and not the position, is what runs short. Where net inventory is positive it is inventory on hand $I(t)$; where it is negative its magnitude is the backorders $B(t)$. While no order is outstanding the two curves coincide, because $IO(t) = 0$ there.

1. Introduction to Inventory Systems

at r , an order is placed, and the position jumps by Q to $r + Q$ while net inventory does not move at all, because nothing has arrived. The two lines run apart for exactly one lead time L , marked by the double arrow. Net inventory continues its decline through that interval, crosses zero, and the shaded region below the axis is the backorders $B(t)$ accumulating while the order is in transit. When the order arrives, net inventory jumps by Q , the backorders are filled first, and the cycle repeats. Thus, the reorder point r does not determine when the system runs out. What determines that is r measured against demand over the lead time. Chapter 8 is built on that calculation.

Each state variable is a **time-persistent** quantity: it holds a value over an interval instead of being observed at isolated instants. Thus, its average is a time average, computed by integrating over the observation interval and dividing by its length. Let T represent the length of that interval, in the same time units as the demand rate, and let $\bar{I}$ and $\bar{B}$ represent the resulting averages, in units:

$$\bar{I} = \frac{1}{T} \int_0^T I(t) dt, \quad \bar{B} = \frac{1}{T} \int_0^T B(t) dt \quad (1.4)$$

and likewise for $\overline{IO}$, $\overline{IN}$, and $\overline{IP}$. Because integration is linear and $IN(t) = I(t) - B(t)$ at every instant, the averages inherit the same relation, $\overline{IN} = \bar{I} - \bar{B}$. Under conditions discussed in Chapter 11, these averages converge as T grows to the system's steady-state performance.

These integrals are where the two halves of the book meet. In a deterministic model $I(t)$ is a known piecewise-linear function and the integral is the area of a triangle. For example, Chapter 3 computes $\bar{I} = Q/2$ this way, and the familiar result is nothing more than the area of the sawtooth in Figure 1.2 divided by its base. In a stochastic model the same integral is an expectation, computed from the distribution of the process or estimated from a simulation run. Thus, the quantity being computed does not change; only the method does.

1.3.4. Little's Law and the Averages It Connects

Several relationships hold regardless of the replenishment policy, and all of them are Little's law applied to a different flow of units. Little's law states that for any system in steady state, the average number of items inside equals the arrival rate times the average time each spends there. Thus, choosing what counts as "inside" is what generates the results below.

Pipeline inventory. Units enter the pipeline when ordered and leave it when received, spending the lead time inside. Applying Little's law to that flow,

$$\overline{IO} = \lambda \overline{L} \quad (1.5)$$

Let $\overline{L}$ represent the average lead time, in the same time units as the demand rate. Average on-order inventory is the demand rate times the average lead time, and nothing else. Notice that no ordering policy appears in Equation 1.5, so no ordering policy can reduce it. You should know this early, because pipeline stock is often mistaken for a target of improvement. It is a consequence of how fast you sell and how long your supplier takes, and it responds only to changing one of those.

Backorders and waiting. Units enter the backorder queue when demanded and unfilled, and leave when supplied, spending the customer's wait inside. The same argument gives

$$\overline{B} = \lambda \overline{W} \quad (1.6)$$

where $\overline{W}$ represents the average time a demanded unit waits, counting zero for units filled immediately. That is, average backorders and average customer waiting are two views of one quantity. This is why Section 1.5.5 can convert a statement management is willing to make, a tolerable wait, into the quantity that the backorder cost b multiplies.

1. Introduction to Inventory Systems

Stocking time and turnover. Apply the same argument to the stock on the shelf. Units enter when a replenishment is received and leave when they are demanded, so if $\bar{T}_s$ denotes the average time a unit spends on hand,

$$\bar{I} = \lambda \bar{T}_s \quad \Longleftrightarrow \quad \bar{T}_s = \frac{\bar{I}}{\lambda} \quad (1.7)$$

Units that arrive to fill an outstanding backorder ship immediately and spend no time on the shelf; they enter the average as zeros, and the relation still holds.

Its reciprocal is a quantity every operations manager already knows. The **inventory turnover ratio**

$$TO = \frac{\lambda}{\bar{I}} = \frac{1}{\bar{T}_s} \quad (1.8)$$

counts how many times the average stock is consumed and replaced per unit time. Turnover is usually quoted annually and computed in dollars, as cost of goods sold divided by average inventory value. Multiplying numerator and denominator by the unit cost c changes nothing, so the dollar ratio and the unit ratio are the same number.

That equivalence is more useful than it looks. Turnover is reported by almost every organization and understood by people who would not sit through a derivation, and Equation 1.8 says it is nothing more than the reciprocal of the average time a unit sits on a shelf. “We turn this item four times a year” and “a unit sits here three months on average” are the same statement. It also supplies a sanity check available before any modeling begins: an item whose computed $\bar{I}$ implies a turnover far from what the business reports has been mis-specified somewhere.

1. Introduction to Inventory Systems

1.3.5. Stockout Indicators

Whether the system is currently able to fill a demand is itself a quantity we can average. Define the **stockout indicator**

$$SO(t) = \begin{cases} 1 & I(t) \leq 0 \\ 0 & I(t) > 0 \end{cases} \quad (1.9)$$

Since $I(t)$ is a count of units on hand it is never negative, so the condition $I(t) \leq 0$ is the event $I(t) = 0$; it is written with the inequality because a demand arriving at the instant the shelf empties cannot be filled, and the convention should make that unambiguous.

The **ready indicator** is its complement, $RR(t) = 1 - SO(t)$. Averaging either over time gives a proportion:

$$\overline{SO} = \frac{1}{T} \int_0^T SO(t) dt \quad (1.10)$$

is the proportion of time the system is out of stock, and $\overline{RR} = 1 - \overline{SO}$ is the proportion of time it has stock on hand. Section 1.6 turns these into service measures and explains why $\overline{RR}$ is *not* interchangeable with the fraction of demand that gets filled, even though the two are frequently reported as though they were.

1.3.6. The Demand Process

Demand arrives as a sequence of **transactions**. Let t_i represent the time at which the i th transaction arrives, and let D_i represent the number of units it requests, a positive integer. Over an interval of length T containing n transactions, the demand rate is

1. Introduction to Inventory Systems

$$\lambda = \frac{1}{T} \sum_{i=1}^n D_i \quad \text{units per unit time,}$$

while n/T is the **transaction rate**, in transactions per unit time. The two differ by the average transaction size, and keeping them distinct matters more than it first appears. For example, a location seeing 500 units a year in five orders of 100 is a different system from one seeing 500 units in 500 orders of one, even though λ is identical in both. The first will run a large order quantity and spend most of its time either well stocked or badly short; the second will behave far more smoothly.

The distinction also divides the service measures of Section 1.6. Some count units, some count transactions, and on a demand stream with $D_i > 1$ they give different answers about the same history.

The state variables defined here are what the next section prices. Nothing in Section 1.4 introduces a new quantity to be measured; it attaches a rate of money to each of the quantities already defined.

1.4. Cost Parameters

An inventory policy is a rule, and a rule has no cost. What costs money is the *behavior* the rule produces: orders placed at some rate, stock sitting on a shelf for some length of time, demands arriving when the shelf is empty. The purpose of a cost model is to price that behavior, so that two rules can be compared on something other than intuition.

1.4.1. What an Inventory Model Charges

Every model in this book computes a total cost that is a **rate**, dollars per unit time, and every term in it has the same shape:

1. Introduction to Inventory Systems

$$\text{cost rate} = \text{price} \times \text{quantity the system exhibits} \quad (1.11)$$

The division of labor matters. The quantities on the right, how often orders are placed and how much stock is on hand and how many units are owed, are properties of the system, and computing them is the technical content of the chapters that follow. The *prices* on the left are what you supply, and no amount of modeling will produce them.

Table 1.4 sets out the division for the terms that appear throughout the book. The state variables in its right column are defined in Section 1.3.

Table 1.4.: The two halves of every cost term. Only the left column requires data about your organization; the right column is what the models in this book are for.

Term	Price (you supply)	Quantity (the model computes)
Ordering	k , dollars per order	order rate, orders per unit time
Holding	h , dollars per unit per unit time	average on-hand inventory $\bar{I}$, units
Backorder	b , dollars per unit per unit time	average backorders $\bar{B}$, units
Stockout	π , dollars per unit	rate of demand not filled from stock, units per unit time
Purchase	c , dollars per unit	demand rate λ , units per unit time
Stocking position	f , dollars per item per location per unit time	<i>none</i>

Read Table 1.4 one row at a time, checking the units as you go. The ordering row multiplies dollars per order by orders per unit time, and the

1. Introduction to Inventory Systems

orders cancel. The holding row multiplies dollars per unit per unit time by a number of units, and the units cancel. Every row in the table reduces to dollars per unit time, so the rows can be added. Notice that the last row has no quantity at all. Three features of the table shape everything that follows.

Some prices are themselves rates. The holding and backorder prices are quoted per unit *per unit time*, so they multiply a **level**, a number of units, and not a rate, and the product is still dollars per unit time. The ordering and stockout prices multiply genuine rates. Thus, this is the commonest source of dimensional error in a first cost model, and the check is the one just performed: every term must reduce to dollars per unit time.

The purchase term is inert. It is $c\lambda$, and when demand is not itself a decision, which through Chapter 7 it is not, this term is identical under every policy and cannot influence the choice among them. Unit cost still matters, but it matters *through the holding cost*, as we are about to see, not on its own account. Including $c\lambda$ in a comparison is a common error, and its symptom is that the alternatives all look suspiciously alike, because a large constant has been added to both sides.

The stocking position term has no quantity at all. It is charged from the moment an item is carried at a location, identically whether one unit or ten thousand pass through it. That absence is not an oversight. It makes this the parameter that decides marginal cases. See Section 1.4.7.

1. Introduction to Inventory Systems

i A note on time units

The models are written in a generic time unit. Whether it is a day, a week, or a year is a matter of convention, and only two rules matter: every rate in a model must use the same unit, and every price quoted per period must be converted to that unit on entry. Accounting data arrives annually and simulations usually run in days, so a conversion is nearly always required. A missed one is the error most likely to survive review, because the answer remains plausible while being wrong by a factor of 365.

1.4.2. The Marginality Test

Almost every difficulty in cost estimation reduces to one question, and it has one answer.

! The test

A cost belongs in a parameter **only if it changes when the quantity that parameter multiplies changes.**

Recall from Equation 1.11 that every cost term is a price times a quantity the system exhibits. Thus, applying the test means naming the quantity first. The holding cost h multiplies average on-hand inventory $\bar{I}$, so for a cost to belong in h it must rise when one more unit is held for one more unit of time. The ordering cost k multiplies the order rate, so a cost belongs in k only if it rises when one more order is placed. A cost that does neither is a **period cost**: real, possibly large, and irrelevant to the decision, because it is incurred identically under every policy you are choosing among.

For example, the classic failure is a fully allocated warehouse cost per unit. The building's lease, its depreciation, its heating, and its management are divided by the units that passed through, producing a defensible-looking

1. Introduction to Inventory Systems

dollars-per-unit figure. That figure is correct as an accounting statement and wrong as a model input, because none of those costs change if you hold one more unit. Charging it as though it did will overstate holding cost, understate order quantities, and recommend ordering more often than is economical.

This is the point at which the approach is usually accused of ignoring real money. It is not. Period costs are real and must be paid; they simply do not depend on the decision at hand, so including them changes the answer without changing reality. An accountant computing the true cost of operating a warehouse and an analyst computing the cost parameters for a lot sizing model are answering different questions, and will legitimately produce different numbers from the same ledger. Neither is wrong. Confusing the two is.

1.4.3. Unit Cost, c

The value of one unit, in dollars per unit. What matters is the money tied up in the asset, which is normally the **latest acquisition cost**, what it would cost to replace the unit now, and not an internal transfer price or a customer-facing standard price.

Unit cost enters the models in two distinct ways, and only one of them affects the policy. It appears in the purchase term $c\lambda$, which is constant across policies and therefore inert. It also appears inside the holding cost, because the dominant cost of holding a unit is the capital it ties up, in proportion to what the unit is worth. Thus, the second route is the one that matters.

1.4.4. Holding Cost, h

The cost of holding one unit in inventory for one unit of time. It is almost never estimated directly. Instead it is built from unit cost and a

1. Introduction to Inventory Systems

dimensionless **carrying charge** i :

$$h = i c \tag{1.12}$$

Let i represent the carrying charge, expressed as dollars per dollar of inventory value per unit time, and let c represent the unit cost in dollars per unit, as defined in Section 1.4.3. That is, a carrying charge of 0.20 per year means it costs twenty cents per year to hold a dollar's worth of stock. Writing h this way is not only convenient. It says that the cost of holding is proportional to value, so a single carrying charge serves an entire portfolio of items with different prices.

Carrying charges are quoted annually by universal convention, so a model running in days uses $h = ic/365$. Section 1.5.3 builds i from accounting data, where the natural period is again the year.

Note that h multiplies **on-hand** inventory $\bar{I}$, not net inventory $\overline{IN}$. The two differ exactly when the system is short, and the difference is charged separately, by b . Treatments that write the holding term on net inventory are assuming backorders are negligible; Chapter 3 does not make that assumption, and Section 1.3 makes the distinction precise.

Which costs belong in i is the question the marginality test exists to answer, and the conventional list is not reliable. Table 1.5 applies the test to each component in turn.

1. Introduction to Inventory Systems

Table 1.5.: Applying the marginality test to the components conventionally listed under carrying cost.

Component	Belongs in i ?	Why
Cost of capital	Yes	Money tied up in stock is unavailable for anything else. Unambiguous, and usually the floor.
Obsolescence, deterioration, shrinkage	Yes	Exposure is proportional to the value held and to how long it is held.
Insurance and inventory taxes	Yes, where assessed on inventory value	Insurers rate on declared value; some jurisdictions levy an ad valorem inventory tax and many do not.
Storage and space	Only where marginal	In an owned facility below capacity, the marginal space cost of one more unit is zero.
Handling	Rarely	Receiving, picking, and shipping scale with <i>throughput</i> , which is set by customer demand, not by the policy.

Notice that the three rows answering an unqualified **Yes** are the three whose exposure grows with the value held and with the time it is held, exactly the quantity h multiplies. The two remaining rows are qualified, and they are the ones that surprise people. We justify each of them.

1. Introduction to Inventory Systems

Storage. In a facility you own, operating below capacity, the rack exists whether or not anything is on it. The lease is signed, the depreciation accrues, and holding one more unit costs nothing in space. Charging a percentage for space in that setting treats a fixed period cost as variable. Storage genuinely *is* marginal in three situations, and belongs in i in each of them: when space is rented from a third party and billed per pallet position per period; when the facility is at or near capacity, so that additional stock forces overflow leasing or displaces something else, in which case the right figure is the shadow price of capacity rather than the average cost per square foot; and when a lease genuinely scales with the space consumed.

Handling. Every unit that arrives is received and put away, and every unit that leaves is picked and shipped. That work is real, but its volume is set by how much customers buy, not by how the replenishment policy is configured. It does not change with the order quantity or the reorder point. Per-receipt handling belongs in the ordering cost k , where it does scale with the decision; per-unit handling belongs in unit cost. Only stock-driven handling, such as re-warehousing, relocation, and congestion caused by holding too much, belongs in i , and it is usually small and difficult to isolate.

1.4.5. Fixed Order Cost, k

The cost of placing one replenishment order, in dollars per order, independent of how large the order is. It covers the administrative work a replenishment triggers: creating and approving the requisition, transmitting it, receiving and inspecting the shipment, matching the invoice, and paying it.

The word *fixed* in “fixed order cost” means fixed **with respect to the order quantity**, not fixed with respect to the number of orders. This trips up nearly everyone once. That is, a cost that scales with the number of units ordered is not part of k and belongs in c , while a cost that is incurred once per order regardless of size is exactly what k is.

1. Introduction to Inventory Systems

Inbound freight is the usual complication. Where a replenishment order causes a shipment, the fixed portion of that shipment's freight behaves exactly like an ordering cost and belongs in k . That fixed portion is often larger than it appears: carriers charge a minimum, and for many items an order sits below the minimum across its entire realistic range, so the freight bill does not vary with order size at all. The variable portion, a rate per pound say, scales with the number of units and behaves like unit cost, with the same inertness as $c\lambda$, because it depends on the demand rate, which the policy does not change. Where a lane is *scheduled* instead of on demand, the truck departs whether or not you ordered, and no part of its cost is caused by your order. Chapter 10 treats freight explicitly, where the choice of where to stock changes which lanes are used. In this chapter it is enough to know that the fixed component belongs in k and must be counted exactly once. See Section 1.5.6.

1.4.6. Stockout and Backorder Costs, π and b

When a demand arrives and cannot be filled from stock, something bad happens. What that something *is* depends on what becomes of the unsatisfied demand, and the two possibilities are priced differently.

If the demand is lost, meaning the customer buys elsewhere and does not return for this order, the harm is a discrete event incurred once per unit not supplied. It is priced by the **stockout cost**

$$\pi = \text{dollars per unit of demand not filled from stock,}$$

and the corresponding cost rate is π times the rate at which demand goes unfilled, the quantity λ_ℓ of Equation 1.3.

If the demand is backordered, meaning the customer waits and is supplied later, the harm is a *duration*, not an event. One unit owed for

1. Introduction to Inventory Systems

ten days is worse than one unit owed for one day, and a cost charged per incident cannot express that. It is priced by the **backorder cost**

b = dollars per unit backordered per unit time,

and the corresponding cost rate is $b\bar{B}$, where $\bar{B}$ is the average number of units backordered. This is the parameter that governs the general deterministic model of Chapter 3, in which the system is deliberately allowed to run short because carrying less stock is worth more than the waiting it causes.

The two are not interchangeable notation for one idea; they describe different physical situations. Some settings need both, a per-incident charge for the expedite a stockout triggers plus a per-unit-time charge for the waiting it imposes, and some models charge both. What must not happen is charging both for the *same* consequence, one of the double counts catalogued in Section 1.5.6.

i Backorders and waiting are the same measurement

Charging b per unit backordered per unit time is the same as charging for customer waiting time. If demand arrives at rate λ and the average customer waits $\bar{W}$, then $\bar{B} = \lambda\bar{W}$, so $b\bar{B} = b\lambda\bar{W}$. A backorder cost and a delay cost are two views of one quantity, and Section 1.3 derives the relationship. This matters in practice: organizations that cannot price a backorder can often state a tolerable customer wait, and that statement is enough.

In practice, organizations rarely produce a defensible number for either π or b . The standard response is not to force one. Instead a **service target** is specified and the policy chosen to meet it at minimum cost. Thus, the two formulations are closely related, since any service target implies a shortage cost and any shortage cost implies a service level. Section 1.6

1. Introduction to Inventory Systems

defines the service measures, and Section 1.5.5 shows how to recover an implied shortage cost from a target the organization already operates to.

1.4.7. Stocking Position Cost, f

The cost, per unit time, of carrying an item at a location at all: the catalogue record, the bin reserved for it, the cycle counts it attracts, the planner attention it consumes. Conventionally quoted as dollars per item per location per year.

! This is not holding cost

Holding cost scales with the quantity on hand. Stocking position cost is **independent of quantity held**, identical whether one unit or a thousand sits in the bin. The two must not be merged, and neither should be folded into the other.

This parameter does not appear in classical single-location treatments, and it is easy to dismiss as a detail. It is not. For a typical mid-value item it is frequently *larger* than the holding cost of the stock it houses, and because it multiplies nothing, it is the only term that penalizes stocking an item in more places. A model without it will forward-position marginal items that no practical organization would ever stock, because it sees no reason not to.

It becomes central in Chapter 10, where the decision is not only how much to stock but *where*. It is introduced here because it is a cost parameter of inventory systems, and leaving it out of the definitions invites the assumption that holding cost is the only cost of carrying an item.

1.4.8. How Accurate Do These Need to Be?

A reasonable worry, having seen how much judgment goes into these numbers, is whether the resulting model can be trusted at all. The answer has a reassuring half and an uncomfortable half, and you should know both before doing the work.

The reassuring half is that the order quantity depends on k/h under a square root, as Chapter 3 derives, so errors are compressed. Let θ represent the factor by which the estimated ratio k/h departs from its true value, so that $\theta = 1$ is a perfect estimate. A parameter wrong by a factor of four moves the order quantity by a factor of two and, as Figure 1.3 shows, costs far less than that in dollars.

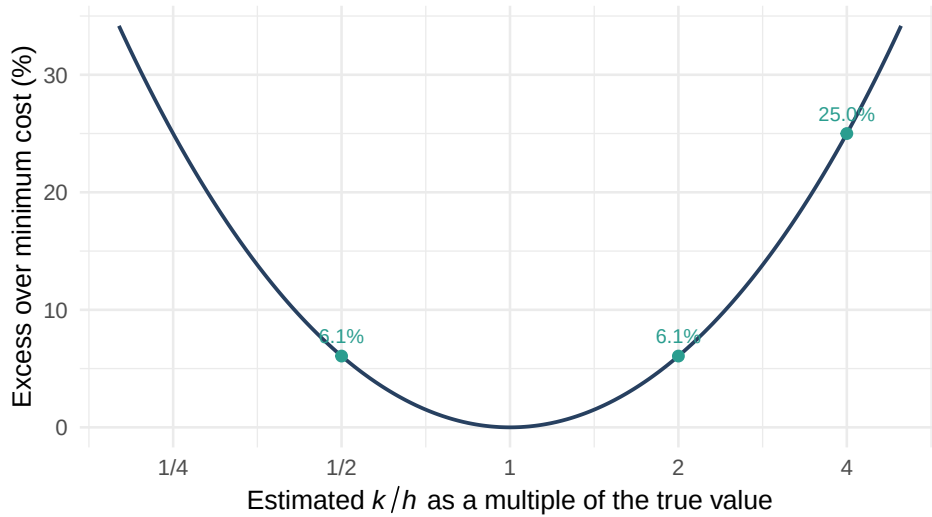


Figure 1.3.: Cost penalty from a misestimated parameter. If the ratio k/h is wrong by a factor θ , the resulting order quantity is wrong by $\sqrt{\theta}$, and the relevant cost rate exceeds its minimum by the amount shown. The curve is flat near the optimum: being wrong by a factor of two costs about 6%.

1. Introduction to Inventory Systems

Read Figure 1.3 from its minimum outward. The horizontal axis is θ on a doubling scale, so equal distances left and right are equal factors of error, and the vertical axis is the percentage by which the resulting cost rate exceeds the minimum. The curve touches zero at $\theta = 1$, where the estimate is exact. Notice how flat it is on both sides of that point. Estimating k/h at half its true value or at twice its true value costs about 6%, and being wrong by a factor of four still costs only about 25%. Notice also that the curve is symmetric on the doubling scale, so overestimating the ratio by a factor and underestimating it by the same factor carry the same penalty. Thus, the square root in the order quantity is doing the protecting, and it protects a great deal.

The uncomfortable half is the same fact seen from the other side. Because the cost surface is flat, a badly wrong parameter produces a cost that looks perfectly acceptable while the *policy* it recommends is substantially wrong. The model will not complain. If the recommendation is a number of units to order, a 6% cost penalty may be tolerable. If the recommendation is which of two configurations to adopt, such as whether to stock an item at a location at all, then a parameter error costing 6% can flip the decision outright, and the flatness that protected the cost is what makes the comparison delicate.

Two practical consequences follow, and both shape the estimation work in Section 1.5. First, effort should go where the answer is sensitive, not uniformly across parameters. Second, when a parameter cannot be pinned down, the right move is often not to defend a point estimate but to report the value at which the recommendation changes.

1.5. Estimating Cost Parameters from Standard Data

The parameters in Section 1.4 are defined precisely enough to be estimated, and every one of them can be built from data an organization already keeps. This section shows how. The examples are notional, but the methods and

1. Introduction to Inventory Systems

the magnitudes are representative of distribution practice. Every figure below is annual, following the convention of the accounting records these numbers come from; converting to the model's time unit is the last step, not the first.

1.5.1. Where the Numbers Come From

Four sources carry essentially everything needed:

- **The general ledger:** write-offs and disposals, insurance premiums, departmental budgets, average inventory value at cost.
- **ERP transaction counts:** purchase orders placed, receipts processed, lines picked. These supply the denominators.
- **The item master:** unit cost, unit of issue, weight and cube.
- **Contracts and instruments:** the insurance policy, the tax assessment, the carrier tariff, the published cost-of-capital rate. These are read, not estimated, and are the most defensible inputs available.

Notice what is absent from that list: no new measurement, no study, no instrumentation. Thus, the work is one of selection and arithmetic, and the selection is governed by the marginality test of Section 1.4.2.

1.5.2. Unit Cost

Take the **latest acquisition cost** from the item master, not the standard or list price.

The distinction matters whenever an internal price carries a cost-recovery surcharge, which is common in organizations that bill their own operations for material.

1. Introduction to Inventory Systems

Example 1.1 (Acquisition cost against standard price). An item's records show two prices: a standard price of \$7.35, which is what internal customers are billed, and a latest acquisition cost of \$6.11, which is what the organization paid. The ratio is

$$\frac{7.35}{6.11} = 1.203$$

so the standard price carries a 20.3% cost-recovery markup. Recall that c is meant to be the money tied up in the asset, what the organization paid and not what it bills internally. Thus, using the standard price as c overstates the capital tied up in each unit by 20.3%, and because $h = ic$ by Equation 1.12 it overstates the holding cost by the same fraction.

That error does not stay put. Holding cost is one of the two quantities the order quantity balances, so an inflated h produces order quantities that are too small, and it does so for every item priced this way.

If only the standard price is available, divide by one plus the cost-recovery rate in force for the period, and record the adjustment as an explicit assumption. It is a single multiplier and easy to revise later.

1.5.3. The Carrying Charge

We build i component by component, applying the test in Table 1.5. We include only the components that survive the test, and we compute each as a fraction of average inventory value.

Cost of capital. Read it off, do not estimate it. Use the published rate on the marginal use of funds: a Treasury or OMB rate in a government setting, the weighted average cost of capital commercially. This is the one component nobody argues about, and it sets a floor under i .

Obsolescence, deterioration, and shrinkage. Usually the largest component, and recoverable directly from accounting:

1. Introduction to Inventory Systems

$$\text{obsolescence rate} = \frac{\text{annual write-offs} + \text{disposals} + \text{net negative inventory adjustments}}{\text{average inventory value at cost}} \quad (1.13)$$

Write-offs and disposals come from the general ledger; average inventory from monthly balances.

Compute this **by commodity class, not portfolio-wide**. Items supporting equipment near the end of its service life run far above consumables, and that spread is the single largest reason two organizations' carrying charges differ. A portfolio average charges the fast-moving consumable for the obsolescence risk of the spare part.

Insurance and taxes. Read off the instruments. Insurers rate on declared inventory value, so the component is the premium attributable to inventory divided by average inventory value. Ad valorem inventory tax applies only where the jurisdiction levies it, which is often nowhere. In practice this component is well under 1%, and it does not repay extended effort.

Storage, where it qualifies. Only in the three marginal cases of Section 1.4.4. Where it does qualify, it is the contracted rate times the positions consumed, divided by inventory value.

Example 1.2 (Building a carrying charge). A depot carries \$70 million of inventory at cost, in a building it owns and operates at about 60% of capacity. Its published cost of capital is 5.0%. Last year it wrote off and disposed of \$4.2 million of stock, and its insurance premium attributable to inventory was \$180,000. Receiving and picking labour cost \$3.1 million.

We take the components that survive the marginality test, each as a fraction of the \$70 million base:

$$\text{obsolescence} = \frac{4,200,000}{70,000,000} = 6.0\%, \quad \text{insurance} = \frac{180,000}{70,000,000} = 0.26\%$$

1. Introduction to Inventory Systems

Table 1.6.: A carrying charge built from components that survive the marginality test.

Component	Value
Cost of capital	5.00%
Obsolescence and shrinkage	6.00%
Insurance	0.26%
Storage	excluded: owned, below capacity
Handling	excluded: throughput-driven
Carrying charge i	11.3% per year

Notice that Table 1.6 lists five components and includes only three of them. We exclude storage because the building exists whether or not the rack is full, so one more unit costs nothing in space. We exclude handling because the \$3.1 million of receiving and picking scales with throughput, which customers set, not with how the replenishment policy is configured. Adding the three surviving components gives $5.00 + 6.00 + 0.26 = 11.26\%$, which we report as 11.3% per year. You should perform that addition every time, because a carrying charge that does not reconcile to the sum of its own table has usually picked up a component twice.

Now change one fact. Suppose the same inventory sits in third-party warehousing at \$18 per pallet position per month, occupying 8,400 positions. Space is now bought by the position and is unambiguously marginal. Carrying the units through the arithmetic,

$$\left(\frac{18 \text{ dollars}}{\text{position-month}}\right) \left(\frac{12 \text{ months}}{\text{year}}\right) (8,400 \text{ positions}) = \frac{1,814,400 \text{ dollars}}{\text{year}}$$

so that

$$\text{storage} = \frac{1,814,400}{70,000,000} = 2.6\%$$

1. Introduction to Inventory Systems

which raises i from 11.3% to 13.9% per year. Notice that the physical inventory has not changed at all. What changed is whether holding one more unit costs anything in space, and that alone moves the carrying charge by a quarter.

We compare the 11.3% figure against the 20% to 25% often quoted as a rule of thumb. The gap is not an error in either direction; it is the storage and handling components, which the rule of thumb includes and the marginality test excludes. Which is right depends entirely on whether those costs are marginal in the setting at hand. In a third-party warehouse billed by the pallet position they certainly are, and i will be much closer to the rule of thumb. In an owned building at 60% occupancy they are not.

 This choice is not neutral

Because Q^* varies as $1/\sqrt{h}$, moving i from 18% to the 11.3% of Example 1.2 raises order quantities across the portfolio by $\sqrt{18/11.3}$, about 26%. An inflated carrying charge does not merely add noise: it systematically penalizes whichever alternative holds more inventory. In any comparison where the alternatives differ in how much stock they carry, the direction of the error is predictable, which makes an unjustified carrying charge worse than an uncertain one.

1.5.4. The Fixed Order Cost

We divide the annual cost of the functions that scale with transaction count by the number of transactions processed:

$$k = \frac{\text{annual transaction-driven cost}}{\text{orders placed per year}} \quad (1.14)$$

1. Introduction to Inventory Systems

Let k represent the fixed order cost in dollars per order, as defined in Section 1.4.5. The numerator is where the judgment lives, because it includes only what moves with the number of orders.

Example 1.3 (Building a fixed order cost). A purchasing organization placed 18,000 orders last year. We review its budget and keep only the costs that move with the number of orders placed:

Table 1.7.: Building the fixed order cost from a purchasing organization's budget.

Component	Annual
Purchasing staff time attributable to placing orders	\$570,000
Receiving and inspection, the per-receipt portion	\$280,000
Systems, transmission, invoice matching	\$50,000
Transaction-driven total	\$900,000

Then from Equation 1.14,

$$k = \frac{900,000}{18,000} = \$50 \text{ per order}$$

Notice what Table 1.7 excludes: management overhead, facility cost, and the contract-negotiation effort that does not vary with how many orders are placed. Each is a real cost of running a purchasing department, and none of them changes if one more order is placed this year. That is, they are period costs in the sense of Section 1.4.2.

Suppose we had included all of those, bringing the numerator to \$1.4 million and k to \$77.78 per order. Because the order quantity varies as $\sqrt{k}$, the inflated figure would raise every order quantity in the portfolio by $\sqrt{77.78/50} = 1.25$, a 25% overstatement, and it would do so invisibly.

1. Introduction to Inventory Systems

Note also that k carries no time unit. It is dollars per order, and the time dimension enters through the order *rate* it multiplies, so it is the one parameter needing no conversion.

Do not blend dissimilar transactions. An automated release against an existing contract, an EDI message or an automatic replenishment signal, may cost under \$5 and sometimes a fraction of a dollar. A manual procurement action involving sourcing, approval, and inspection commonly runs \$30 to \$150. The gap is an order of magnitude, and a single average is wrong for both populations: it makes automated items order too much and manual items order too little. Where both exist, estimate two values and assign each item by its acquisition method. The same logic applies across echelons. An internal requisition from a branch to its own warehouse is usually far cheaper than an external purchase and should carry its own k .

1.5.5. Stockout and Backorder Costs, and What to Do Instead

There is no ledger account for a disappointed customer, so neither π nor b can be built the way i and k can. Three approaches, in increasing order of practicality:

Enumerate the consequences. Where the unsatisfied demand triggers a measurable event, such as an emergency shipment at a known premium, a production line idled at a known hourly cost, or a contractual late penalty, those costs can be priced directly. Notice that the natural unit falls out of the enumeration. An expedite fee is a one-time charge and belongs in π , while an idled line accrues at an hourly rate and belongs in b . This works in narrow settings and fails in most.

Substitute a service target. Specify the required fill rate, ready rate, or tolerable customer wait, and choose the cheapest policy meeting it. This is what most organizations actually do, and it is not a retreat: it moves the judgment from a dollar figure nobody can defend to a service

1. Introduction to Inventory Systems

statement management is already prepared to make. A stated tolerable wait is particularly convenient. Recall that $\bar{B} = \lambda \bar{W}$ from Equation 1.6, so a tolerable wait converts directly into the quantity b would have multiplied. Chapter 8 develops the machinery.

Invert an existing policy. If the organization already operates a rule, a service target it holds to or a stocking threshold such as “do not stock at a branch below 12 demands a year”, then that rule embodies a shortage cost, whether or not anyone has written it down. Solving for the π or b that makes the model reproduce the existing rule recovers the organization’s own revealed valuation. It is often the most defensible number available, precisely because it is not an outside opinion about what a stockout is worth. It also serves as a cross-check: if the implied value is absurd, the existing rule deserves scrutiny.

1.5.6. Avoiding Double Counts

Cost accounting does not partition costs the way a model needs them to be partitioned, and the same dollar can easily land in two parameters. The model will not detect this; it will simply charge twice and recommend accordingly. Four overlaps account for nearly all such errors.

Table 1.8.: The four overlaps that produce double counts, and how to resolve each.

Overlap	The test
k against inbound freight	Many organizations already bundle inbound freight into a standard cost per order. If k includes it, do not charge freight again as a separate term.

1. Introduction to Inventory Systems

Overlap	The test
i against f	The holding term scales with inventory value ; the stocking position term scales with nothing . If i already contains a flat per-item storage allocation, subtract it before setting f .
π against b	A backordered demand is charged for the <i>waiting</i> by b . Adding a per-unit π on top charges the same failure twice, unless π prices a genuinely separate one-time consequence such as an expedite fee.
Obsolescence	Belongs in i , not f , because write-off exposure scales with the value sitting there, not with the fact of the item being listed.

Notice that every row of Table 1.8 is resolved the same way, by asking which quantity the disputed dollar scales with. Thus, the general form of the test is the one that governs everything else in this section. Ask what quantity the cost scales with, and put it in the parameter that multiplies that quantity. A cost that scales with value goes in i . A cost that scales with orders goes in k . A cost that scales with units-owed-times-time goes in b . A cost that scales with nothing goes in f . A cost that scales with throughput goes in none of them.

1.5.7. When a Point Estimate Is Not Defensible

Some parameters resist estimation no matter how the ledger is interrogated, and f is usually one of them, because the costs that make it up are recorded

1. Introduction to Inventory Systems

as departmental totals rather than per item.

The productive move is to stop defending a number and report the value at which the decision changes.

Example 1.4 (When the slot costs more than the stock). A mid-value item has unit cost $c = \$6.11$ per unit, carrying charge $i = 0.18$ per year, and average on-hand inventory $\bar{I} = 10$ units. Using Equation 1.12, we compute its holding cost rate as

$$h \bar{I} = i c \bar{I} = (0.18)(6.11)(10) = \$11.00 \text{ per year}$$

A plausible stocking position cost of $f = \$20$ per item per year is $20/11.00$, or 1.8 times that. That is, the cost of *having a slot* exceeds the cost of the inventory sitting in it.

Thus, the stocking decision for this item turns almost entirely on f , the parameter that is hardest to estimate, and hardly at all on the holding cost, which can be computed to the cent. Defending \$20 over \$15 would be false precision. Reporting the break-even is not.

The alternative is a statement of the form: *forward stocking this item pays as long as a stocking position costs less than \$X per year*. That is more defensible than a point estimate, more useful to whoever must act on it, and it converts an argument about an unknowable number into a question the organization can answer from its own experience. Where a parameter is both uncertain and decisive, report the break-even.

1.6. Performance Measures

A performance measure reduces the behavior of a system to a number, so that two policies can be compared. Inventory systems are judged on two families of them. **Cost measures** ask what the system consumes.

1. Introduction to Inventory Systems

Service measures ask how well it meets demand. Both are built from the state variables of Section 1.3 and the prices of Section 1.4, and neither is complete on its own.

1.6.1. Cost as a Performance Measure

The cost terms of Table 1.4 assemble into a single rate. Let $\bar{N}$ represent the average number of orders placed per unit time, in orders per unit time, the quantity the ordering price multiplies.

$$TC = \underbrace{k\bar{N}}_{\text{ordering}} + \underbrace{h\bar{I}}_{\text{holding}} + \underbrace{b\bar{B}}_{\text{backorder}} + \underbrace{\pi\lambda_\ell}_{\text{stockout}} + \underbrace{c\lambda}_{\text{purchase}} + \underbrace{f}_{\text{position}} \quad (1.15)$$

Every symbol on the right has been defined, the prices in Section 1.4 and the quantities in Section 1.3. Notice that each of the six terms reduces to dollars per unit time, the check of Section 1.4.1 applied to the assembled expression. Three features of Equation 1.15 matter before we use it.

The two shortage terms are alternatives. Under backordering nothing is lost, so $\lambda_\ell = 0$ and the π term vanishes. Under lost sales nothing is owed, so $\bar{B} = 0$ and the b term vanishes. Both appear only under partial backordering, where some demand waits and some departs.

Two terms are inert. Recall that Section 1.4.1 showed $c\lambda$ to be identical under every policy, and that f is charged for carrying the item at all rather than for how it is replenished. Both belong in TC , which is a total, and both drop out of any comparison between replenishment policies. That last claim has one exception, and Section 3.8 is where it appears. When the supplier prices in tiers, c depends on the order quantity and the purchase term stops being common to the alternatives being compared. What remains,

$$C = k\bar{N} + h\bar{I} + b\bar{B} + \pi\lambda_\ell \quad (1.16)$$

1. Introduction to Inventory Systems

is the **relevant cost**, and it is what Section 3.7 measures penalties against.

TC is a rate. It is dollars per unit time, so it can be compared across systems of different sizes only after normalizing, usually by dividing by λ to get cost per unit demanded.

Three other cost-based measures appear constantly in practice, and all are functions of quantities already defined:

- **Average inventory investment**, $c\bar{I}$, in dollars. This is what appears on a balance sheet and what a directive to reduce inventory usually refers to.
- **Cost per unit demanded**, TC/λ , which normalizes for volume.
- **Turnover**, $\lambda/\bar{I}$ from Equation 1.8, which normalizes for the size of the item.

Cost has one property that no service measure has. It is a single number, so it ranks alternatives without further judgment, and that is why Equation 1.15 is the objective for the rest of the book. It also has one weakness, and it is decisive. Two of its six terms require b or π , and Section 1.5.5 argued that most organizations cannot defend either. Thus, a measure that cannot be computed is not a measure.

Service measures are the response. They replace the term that cannot be priced with a constraint that can be stated.

1.6.2. Service Measures

Service measures differ from each other in one respect, and it explains nearly every disagreement about them in practice:

1. Introduction to Inventory Systems

! The distinction that matters

Service measures differ in **what they average over**. Some average over *time*, some over *units demanded*, some over *demand transactions*, and some over *replenishment cycles*. Two measures from different families can be far apart on the same history, and neither is wrong.

1.6.2.1. The Ready Rate

Recall that Equation 1.9 defines $SO(t)$ to equal 1 whenever the shelf is empty. The **ready rate** is the proportion of time the system has stock on hand:

$$\overline{RR} = 1 - \overline{SO} = \frac{1}{T} \int_0^T RR(t) dt \quad (1.17)$$

It averages over **time**, weighting every instant equally. That is its strength and its weakness. It is the natural measure when what matters is whether the item is available at an arbitrary moment, such as a spare part supporting equipment that could fail at any time. However, it counts the long quiet stretches when nothing was demanded as heavily as the busy ones, so a system can post a high ready rate while having failed the demands that actually arrived.

1.6.2.2. Fill Rates

A **fill rate** averages over demand rather than time, and there are two of them because demand can be counted in transactions or in units.

Let Y_i represent an indicator that takes the value 1 if the i th demand transaction is filled immediately and completely from stock, and 0 if any

1. Introduction to Inventory Systems

part of it is backordered. The **transaction fill rate** over n transactions is

$$\overline{FR} = \frac{1}{n} \sum_{i=1}^n Y_i \quad (1.18)$$

The **unit fill rate** counts units instead, giving partial credit when a transaction is filled in part:

$$\overline{FR}_u = \frac{\text{units filled from stock on arrival}}{\text{units demanded}} \quad (1.19)$$

When every transaction is for a single unit the two coincide. When transactions are larger they do not, and the gap is not small. For example, a ten-unit order filled to nine units scores 0.9 on the unit fill rate and 0 on the transaction fill rate. Which is right depends on the customer. If the ten units are ten independent issues from a stockroom, nine of them were served and the unit measure is fair. If the ten units are a kit and nine of them assemble nothing, the transaction measure is the right one.

1.6.2.3. Cycle Service Level

The **cycle service level** is the proportion of replenishment cycles in which no stockout occurs, equivalently the probability that demand over a replenishment cycle does not exceed the stock available to meet it. It averages over **cycles**.

This measure is the odd one in this section, and the reason matters. A *cycle* is defined by the replenishment policy: it is the interval between successive order arrivals. An observer who knew nothing about how the system was controlled could measure the ready rate, both fill rates, and the customer wait, but could not identify a cycle at all. The cycle service

1. Introduction to Inventory Systems

level is therefore a property of the system *together with* its policy, not of the system alone.

Thus, it is convenient for setting a reorder point, since it translates directly into a quantile of the lead-time demand distribution, and treacherous for comparing alternatives. Two systems at 95% cycle service can deliver very different service to customers if their cycles differ in length, because the one ordering more often is exposed to more cycles and therefore to more opportunities to fail.

1.6.2.4. Customer Wait

The average time a demanded unit waits to be supplied, $\bar{W}$, counting zero for units filled immediately. By Equation 1.6 it is $\bar{B}/\lambda$, so it requires no new measurement.

It is often the measure management can state most confidently. For example, a purchasing manager who cannot price a stockout can usually say whether a two-week wait is acceptable, the property Section 1.5.5 exploits. Reporting the wait *conditional* on having waited is also useful, and is a larger number, because it excludes the immediately-filled demands that pull the unconditional average toward zero.

1.6.2.5. The Measures Disagree

Example 1.5 (Three service measures from one demand history). A location opens the window $[0, 5]$ with 10 units on hand and receives no replenishment during it. Demands arrive at $t = 1, 2, 3, 4, 5$ for the quantities shown. Unfilled demand is backordered, and $Y_i = 1$ when transaction i is filled completely on arrival, as defined in Section 1.6.2.2.

We work the history one transaction at a time, carrying the on-hand balance forward from each row to the next. At $t = 1$ the shelf holds 10 units and 3 are demanded, so all 3 are filled and 7 remain. At $t = 2$ four more are

1. Introduction to Inventory Systems

demanded against 7 on hand, so all 4 are filled and 3 remain. At $t = 3$ five are demanded against 3 on hand, so 3 are filled, 2 are backordered, and the shelf empties. From that point the shelf stays empty, so the demands at $t = 4$ and $t = 5$ are backordered in full. Table 1.9 records the result.

Table 1.9.: A five-transaction demand history against an opening stock of 10 units.

t_i	D_i	On hand before	Filled now	Backo- rdered	$I(t)$ after	Y_i
1	3	10	3	0	7	1
2	4	7	4	0	3	1
3	5	3	3	2	0	0
4	2	0	0	2	0	0
5	1	0	0	1	0	0
Total	15		10	5		2

Ready rate. On hand is positive over $[0, 3)$ and zero over $[3, 5]$, so by Equation 1.17

$$\overline{RR} = \frac{3}{5} = 60\%$$

Unit fill rate. Of the 15 units demanded, 10 were supplied from stock on arrival, so by Equation 1.19

$$\overline{FR}_u = \frac{10}{15} = 66.7\%$$

Transaction fill rate. Two of the five transactions were filled completely, so by Equation 1.18

1. Introduction to Inventory Systems

$$\overline{FR} = \frac{1}{5} \sum_{i=1}^5 Y_i = \frac{2}{5} = 40\%$$

Three numbers between 40% and 67%, from one history, none of them wrong. Notice that the transaction measure is harshest because it gives no credit for the three units of transaction 3 that were supplied. The ready rate is highest because it counts the quiet early periods, when the shelf was full and nothing much was being asked of it, as heavily as the busy later ones. That is, the three measures differ here for exactly the reason Section 1.6.2 gave, which is that each averages over a different thing.

A report claiming “95% service” without saying which of these it means is not telling you very much.

1.6.2.6. When Do They Agree?

The ready rate and the unit fill rate coincide under a specific and important condition: when demand arrives one unit at a time in a Poisson stream. The reason is that Poisson arrivals see time averages: the fraction of arrivals finding the system out of stock equals the fraction of *time* it is out of stock, so averaging over demand and averaging over time give the same answer.

Thus, the two are so often used interchangeably, and doing so is usually safe for slow-moving items demanded singly. It stops being safe as soon as demand arrives in batches. With batch demand a stockout tends to be *caused* by a large transaction, so the arrivals that find the system short are systematically the big ones, and the demand-weighted measures fall below the time-weighted one. That is the pattern in Example 1.5, where the transaction fill rate sits well below the ready rate.

1. Introduction to Inventory Systems

1.6.2.7. Choosing a Measure

Two questions settle it in practice.

What does the customer actually experience? If a partially filled order is useless, the transaction fill rate is the measure. If units are consumed independently, the unit fill rate is. If the item's value is being available whenever it is needed, the ready rate is. If the customer will wait but cares how long, it is $\bar{W}$.

What can the organization state without inventing a number? The point of substituting a service target for a shortage cost is that management can commit to the target. A target nobody can defend is no better than the cost estimate it replaced. In practice a tolerable wait and a fill rate are the two statements organizations make most readily.

Whichever is chosen, state it precisely and use it consistently. Most of the disagreement about inventory performance in practice is not disagreement about the system. It is two people reporting different averages of the same history, each assuming the other means what they mean.

1.6.3. The Problem the Rest of the Book Solves

The pieces now assemble. A policy sets the values of a few parameters, those parameters determine the quantities in Equation 1.15 and the service measures above, and the problem is to choose the parameters. It takes one of two forms.

The **cost form** prices every consequence and minimizes:

$$\min_{\text{policy}} k \bar{N} + h \bar{I} + b \bar{B} + \pi \lambda_\ell + f \quad (1.20)$$

The **service-constrained form** minimizes only the costs that can be defended, and requires a service level instead of pricing failure:

1. Introduction to Inventory Systems

$$\min_{\text{policy}} k \bar{N} + h \bar{I} + f \quad \text{subject to} \quad \overline{FR} \geq \beta \quad (1.21)$$

for a stated target β , with any of the measures of this section in place of $\overline{FR}$.

These are the same problem. A shortage cost implies the service level that optimizing against it produces, and a service target implies the shortage cost for which it is optimal, which is the inversion Section 1.5.5 uses to recover b from an existing policy. Thus, which form to use is a question about which input the organization can defend, not about which is correct.

What changes from chapter to chapter is the policy being chosen and which terms survive, as Table 1.10 shows.

Table 1.10.: The general problem specialized. Every chapter chooses different parameters against the same objective.

Chapter	Policy parameters	Terms that remain
Chapter 3	Q	$k\bar{N} + h\bar{I}$, plus $b\bar{B}$ when backorders are permitted
Chapter 5	An order quantity per period	$k\bar{N} + h\bar{I}$, with demand varying by period
Chapter 7	Q , chosen once	Overage against underage; k is irrelevant with one order
Chapter 8	r and Q	All of Equation 1.15, or Equation 1.21
Chapter 9	R and S	The same, with R a decision

1. Introduction to Inventory Systems

Chapter	Policy parameters	Terms that remain
Chapter 10	Levels at every location	The same, with f deciding where to stock

Read Table 1.10 down its third column, not across. Notice that the holding term $h\bar{I}$ appears in every row, and that the ordering term $k\bar{N}$ appears in every row but one, the single-period problem, where there is only one order and its cost is unavoidable. What varies is which shortage term survives, and whether f is a decision or a constant. Thus, the rest of the book is one objective, one set of state variables, and a different set of assumptions in each chapter about what is known and what may be decided.

1.7. Exercises

Unless an exercise says otherwise, use these conventions so that your answer and the instructor's agree. Take the time unit to be one year, and convert any figure quoted per month or per day to an annual basis before using it. Report dollar amounts to the nearest cent, carrying charges and service measures as percentages to one decimal place, and all other ratios and intervals to four significant figures. Assume unmet demand is backordered unless the exercise says it is lost, and assume that arriving replenishments fill outstanding backorders before any of the units reach the shelf. Where an exercise asks which cost parameter a disputed cost belongs in, state the quantity the cost scales with, since that is what the marginality test of Section 1.4.2 turns on.

Exercise 1.1. A distributor stocks a single item with steady demand and a reliable two-week lead time. Classify this system along each of the dimensions introduced in this chapter, and identify which cost parameters you would expect to dominate.

1. Introduction to Inventory Systems

Exercise 1.2. Explain, in your own words, why a fill rate of 95% and a ready rate of 95% are not the same requirement. Then construct a demand history of your own, in the style of Example 1.5, in which the two measures disagree by at least twenty percentage points, and report all three measures for it.

Exercise 1.3. A distribution center reports the following for the past year: average inventory at cost \$42,000,000; inventory write-offs and disposals \$1,850,000; insurance premium attributable to inventory \$96,000; warehouse operating cost \$3,100,000 for an owned building running at roughly 55% of capacity; receiving and picking labor \$2,400,000. The organization's cost of capital is 4.5%.

Following Example 1.2, determine the carrying charge i . State which components you excluded and justify each exclusion using the marginality test. Then state what your answer would become if the same operation used third-party warehousing billed at \$16 per pallet position per month for the 5,500 positions it occupies.

Exercise 1.4. A purchasing manager proposes the following fixed order cost: the purchasing department's total annual budget of \$1,400,000 divided by the 20,000 orders it placed, giving \$70 per order. The budget comprises buyer salaries of \$780,000, of which roughly two-thirds is spent placing and expediting orders and one-third on contract negotiation and supplier development; departmental management of \$210,000; an allocated share of facility cost of \$160,000; and purchasing systems and transaction processing of \$250,000.

Following Example 1.3, determine a defensible value of k and explain each inclusion and exclusion. By what factor does the manager's figure overstate the order quantity the model will recommend?

Exercise 1.5. An analyst estimating parameters for a distribution network reports: a carrying charge of 22% per year, which includes a \$14 per item

1. Introduction to Inventory Systems

per year allocation for shelf space and cataloguing; a fixed order cost of \$85, which the analyst notes “includes the inbound freight, since we always pay a \$60 minimum on that lane”; and a stocking position cost of \$14 per item per year. The model will also charge inbound freight explicitly at \$60 per shipment. The average item carries \$350 of inventory value at the location.

Identify every double count in this set of parameters, state which parameter each disputed cost belongs in, and give a corrected set.

Exercise 1.6. An item is backordered when out of stock. Demand arrives at $\lambda = 40$ units per year, and under the current policy the average number of units on backorder is $\bar{B} = 0.8$.

Determine the average time a customer waits for an item that is not filled immediately, stating the relationship you used. If management says a two-week wait is tolerable but a one-month wait is not, what does that imply about the backorder cost b relative to the holding cost h ? Explain why a per-unit stockout cost π could not express the same requirement.

Exercise 1.7. A location operates an (r, Q) policy with $r = 8$, $Q = 24$, a constant demand rate of $\lambda = 3$ units per day, and a constant lead time of $L = 4$ days.

Sketch $IP(t)$ and $IN(t)$ over three replenishment cycles. Determine the cycle length, the minimum net inventory reached, the average on-hand inventory $\bar{I}$, the average backorders $\bar{B}$, and the ready rate. Then determine the average on-order inventory $\bar{IO}$ two ways, from your sketch and from Equation 1.5, and confirm they agree. Finally, report the average stocking time $\bar{T}_s$ and the turnover ratio, and state the turnover in turns per year assuming the demand rate is in units per day.

Exercise 1.8. A stockroom opens a week with 12 units on hand and receives no replenishment during it. Demands arrive on successive days for 4, 3, 7, 2, and 5 units.

1. Introduction to Inventory Systems

Compute the ready rate over the five-day window, the unit fill rate, and the transaction fill rate. Explain which of the three you would report to a customer whose orders are kits that are useless if incomplete, and why.

Exercise 1.9. A regional distributor has been told by its parent company to reduce inventory investment by 25% within the quarter. Its stock of a representative item decomposes roughly as: 30% pipeline, 25% cycle, 40% safety, and 5% anticipation held against a seasonal peak three months away.

Identify which interventions could plausibly deliver the reduction within a quarter and which could not, and explain why in terms of what creates each kind of inventory. Then state what you would expect to observe if the target were met on schedule with no other change, and when you would expect to observe it.

2. Managing Inventory

i Learning objectives

After reading this chapter you should be able to:

- describe the recurring decisions an inventory manager faces across a large item population
- construct an ABC classification and defend its cut points
- explain what a single-criterion classification misses, and apply a multi-criteria alternative
- classify items by demand pattern, and say which model each pattern selects
- explain why inventory records drift, and measure record accuracy
- design a cycle counting program and set count frequencies by class

2.1. The Manager's Problem

Recall that Section 1.6.3 ends Chapter 1 with a single problem: choose the policy parameters that minimize Equation 1.15, or that meet a service target at least cost. Everything from Chapter 3 onward solves that problem under different assumptions about demand, lead times, and review.

2. Managing Inventory

All of it concerns one item. A manager does not have an item. In this chapter we take up the decisions that arise only when there are thousands of them, and none of those decisions is a lot sizing formula.

2.1.1. What Scale Changes

A distributor carries tens of thousands of stock keeping units. A hospital system carries more. Each one has a demand history, a lead time, a unit cost, a supplier, and a policy that has to come from somewhere.

Computation is not what breaks. A spreadsheet will produce forty thousand order quantities in a few seconds, and the models in this book are not expensive to evaluate. What breaks is everything around the computation. Someone has to decide which items deserve a considered decision instead of a default, keep the inputs current as demand and costs move, investigate the exceptions the system raises, and answer for the results. Those activities consume people, and people are finite in a way that arithmetic is not.

Example 2.1 (The attention budget). A distributor stocks 12,000 items. One planner is responsible for them, and about 30 hours a week of that planner's time is genuinely available for reviewing items, the rest going to expediting, supplier problems, and meetings. A considered review of one item, looking at its recent demand, checking its parameters, and deciding whether to change anything, takes about six minutes. We work out what that buys.

The weekly capacity is

$$\frac{(30)(60)}{6} = 300 \text{ reviews per week}$$

or 15,000 reviews a year over 50 working weeks. Spread evenly across the portfolio that is

2. Managing Inventory

$$\frac{15,000}{12,000} = 1.25 \text{ reviews per item per year}$$

Now we try to allocate that budget sensibly. Suppose 8% of items are important enough to want monthly attention. Then 960 items at 12 reviews a year consumes 11,520 reviews and leaves 3,480. Give the next 17%, some 2,040 items, a quarterly review and the requirement is 8,160 more, for a total of 19,680 against a budget of 15,000. It does not fit.

Falling back to reviewing that middle group once a year costs 2,040 and leaves 1,440 reviews for the remaining 9,000 items. That works out to one review per item every six years.

Two conclusions follow, and neither depends on the particular numbers.

Most items will never be looked at. That is not through neglect but by arithmetic. Whatever the portfolio size and however diligent the planner, the great majority of items must be handled by a rule that runs without anyone examining the item.

The scarce resource is attention, not computation. For example, giving all 12,000 items even a quarterly review would take 48,000 reviews a year, i.e. 3.2 planners doing nothing else. The order quantities, by contrast, could be recomputed nightly at no meaningful cost.

A manager's real decisions are not the ones the models answer. The models answer how much and when, for an item, once the inputs are known. The manager decides which items get a considered answer and which get a default, and whether the inputs can be trusted at all.

2.1.2. What the Manager Decides

Table 2.1 lists the recurring responsibilities and where each is treated.

2. Managing Inventory

Table 2.1.: The recurring responsibilities of inventory management. Two of them are the subject of this chapter.

Responsibility	The question	Treated in
Segmentation	Which items deserve attention, and which can share a policy?	Section 2.2
Record accuracy	Do the quantities the policies read match the shelf?	Section 2.3
Policy parameters	How much to order, and when?	Parts I and II
Service targets	What availability is promised, and measured how?	Section 1.6
Stocking decisions	Which items are held where?	Chapter 10
Excess and obsolete	What happens to stock that will not sell?	Section 2.4
Performance reporting	What is measured, and to whom?	Section 2.4

The first two are the subject of this chapter, and we place them before the models, not after, for the same reason.

Segmentation comes first because the models assume an allocation of attention that somebody has to make. Every result in Parts I and II is derived for an item whose parameters are known and current. Recall that Example 2.1 shows this condition cannot hold for the whole portfolio. Thus, deciding which items it will hold for is a prior decision, and it is not a technical one.

Record accuracy comes first because the models read a record, not a shelf. Section 1.3 defines the state of an inventory system as though

2. Managing Inventory

$I(t)$ were directly observable. It is not. Every policy in this book triggers on a number the system believes, and Section 2.3 is about the gap between that number and the truth. A correct policy applied to a wrong quantity is wrong, and nothing in its output says so.

Both are prerequisites in the strong sense. That is, a policy can be derived perfectly and still fail, either because it was applied to items that never should have had individual treatment, or because it was computed from quantities that were not true. Notice that neither failure is visible in the policy itself.

2.1.3. What This Chapter Does Not Cover

Two large topics are named here and treated elsewhere, because both need machinery this chapter does not have.

What to stock, and where, is a decision about the network, not about an item, and it turns on the stocking position cost f of Section 1.4.7 together with the structure of the supply network. Chapter 10 takes it up.

How much and when is the rest of the book.

What remains here is the work that has to be done before either of those questions can be answered sensibly: dividing the portfolio so that attention can be allocated, and establishing that the numbers everything else depends on deserve that reliance.

2.2. Grouping and Segmentation

A manager responsible for forty thousand items cannot make forty thousand decisions. Something has to reduce the problem, and that something is grouping. That is, we partition the item population so that decisions are made once per group instead of once per item.

2. Managing Inventory

2.2.1. Two Reasons to Group

Grouping is done for two different purposes, and confusing them causes most of the trouble in practice (Abaydulla 2016).

Grouping for attention. Management effort is scarce and should go where it matters. An importance-based scheme ranks items so that the few worth individual review can be identified and the many that are not can be left to a rule. ABC classification is the standard instrument, and Section 2.2.2 takes it up.

Grouping for policy. Setting and maintaining a policy costs effort whether the item is important or not. An operations-based scheme puts items with similar operating characteristics together so that one policy can serve them all without much loss. Clustering methods are the standard instrument, and Section 2.2.7 takes them up.

These are not the same problem. They use different attributes, produce different groups, and are judged by different criteria. An importance ranking says nothing about whether two items can share a reorder policy, and a scheme that groups items by operating similarity says nothing about which deserve a planner's attention.

Applying one where the other is needed is the commonest failure in practice. It is why Ernst and Cohen (1990) observe that classifications built on cost and volume alone “may provide unacceptable performance when evaluated with respect to cost and service measures in complex inventory environments.”

Table 2.2 sets out the distinction. Reading down the two columns, notice that they share no row: not the question, not the attributes, not the method, not even the criterion by which the result is judged. Thus, a scheme built for one column is not evidence about the other.

2. Managing Inventory

Table 2.2.: The two purposes of grouping. A scheme built for one column is not evidence about the other.

	Grouping for attention	Grouping for policy
Question answered	Which items deserve review?	Which items can share a policy?
Typical attributes	Annual dollar usage, criticality	Demand rate, unit cost, lead time, demand pattern
Typical method	ABC classification	Clustering
Number of groups	Three to six	Whatever the data supports
Judged by	Whether attention lands on the right items	Cost penalty against individual policies

2.2.2. Pareto Analysis and ABC Classification

The observation behind ABC classification is that annual dollar usage is distributed very unevenly across items. A small fraction of items accounts for most of the money.

- Let λ_i represent the annual demand for item i , and c_i its unit cost, so that their product is the **annual dollar usage** of that item. Ranking by

$$\text{annual dollar usage} = \lambda_i c_i \quad (2.1)$$

and cutting the ranked list into classes concentrates attention where the money is. Class A gets individual review and tight control, class C gets a simple rule and infrequent attention, and class B sits between them.

2. Managing Inventory

Example 2.2 (Constructing an ABC classification). A location carries twelve items with the annual demands and unit costs below. The third column is Equation 2.1, and the rows are already sorted by it. We construct the classification in two steps, first accumulating the value and then choosing cut points.

Item	Annual demand λ	Unit cost c	Annual dollar usage	Cumulative	Cumulative %
IT-01	1,200	\$75.00	\$90,000	\$90,000	51.4%
IT-02	60	\$520.00	\$31,200	\$121,200	69.3%
IT-03	9,000	\$2.40	\$21,600	\$142,800	81.6%
IT-04	300	\$38.00	\$11,400	\$154,200	88.1%
IT-05	40	\$180.00	\$7,200	\$161,400	92.2%
IT-06	2,500	\$1.80	\$4,500	\$165,900	94.8%
IT-07	150	\$22.00	\$3,300	\$169,200	96.7%
IT-08	18,000	\$0.15	\$2,700	\$171,900	98.2%
IT-09	25	\$64.00	\$1,600	\$173,500	99.2%
IT-10	800	\$1.10	\$880	\$174,380	99.7%
IT-11	5,000	\$0.08	\$400	\$174,780	99.9%
IT-12	90	\$2.20	\$198	\$174,978	100.0%

Total annual dollar usage is \$174,978. Cutting at 80% and 95% of cumulative value gives Table 2.3.

Table 2.3.: The resulting classification.

Class	Items	Share of items	Share of value
A	IT-01 to IT-03	25%	81.6%
B	IT-04 to IT-06	25%	13.2%
C	IT-07 to IT-12	50%	5.2%

2. Managing Inventory

Notice that three items out of twelve carry 82% of the annual expenditure, while half the items carry 5%. The technique depends on that concentration, because a planner who reviews only IT-01, IT-02, and IT-03 is watching most of the money.

Notice also that the cut points were chosen, not derived. For example, moving the A boundary up one item would put 88% of the value in class A, at the cost of reviewing a third of the portfolio. Nothing in the data says where to stop. Thus, the conventional 80/15/5 split is a convention, not a result, and you should be prepared to defend the cut points you choose.

Figure 2.1 shows the same data as a cumulative curve, which is how the concentration is usually presented and how the cut points are usually chosen.

Read the curve from left to right. It rises steeply over the first quarter of the items, reaching 82% of the value by the third item, and then flattens as each further item adds less. The diagonal is what the curve would look like if every item carried the same annual dollar usage. Thus, the gap between the curve and the diagonal is the concentration itself.

Notice what that gap tells you before any cut point is chosen. An item file that plots close to the diagonal has no concentration to exploit, and ABC classification will not help there. The steeper the early rise, the more a small class A buys.

2.2.3. What One Criterion Misses

Ranking by annual dollar usage answers one question, where the money is. That is not always the right question.

The failure runs in both directions (Flores and Whybark 1986). For example, a high dollar usage item may be entirely undemanding, i.e. readily available, substitutable, with a short lead time and no consequence to running out. It will be reviewed weekly because it is expensive. A low dollar usage

2. Managing Inventory

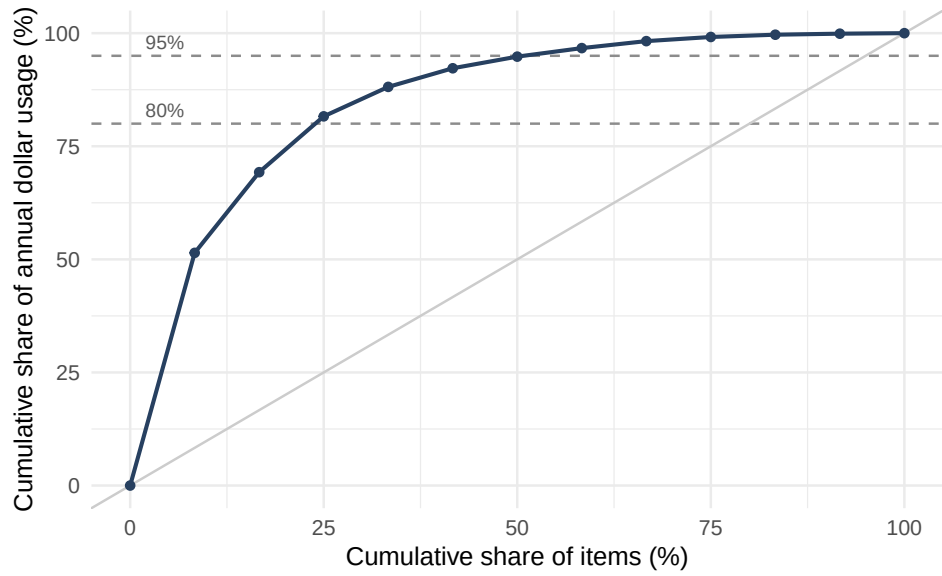


Figure 2.1.: Cumulative share of annual dollar usage against cumulative share of items, for the twelve items of Example 2.2. The curve's steepness at the left is the concentration that makes ABC classification useful. The dashed lines mark the 80% and 95% cuts.

2. Managing Inventory

item may be a fastener that halts an assembly line, available from one supplier on a four month lead time. It will be reviewed annually because it is cheap.

Flores and Whybark (1986) propose that more than one criterion be used, and the criteria proposed since then are numerous: lead time, criticality, commonality, obsolescence, substitutability, reparability, scarcity, durability, order size requirements, stockability, and stockout penalty cost (Ramanathan 2006). Criticality is the most important of these, because it captures the consequence of not being able to supply, exactly what dollar usage cannot see.

Example 2.3 (Reclassifying on two criteria). Recall the twelve items of Example 2.2, and add a criticality rating on a five-point scale, where 5 means an item whose absence halts an operation.

We normalize each criterion to the unit interval, dollar usage by its maximum and criticality by $(\text{rating} - 1)/4$, and then score each item with equal weights:

$$\text{score}_i = 0.5 \left(\frac{\lambda_i c_i}{\max_j \lambda_j c_j} \right) + 0.5 \left(\frac{\text{criticality}_i - 1}{4} \right)$$

Table 2.4.: The twelve items reranked, taking the top three as A and the next three as B.

Item	Dollar usage	Criticality	Score	New class	Old class
IT-01	\$90,000	3	0.750	A	A
IT-08	\$2,700	5	0.515	A	C
IT-11	\$400	5	0.502	A	C
IT-05	\$7,200	4	0.415	B	B
IT-03	\$21,600	3	0.370	B	A
IT-07	\$3,300	3	0.268	B	C

2. Managing Inventory

Item	Dollar usage	Criticality	Score	New class	Old class
IT-04	\$11,400	2	0.188	C	B
IT-02	\$31,200	1	0.173	C	A
IT-06	\$4,500	2	0.150	C	B
IT-09	\$1,600	2	0.134	C	C
IT-10	\$880	1	0.005	C	C
IT-12	\$198	1	0.001	C	C

Seven of the twelve items change class. Two of the movements matter most.

IT-11 rises from C to A. It is an eight cent part with \$400 of annual usage, ranked eleventh of twelve on money and third overall once criticality is counted. Thus, under the single-criterion scheme it would be counted once a year and reviewed never.

IT-02 falls from A to C. It is the second most expensive line in the file at \$31,200 a year, and it is a criticality 1 item, i.e. substitutable, with no operational consequence to a stockout. It was receiving weekly attention because of its price tag alone.

Notice that the weights were chosen, and that a different pair would move different items. That is the weakness of this approach, and Section 2.2.4 is about avoiding it.

2.2.4. Deriving the Criterion Instead of Choosing Weights

The difficulty with Example 2.3 is the 0.5. Nothing justified it, a manager who preferred 0.7 would get different classes, and no experiment settles the question.

A better line of attack derives the ranking statistic from the cost model rather than assembling it from opinions. Zhang et al. (2001) formulate

2. Managing Inventory

the inventory problem as minimizing investment subject to service and order frequency constraints, and extract from the resulting expression for the reorder point a single classifying statistic combining unit cost, lead time, and demand. Teunter et al. (2010) extend this with a cost criterion that includes shortage cost, so that criticality enters through the cost of failing to supply instead of through a subjective rating, and report that it outperforms both traditional ABC and the earlier criterion across several real datasets.

The appeal is that several parameters are folded into one statistic without anyone choosing weights, and that the statistic is the one the objective function implies. Thus, where a defensible cost model exists, this is the better approach. Where one does not, weighted scoring remains, and you should record its weights as the assumptions they are.

2.2.5. Which Attributes, and Which Forbid Grouping

Not every attribute plays the same role. Lenard and Roy (1995) separate three kinds, given in Table 2.5, and the distinction is the most practically useful in this literature.

Table 2.5.: Three roles an attribute can play in a grouping (Lenard and Roy 1995).

Role	Meaning	Examples
Prevent grouping	Items differing here must not share a group at all	Storage structure, strategic importance
Weaken grouping	Differences degrade a shared policy without forbidding it	Demand dispersion, lead time

2. Managing Inventory

Role	Meaning	Examples
Useful to the manager	Make the resulting groups actionable	Nature of the item, supplier, existing functional groups

The first row of Table 2.5 is the one that gets skipped. For example, two items stocked at different echelons serve different functions, so grouping them produces a policy correct for neither, however similar their demand rates. Thus, you should identify the preventing attributes first and partition on them before any statistical method is applied.

Abaydulla (2016) draws a related line between **structural** attributes, which describe an item's position in the supply network, and **non-structural** attributes, which do not. You should carry that finding forward. Structural attributes should not be fed into a clustering algorithm as ordinary numeric variables, because doing so raised both the cost penalty and the computation time. It was also unnecessary, since items sharing a network structure tended to cluster together on their other attributes anyway. Thus, partition on structure first, then cluster within.

For operational attributes, Kampen et al. (2012) offer four categories: **volume** (demand quantity and demand value), **product** (unit cost, lead time), **customer**, and **timing**. Timing is the least used of the four, and its most important member is the interval between demands. The next section takes that up, and the criterion there selects the model instead of merely describing the item.

2.2.6. Demand Pattern Segmentation

Two items with identical annual demand can behave entirely differently. For example, one sells four units every week while the other sells two hundred units twice a year. Annual dollar usage cannot tell them apart, and no policy that suits one will suit the other.

2. Managing Inventory

We describe the pattern with two quantities.

- Let ADI represent the **average demand interval**, which measures intermittence, i.e. how often a demand occurs.
- Let CV^2 represent the squared coefficient of variation of the non-zero demand sizes, which measures lumpiness, i.e. how variable a demand is when it does occur.

The first is the average interval between non-zero demands,

$$ADI = \frac{\text{number of periods}}{\text{number of periods with non-zero demand}} \quad (2.2)$$

and the second is

$$CV^2 = \left(\frac{s}{\bar{x}} \right)^2 \quad (2.3)$$

where $\bar{x}$ and s are the mean and standard deviation of the non-zero demands. Cutting each at a threshold gives the four classes of Table 2.6.

Table 2.6.: Demand classes by intermittence and lumpiness, with the conventional cut points.

	Frequent ($ADI \leq 1.32$)	Infrequent ($ADI > 1.32$)
Variable ($CV^2 > 0.49$)	Erratic	Lumpy
Steady ($CV^2 \leq 0.49$)	Smooth	Intermittent

Example 2.4 (Classifying five demand histories). Five items each have twelve periods of demand. We classify them by computing Equation 2.2 and Equation 2.3 for each.

2. Managing Inventory

Item	Demand by period
S-1	20, 22, 19, 21, 23, 20, 18, 22, 21, 19, 20, 21
E-1	5, 40, 2, 60, 10, 35, 1, 50, 8, 45, 3, 30
I-1	0, 10, 0, 0, 9, 0, 11, 0, 0, 10, 0, 0
L-1	0, 0, 45, 0, 0, 3, 0, 0, 0, 80, 0, 12
B-1	0, 15, 12, 0, 18, 14, 0, 16, 13, 0, 17, 15

Take I-1 first. Four of the twelve periods carry demand, so from Equation 2.2 $ADI = 12/4 = 3.00$. The non-zero demands are 10, 9, 11, and 10, with mean 10.0 and standard deviation 0.816, so from Equation 2.3 $CV^2 = (0.816/10.0)^2 = 0.007$. That is infrequent and steady, so I-1 is **intermittent**.

Repeating for all five gives Table 2.7.

Table 2.7.: The five histories classified.

Item	Non-zero periods	ADI	CV^2	Class
S-1	12	1.00	0.005	Smooth
E-1	12	1.00	0.798	Erratic
I-1	4	3.00	0.007	Intermittent
L-1	4	3.00	1.001	Lumpy
B-1	8	1.50	0.018	Intermittent

B-1 is the instructive one. It has demand in eight of twelve periods and very steady sizes, which reads as a well behaved item. However, $ADI = 1.50$ puts it just past the 1.32 cut and into the intermittent class. Thus, you should not let an item sitting that close to a boundary have its treatment decided by which side it fell on.

The classification matters because it selects the model and does not merely describe the item. The stochastic policies of Chapter 8 and Chapter 9

2. Managing Inventory

assume a demand distribution that a smooth item satisfies and a lumpy one does not. The fill rate computed for a lumpy item from a normal approximation can be badly wrong.

Two cautions apply. First, the thresholds are conventions, and the literature does not agree on when a series becomes intermittent. Proposals include at least 30% of periods with zero demand, no more than 60 to 70% of periods with demand, and a mean interval exceeding 1.25 review periods (Johnston and Boylan 1996).

Second, the class depends on the period length chosen. The same transactions bucketed daily rather than monthly will show more zero periods and a longer inter-demand interval. Thus, an item can be moved between classes by changing nothing but the reporting calendar. You should never allow such a result to drive a policy.

A portfolio is rarely mostly smooth. For example, in a hospital pharmacy study of roughly two thousand items, most items fell outside the smooth class (Varghese et al. 2012). That is the normal case and not a pathology, and it explains much of why fill rate estimates disappoint in practice.

2.2.7. Clustering for Policy

Classification schemes assign items to a small number of predetermined classes. Clustering instead lets the data determine the groups. That is, we place items in a space whose coordinates are their operational attributes and group those that lie close together.

The practical algorithm at scale is **k-means**, which partitions items into k groups by repeatedly assigning each item to the nearest group centre and recomputing the centres. Alternatives exist, including genetic algorithms, simulated annealing, and tabu search. However, they are far slower on problems of this size, and k-means and its close relatives are essentially the only methods that have been applied to inventory populations of realistic scale (Abaydulla 2016).

2. Managing Inventory

Three practical points.

Attributes must be scaled. Demand rate in thousands and unit cost in dollars are not comparable distances, so an unscaled clustering is dominated by whatever attribute happens to carry the largest numbers.

Choosing k is a judgment. More groups reduce the penalty of a shared policy and increase the work of maintaining policies. For example, Abaydulla (2016) found seven groups outperformed three on penalty cost, which is unsurprising and is exactly the trade Section 2.2.8 makes explicit.

Group first, then decide how far to share. Rossetti and Achlerkar (2011) distinguish two uses of the same clustering. Under **group policies**, every item in a cluster uses a single policy computed from the group's average attributes. Under **grouped individual policies**, the clustering organizes the work but each item still receives its own policy. The first is fast and carries a real penalty, besides costing items their individual identity. The second lands close to individual optimization and costs computation. Both beat ABC on cost and service.

You should keep that distinction in view, because it separates two things that are often run together. Grouping is a way of organizing decisions. It does not oblige you to give every member of a group the same answer.

2.2.8. Evaluating a Grouping

A classification is not right or wrong. It is more or less expensive, and the cost is measurable.

The reference point is what the items would cost under individual policies. Any grouping that forces items to share a policy gives something up against that benchmark.

- Let the **grouping penalty** represent that loss, as a fraction of the individual policy cost.

2. Managing Inventory

That is,

$$\text{penalty} = \frac{C(\text{group policies}) - C(\text{individual policies})}{C(\text{individual policies})} \quad (2.4)$$

Notice that Equation 2.4 uses the relevant cost of Equation 1.16 rather than TC , and that the choice matters. Purchase cost is identical under every grouping, so it cancels in the numerator and only inflates the denominator. Dividing by TC would make the same grouping look cheap for an expensive item and dear for a cheap one, a statement about unit cost and not about grouping.

Set against the penalty is the effort saved, which is roughly the reduction in the number of policies to set, review, and maintain. A decision about grouping is made on those two numbers.

Example 2.5 (Is the grouping worth it?). Six items have been costed three ways: under individual policies, under a two-group scheme, and under a three-group scheme. All figures are annual relevant cost, so purchase cost is excluded. We compute the penalty for each scheme.

Item	Individual	Two groups	Three groups
A	\$4,200	\$4,310	\$4,250
B	\$3,800	\$3,905	\$3,845
C	\$5,100	\$5,180	\$5,140
D	\$1,150	\$1,260	\$1,200
E	\$980	\$1,090	\$1,030
F	\$1,340	\$1,415	\$1,370
Total	\$16,570	\$17,160	\$16,835

From Equation 2.4, the two-group scheme costs

2. Managing Inventory

$$\frac{17,160 - 16,570}{16,570} = 3.56\%$$

and the three-group scheme costs

$$\frac{16,835 - 16,570}{16,570} = 1.60\%$$

The decision is worth \$590 a year against \$265 a year, set against maintaining two policies, three, or six.

For six items this is not a decision anybody needs to make, because anyone can maintain six policies. Scale it to forty thousand items and the same 3.56% is the price of reducing forty thousand policy calculations to two hundred, the difference between a procedure that runs and one that does not. Notice that the penalty is the same number either way. What changes is what it buys.

2.2.9. Combining Schemes

Nothing requires choosing one method. Importance classification and operational clustering answer different questions, so applying both yields groups that are identified by importance and homogeneous in operation. However, the order in which they are applied matters (Abaydulla 2016).

Importance first, then clustering. Classify into A, B, and C, then cluster within each class. Items keep a visible priority label, and the clusters inside each class support a shared policy. It is the natural choice when management reporting is organized around the classes.

Clustering first, then importance. Cluster on operational attributes, then rank within each cluster. The number of operational groups is no longer constrained to three or six, and the importance label is applied inside groups that are already homogeneous.

2. Managing Inventory

The simplest version of the first is a cross-tabulation, classifying items by dollar usage and by demand pattern and then reading the table. For example, an A item with lumpy demand and a C item with smooth demand are different management problems, and neither classification alone distinguishes them.

2.2.10. Using a Classification

A classification earns its place only through the decisions it changes. Table 2.8 lists the usual ones. Notice that a class appearing in none of those rows is changing nothing.

Table 2.8.: What a classification is for. A class that changes none of these is decoration.

Decision	How the class enters
Review frequency	A items reviewed individually and often, C items on a rule
Service target	Higher fill rates for critical or high value classes
Count frequency	More frequent cycle counts for A items (Section 2.3.7)
Stocking decision	Whether to hold the item at a location at all
Policy sharing	Whether a group policy is applied or each item is set individually

Three failures recur in practice.

Classes become ends in themselves. A quarterly report of how many items are in each class, with no decision attached, is a measurement that costs money and returns nothing.

2. Managing Inventory

Boundaries are treated as cliffs. The item just below the A cut and the item just above it are nearly identical, so giving them materially different treatment is arbitrary. Thus, where a decision is sensitive at a boundary, you should look at the items near it, not at the rule.

Classes go stale. Demand moves, products reach end of life, and criticality changes when equipment is retired. Thus, a classification built three years ago and never revisited is directing today's attention using yesterday's item file. Reclassification should be scheduled, and its frequency is a decision of the same kind as count frequency in Section 2.3.7.

2.3. Inventory Record Analysis

Nobody sets a reorder point by walking to the shelf and counting. The policy reads a **record**, and the record is a claim about the shelf, not the shelf itself. Claims decay. In this section we take up how far they decay, how to measure it, and what to do about it.

2.3.1. What a Record Claims, and Why It Drifts

An inventory record asserts, at a minimum, four things: a **stock number**, a **location**, a **quantity on hand**, and a **condition** (Rossetti et al. 2001). The record is inaccurate if any one of them is wrong. That is a stricter definition than it first appears, and it is the right one. For example, an item recorded in the correct quantity at the wrong location cannot be picked, and an item in the right place with the right count but the wrong condition code will be issued to a customer who cannot use it.

Table 2.9 lists where the errors come from.

2. Managing Inventory

Table 2.9.: Where record errors come from, and the signature each leaves.
The signature matters because it is what makes a cause findable.

Source	What happens	How it shows up
Transaction error	A receipt, issue, or transfer is keyed with the wrong quantity, item, or location	A discrepancy with no offsetting entry
Unrecorded movement	Stock is taken, borrowed, or returned without a transaction	Record higher than shelf, drifting one way
Shrinkage	Theft, breakage, spoilage	Record higher than shelf
Misidentification	Two similar items confused at receipt or pick	Paired errors, one item high and another low
Unit of issue	Each against case, pack against piece	Errors that are exact multiples
Timing	The count and the transaction fall on opposite sides of a cutoff	A discrepancy that resolves itself
Location error	Stock physically moved without a transfer	Correct total, wrong place

Those sources fall into two mechanisms, and we separate them because they call for different responses (Rossetti et al. 2006). **Transaction errors** occur when a movement is recorded incorrectly, e.g. a receipt keyed at the wrong quantity or a transfer entered against the wrong item. They are process defects and can be designed out. **Stock loss errors** are unrecorded losses through shrinkage, damage, or theft, where the movement was real and no transaction was ever generated. No amount of transaction discipline

2. Managing Inventory

prevents them, and they drift in one direction, with the record always above the shelf.

Their relative weight depends on the setting. In retail store operations the errors concentrate at receiving and at the point of sale, followed by database processing, physical counting, ticketing, stocking, and shelf alignment (Rossetti et al. 2006). Knowing the order matters, because it says where process improvement effort returns the most.

Notice one feature of Table 2.9 that organizes the whole subject. Every source is a **transaction**, or the absence of one. That is, a record for an item that nothing has happened to cannot become wrong. Orozco (1993) pressed this point hard, arguing that counting effort spent on items with no activity since the last count is wasted, which is the reasoning behind the transaction-based and opportunity-based counting methods of Section 2.3.6.

2.3.2. Why Accuracy Comes First

Recall that Section 1.3 defined the on-hand inventory $I(t)$ as though it were a single quantity. It is two.

- Let $I_a(t)$ represent the amount actually on the shelf at time t .
- Let $I_r(t)$ represent the amount the system believes is there.
- Let $D(t)$ represent the **discrepancy** between them.

That is,

$$D(t) = I_a(t) - I_r(t) \tag{2.5}$$

The record is accurate when $D(t) = 0$. Notice that shrinkage and unrecorded issues make $D(t)$ negative, with the record standing above the shelf.

2. Managing Inventory

The distinction matters because the two quantities are used for different things (Rossetti et al. 2008). Demand is filled from $I_a(t)$, because that is what physically exists. However, the reorder decision is made on the inventory position, and the position is computed from the record:

$$IP(t) = I_r(t) + IO(t) - B(t) \quad (2.6)$$

Every policy in this book compares that quantity against a reorder point. When $D(t) \neq 0$ the comparison is made against the wrong number. DeHoratius and Raman (2004) note that the error runs both ways, since it can trigger an order that is not needed and it can suppress one that is.

A policy computed from a wrong record is wrong, and it fails in a way that is difficult to detect.

Consider a continuous review system holding a reorder point r chosen to give some service level. The policy triggers when the recorded position reaches r . If the record overstates what is on the shelf, then the trigger fires late, the effective reorder point is lower than the one designed, and the item runs short more often than the design allows. Notice that nothing in the system reports this. The model reports the service it was designed to deliver, and the warehouse delivers something worse.

Kumar and Arora (1992) studied exactly this at a service parts operation and found that service levels were not being met, with record inaccuracy the cause. We dwell on that for a moment. The organization had a defensible policy, correctly derived, and the policy was not delivering, because the quantity it was reading was not the quantity it had.

There is a second way to see it. Chapter 8 will show that safety stock exists to absorb uncertainty in demand over the lead time. Record error is a second source of uncertainty, and the safety stock absorbs it too, whether or not anyone intended that. Morey (1985) formalized this by defining a minimum actual protection level, i.e. the service a policy really achieves once record error is accounted for alongside demand variability. Thus, an

2. Managing Inventory

organization with poor records is paying for safety stock twice, once for the demand uncertainty it knows about and again for the record uncertainty it does not.

2.3.3. What Error Actually Costs

Simulation of a two-echelon chain, a supplier feeding a distribution centre feeding a retailer, each running an (r, Q) policy with backordering, separates the responses available to an organization whose records are wrong (Rossetti et al. 2008). There are three cases, and the comparison between them is the argument for this whole section.

Leave the policy alone. Running the parameters that were correct for accurate records, against records that are not, gives up service. For example, in the reported experiments the high error case lost roughly half its fill rate, and system backorders rose from about ten items to about three hundred and seventy.

Re-optimize the policy around the error. Recomputing r and Q to hit the same 90% fill rate target does restore the service, and pays for it in stock. The system carries substantially more inventory purely to mask the discrepancy, and backorders still rise sharply, by roughly 300% in the high error case for fast moving items.

This second case is the one that explains why record inaccuracy survives for years without being raised. The customer facing measure is back at target, while the cost has moved into inventory, transportation, and handling, where nobody attributes it to record keeping (Rossetti et al. 2006). That is, the symptom that would have prompted an investigation has been bought off.

Count. Adding cycle counting raised fill rates while holding inventory the same or slightly lower. Notice that this is not a trade. The first two responses trade service against inventory, while counting improves service without buying it with stock, because it attacks the discrepancy instead

2. Managing Inventory

of compensating for it. This section therefore comes before the policies. There is no point tuning a reorder point to the second decimal place against a record that is right eight times in ten.

2.3.4. Measuring Record Accuracy

The standard measure is the fraction of records examined that are accurate:

$$\text{record accuracy} = \frac{\text{number of accurate records}}{\text{number of records examined}} \quad (2.7)$$

That is simple to state. However, it hides three choices that change the answer.

What counts as accurate. Requiring the count to match the record exactly is one convention. The other is a **tolerance**, under which a count within some percentage of the recorded quantity is treated as accurate. Tolerances exist because exact agreement on a bin of ten thousand washers is not worth what it costs to obtain. APICS (1980) recommends tolerances of 0.5%, 1%, and 5% for A, B, and C items respectively. Stahl (1998) argues the tolerance should be set from value, usage, lead time, method of counting, criticality, and bill of material level, not from value alone, on the reasoning that a cheap item that halts an assembly deserves a tight tolerance whatever it costs.

The choice is not cosmetic. The same warehouse reports very different accuracy under a strict match rule and under a 5% tolerance. Thus, comparisons across organizations that do not state their tolerance are meaningless.

What is weighted. Equation 2.7 weights every record equally. An alternative weights by value, the thing a financial auditor cares about. These diverge sharply, and the divergence has a direction. A warehouse can be accurate in dollars while being inaccurate in records, because

2. Managing Inventory

the large errors fall on cheap items and the expensive items are counted carefully. Aggregate dollar value therefore masks individual errors, the central criticism of accuracy programs built on financial sampling. Picking an item to fill an order requires that item's record to be right, and no amount of dollar accuracy supplies it.

Example 2.6 (Three accuracy figures from one count sheet). A cycle count of ten records returns the results below, where the error column is the absolute difference as a percentage of the recorded quantity. We compute the accuracy three ways.

Item	Unit cost	Recorded	Counted	Difference	Error
A-100	\$80.00	120	120	0	0.0%
A-101	\$150.00	45	44	-1	2.2%
A-102	\$2.00	500	495	-5	1.0%
B-200	\$400.00	12	12	0	0.0%
B-201	\$0.40	3,000	2,970	-30	1.0%
B-202	\$25.00	8	0	-8	100.0%
C-300	\$6.00	250	250	0	0.0%
C-301	\$0.75	1,000	990	-10	1.0%
C-302	\$3.00	60	66	+6	10.0%
C-303	\$95.00	15	15	0	0.0%

Strict match. Four records agree exactly, so

$$\text{accuracy} = \frac{4}{10} = 40\%$$

Within a 2% tolerance. The four exact records qualify, and so do A-102, B-201, and C-301 at 1.0% each. However, A-101 misses at 2.2%, C-302 at 10%, and B-202 at 100%. Therefore,

2. Managing Inventory

$$\text{accuracy} = \frac{7}{10} = 70\%$$

Weighted by value. Recorded value is $80(120) + 150(45) + \dots + 95(15) = \$27,405$, and the absolute dollar discrepancy is $150(1) + 2(5) + 0.40(30) + 25(8) + 0.75(10) + 3(6) = \397.50 . Therefore,

$$\text{accuracy} = 1 - \frac{397.50}{27,405} = 98.55\%$$

Notice that the same shelf, counted once, reports 40%, 70%, or 98.6% depending only on which definition is used. The value-weighted figure is highest because the errors sit on cheap items. Recall the masking effect described in Section 2.3.3; this is that effect seen at the level of a single count sheet.

Notice also what the dollar figure does to B-202. The item is missing entirely, with eight units recorded and none on the shelf, and it moves the value-weighted accuracy by \$200 out of \$27,405. Thus, the next customer who wants B-202 will not care that the warehouse is 98.6% accurate by value.

Which fields. Quantity accuracy alone is the usual reported figure. Location accuracy and condition accuracy are separate measures. A program that reports only quantity is reporting on one of the four fields a record asserts.

Benchmarks. A survey of 410 manufacturing companies found roughly 20% of firms using cycle counting achieving 98% accuracy or better, with more than 60% between 90% and 97% (Rossetti et al. 2001). Materials requirements planning is usually said to require 95% or better in order to function, because Chapter 6 explodes a schedule through records and an error at one level propagates to every level below it. Best in class is 99% and above.

2. Managing Inventory

2.3.5. Physical Inventory and Cycle Counting

There are two ways to find out what is really there.

An **annual physical inventory**, counting everything at once, is the older practice and is often required for financial reporting. As a way of managing accuracy it has three defects. It is disruptive, usually requiring operations to stop. It is infrequent, so an error introduced in February is corrected in December after eleven months of decisions made on it. And it diagnoses nothing, because a wall to wall count produces a corrected balance and no information about how the balance became wrong.

Cycle counting counts a portion of the population continuously, on a schedule, without stopping operations. It corrects records as a by-product. Its actual purpose, as Tersine (1994) sets out, is a list in a deliberate order:

1. Identify the causes of errors.
2. Correct the conditions causing the errors.
3. Maintain a high level of record accuracy.
4. Provide a correct statement of assets.

Notice that accuracy is third on that list and the balance sheet is fourth. A cycle counting program judged only on the number of records it corrected is being judged on its by-product.

The two are not exclusive, and in practice most organizations run both, using cycle counting to manage accuracy and a physical inventory to satisfy an auditor. Where cycle counting is well established the physical inventory can often be dispensed with, which is a large part of its financial justification.

2. Managing Inventory

2.3.6. Cycle Counting Methods

The methods differ in how the sample is formed, and the choice has consequences for cost, coverage, and statistical validity. Table 2.10 summarizes the main ones (Rossetti et al. 2001; Brooks and Wilson 1995).

Table 2.10.: Cycle counting methods. No method dominates, and large operations normally combine several.

Method	How the sample is formed	Strengths and weaknesses
Random sample	Items drawn at random from the population, with or without replacement	Statistically clean and easy to defend; ignores everything known about which items are likely to be wrong
ABC	Population stratified by class, each class counted at its own frequency	Focuses effort where value is; workload scales with the number of items; classifies on financial criteria that may not track error risk
Process control	The counter counts what is easy to count and skips the rest	Fast, many counts per hour; the sample is biased, but biased toward the items most likely to be wrong
Location based	An area is chosen and every item in it is counted	Large economies of movement; the sample is formed by geography, so item characteristics play no part

2. Managing Inventory

Method	How the sample is formed	Strengths and weaknesses
Opportunity based	Counts triggered by events: a reorder, a put away, a balance near the reorder point, or a demand that exposes a discrepancy	Counts at the moment the count is cheapest and most informative; coverage is uneven
Transaction based	An item is counted after every n transactions	Targets the mechanism that creates errors; items with no activity are never counted

Two entries in Table 2.10 deserve comment, because they look wrong at first.

Process control counting is deliberately biased, and its advocates know it. The counter is free to skip an item that is hard to count, so the sample is not random and the resulting accuracy figure is not an unbiased estimate of population accuracy. Brooks and Wilson (1995) recommend it anyway, on the argument that the bias runs toward items with the greatest probability of containing errors, where the counting effort should go. That is, if the purpose is to find and fix causes rather than to estimate a population parameter, then a biased sample aimed at the problem beats an unbiased one that is mostly clean records. The counter is also not performing a blind count, since the record is visible, which is a separate and more serious objection.

Transaction and opportunity based counting follow from the observation in Section 2.3.1 that errors enter through transactions. Counting an item that has not moved since it was last counted correct is close to guaranteed to find nothing.

2. Managing Inventory

Opportunity counting is more specific than a general instruction to count at convenient moments. Ordinary operation exposes discrepancies for free, and each exposure is a trigger (Rossetti et al. 2006):

- A demand arrives, the record says there is not enough to fill it, and the stock is visibly on the shelf. The record is understating and can be corrected on the spot.
- A demand arrives, the record shows a positive balance, and the shelf cannot fill it. The record is overstating, and the stockout just made that visible.
- The recorded balance approaches the reorder point, so a count is about to matter for a decision, not for a report.

Notice that the third trigger is the most useful of the three, because it verifies the record precisely when a wrong one would cause a wrong order.

2.3.7. Setting Count Frequency

Under ABC counting the conventional frequencies are four counts per year for A items, two for B, and one for C (Rossetti et al. 2001). Those numbers are a starting convention and not a result. We consider two ways to do better.

Work back from the error rate.

- Let p represent the probability that a record, once verified correct, becomes wrong in any one period, and assume it stays wrong until the next count finds it.
- Let n represent the number of periods between counts, and $A(n)$ the average accuracy over the count cycle.

A record counted every n periods is correct for the first few periods and increasingly likely to be wrong later. Therefore,

2. Managing Inventory

$$A(n) = \frac{1}{n} \sum_{t=0}^{n-1} (1-p)^t = \frac{1 - (1-p)^n}{np} \quad (2.8)$$

Equation 2.8 is the simplest useful version of the two-state Markov model Neeley (1987) develops, in which a record is either accurate or inaccurate and counting returns it to the accurate state.

Figure 2.2 inverts the relation, so that given an error rate and a target accuracy it returns the count interval required. Read it by fixing a target accuracy and following that curve from left to right. As the error rate rises the required interval falls steeply at first and then flattens, so an item that goes wrong twice as often needs counting roughly twice as often. Notice how far apart the curves for different targets lie at low error rates and how they converge at high ones. That is, once a record is going wrong often enough, the target accuracy barely changes the schedule, because the record must be counted almost every period whatever the target is.

Notice that Figure 2.2 makes the case for differentiated frequencies without appealing to value at all. Items differ in p , and it is p , not annual dollar volume, that determines how often a record needs verifying. Thus, where the two disagree, counting by value alone spends effort in the wrong place.

They do disagree. The discriminant analysis of a field experiment by Sheppard and Brown (1993) found item value, dollar value of stock, and quantity on hand all predictive of error, with low value high volume items showing the *lowest* accuracy. That is precisely the population an ABC scheme counts least often.

Example 2.7 (Choosing a count interval from an error rate). An item's record goes wrong with probability $p = 0.03$ in any month, and the target is 95% average accuracy. We evaluate Equation 2.8 at successive intervals.

2. Managing Inventory

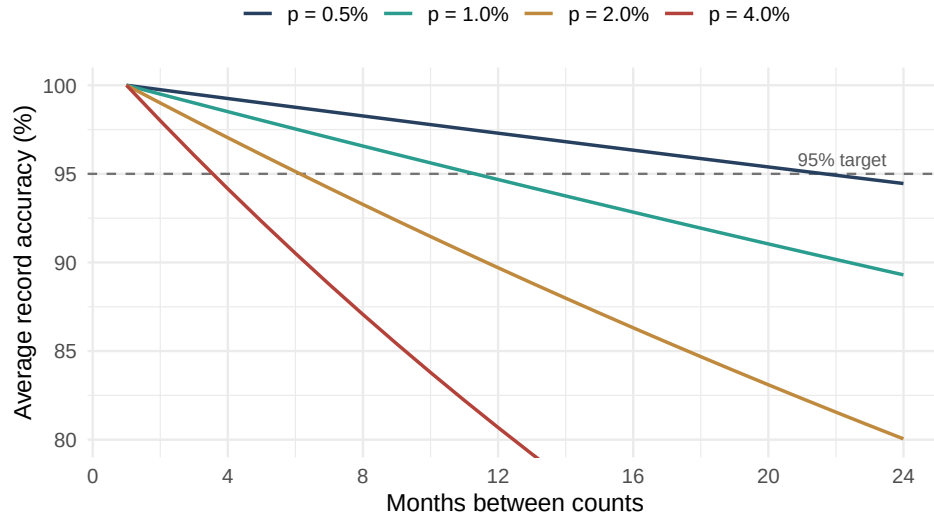


Figure 2.2.: Average record accuracy over a count cycle, from Equation 2.8, against the interval between counts. Each curve is one monthly error rate p . The dashed line marks a 95% target. Holding that target needs a count roughly every six months at $p = 2\%$ and every quarter at $p = 4\%$, against every eleven months at $p = 1\%$.

2. Managing Inventory

n months	1	2	3	4	5	6
$A(n)$	100.0%	98.5%	97.0%	95.6%	94.2%	92.8%

Counting every four months holds 95.6%, while every five months falls to 94.2%. Thus, the item should be counted every four months, i.e. three times a year.

Now suppose a second item is twice as error prone, with $p = 0.06$. Then $A(2) = 97.0\%$ and $A(3) = 94.1\%$, so it needs counting every two months, six times a year, in order to hold the same target. Notice that doubling the error rate roughly doubles the required count frequency, the flattening seen in Figure 2.2.

Notice also that the first answer lands near the conventional four counts a year for an A item. The difference is that it was derived from a measured error rate rather than assumed, and that it applies whatever the item's annual dollar volume happens to be.

Example 2.8 (Sizing a sample to estimate accuracy). Accuracy is believed to be near 92%, and the goal is to estimate it within two percentage points at 95% confidence. From Equation 2.9 with $z_{0.025} = 1.96$, $\hat{a} = 0.92$, and $e = 0.02$, we have

$$m = \frac{(1.96)^2(0.92)(0.08)}{(0.02)^2} = \frac{(3.8416)(0.0736)}{0.0004} = 706.9$$

Therefore, 707 records must be counted. Notice that tightening the margin to one percentage point quadruples the requirement, to 2,828 records, because m varies as $1/e^2$. For that practical reason, accuracy is usually reported to the nearest percentage point and no finer.

Ask which items counting pays for. Accuracy and the value of accuracy are different things, so the item classes that score worst on one may not be

2. Managing Inventory

the ones worth counting. For example, simulation of a retail supply chain found cycle counting essential for **low demand, high cost** items, where it produced substantial savings while holding fill rates, and close to worthless for **high demand, low cost** items, where the savings were negligible and sometimes negative (Rossetti et al. 2006). For fast moving cheap items it is usually cheaper to hold more stock than to count, and where counting them is worthwhile at all, opportunity counting outperforms a schedule.

Set that beside Sheppard and Brown (1993), who found low value high volume items had the *lowest* accuracy. Both results hold. The cheap fast items have the worst records and are the least worth correcting, because the consequence of an error on them is small and a little extra stock covers it. Thus, counting effort should follow the cost of being wrong, not the probability of being wrong.

Optimize against cost. The alternative is to price counting and to price its absence. Young and Nie (1992) build a total cost model with stockout cost, counting cost, ordering cost, and holding cost, and study frequency by simulation. Iglehart and Morey (1971) go further and choose the counting policy and the stocking policy together, minimizing total cost per unit time subject to a bound on the probability of a stock denial. Both frame the count frequency as one more term in Equation 1.15, the right way to see it. Counting is a cost incurred in order to protect the accuracy that every other term depends on.

Sizing a sample. Estimating population accuracy is a separate problem from maintaining it.

- Let m represent the number of records to count, e the margin of error, and $\hat{a}$ a prior estimate of accuracy.

To estimate a proportion within a margin e at confidence $1 - \alpha$, the usual expression applies:

$$m = \frac{z_{\alpha/2}^2 \hat{a}(1 - \hat{a})}{e^2} \quad (2.9)$$

2. Managing Inventory

Notice the awkward consequence of high accuracy. As $\hat{a}$ approaches one the variance falls, and yet demonstrating that accuracy is 99% rather than 97% still requires a large sample. That is one reason mature programs stop trying to measure accuracy precisely and concentrate on causes.

Stratified sampling reduces the sample needed, and Buck and Sadowski (1983) give a method for choosing strata and allocating counts to them for a target confidence. However, their approach and its successors have been criticized for resting on aggregate dollar value, which returns to the masking problem of Section 2.3.4.

2.3.8. Monitoring Accuracy Over Time

Everything above treats accuracy as a quantity to be measured and a schedule to be set. Neither answers the operational question a manager actually has, which is whether accuracy is holding steady, drifting, or responding to a change that was made last month.

Record accuracy is an attribute. That is, each record sampled is either accurate or it is not, so a sample of n records yields a proportion. A *p* **chart** consumes exactly that data. We plot the sampled accuracy period by period against a center line and control limits, and the chart then separates ordinary sampling noise from a real change. A point inside the limits means the process is behaving as it has been, whatever its level. A point outside means something has changed, and you should investigate it.

Perpetual inventory record sampling is built on that, combining control group sampling with statistical process control (Rossetti et al. 2006). Rather than counting every record on a cycle, which is infeasible where a single department may carry twenty to fifty thousand items, a sample is drawn each period from a prioritized control group and charted. The recommendation from that work is a sample of 200 to 300 records per

2. Managing Inventory

week, enough to make the chart informative without consuming a counting team.

Figure 2.3 shows such a chart. The center line is the process average and the two outer lines are the control limits, set three standard errors away. Following the points from left to right, the first eighteen weeks scatter within the limits with no pattern, which is what a stable process looks like however high or low its level. Notice that they range from 88% to 95%. That is, a week four points below the center line is ordinary, and on its own it means nothing.

From week 19 the points drop, and nine of the last twelve fall below the lower limit. That run is the signal. Notice that it is a run and not a point. The chart is reporting a level that has moved and stayed moved, which is what separates a process change from the scatter above it.

Example 2.9 (Reading a control chart). A program samples $n = 250$ records a week, and accuracy has been running at $\bar{p} = 0.92$. We compute the limits and then read two weeks against them.

The standard error of a sampled proportion is

$$s = \sqrt{\frac{\bar{p}(1 - \bar{p})}{n}} = \sqrt{\frac{(0.92)(0.08)}{250}} = 0.0172$$

Therefore, the control limits are $0.92 \pm 3(0.0172)$, giving a lower limit of 0.869 and an upper limit of 0.972.

This week's count finds 214 of 250 records accurate, i.e. 85.6%. That falls below the lower limit, so the process has changed and the cause should be investigated.

Contrast a week returning 220 of 250, i.e. 88.0%. That is four percentage points below the center line and looks alarming. However, it sits inside the limits. A sample of 250 drawn from a process running at 92% produces readings that low often enough that one of them means nothing on its own.

2. Managing Inventory

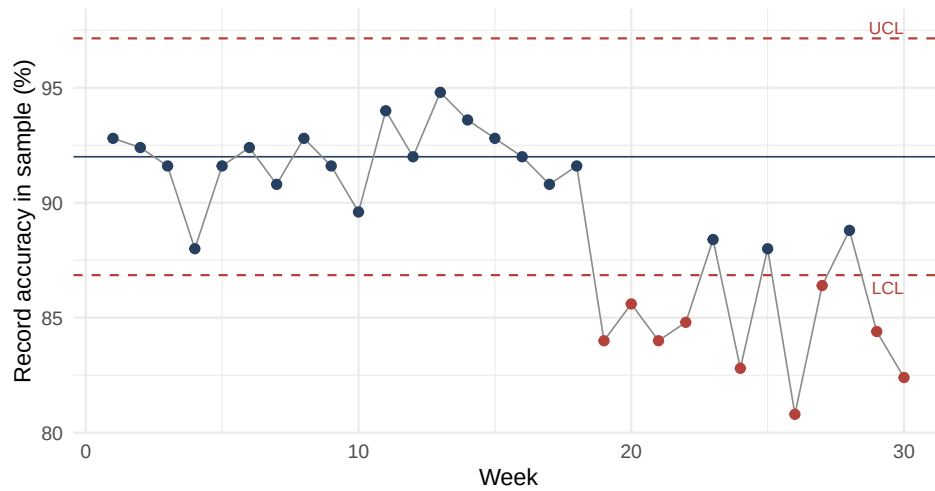


Figure 2.3.: A p chart of weekly record accuracy, 250 records sampled per week. Accuracy runs at 92% for eighteen weeks and then drops to 86% after a process change. Sampling alone moves the weekly reading between 88% and 95% over the stable period, so no single low week means anything on its own. The chart separates that noise from the shift, which breaks the lower control limit in week 19 and stays out for most of the weeks that follow.

2. Managing Inventory

Thus, acting on it would send a team looking for a cause that does not exist.

Two cautions come with the chart, and you should keep both in view.

The population is not homogeneous. A p chart assumes every unit sampled comes from the same process with the same probability of conforming. Inventory records do not satisfy that assumption. Each item has its own propensity to go wrong, for the reasons in Section 2.3.1, so the sample mixes items with different underlying accuracies, so the false alarm rate is inflated. Simulation of the effect found the inflation slight, small enough that the chart remains usable in practice (Rossetti et al. 2006). However, the assumption is violated, and the chart's stated false alarm rate is optimistic.

A signal is a prompt to investigate, not to adjust. An out of control point says the process producing records has changed. Correcting the sampled records addresses none of that. The response is a root cause investigation, the subject of Section 2.3.10.

The chart's value is not primarily in detecting deterioration. It is in demonstrating stability, and in showing whether a process improvement actually moved the level. Without a baseline and control limits, an improvement of two percentage points is indistinguishable from a good week.

2.3.9. Accuracy Is Not a Local Property

Everything so far treats record accuracy as something a location manages for itself. Across a supply chain it is not.

A location's replenishment orders are the demand its supplier sees. Thus, a retailer ordering against bad records sends distorted demand upstream, and the distribution centre serving it responds to orders that were triggered when they should not have been, or that failed to trigger when they should have.

2. Managing Inventory

The same two-echelon experiments measured this directly. A single retailer declining to cycle count, while every other location did, cost that chain roughly 6% of fill rate, added about 2% to inventory, and raised backorders by roughly 400% (Rossetti et al. 2008). That is, the cost of one partner's records lands on partners who have no control over them.

Two consequences follow. First, record accuracy belongs in the performance measures a supply chain agrees on, and not only in each location's internal reporting. Brooks and Wilson (1995) make tying inventory managers' evaluation to record accuracy a basic tenet of a working program. Second, where locations are separate firms, the natural instrument is contractual, i.e. performance clauses specifying the accuracy each partner will maintain, on the same footing as delivery or quality terms (Rossetti et al. 2008).

Chapter 10 takes up the analysis of these systems, where the assumption that each location knows its own stock is doing more work than it appears to.

2.3.10. Correction Is Not the Point

A count that finds a discrepancy and adjusts the record has closed a symptom. If the condition that produced the discrepancy is still in place, then the same record will be wrong again, and the program will count the same errors indefinitely at a steady cost.

Control group counting is the standard instrument for finding causes. A small representative set of items is counted repeatedly and frequently, often before a full program is deployed. Because the same items are counted again and again, the errors that appear are process errors, not history, and the short interval between counts localizes when each one entered (Stahl 1998). Control group counting is therefore diagnostic, not corrective, and it is the part of a program most often skipped.

The most useful finding in the practitioner literature is a negative one. In benchmarking more than twenty companies, Rossetti et al. (2001) report

2. Managing Inventory

that best in class accuracy, 99% and above, was achieved by organizations that dedicated resources to counting, had strong system support, and concentrated on finding and eliminating common process errors. Those organizations did **not** emphasize statistical methods. Stratified sampling and control charting were not what carried them past 99%. Their attention went to enforcing procedures and resourcing the work.

You should read that carefully, because it is easy to over-read. You should know the statistical material above, since it tells you how often to count, what a measured accuracy figure does and does not support, and how large a sample an estimate requires. What the benchmarking says is that statistical sophistication is not sufficient and is not the binding constraint. That is, an organization counting diligently on crude rules and fixing what it finds will outperform one sampling optimally and adjusting records without asking why they were wrong.

The same conclusion is reached from the other direction in Rossetti et al. (2006), whose recommendation for a sampling program is explicit that a program producing only counts and adjustments delivers no real benefit. The counting is the instrument. Finding and preventing the causes is the work.

The difference is also measurable. The two-echelon experiments included a **learning effect**, in which counting not only corrects records but reduces the rate at which errors are subsequently introduced. Its benefits ran both locally and across the chain, and they were larger than those from correction alone (Rossetti et al. 2008). That states the difference between the two activities quantitatively. Correcting a record improves one number once, while removing its cause improves every future period, at every location downstream.

2.4. Other Responsibilities

Two responsibilities from Table 2.1 remain. Neither gets a chapter of its own, and both arise constantly enough to state here.

2.4.1. Excess and Obsolete Stock

Excess stock is more than will be used in any reasonable horizon. **Obsolete** stock will not be used at all, e.g. the product it served is gone, the equipment it repaired is retired, or the specification changed.

It accumulates because every model in this book assumes demand continues. A replenishment policy has no mechanism for noticing that an item's demand has stopped. It sees inventory position fall below a reorder point and orders, exactly as designed, until someone intervenes. Thus, excess is not a failure of the policy. It is the absence of a decision the policy was never asked to make.

Disposing of it is a forward-looking decision, and the difficulty is that it does not feel like one.

Example 2.10 (Keeping stock that will never be used). A location holds 400 units of an item that cost \$28 each. Annual demand has fallen to 12 units, so the stock represents more than 33 years of supply. The carrying charge is $i = 0.20$ per year and the stocking position cost is $f = \$30$ per year. A liquidator offers \$6 per unit. We compare keeping the excess against disposing of it.

Suppose two years of supply is retained, i.e. 24 units, leaving 376 units in excess. The holding cost per unit per year is $h = ic = (0.20)(28) = \$5.60$, so keeping that excess costs

$$(376)(5.60) + 30 = \$2,135.60 \text{ per year}$$

2. Managing Inventory

Disposing of it returns $(376)(6.00) = \$2,256$ now. Notice that one year of holding costs very nearly the entire salvage value, so the break-even is about 1.06 years. Since the stock represents three decades of demand, disposal dominates by a wide margin.

What makes this decision hard is a number that does not appear above. The excess cost $(376)(28) = \$10,528$ to buy, and disposing of it at \$6 realizes a write-off of over \$8,000 that someone must sign. That \$10,528 is sunk. It is gone whether the stock is sold, held, or scrapped, and it is not a reason to keep anything. However, the write-off is visible and the \$2,135 a year is not, so the common outcome is to spend the larger amount quietly rather than recognize the smaller one publicly.

Notice where the pieces of this decision came from. Holding cost is Equation 1.12, the stocking position cost is Section 1.4.7, and obsolescence risk is already inside the carrying charge through Example 1.2. Excess disposition is not a separate subject. It is the same cost model applied to an item whose demand has ended.

Detecting it is harder than it looks, and Section 2.2.6 explains why. The obvious rule flags items with no movement in some number of months. However, an intermittent item with $ADI = 4$ has demand in one month of four, so three consecutive zero months is its normal behavior, not evidence of death. Thus, a no-movement rule applied without regard to demand pattern will flag healthy slow movers as obsolete and will be slow to catch a smooth item that has genuinely stopped. You should set detection thresholds by demand class.

2.4.2. Performance Reporting

What an organization measures is what it manages, and inventory reporting is usually built from quantities already defined: fill rate and ready rate from Section 1.6, average inventory and turnover from Section 1.3, and backorders, inventory investment, and record accuracy from Section 2.3.

2. Managing Inventory

Two cautions apply to all of them.

Aggregation hides. For example, a portfolio fill rate of 96% is consistent with a class of critical items running at 60%, because the fast movers that dominate the average are the easy ones. Any measure reported for a portfolio should also be reported by class, which is one more thing the segmentation of Section 2.2 is for.

Measures drive behavior, including behavior nobody wanted.

Recall that Section 1.1 described what happens when inventory reduction is mandated without saying which kind. Safety stock is the only component that can be cut on a deadline, so it is the one that gets cut, and the consequence appears a lead time later. Every inventory measure has an equivalent failure mode. Thus, the defense is to pair each with a counterweight that moves the other way, as in Table 2.11.

Table 2.11.: Inventory measures and their counterweights. A measure reported alone will eventually be met alone.

Measure	Improved cheaply by	Counterweight
Inventory turnover	Cutting safety stock	Fill rate, backorders
Inventory investment	Deferring replenishment	Fill rate, stockout frequency
Fill rate	Holding more of everything	Turnover, investment
Order frequency	Ordering in larger lots	Average inventory
Record accuracy	Counting only easy items	Accuracy on a random sample

The last row of Table 2.11 is the one Section 2.3.6 already anticipated. Process control counting is defensible as a way of finding errors precisely because it is biased toward items likely to be wrong, and it is indefensible as a way of reporting a population accuracy figure, for the same reason.

2.5. Summary

The models in the rest of this book take an item and produce a policy. A manager has neither one item nor the attention to treat forty thousand of them individually. In this chapter we settled the two things that have to be in place before those models can be used at scale.

Attention is the binding constraint, not computation. Example 2.1 shows that a realistic review capacity covers a small fraction of a portfolio. Thus, most items must be handled by a rule that runs without anyone examining them, and deciding which items escape the rule is a prior decision. It is a management decision rather than a technical one.

Grouping serves two purposes that should not be confused. Grouping for attention ranks items so effort lands where it matters, and ABC classification by annual dollar usage is the standard instrument. Grouping for policy puts operationally similar items together so one policy can serve them, and clustering is the standard instrument. Notice that a ranking says nothing about whether two items can share a policy.

Single-criterion ABC misses in both directions, promoting expensive items that nothing depends on and burying cheap items that halt operations. Example 2.3 demonstrates the problem, and multi-criteria schemes and cost-derived criteria address it. Whatever the method, a grouping is judged by the cost penalty it carries against individual policies, set against the policies it saves maintaining.

A policy reads a record, not a shelf. Section 1.3 defines on-hand inventory as though it were observable, while Section 2.3 separates what is on the shelf from what the system believes and shows that the reorder decision uses the second.

Record accuracy is not one number. The same count sheet in Example 2.6 gives 40%, 70%, or 98.6% depending on the definition, so a figure reported without its definition says little. Errors enter through transactions, which is

2. Managing Inventory

why counting effort belongs where activity is and why opportunity counting works.

Counting is not a trade against inventory. An organization with bad records can give up service or hold more stock in order to mask the problem. The second is why inaccuracy survives for years unremarked, because the customer facing measure returns to target and the cost moves somewhere nobody attributes to record keeping.

Counting raises service without buying it with stock, because it removes the discrepancy instead of compensating for it. Notice also that correcting records is the by-product, not the purpose. A program that only adjusts will find the same errors indefinitely.

Both subjects are prerequisites in a strong sense. A policy derived correctly can still fail, either because it was applied to items that should never have had individual treatment, or because it was computed from quantities that were not true. Neither failure is visible in the policy itself, which is why both belong before Part I rather than after it.

In the next chapter we take up the first of the models themselves, under the strongest assumption available: that demand is known and constant.

2.6. Exercises

Unless an exercise says otherwise, use these conventions so that your answer and the instructor's agree. Take the time unit to be one year. Report dollar usage and costs to the nearest cent, percentages and accuracy figures to one decimal place, and indices, ratios, and intervals to four significant figures. Where an exercise asks for a classification, state the cut points you used, since nothing in the data supplies them.

Exercise 2.1. A regional distribution center stocks 28,000 items. Three planners are responsible for them, each with about 26 hours a week available

2. Managing Inventory

for item review over 48 working weeks a year. A considered review takes eight minutes.

Determine the annual review capacity, and the average number of reviews per item per year if the capacity were spread evenly.

Management proposes reviewing the top 6% of items monthly and the next 14% quarterly. Show that this proposal cannot be met with the present staffing, and determine by how much it exceeds capacity. Then find a feasible allocation that preserves monthly review for the top 6%, and report how often a typical item in the remaining group would be reviewed under it.

Finally, determine how many planners would be needed to review every item at least twice a year, and comment on whether that is the right response to the finding.

Exercise 2.2. Following Example 2.2, and given the item file in the accompanying spreadsheet, construct an ABC classification by annual dollar usage. Report the fraction of items and the fraction of annual dollar volume falling in each class, plot the cumulative curve, and defend your cut points. State what would change if the cuts were moved by five percentage points in either direction.

Exercise 2.3. Following Example 2.3, reclassify the same item file using at least two criteria, one of which is not dollar usage. Identify the items whose class changes, explain why each moved, and say which classification you would give to a planner.

Exercise 2.4. Four items have the following demand histories over twelve months.

Item	Demand by month
P-1	0, 0, 8, 0, 0, 0, 12, 0, 0, 9, 0, 0

2. Managing Inventory

Item	Demand by month
Q-1	4, 6, 5, 7, 4, 6, 5, 6, 4, 7, 5, 6
R-1	0, 30, 0, 2, 0, 45, 0, 1, 0, 38, 0, 3
S-1	12, 1, 20, 3, 15, 2, 18, 1, 22, 2, 14, 3

Following Example 2.4, compute ADI and CV^2 for each item and assign it to a demand class.

Now suppose the same transactions are reported in two-month buckets rather than monthly, so that each item's history becomes six periods. Recompute both statistics and reclassify. Two of the four items change class. Identify which, explain why the change occurred, and state what this implies about reporting a demand classification without stating the period length it was computed on.

Exercise 2.5. Eight items have been costed under individual policies and under two candidate groupings, one forming two groups and one forming four. All figures are annual.

Item	Individual	Two groups	Four groups
A	\$6,400	\$6,610	\$6,490
B	\$5,900	\$6,090	\$5,980
C	\$7,200	\$7,395	\$7,280
D	\$2,100	\$2,290	\$2,185
E	\$1,850	\$2,040	\$1,930
F	\$2,400	\$2,585	\$2,480
G	\$980	\$1,145	\$1,050
H	\$1,120	\$1,290	\$1,195

Using Equation 2.4, compute the grouping penalty for each scheme, as in Example 2.5.

2. Managing Inventory

Then recommend a scheme in each of two settings, justifying each recommendation in terms of what the penalty buys. In the first, these eight items are the entire portfolio. In the second, they are a representative sample of a portfolio of 30,000 items, and a planner can maintain roughly 150 distinct policies.

Exercise 2.6. A cycle count of 200 item records finds 34 with a quantity discrepancy. Of those, 11 are within a 2% tolerance of the recorded quantity and the rest are not.

Following Example 2.6, report record accuracy under a strict count-match definition and under the tolerance-based definition, and explain which you would report to management and why. Then, following Example 2.8, determine the sample size needed to estimate accuracy within two percentage points at 95% confidence, using your tolerance-based figure as the prior estimate.

Exercise 2.7. Three items have monthly record error rates of $p = 0.01$, $p = 0.02$, and $p = 0.05$.

Using Equation 2.8, determine the longest count interval that holds average record accuracy at 98% for each, as in Example 2.7. Express each answer as counts per year. Then determine the total annual counting workload for a population of 400 items split evenly among the three error rates, and compare it with the workload under a conventional schedule of four counts a year for A items, two for B, and one for C, assuming the population splits 20/30/50 across those classes.

Exercise 2.8. Design a cycle counting schedule for the item population of Exercise 2.2, given a counting team able to perform 60 counts per week. State the count frequency for each class, show that the schedule fits the available capacity, and justify the frequencies in terms of what a count is for.

2. Managing Inventory

Exercise 2.9. A location holds 900 units of a component that cost \$14.50 each. The product it supported was discontinued eighteen months ago, and demand over the past year was 30 units, all from a single customer still running the old equipment. The carrying charge is 0.22 per year and the stocking position cost is \$25 per year. A broker offers \$2.75 per unit for the entire quantity.

Determine the annual cost of retaining the stock beyond a three-year supply, the proceeds of disposing of that excess, and the break-even holding period. State your recommendation.

Then identify the sunk cost in this problem, state its correct role in the decision, and explain why a manager might nonetheless act as though it mattered.

Part I.

Deterministic Models

3. Deterministic Lot Sizing

i Learning objectives

After reading this chapter you should be able to:

- describe one cycle of a general deterministic inventory system and identify its four segments
- derive the average on-hand and backorder levels from the geometry of the cycle
- assemble the total cost rate and optimize it over the order quantity and the backorder level
- obtain the classical lot sizing models as special cases of the general one
- compute the reorder point when the lead time is shorter or longer than a cycle
- assess how much a departure from the optimal order quantity costs
- determine order quantities under all-units and incremental quantity discounts
- compute the performance measures of Section 1.6 in closed form for this model
- build and audit a spreadsheet implementation of each of these models
- reproduce the same analysis in code, and say what code does that a worksheet cannot

3. *Deterministic Lot Sizing*

Recall the pair of questions from Section 1.1: how much to order, and when. In this chapter we answer them for the simplest interesting case, in which demand is known and constant, and so is everything else. That assumption is never exactly true, and Section 1.2.2 explained when it is nonetheless the right one to work under.

Our approach is to build one model that contains all the classical results, and then to obtain each of them by switching an assumption off. That is more work at the start and much less work afterwards, because the cycle geometry is derived once instead of four times.

3.1. The General Cycle

3.1.1. Assumptions

A single item is stocked at a single location. We need notation for the item before anything can be derived.

- Let λ represent the demand rate, which is known and constant, so that demand occurs continuously at that rate.
- Let p represent the replenishment rate, also known and constant, with $p > \lambda$, so that stock builds while replenishment is in progress and depletes when it is not.

Unsatisfied demand is backordered, not lost, and everything repeats, i.e. each cycle is identical to the last.

Two quantities are ours to choose.

- Let Q represent the **order quantity**, the amount brought in each time replenishment occurs.
- Let $\hat{B}$ represent the **maximum backorder level**, the depth to which the system is allowed to run short before replenishment arrives.

3. Deterministic Lot Sizing

i Two symbols for peaks

Recall that Section 1.3 writes $I(t)$ for on-hand inventory and $B(t)$ for backorders at an instant, with $\bar{I}$ and $\bar{B}$ for their time averages. In this chapter we also need their **peaks** within a cycle, which are written with hats:

$$\hat{I} = \max_t I(t), \quad \hat{B} = \max_t B(t)$$

Bar for average, hat for maximum. The peak $\hat{B}$ is a decision, while $\bar{B}$ is a consequence of that decision.

3.1.2. One Cycle, Four Segments

Figure 3.1 shows one cycle, and we follow it from left to right. Net inventory $IN(t)$ starts at its lowest point, $-\hat{B}$. Replenishment begins, and since production runs at p while demand continues at λ , net inventory climbs at the net rate $p - \lambda$. Notice that the line crosses zero partway up the first rise. At that instant the accumulated backorders have all been filled, and every unit arriving after it becomes stock, not a shipment to a waiting customer. Net inventory continues to its peak $\hat{I}$.

Replenishment then stops and demand alone acts, so net inventory falls at rate λ . It passes through zero a second time, at which point the shelf is empty and backorders begin to accumulate, and it reaches $-\hat{B}$, where the cycle repeats. The two rising segments have the same slope, and the two falling segments have the same slope, because only two rates act anywhere in the cycle.

Two relations set the scale of the cycle.

- Let T_p represent the replenishment period, the length of time during which replenishment is in progress.

3. Deterministic Lot Sizing

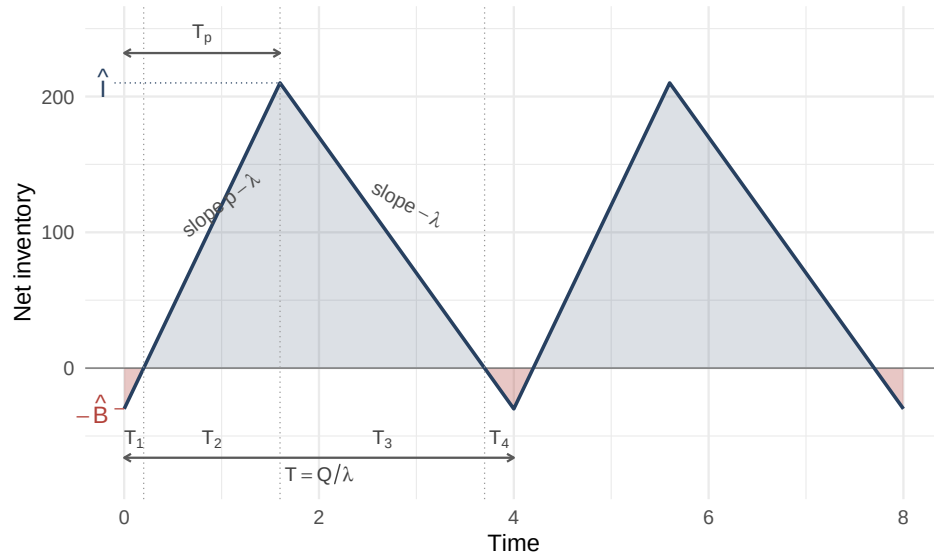


Figure 3.1.: One cycle of the general deterministic model, with $\lambda = 100$, $p = 250$, $Q = 400$, and $\hat{B} = 30$. Net inventory rises at the net rate $p - \lambda$ while replenishment is in progress and falls at λ when it is not. The four segments are the times to clear the backorders, build to the peak, deplete the peak, and accumulate the backorders again.

3. Deterministic Lot Sizing

The quantity Q is produced at rate p , so that

$$Q = p T_p \quad \Rightarrow \quad T_p = \frac{Q}{p} \quad (3.1)$$

During T_p the net inventory must climb the whole distance from $-\hat{B}$ to $\hat{I}$ at the net rate $p - \lambda$. That is,

$$\hat{I} + \hat{B} = (p - \lambda) T_p = (p - \lambda) \frac{Q}{p}$$

Solving for the peak in terms of the two decisions, we obtain

$$\hat{I} = Q \left(1 - \frac{\lambda}{p} \right) - \hat{B} \quad (3.2)$$

Notice the factor $(1 - \lambda/p)$, which recurs throughout this chapter. It is the fraction of the order quantity that survives to become stock, the rest having been consumed by demand while it was still being produced. For example, an item produced at twice its demand rate keeps half of each order as stock.

The four segment lengths follow from the same two rates.

- Let T_1, T_2, T_3, T_4 represent the lengths of the four segments, in the order they occur.

Over T_1 the backorders are cleared at the net rate, over T_2 the peak is built at the net rate, over T_3 the peak is consumed at the demand rate, and over T_4 the backorders accumulate at the demand rate. Therefore,

$$T_1 = \frac{\hat{B}}{p - \lambda}, \quad T_2 = \frac{\hat{I}}{p - \lambda}, \quad T_3 = \frac{\hat{I}}{\lambda}, \quad T_4 = \frac{\hat{B}}{\lambda} \quad (3.3)$$

3. Deterministic Lot Sizing

The cycle length is their sum, and it must also be the time to consume Q at rate λ . That is,

$$T = T_1 + T_2 + T_3 + T_4 = \frac{Q}{\lambda} \quad (3.4)$$

3.2. Average Inventory and Backorders

Section 1.3 defines the averages as time integrals over the cycle. Here the integrands are triangles, so the integrals are areas and no calculus is needed.

Notice in Figure 3.1 that the on-hand inventory is positive over T_2 and T_3 , forming a triangle of height $\hat{I}$ on a base of $T_2 + T_3$. The backorders are positive over T_4 and T_1 , forming a triangle of height $\hat{B}$ on a base of $T_1 + T_4$, so that

$$\bar{I} = \frac{1}{T} \int_0^T I(t) dt = \frac{1}{T} \cdot \frac{\hat{I}}{2} (T_2 + T_3), \quad \bar{B} = \frac{1}{T} \int_0^T B(t) dt = \frac{1}{T} \cdot \frac{\hat{B}}{2} (T_1 + T_4) \quad (3.5)$$

Substituting Equation 3.3 and Equation 3.4 and simplifying, we obtain closed forms in the decision variables:

$$\bar{I} = \frac{\hat{I}^2}{2Q \left(1 - \frac{\lambda}{p}\right)} = \frac{\left[Q \left(1 - \frac{\lambda}{p}\right) - \hat{B}\right]^2}{2Q \left(1 - \frac{\lambda}{p}\right)} \quad (3.6)$$

$$\bar{B} = \frac{\hat{B}^2}{2Q \left(1 - \frac{\lambda}{p}\right)} \quad (3.7)$$

3. Deterministic Lot Sizing

Example 3.1 (Working one cycle). An item is demanded at $\lambda = 100$ units per year and replenished at $p = 250$ units per year. The order quantity is $Q = 400$ units and the policy permits a maximum backorder level of $\hat{B} = 30$ units. We work the cycle through.

The peak on-hand level, from Equation 3.2, is

$$\hat{I} = 400 \left(1 - \frac{100}{250} \right) - 30 = 400(0.6) - 30 = 210 \text{ units}$$

The four segments follow from Equation 3.3, with $p - \lambda = 150$ units per year:

$$T_1 = \frac{30}{150} = 0.2, \quad T_2 = \frac{210}{150} = 1.4, \quad T_3 = \frac{210}{100} = 2.1, \quad T_4 = \frac{30}{100} = 0.3$$

Notice that these sum to 4.0 years, which is $Q/\lambda = 400/100$, as Equation 3.4 requires. You should perform that check every time, because it catches most algebra errors immediately.

We compute the averages twice, first from the areas directly:

$$\begin{aligned} \bar{I} &= \frac{1}{4} \cdot \frac{210}{2} (1.4 + 2.1) = \frac{1}{4} (105)(3.5) = 91.875 \\ \bar{B} &= \frac{1}{4} \cdot \frac{30}{2} (0.2 + 0.3) = \frac{1}{4} (15)(0.5) = 1.875 \end{aligned}$$

and then from the closed forms of Equation 3.6 and Equation 3.7, with $Q(1 - \lambda/p) = 240$:

$$\bar{I} = \frac{210^2}{2(240)} = \frac{44,100}{480} = 91.875, \quad \bar{B} = \frac{30^2}{2(240)} = \frac{900}{480} = 1.875$$

The two routes agree, the second check we make.

3. Deterministic Lot Sizing

The system holds about 92 units on average and owes about 2. Notice the asymmetry. A backorder allowance of 30 units, a seventh of the peak, produces an average backorder level of under two units. The reason is visible in the segments: the system spends $T_1 + T_4 = 0.5$ years of every four in a shortage position, so the backorder triangle is short as well as low. Thus, you should not read a maximum backorder level as though it described the usual state of the system.

3.3. The Total Cost Rate

Equation 1.15 gives the shape of the total cost rate. Each term is a price from Section 1.4 times a quantity the cycle produces, so we take them one at a time.

Ordering. One order per cycle at k dollars each, and $1/T = \lambda/Q$ cycles per unit time, giving an ordering cost rate of $k\lambda/Q$.

Purchasing. Q units per cycle at c each, giving $c\lambda$. Recall from Section 1.4.1 that this term does not depend on Q or $\hat{B}$, and so it cannot affect the optimization. It is part of TC and not part of the relevant cost of Equation 1.16.

Holding. h per unit per unit time on the average on-hand level, $h\bar{I}$.

Backordering. b per unit per unit time on the average backorder level, $b\bar{B}$.

Stockout. The price π is charged on every unit of demand that arrives to an empty shelf, the λ_ℓ of Equation 1.15. Demand arrives at rate λ throughout the shortage, and the shortage lasts $T_4 + T_1$. Thus, the count in a cycle is $\lambda(T_4 + T_1)$ and not $\hat{B}$.

We dwell on the distinction, because it is easy to get wrong. The peak $\hat{B}$ is the *depth* the queue of waiting demand reaches, while units keep arriving

3. Deterministic Lot Sizing

as it drains. Over T_1 another λT_1 of them join the queue and wait. From Equation 3.3,

$$\lambda(T_4 + T_1) = \lambda \left(\frac{\hat{B}}{\lambda} + \frac{\hat{B}}{p - \lambda} \right) = \frac{\hat{B}}{1 - \frac{\lambda}{p}} \quad (3.8)$$

At λ/Q cycles per unit time, the cost rate is therefore $\pi \hat{B} \lambda / [Q(1 - \lambda/p)]$.

The backorder term needs no such correction. The average $\bar{B}$ integrates the queue over both segments, so the waiting of the units that arrive during T_1 is already counted there.

Collecting the five terms, we obtain

$$TC(Q, \hat{B}) = \frac{k\lambda}{Q} + c\lambda + h \frac{[Q(1 - \frac{\lambda}{p}) - \hat{B}]^2}{2Q(1 - \frac{\lambda}{p})} + b \frac{\hat{B}^2}{2Q(1 - \frac{\lambda}{p})} + \frac{\pi \hat{B} \lambda}{Q(1 - \frac{\lambda}{p})} \quad (3.9)$$

Notice that both shortage prices appear in Equation 3.9. Recall that Section 1.4.6 described π and b as belonging to different physical situations. In a backordering model with an expedite charge both are present, because π prices the incident of being short and b prices the waiting that incident causes. Setting either to zero is a modeling choice, and Section 3.5 makes several of them.

3.4. Optimizing the General Model

We minimize Equation 3.9 over the two decisions. Setting the partial derivatives to zero,

3. Deterministic Lot Sizing

$$\frac{\partial TC}{\partial Q} = 0, \quad \frac{\partial TC}{\partial \hat{B}} = 0$$

and solving the pair, we obtain

$$Q^* = \sqrt{\frac{h+b}{b}} \sqrt{\frac{2k\lambda}{h\left(1 - \frac{\lambda}{p}\right)} - \frac{(\pi\lambda)^2}{h(h+b)\left(1 - \frac{\lambda}{p}\right)^2}} \quad (3.10)$$

$$\hat{B}^* = \frac{hQ^*\left(1 - \frac{\lambda}{p}\right) - \pi\lambda}{h+b} \quad (3.11)$$

Every classical lot sizing result in this chapter is Equation 3.10 and Equation 3.11 with something switched off.

Two structural features can be read off before we specialize. Notice first that the factor $\sqrt{(h+b)/b}$ is greater than one and increases as b falls. Thus, cheaper backordering means larger orders, because the system is willing to run short for longer between replenishments. Notice second that $\hat{B}^*$ is proportional to $h/(h+b)$ through Q^* . That is, it depends on the ratio of the cost of holding to the cost of holding plus the cost of owing, the same trade-off that the newsvendor's critical ratio expresses in Chapter 7.

Neither formula is unconditionally usable, and you should see the reason now rather than discover it later. A large π drives the second term under the inner root above the first, leaving no real root. A little below that threshold, Equation 3.11 returns a negative $\hat{B}^*$. In both cases the stationary point lies outside the feasible region, so the optimum is on the boundary $\hat{B} = 0$, where the model reduces to Equation 3.14.

Neither case is exotic. For example, with $\lambda = 100$, $p = 250$, $k = 20$, $h = 4$, and $b = 6$, the backorder level turns negative by $\pi = 1$ and the root disappears by $\pi = 1.55$. A high enough price on being short means the right answer is never to be short.

3. Deterministic Lot Sizing

3.5. The Classical Models as Special Cases

Four assumptions can be relaxed or imposed: whether replenishment is instantaneous, whether backorders are permitted, and whether each of the two shortage prices is charged. Table 3.1 collects the results, and we derive each of them in turn.

Table 3.1.: The classical lot sizing models, each obtained from the general model by imposing an assumption.

Assumptions	Model	Order quantity
p finite, backorders, both π and b	General	Equation 3.10
p finite, backorders, $\pi = 0$	General, no expedite charge	$\sqrt{\frac{h+b}{b}} \sqrt{\frac{2k\lambda}{h(1-\frac{\lambda}{p})}}$
p finite, no backorders	Economic production quantity	$\sqrt{\frac{2k\lambda}{h(1-\frac{\lambda}{p})}}$
$p \rightarrow \infty$, backorders, $\pi = 0$	EOQ with planned backorders	$\sqrt{\frac{h+b}{b}} \sqrt{\frac{2k\lambda}{h}}$
$p \rightarrow \infty$, no backorders	Classical EOQ	$\sqrt{\frac{2k\lambda}{h}}$

3.5.1. No Charge Per Unit Short

Setting $\pi = 0$ removes the subtracted term in Equation 3.10, which leaves

$$Q^* = \sqrt{\frac{h+b}{b}} \sqrt{\frac{2k\lambda}{h(1-\frac{\lambda}{p})}}, \quad \hat{B}^* = Q^* \frac{h(1-\frac{\lambda}{p})}{h+b} \quad (3.12)$$

3. Deterministic Lot Sizing

Equation 3.12 is the usual formulation when shortages are paid for by the waiting they cause and nothing else. That is the common case when backorders are filled from the next scheduled replenishment and not by an expedited shipment.

3.5.2. No Backorders

Forbidding shortages means $\hat{B} = 0$, so both $\bar{B}$ and the stockout term vanish, and Equation 3.6 collapses to

$$\bar{I} = \frac{\left[Q \left(1 - \frac{\lambda}{p}\right)\right]^2}{2Q \left(1 - \frac{\lambda}{p}\right)} = \frac{Q}{2} \left(1 - \frac{\lambda}{p}\right)$$

This is the familiar half-of-the-peak result, adjusted for the fraction of the order that survives production. The cost rate becomes

$$TC(Q) = \frac{k\lambda}{Q} + c\lambda + \frac{hQ}{2} \left(1 - \frac{\lambda}{p}\right) \quad (3.13)$$

Minimizing Equation 3.13 over Q , we obtain the **economic production quantity**

$$Q^* = \sqrt{\frac{2k\lambda}{h \left(1 - \frac{\lambda}{p}\right)}} \quad (3.14)$$

How backorders are forbidden

You may be tempted to obtain Equation 3.14 by setting $b = 0$, and that is wrong. A zero backorder cost makes shortages free, not forbidden, and Equation 3.12 then makes Q^* unbounded.

Forbidding shortages is instead the limit $b \rightarrow \infty$. In Equation 3.12,

3. Deterministic Lot Sizing

$\sqrt{(h+b)/b} \rightarrow 1$ and $\hat{B}^* \rightarrow 0$, which recovers Equation 3.14 exactly. Thus, charging an infinite price for owing a unit is how a model is told never to owe one.

3.5.3. Instantaneous Replenishment

If a replenishment arrives all at once instead of being produced over time, then $p \rightarrow \infty$ and $\lambda/p \rightarrow 0$. The factor $(1 - \lambda/p)$ becomes one and T_p becomes zero. Thus, the cycle loses its build-up segments, and the sawtooth of Figure 3.1 becomes vertical on the way up.

With backorders still permitted and $\pi = 0$,

$$Q^* = \sqrt{\frac{h+b}{b}} \sqrt{\frac{2k\lambda}{h}}, \quad \hat{B}^* = Q^* \frac{h}{h+b} \quad (3.15)$$

3.5.4. The Classical Economic Order Quantity

Imposing both instantaneous replenishment and no backorders leaves the oldest result in the subject. The cost rate is

$$TC(Q) = \frac{k\lambda}{Q} + c\lambda + \frac{hQ}{2} \quad (3.16)$$

Notice the three terms: an ordering term falling in Q , a holding term rising in Q , and a purchase term that does not depend on Q at all. Differentiating,

$$\frac{d TC}{dQ} = -\frac{k\lambda}{Q^2} + \frac{h}{2} = 0 \quad \Rightarrow \quad Q^* = \sqrt{\frac{2k\lambda}{h}} \quad (3.17)$$

3. Deterministic Lot Sizing

Substituting back, we obtain a total cost with a memorable form:

$$TC^* = \sqrt{2k\lambda h} + c\lambda \quad (3.18)$$

Notice that at the optimum the ordering and holding terms are equal, each contributing $\sqrt{2k\lambda h}/2$. That is not a coincidence, and it is the fastest way to check a numerical answer. If the two terms differ, then Q is not optimal.

Figure 3.2 shows the three terms for an item with $\lambda = 2400$ units per year, $k = \$75$ per order, and $h = \$4$ per unit per year. Reading it from left to right, the ordering curve falls steeply at small quantities, because a small order means many orders a year. The holding line rises steadily, since average stock is proportional to the order quantity. Notice that the two cross exactly at the dashed line marking Q^* , the equal-terms property stated above made visible.

Notice also the shape of the total curve near its minimum. It is flat over a wide range of quantities, so a quantity some way from Q^* costs very little more than Q^* itself. That flatness is the subject of Section 3.7 and it is relied on repeatedly later in the book.

Because Q^* is fixed and demand is constant, the optimal cycle length follows immediately as

$$T^* = \frac{Q^*}{\lambda} = \sqrt{\frac{2k}{h\lambda}} \quad (3.19)$$

Choosing Q and choosing T are the same decision. Which one you present to an operating group depends on what they can act on. For example, a warehouse that ships on a weekly cycle finds a time far easier to use than a quantity.

3. Deterministic Lot Sizing

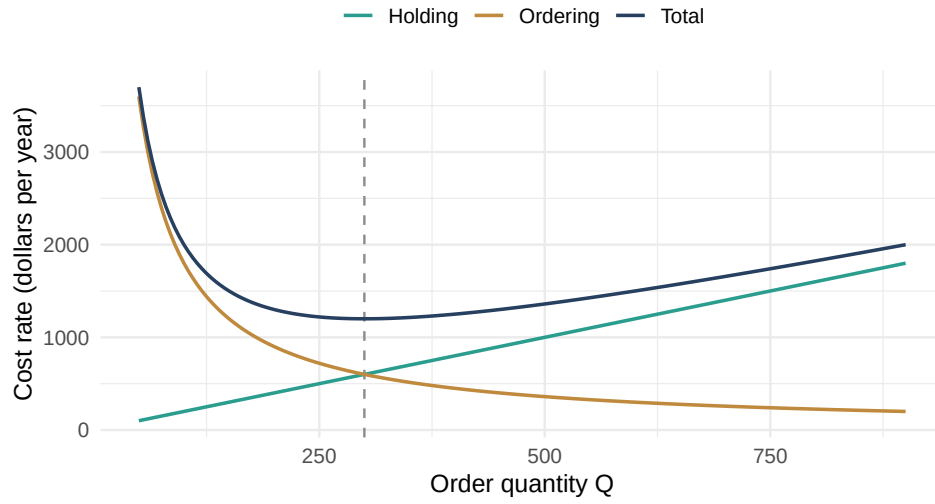


Figure 3.2.: Ordering, holding, and total relevant cost rate against the order quantity, for $\lambda = 2400$, $k = 75$, and $h = 4$. The dashed line marks Q^* , where the two curves cross and the total is at its minimum. The total is visibly flat near the minimum, which is the subject of Section 3.7.

3. Deterministic Lot Sizing

Example 3.2 (The economic order quantity). A product is demanded at $\lambda = 220$ units per year and has a unit cost of $c = \$1,200$ per unit. The ordering cost is $k = \$800$ per order and the carrying charge is $i = 0.18$ per year. No shortages are allowed. We work the four parts in turn.

The holding cost, from Equation 1.12, is

$$h = ic = (0.18)(1200) = \$216 \text{ per unit per year}$$

a) The optimal order quantity. From Equation 3.17,

$$Q^* = \sqrt{\frac{2(800)(220)}{216}} = \sqrt{\frac{352,000}{216}} = 40.37 \text{ units}$$

b) The optimal time between orders. From Equation 3.19,

$$T^* = \frac{40.37}{220} = 0.1835 \text{ year} = 66.98 \text{ days}$$

c) The optimal total cost. From Equation 3.18,

$$TC^* = \sqrt{2(800)(220)(216)} + (1200)(220) = 8,719.63 + 264,000 = \$272,719.63$$

Notice the proportions. The purchase cost is 97% of the total and is untouched by the decision. The \$8,719.63 is the entire quantity the lot sizing decision can influence, and half of it is ordering cost and half is holding cost. That equal split is the check of Section 3.5.4, and you should perform it every time, because unequal halves mean Q is not optimal.

d) The reorder point for a 7 day lead time. Seven days is well under the 66.98 day cycle. Thus, Section 3.6 gives $r = \lambda L$:

3. Deterministic Lot Sizing

$$r = 220 \left(\frac{7}{365} \right) = 4.22 \text{ units}$$

An order is placed whenever the inventory position falls to about 4 units.

A closing observation. The optimal cycle $T^* = 66.98$ days is 9.6 weeks. If this item ships on a weekly truck, then ordering every ten weeks fits an operating rhythm the warehouse already has, and it costs almost nothing against the optimum, as Section 3.7 will confirm. You will usually find that the better answer.

3.6. Lead Time and the Reorder Point

Nothing so far has said when to order, only how much. With demand and lead time both known the timing is exact. We place the order so that it arrives at the moment stock would otherwise reach $-\hat{B}$.

- Let L represent the replenishment lead time, which is known and constant.
- Let r represent the **reorder point**, the inventory position at which the order is placed.

When $L \leq T$. Demand over the lead time is λL . Thus, ordering when the position reaches

$$r = \lambda L - \hat{B} \tag{3.20}$$

brings the replenishment in exactly on time. With no backorders permitted this reduces to $r = \lambda L$.

When $L > T$. A lead time longer than a cycle means one or more replenishment orders are outstanding when the next is placed. Thus, the reorder point must account only for the demand not already covered by

3. Deterministic Lot Sizing

those outstanding orders. The number of whole cycles inside the lead time is $\lfloor L/T \rfloor$, so the uncovered portion of the lead time is the remainder $L - \lfloor L/T \rfloor T$, which gives

$$r = \left(L - \left\lfloor \frac{L}{T} \right\rfloor T \right) \lambda - \hat{B} \quad (3.21)$$

Equation 3.21 reduces to Equation 3.20 when $L < T$, because the floor is then zero. Recall from Section 1.3.3 that what matters is the inventory **position**, which counts outstanding orders, and not the on-hand level. Equation 3.21 is the deterministic version of that point.

3.7. Sensitivity of the Order Quantity

The formulas above return a number like 40.37, and no warehouse orders 40.37 of anything. It matters how much is lost by rounding that number, or by ordering a case quantity, or by fitting a shipping schedule.

- Let $C(Q)$ represent the relevant cost of Equation 1.16 at an order quantity Q , which for this model is the part of Equation 3.16 that depends on Q .

That is,

$$C(Q) = \frac{k\lambda}{Q} + \frac{hQ}{2}, \quad C^* = C(Q^*) = \sqrt{2k\lambda h}$$

Taking the ratio and substituting $Q^* = \sqrt{2k\lambda/h}$, we obtain

$$\frac{C(Q)}{C^*} = \frac{1}{2} \left(\frac{Q}{Q^*} + \frac{Q^*}{Q} \right) \quad (3.22)$$

3. Deterministic Lot Sizing

Notice that the penalty depends only on the **ratio** of the chosen quantity to the optimal one, and not on k , λ , h , or the size of the item. Recall that the same expression appeared in Section 1.4.8, where the ratio came from an error in the parameters and not from a deliberate rounding.

Example 3.3 (What a wrong order quantity costs). Suppose the order quantity used is 50% larger than optimal, $Q/Q^* = 3/2$. From Equation 3.22,

$$\frac{C}{C^*} = \frac{1}{2} \left(\frac{3}{2} + \frac{2}{3} \right) = \frac{1}{2} \left(\frac{9+4}{6} \right) = \frac{13}{12} = 1.083$$

That is an increase of 8.3% in the relevant cost. For example, for the item of Example 3.2, whose relevant cost is \$8,719.63, it amounts to about \$727 a year against a total spend of \$272,720.

Ordering half the optimal quantity, $Q/Q^* = 1/2$, gives $\frac{1}{2}(0.5 + 2) = 1.25$, a penalty of 25%. Ordering double gives the same 25%, because Equation 3.22 is symmetric in Q/Q^* and its reciprocal.

You should read this as saying that the EOQ is a target to round toward, not a value to honor. Being wrong by half is expensive; being wrong by a fifth is not. That tolerance is the reason it is reasonable to order in case quantities, to fit a delivery schedule, and to group items onto a shared order, all of which Chapter 4 does deliberately.

3.8. Quantity Discounts

Everything so far has treated the unit cost c as a constant, which made the purchase term $c\lambda$ inert. Suppliers routinely price in tiers, and once c depends on Q the purchase term is no longer inert. The analysis changes.

3. Deterministic Lot Sizing

Two schemes are in general use. Under **all-units** discounting the whole order is priced at the rate reached by its size. Under **incremental** discounting only the units above each break point receive the lower price. For example, on a schedule breaking at 500 units, an order of 600 units pays the lower price on all 600 units under all-units discounting and on only the last 100 under incremental discounting. They lead to different models and different answers, so we treat them separately.

The setting is common to both.

- Let m represent the number of discount levels.
- Let q_j represent the break point beginning level j , with

$$q_1 = 0 < q_2 < q_3 < \cdots < q_m$$

- Let c_j represent the unit price applying in interval j , i.e. for $q_j \leq Q < q_{j+1}$, with $c_1 > c_2 > \cdots > c_m$ so that larger orders are cheaper per unit.

We assume a single item at a single location, no shortages, and instantaneous replenishment, so that the underlying model is the classical EOQ of Section 3.5.4.

3.8.1. All-Units Discounting

If the entire order is priced at c_j , then the cost rate within interval j is Equation 3.16 with that price:

$$TC_j(Q) = c_j\lambda + \frac{k\lambda}{Q} + \frac{h_jQ}{2}, \quad h_j = ic_j, \quad q_j \leq Q < q_{j+1} \quad (3.23)$$

3. Deterministic Lot Sizing

The important feature is that $TC(Q)$ is **not continuous**. Each interval carries its own curve, lower prices give lower curves, and at each break point the cost drops to the next curve down.

Figure 3.3 shows the shape for an item with $\lambda = 8000$ units per year, $k = \$30$ per order, $i = 0.30$ per year, and a price of \$10 below 500 units and \$9 at or above. Each price is drawn solid where it applies and dotted where it does not, so the cost function the buyer actually faces is the solid pieces only. Notice that the \$9 curve lies wholly below the \$10 curve, because a lower price lowers both the purchase term and the holding term. Notice also what happens at the dashed line marking the break point. The solid path drops vertically from the upper curve to the lower one, and that drop is why the minimum cannot be found by differentiating.

The two marked points are the candidates the procedure below produces. The left one is the unconstrained minimum of the \$10 curve, feasible because it lies below 500. The right one sits exactly at the break point, because the \$9 curve is still falling when it becomes available.

Because the function is discontinuous, we cannot find the minimum by differentiating alone. There are three cases, collected in Table 3.2, and the procedure below handles all of them.

Table 3.2.: The three cases for all-units discounting. The third is why every candidate must be evaluated instead of the first feasible one being accepted.

Case	What happens	Why
Minimum at a break point q_j	The unconstrained Q_j^* lies below q_j , so the interval's best feasible point is its left endpoint	The discount reduces cost by more than the extra stock costs to hold

3. Deterministic Lot Sizing

Case	What happens	Why
Minimum inside the interval	Q_j^* falls within $[q_j, q_{j+1})$	The extra carrying cost outweighs the price reduction
A local minimum only	The interval contains a minimum that a lower interval beats	Must compare across intervals, not within one

The procedure follows from Table 3.2:

1. For each price level j , compute $Q_j^* = \sqrt{2k\lambda/h_j}$.
2. Discard any Q_j^* that falls outside its own interval $[q_j, q_{j+1})$; that price is not available at that quantity.
3. For each discarded level whose price is lower than a feasible one, add the break point q_j as a candidate, since the interval's cost is decreasing up to its unconstrained minimum.
4. Evaluate Equation 3.23 at every candidate and take the smallest.

Example 3.4 (An all-units discount). An item has $\lambda = 8000$ units per year, $k = \$30$ per order, and a carrying charge of $i = 0.30$ per year. The supplier prices it at \$10 per unit for orders below 500 and \$9 per unit for orders of 500 or more. This is the item drawn in Figure 3.3, and we now work its four steps.

Step 1, the candidate quantities. With $h_j = ic_j$,

$$Q_1^* = \sqrt{\frac{2(30)(8000)}{10(0.3)}} = \sqrt{160,000} = 400 \quad Q_2^* = \sqrt{\frac{2(30)(8000)}{9(0.3)}} = \sqrt{177,778} = 422$$

Step 2, feasibility. The quantity $Q_1^* = 400$ lies in $[0, 500)$, so it is feasible at the \$10 price. However, $Q_2^* = 422$ does not lie in $[500, \infty)$, so 422 units cannot be bought at \$9. That candidate is discarded.

3. Deterministic Lot Sizing

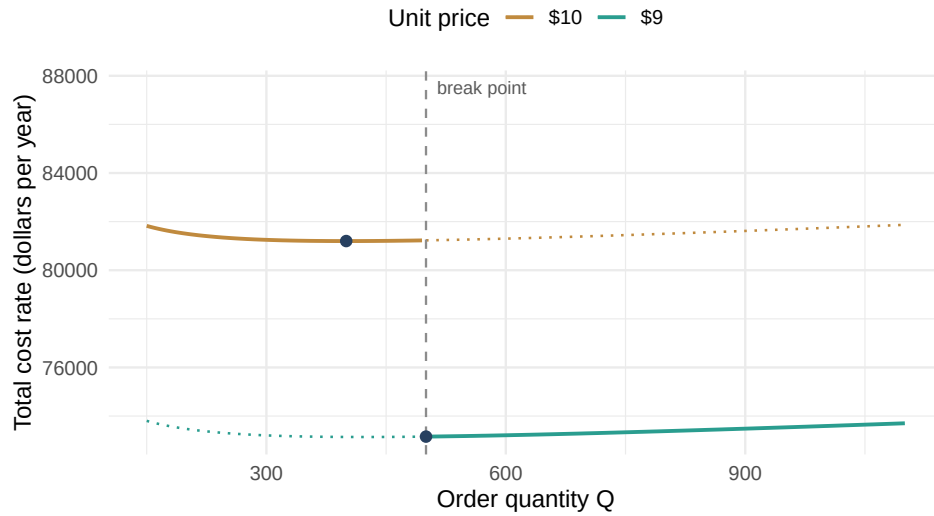


Figure 3.3.: All-units discounting with $\lambda = 8000$, $k = 30$, $i = 0.30$, and a price of \$10 below 500 units and \$9 at or above. Each price has its own cost curve, drawn solid where that price applies and dotted where it does not. The realized cost function is the solid pieces, and it drops discontinuously at the break point.

3. Deterministic Lot Sizing

Step 3, the break point. Since the \$9 curve is still decreasing at 500, the best available quantity at that price is the break point itself. 500 joins the candidate list.

Step 4, evaluate.

$$TC(400) = 8000(10) + \frac{30(8000)}{400} + \frac{(0.3)(10)(400)}{2} = 80,000 + 600 + 600 = \$81,200$$

$$TC(500) = 8000(9) + \frac{30(8000)}{500} + \frac{(0.3)(9)(500)}{2} = 72,000 + 480 + 675 = \$73,155$$

Order 500. Buying a quarter more than the EOQ at the \$10 price saves \$8,045 a year, almost all of it on the purchase term. The holding cost does rise, from \$600 to \$675. However, \$75 of extra holding against \$8,000 of price reduction is not a contest.

This is the first case in Table 3.2, and it is the common one. When a discount is worth taking at all, it is usually worth taking exactly at the break point and no further.

3.8.2. Incremental Discounting

Under incremental discounting the unit price falls only on the units above each break point. An order of size Q in interval j pays c_1 for the first $q_2 - q_1$ units, c_2 for the next $q_3 - q_2$, and so on, with c_j applying only to the $Q - q_j$ units above the last break point crossed.

- Let R_j represent the cost of filling all the intervals below j completely.

That is,

$$R_j = \sum_{i=2}^j c_{i-1} (q_i - q_{i-1}), \quad R_1 \equiv 0 \quad (3.24)$$

3. Deterministic Lot Sizing

The purchase cost of an order of size Q in interval j is then

$$C(Q) = R_j + c_j (Q - q_j) \quad (3.25)$$

Unlike the all-units case, Equation 3.25 is continuous. Crossing a break point changes the slope of the purchase cost, not its level.

The consequence for the model is that the **average price per unit** now depends on Q :

$$\frac{C(Q)}{Q} = \frac{R_j}{Q} + c_j - \frac{c_j q_j}{Q}$$

Holding cost is charged on the value of what is held. The holding cost per unit becomes $i C(Q)/Q$ instead of ic_j . Substituting into the cost rate and collecting terms in Q , we obtain

$$TC(Q) = \frac{k_j \lambda}{Q} + \frac{h_j Q}{2} + \underbrace{c_j \lambda + \frac{i(R_j - c_j q_j)}{2}}_{\text{independent of } Q} \quad (3.26)$$

where

$$k_j = R_j - c_j q_j + k, \quad h_j = ic_j \quad (3.27)$$

This is the point of the manipulation. Notice that the Q -dependent part of Equation 3.26 has exactly the form of Equation 3.16, with an **effective** ordering cost k_j in place of k . Thus, the ordinary square root formula applies level by level:

$$Q_j^* = \sqrt{\frac{2k_j \lambda}{h_j}} \quad (3.28)$$

3. Deterministic Lot Sizing

The procedure is then to compute Q_j^* for each level, keep those that fall inside their own interval, evaluate Equation 3.26 at each survivor, and take the smallest. Unlike the all-units case, there are no break-point candidates, because the cost function is continuous and has no downward jumps to exploit.

Example 3.5 (An incremental discount). An item has $\lambda = 3000$ units per year, $k = \$50$ per order, and $i = 0.30$ per year. The supplier's incremental schedule is given in Table 3.3, and we work the levels in turn.

Table 3.3.: An incremental discount schedule with three levels.

Interval	Break point q_j	Price c_j
1	0	\$3.00
2	500	\$2.97
3	1500	\$2.95

The filled-interval costs. From Equation 3.24,

$$R_1 = 0, \quad R_2 = 3.00(500-0) = 1500, \quad R_3 = 1500 + 2.97(1500-500) = 4470$$

The effective ordering costs. From Equation 3.27,

$$k_1 = 0 - 3.00(0) + 50 = 50, \quad k_2 = 1500 - 2.97(500) + 50 = 65, \quad k_3 = 4470 - 2.95(1500) + 50 = 95$$

The effective ordering cost **rises** with the discount level. That is the mechanism by which incremental discounting encourages larger orders. Reaching a deeper level means having already paid for everything below it, and that paid amount behaves exactly like a larger fixed charge.

3. Deterministic Lot Sizing

The candidate quantities. From Equation 3.28 with $h_j = ic_j$,

$$Q_1^* = \sqrt{\frac{2(50)(3000)}{0.3(3.00)}} = 577.4, \quad Q_2^* = \sqrt{\frac{2(65)(3000)}{0.3(2.97)}} = 661.6, \quad Q_3^* = \sqrt{\frac{2(95)(3000)}{0.3(2.95)}} = 802.5$$

Feasibility. The quantity $Q_1^* = 577.4$ must lie in $[0, 500)$ and does not, and $Q_3^* = 802.5$ must lie in $[1500, \infty)$ and does not. Only $Q_2^* = 661.6$ lies in its own interval $[500, 1500)$.

The answer. One candidate survives. Thus, $Q^* = 661.6$ units, and from Equation 3.26

$$TC = \frac{65(3000)}{661.6} + \frac{0.891(661.6)}{2} + 2.97(3000) + \frac{0.3(1500 - 2.97(500))}{2} = \$9,501.73$$

Notice that the first two terms are \$294.74 each. The effective ordering cost substitution has turned this into an ordinary EOQ problem, so the balance property of Section 3.5.4 still holds. You should perform that check every time, because it confirms both the effective ordering cost and the quantity computed from it.

3.9. Performance Measures for the Cycle

Section 1.6 defined the service measures without reference to any model. Because the deterministic cycle is fully known, each of them has a closed form here, and computing them is a good way to see what the definitions mean.

Throughout this section we use the general cycle of Section 3.1 with its four segments.

3. Deterministic Lot Sizing

Order frequency. One order is placed per cycle, so that

$$\overline{N} = \frac{1}{T} = \frac{\lambda}{Q} \quad (3.29)$$

Inventory turnover. From Equation 1.8, with $\bar{I}$ from Equation 3.6:

$$TO = \frac{\lambda}{\bar{I}}$$

Fraction of time out of stock. The system is short over T_4 and T_1 . Thus, from the stockout indicator of Equation 1.9,

$$\overline{SO} = \frac{T_1 + T_4}{T} = \frac{\frac{\hat{B}}{p - \lambda} + \frac{\hat{B}}{\lambda}}{\frac{Q}{\lambda}} \quad (3.30)$$

Ready rate. Its complement, $\overline{RR} = 1 - \overline{SO}$.

Rate of unfilled demand. Demand arrives at a constant rate, so the rate at which it arrives to an empty shelf is $\lambda \overline{SO}$. That is λ_ℓ of Equation 1.3, computable exactly here.

Average customer wait. From Equation 1.6, $\overline{W} = \bar{B}/\lambda$.

Example 3.6 (Performance of the general cycle). Recall the cycle of Example 3.1, with $\lambda = 100$, $p = 250$, $Q = 400$, and $\hat{B} = 30$, giving $T = 4$ years, segments 0.2, 1.4, 2.1, 0.3, and averages $\bar{I} = 91.875$ and $\bar{B} = 1.875$ units. We compute each measure in turn.

$$\overline{N} = \frac{100}{400} = 0.25 \text{ orders per year}$$

3. Deterministic Lot Sizing

$$TO = \frac{100}{91.875} = 1.09 \text{ turns per year}$$

$$\overline{SO} = \frac{0.2 + 0.3}{4} = 0.125, \quad \overline{RR} = 0.875$$

$$\lambda \overline{SO} = 12.5 \text{ units per year arrive to an empty shelf}$$

$$\overline{W} = \frac{1.875}{100} = 0.01875 \text{ year} = 6.8 \text{ days}$$

We compare the last two. The system is out of stock one eighth of the time, which sounds poor, and a customer who is made to wait waits about a week. Whether that is acceptable is not a question the model answers. Section 1.6.2.7 is about who does answer it.

Notice also that the ready rate of 87.5% and the fill rate are the **same** here. Demand arrives continuously at a constant rate, so the fraction of demand arriving during a stockout equals the fraction of time in stockout. You should treat that equality as a consequence of the deterministic assumption and not as a general fact. Section 1.6.2.6 gives the condition under which it survives into stochastic models.

3.10. Building These Models in a Spreadsheet

Every result in this chapter is a formula in three or four inputs, so a spreadsheet is the natural place to put it. The workbook that accompanies this chapter, `DeterministicInventory.xlsx`, carries one worksheet per model, and all of them are built to the same plan. That plan is worth more than any single worksheet, because it lets someone who did not write the implementation audit it. In this section we take the plan first and then read each worksheet against it.

3. Deterministic Lot Sizing

3.10.1. The Four-Block Layout

Each worksheet is divided into blocks, in the order given in Table 3.4.

Table 3.4.: The blocks common to every worksheet in the workbook. The separation of the evaluator from the optimizer is what makes the sensitivity analysis of Section 3.7 a matter of typing a number.

Block	Contents	Role
Inputs	λ, c, i, k, L , and p where the model has one, each with its units beside it	the only cells anyone types into
Intermediate calculations	quantities derived from the inputs alone, such as $h = ic$ and $1 - \lambda/p$	names the pieces the formulas are assembled from
Outputs for any Q	frequency, cycle time, average inventory, and each cost component, all evaluated at a quantity that is typed in	the evaluator
Outputs for Q^*	the same rows recomputed at the optimum	the optimizer

Two conventions run through all of it. First, every input carries a units label in the adjacent cell. That is the dimensional discipline of Section 1.4.1 enforced by the layout, because a carrying charge quoted per year cannot be dropped silently into a model whose demand rate is per month when the two labels sit side by side. Second, every quantity has a rounded companion computed with **ROUNDUP**, so the integer that would actually be ordered is visible next to the number the formula produced.

3. Deterministic Lot Sizing

	A	B	C	D
1	Inputs			
2	Demand per time (λ)	100 (units/time)		
3	Cost per unit (c)	8 (\$)		
4	Holding interest rate (i)	0.020833 (\$/\$/time)		
5	Cost per order (K)	20 (\$/order)		
6	Holding cost per unit ($h=i*c$)	0.166667 (\$/unit/time)		
7	Lead time (τ)	0.25 (time)		
8				
9	Outputs for any Q			
10	Order Quantity (Q)	154.9193 (units)		155
11	Ordering Frequency (λ/Q)	0.645497 (orders/time)		
12	Cycle time (Q/λ)	1.549193 time		
13	Average Inventory Onhand (I)	77.45967 (units)		
14	Purchase cost per time ($\lambda*c$)	\$800.00 (\$/time)		
15	Ordering cost per time	\$12.91 (\$/time)		
16	Holding cost per time	\$12.91 (\$/time)		
17	Ordering + Holding Cost per time	\$25.82 (\$/time)		
18	Total Cost per time	\$825.82 (\$/time)		
19	Reorder point (R)	25 (units)		25
20	Ratio of cost to optimal cost	1		
21	Cost difference from optimal	0 \$/time		
22	Relative Cost Error from Optimal	0		
23				
24	Outputs for EOQ (Q^*)			Rounded
25	Economic Order Quantity (Q^*)	154.9193 (units)		155
26	Optimal OC + HC at Q^*	\$25.82 (\$/time)		
27	Optimal Ordering Frequency (λ/Q^*)	0.645497 (orders/time)		
28	Optimal Cycle time (Q^*/λ)	1.549193 time		
29	Optimal Average Inventory Onhand (I)	77.45967 (units)		
30	Purchase cost per time ($\lambda*c$)	\$800.000 (\$/time)		
31	Optimal Ordering cost per time	\$12.910 (\$/time)		
32	Optimal Holding cost per time	\$12.910 (\$/time)		
33	Optimal Ordering + Holding Cost	\$25.820 (\$/time)		
34	Total Cost per time	\$825.820 (\$/time)		
35	Reorder point (R)	25 (units)		25

Figure 3.4.: The Classic EOQ worksheet. Rows 1 to 7 hold the inputs and their units, rows 9 to 22 the evaluator, and rows 24 to 35 the optimizer. Column D carries the rounded companions. The three rows below the evaluator report no penalty here because the evaluator has been given Q^* itself.

3. Deterministic Lot Sizing

Figure 3.4 shows the plan in place on the Classic EOQ worksheet. Reading it top to bottom, rows 1 to 7 are the input block, with every rate carrying its unit in the cell to its right. Rows 9 to 22 are the evaluator, which computes every quantity at whatever order quantity is typed into it. Rows 24 to 35 are the optimizer, the same list of rows recomputed at Q^* . Notice that column D repeats several quantities as **ROUNDUP** companions, so the integer that would actually be ordered sits beside the number the formula produced.

Notice the three rows that join the two output blocks. They are the whole of Section 3.7 built into the sheet: the ratio of the evaluated cost to the optimal cost, the difference, and the relative error. In Figure 3.4 they read 1, 0, and 0, because the evaluator has been given Q^* itself. Type any other quantity and the penalty appears immediately.

The worksheet labels are not identical to the notation of this book, mostly for reasons of what a spreadsheet can display. Table 3.5 gives the translation, and you should keep it at hand when reading any of the worksheet figures.

Table 3.5.: Worksheet labels and the corresponding symbols in the text.

Worksheet label	This book	Meaning
Demand per time (λ)	λ	demand rate
Cost per unit (c)	c	unit purchase or production cost
Holding interest rate (i)	i	carrying charge
Cost per order (K)	k	fixed cost of an order or setup
Holding cost per unit ($h = i * c$)	h	holding cost rate
Lead time (τ)	L	replenishment lead time
Production rate (P)	p	production rate
Average Inventory Onhand (I)	$\bar{I}$	average on-hand level
Max On Hand Inventory (H)	$\hat{I}$	peak on-hand level
Reorder point (R)	r	reorder point

3. Deterministic Lot Sizing

Example 3.7 (Reading the classic EOQ worksheet). The Classic EOQ worksheet of Figure 3.4 is set up in **months**, with $\lambda = 100$ units per month, $c = \$8$ per unit, a carrying charge of 25% per year entered as $i = 0.25/12$ per month, $k = \$20$ per order, and $L = 0.25$ month. We follow the sheet block by block.

The intermediate cell gives the holding cost rate,

$$h = ic = \frac{0.25}{12}(8) = \$0.16667 \text{ per unit per month}$$

The optimizer block then applies Equation 3.17, Equation 3.19, and Equation 3.18:

$$Q^* = \sqrt{\frac{2(20)(100)}{0.16667}} = \sqrt{24,000} = 154.92 \text{ units}, \quad T^* = \frac{154.92}{100} = 1.5492 \text{ months}$$

$$\frac{k\lambda}{Q^*} = \frac{hQ^*}{2} = \$12.91 \text{ per month}, \quad TC^* = 25.82 + (8)(100) = \$825.82 \text{ per month}$$

The two cost components agree to five figures. That is the balance check of Section 3.5.4, performed by the sheet itself and not by the reader.

The evaluator. No one orders 154.92 units. Typing the ROUNDUP value 155 into the evaluator returns a relative cost error of 1.4×10^{-7} , effectively nothing. Typing 150 or 160 returns 0.052% in either case, about a penny a month on a relevant cost of \$25.82. Typing 200 returns 3.3%. The three sensitivity rows turn Equation 3.22 into an experiment instead of an algebraic exercise.

Why the units column matters. Suppose the annual charge of 0.25 had been entered directly against a monthly demand rate. Then h would have been twelve times too large, and Q^* would have come out as 44.7

3. Deterministic Lot Sizing

units, an error of a factor of $\sqrt{12}$. Nothing in the arithmetic would have signaled it. The units label beside the cell is the only guard, so you should never omit it.

3.10.2. The Reorder Point Cell

One cell in the worksheet deserves attention on its own. The reorder point is computed as

`= MOD(lead time, cycle time) * demand rate`

which is Equation 3.21 written in the one function that already does the work. `MOD(L, T)` returns $L - \lfloor L/T \rfloor T$, i.e. the part of the lead time not covered by orders already outstanding. When $L < T$ the floor is zero and the expression collapses to λL . Thus, the single cell handles both cases of Section 3.6 without an IF.

Figure 3.5 shows the same worksheet with its formulas displayed instead of its values, the view to use when auditing someone else's sheet. Reading down the optimizer block, every formula there is one of the closed forms of Section 3.5.4, written once and referring only to the input and intermediate cells above it. Notice the reorder point cell, which is the single `MOD` call discussed above.

Notice one further detail that only the formula view reveals. The order quantity cell of the evaluator holds a reference to the optimizer's Q^* and not a typed number. The evaluator opens at the optimum, and typing over that one cell is the intended way to use it.

Example 3.8 (A lead time longer than the cycle). Recall the item of Example 3.7, whose cycle is $T^* = 1.5492$ months, and suppose the supplier now quotes a lead time of $L = 4$ months.

3. Deterministic Lot Sizing

	A	B	C	D
1	Inputs			
2	Demand per time (λ)	100	(units/time)	
3	Cost per unit (c)	8	(\$)	
4	Holding interest rate (i)	=0.25/12	(\$/\$/time)	
5	Cost per order (K)	20	(\$/order)	
6	Holding cost per unit ($h=i*c$)	=B3*B4	(\$/unit/time)	
7	Lead time (τ)	0.25	(time)	
8				
9	Outputs for any Q			
10	Order Quantity (Q)	=B25	(units)	=ROUNDUP(B10,0)
11	Ordering Frequency (λ/Q)	=B2/B10	(orders/time)	
12	Cycle time (Q/ λ)	=1/B11	time	
13	Average Inventory Onhand (I)	=B10/2	(units)	
14	Purchase cost per time ($\lambda*c$)	=B2*B3	(\$/time)	
15	Ordering cost per time	=B11*B5	(\$/time)	
16	Holding cost per time	=B6*B13	(\$/time)	
17	Ordering + Holding Cost per time	=B15+B16	(\$/time)	
18	Total Cost per time	=B17+B14	(\$/time)	
19	Reorder point (R)	=MOD(B7,B12)*B2	(units)	=ROUNDUP(B19,0)
20	Ratio of cost to optimal cost	=B17/B33		
21	Cost difference from optimal	=B17-B33	\$/time	
22	Relative Cost Error from Optimal	=B21/B33		
23				
24	Outputs for EOQ (Q*)			Rounded
25	Economic Order Quantity (Q*)	=SQRT(2*B5*B2/B6)	(units)	=ROUNDUP(B25,0)
26	Optimal OC + HC at Q*	=SQRT(2*B5*B2*B6)	(\$/time)	
27	Optimal Ordering Frequency (λ/Q^*)	=B2/B25	(orders/time)	
28	Optimal Cycle time (Q*/ λ)	=1/B27	time	
29	Optimal Average Inventory Onhand (I)	=B25/2	(units)	
30	Purchase cost per time ($\lambda*c$)	=B2*B3	(\$/time)	
31	Optimal Ordering cost per time	=B27*B5	(\$/time)	
32	Optimal Holding cost per time	=B29*B6	(\$/time)	
33	Optimal Ordering + Holding Cost	=B31+B32	(\$/time)	
34	Total Cost per time	=B33+B30	(\$/time)	
35	Reorder point (R)	=MOD(B7,B28)*B2	(units)	=ROUNDUP(B35,0)

Figure 3.5.: The Classic EOQ worksheet with formulas displayed. The reorder point is a single MOD call, the order quantity in the evaluator is a reference to the optimum below it, and every formula in the optimizer block is one of the closed forms of Section 3.5.4.

3. Deterministic Lot Sizing

The naive calculation gives $\lambda L = 100(4) = 400$ units, more than twice the order quantity, so it cannot be a position at which to order. The worksheet cell instead gives

$$\left\lfloor \frac{4}{1.5492} \right\rfloor = \lfloor 2.582 \rfloor = 2, \quad 4 - 2(1.5492) = 0.9016 \text{ month}, \quad r = 100(0.9016) = 90.16 \text{ units}$$

Two replenishment orders are outstanding at the moment the third is placed, and they cover $2T$ of the four months. Only the remaining 0.9016 month has to be covered by stock on hand. Thus, the order is placed when the inventory **position** reaches 90.16 units. Recall that Section 1.3.3 distinguishes the position carefully from the on-hand level, and this is a case where the distinction decides the answer.

Sweeping L upward makes the point visible. For example, at $L = 2$ months the same cell returns $r = 45.08$ units, far less than $\lambda L = 200$. The reorder point is not monotone in the lead time. It climbs to Q and drops back to zero once per cycle length of lead time.

3.10.3. Reducing the Production Model to the Order Model

The Classic POQ worksheet is the clearest illustration of what Section 3.5 is claiming. Rather than coding Equation 3.14 as a new formula, the worksheet computes three intermediate cells,

$$\frac{\lambda}{p}, \quad 1 - \frac{\lambda}{p}, \quad h' = h \left(1 - \frac{\lambda}{p} \right)$$

and then applies the **ordinary** EOQ formula to h' . The production model is not implemented at all. It is the order model with a different holding cost rate. The same substitution turns Equation 3.18 into the production version, so one set of formulas serves both worksheets.

3. Deterministic Lot Sizing

The worksheet also guards its own assumption. A cell beside the production rate reads

```
= IF(demand rate >= production rate,
    "Infeasible production rate", "Production rate feasible")
```

because the cycle of Section 3.1 exists only when $p > \lambda$. Without the guard, $p = \lambda$ produces $h' = 0$ and a division by zero, while $p < \lambda$ produces a negative holding cost and a square root of a negative number. Neither failure explains itself, so the sheet explains it first.

Figure 3.6 shows both the substitution and the guard. Reading the intermediate block downward, the three cells build λ/p , then $1 - \lambda/p$, then h' , each referring only to the cell above it. Notice that the optimizer block below contains no production formulas at all, only the ordinary EOQ formulas pointed at h' . The guard sits beside the production rate input, where someone typing a bad rate will see it.

Example 3.9 (Reading the production worksheet). The worksheet is set up with $\lambda = 2500$ units per year, $p = 10,000$ units per year, $c = \$2$ per unit, $i = 0.30$ per year, and a setup cost of $k = \$50$. We read it block by block.

The intermediate block gives

$$h = ic = 0.60, \quad \frac{\lambda}{p} = 0.25, \quad 1 - \frac{\lambda}{p} = 0.75, \quad h' = 0.60(0.75) = \$0.45$$

The optimizer block then runs the ordinary EOQ formula on h' :

$$Q^* = \sqrt{\frac{2(50)(2500)}{0.45}} = 745.36 \text{ units}, \quad \frac{k\lambda}{Q^*} = \frac{h'Q^*}{2} = \$167.71 \text{ per year}$$

3. Deterministic Lot Sizing

	A	B	C	D
1	Inputs			
2	Demand per time (λ)	2500	(units/time)	
3	Production rate (P)	10000	(units/time)	=IF(B2>=B3, "Infeasible production rate", "Production rate feasible")
4	Cost per unit (c)	2	(\$)	
5	Holding interest rate (i)	0.3	(\$/\$/time)	
6	Setup Cost per order (K)	50	(\$/order)	
7	Holding cost per unit ($h=i*c$)	=B4*B5	(\$/unit/time)	
8				
9	Intermediate Calculations			
10	Demand/Production Rate Ratio (λ/P)	=B2/B3		
11	$1 - (\lambda/P)$	=1-B10		
12	$h' = h * (1 - \lambda/P)$	=B7*B11		
13				
14	Outputs for any Q			
15	Production Quantity (Q)	=B30	(units)	=ROUNDUP(B15,0)
16	Cycle time (Q/ λ)	=1/B20	(time)	
17	Time spent in production (Q/P)	=B15/B3	(time)	
18	Time Spent using production	=B16-B17	(time)	
19	Max On Hand Inventory (H)	=B15*B11	(units)	
20	Ordering Frequency (λ/Q)	=B2/B15	(orders/time)	
21	Average Inventory Onhand (I)	=B19/2	(units)	
22	Ordering cost per time	=B20*B6	(\$/time)	
23	Holding cost per time	=B7*B21	(\$/time)	
24	Ordering + Holding Cost per time	=B22+B23	(\$/time)	
25	Ratio of cost to optimal cost	=(B24/B40)		
26	Cost difference from optimal	=B24-B40	\$/time	
27	Relative Cost Error from Optimal	=B26/B40		
28				
29	Outputs for POQ (Q*)			Rounded
30	Economic Production Quantity (Q*)	=SQRT(2*B6*B2/B12)	(units)	=ROUNDUP(B30,0)
31	Optimal OC + HC at Q*	=SQRT(2*B6*B2*B12)	(\$/time)	
32	Cycle time (Q/ λ)	=B30/B2	(time)	
33	Time spent in production (Q/P)	=B30/B3	(orders/time)	
34	Time Spent using production	=B32-B33	(time)	
35	Max On Hand Inventory (H)	=B30*B11	(units)	
36	Ordering Frequency (λ/Q)	=1/B32	(units)	
37	Average Inventory Onhand (I)	=B35/2	(\$/time)	
38	Ordering cost per time	=B36*B6	(\$/time)	
39	Holding cost per time	=B37*B7	(\$/time)	
40	Ordering + Holding Cost per time	=B38+B39		

Figure 3.6.: The production worksheet with formulas displayed. The three intermediate cells build h' , the optimizer then applies the ordinary EOQ formulas to it, and the cell beside the production rate carries the feasibility guard.

3. Deterministic Lot Sizing

The cycle quantities then follow from Equation 3.2 with $\hat{B} = 0$ and from Equation 3.1:

$$\hat{I} = 745.36(0.75) = 559.02, \quad \bar{I} = 279.51, \quad T = \frac{745.36}{2500} = 0.2981, \quad T_p = \frac{745.36}{10,000} = 0.0745$$

Notice that the machine runs for 0.0745 year of every 0.2981 year cycle, i.e. a quarter of the time, which is λ/p as it must be. You should perform that check every time, because it catches an error in either the cycle length or the replenishment period.

Figure 3.7 shows the completed sheet with its values. Reading across the two output blocks, the ordering and holding costs agree at \$167.71, the balance check of Section 3.5.4 surviving the substitution intact.

For comparison, ignoring the finite production rate entirely gives $\sqrt{2(50)(2500)}/0.60 = 645.50$ units. Spreading the receipt over time lowers the average inventory for a given lot, which makes larger lots affordable. Thus, the production quantity exceeds the order quantity, by the factor $1/\sqrt{1 - \lambda/p} = 1/\sqrt{0.75} = 1.155$.

3.10.4. Discount Schedules as Columns

The discount models of Section 3.8 differ from the others in that they require a comparison across levels instead of a single formula. Both worksheets handle this the same way. The price schedule is entered as a table, one row per level, and each step of the procedure becomes a computed column.

For all-units discounting the columns are those of Table 3.6.

3. Deterministic Lot Sizing

	A	B	C	D
1	Inputs			
2	Demand per time (λ)	2500 (units/time)		
3	Production rate (P)	10000 (units/time)	Production rate feasible	
4	Cost per unit (c)	2 (\$)		
5	Holding interest rate (i)	0.3 (\$/\$/time)		
6	Setup Cost per order (K)	50 (\$/order)		
7	Holding cost per unit ($h=i*c$)	0.6 (\$/unit/time)		
8				
9	Intermediate Calculations			
10	Demand/Production Rate Ratio (λ/P)	0.25		
11	$1 - (\lambda/P)$	0.75		
12	$h' = h*(1-\lambda/P)$	0.45		
13				
14	Outputs for any Q			
15	Production Quantity (Q)	745.3559925 (units)		746
16	Cycle time (Q/ λ)	0.298142397 (time)		
17	Time spent in production (Q/P)	0.074535599 (time)		
18	Time Spent using production	0.223606798 (time)		
19	Max On Hand Inventory (H)	559.0169944 (units)		
20	Ordering Frequency (λ/Q)	3.354101966 (orders/time)		
21	Average Inventory Onhand (I)	279.5084972 (units)		
22	Ordering cost per time	\$167.71 (\$/time)		
23	Holding cost per time	\$167.71 (\$/time)		
24	Ordering + Holding Cost per time	\$335.41 (\$/time)		
25	Ratio of cost to optimal cost	1		
26	Cost difference from optimal	0 \$/time		
27	Relative Cost Error from Optimal	0		
28				
29	Outputs for POQ (Q*)			Rounded
30	Economic Production Quantity (Q*)	745.3559925 (units)		746
31	Optimal OC + HC at Q*	335.4101966 (\$/time)		
32	Cycle time (Q/ λ)	0.298142397 (time)		
33	Time spent in production (Q/P)	0.074535599 (orders/time)		
34	Time Spent using production	0.223606798 (time)		
35	Max On Hand Inventory (H)	559.0169944 (units)		
36	Ordering Frequency (λ/Q)	3.354101966 (units)		
37	Average Inventory Onhand (I)	279.5084972 (\$/time)		
38	Ordering cost per time	\$167.71 (\$/time)		
39	Holding cost per time	\$167.71 (\$/time)		
40	Ordering + Holding Cost per time	\$335.41		

Figure 3.7.: The production worksheet evaluated. The intermediate block reduces the model to an ordinary EOQ in $h' = 0.45$, and the ordering and holding costs agree at \$167.71, the balance check of Section 3.5.4 surviving the substitution.

3. Deterministic Lot Sizing

Table 3.6.: The all-units worksheet's computed columns, one per step of the four-step procedure. The answer is the smallest cost among the feasible EOQ entries and the break points of the infeasible ones.

Column	Formula	Step of Section 3.8.1
Lower limit, Upper limit, \$/unit	typed in	the schedule q_j and c_j
EOQ	$\sqrt{2k\lambda/(ic_j)}$	step 1, the candidate at each price
TC(EOQ)	$c_j\lambda + \sqrt{2k\lambda ic_j}$	the cost of that candidate
Feasible?	IF(AND(lower <= EOQ, EOQ <= upper), "Yes", "No")	step 2, is the price available at that quantity
TC(Lower Limit)	$c_j\lambda + k\lambda/q_j + ic_jq_j/2$	step 3, the cost at the break point

Laid out this way the whole procedure is visible at once. Thus, the third case of Table 3.2, in which a lower interval beats a feasible candidate, cannot be missed by stopping early.

Figure 3.8 shows the result. The schedule occupies columns E through G, with one row per price level, and the four computed columns to its right carry out the procedure one step per column. Reading across a row gives that level's candidate quantity, its cost, whether the price is actually available at that quantity, and the cost at the break point if it is not. Notice the evaluator block at the left, which prices whatever quantity is chosen and is the same evaluator that appears on every other worksheet in the workbook.

Notice one detail of the schedule table. The first lower limit is entered as 1

3. Deterministic Lot Sizing

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	Inputs												
2	Demand per time (A)	600 (units/time)			Cost Schedule Per Unit								
3	Holding interest rate (i)	0.2 (\$/\$/time)			Lower Limit	Upper Limit	\$/unit	EOQ	TC(EOQ)	Feasible?	TC(Lower Limit)		
4	Cost per order (K)	8 (\$/order)			1	1	500	0.3	400	204	Yes	4980.03	
5					2	500	1000	0.29	406.838102	197.59661	No	198.1	
6					3	1000	100000	0.28	414.039336	191.186203	No	200.8	
7													
8	Outputs for any Q				Please see the help on the VLOOKUP function. It is critical to understanding this spreadsheet. The cost per unit is selected using the VLOOKUP function on the discount schedule. The cost schedule per unit can be added to by adding additional rows e.g. starting at D7. The schedule can be easily changed in E through G. However, you must then adjust the VLOOKUP expression in cell B10 to include the whole schedule. To solve for the optimal order quantity you can do the following: 1) Check every feasible EOQ (given in column H, see J for feasibility) 2) Compare Total costs of feasible EOQs (column I) with total costs of lower limits (column K) pick quantity with lowest cost Alternatively, you can run Solver, see picture for correct setup. Sometimes it may not converge.								
9	Order Quantity (Q)	500.003087 (units)											
10	Cost per unit	0.29											
11	Holding cost per unit (h=i*c)	0.058 (\$/unit/time)											
12	Ordering Frequency (A/Q)	1.19999259 (orders/time)											
13	Cycle time (Q/A)	0.83333848 time											
14	Average Inventory Onhand (I)	250.001544 (units)											
15	Purchase cost per time (A*c)	\$174.00 (\$/time)											
16	Ordering cost per time	\$9.60 (\$/time)											
17	Holding cost per time	\$14.50 (\$/time)											
18	Ordering + Holding Cost per time	\$24.10 (\$/time)											
19	Total Cost per time	\$198.10 (\$/time)											
20													
21													
22													
23													
24													
25													
26													
27													

Figure 3.8.: The all-units worksheet. The schedule occupies columns E through G and the four computed columns to its right carry out the procedure of Section 3.8.1 one step per column. The evaluator at the left prices the chosen quantity.

3. Deterministic Lot Sizing

and not 0, because a zero would leave the VLOOKUP without a match and would make the λ/Q terms infinite. The model of Section 3.8 sets $q_1 = 0$. Nothing depends on the difference, because an order of size zero is never a candidate.

The worksheet offers a second route to the same answer. You can name the total cost cell as an objective, the order quantity cell as the variable, and minimize with a solver. It is the less reliable route, for a reason the model has already established. Equation 3.23 is discontinuous, dropping to a lower curve at each break point, and that a gradient method assumes it is not. Started inside one interval, such a method finds that interval's minimum and stops, exactly the third case of Table 3.2. Thus, the columns enumerate the candidates and cannot miss one, while a solver searches and can.

Example 3.10 (Reading the all-units worksheet). The worksheet has $\lambda = 600$ units per time, $k = \$8$ per order, and $i = 0.20$, together with the three level schedule below. The computed columns of Figure 3.8 fill in as shown.

Level	q_j	c_j	$h_j = ic_j$	Q_j^*	Feasi-ble?	$TC(Q_j^*)$	$TC(q_j)$
1	0	\$0.30	0.060	400.00	Yes	204.00	
2	500	\$0.29	0.058	406.84	No		198.10
3	1000	\$0.28	0.056	414.04	No		200.80

The candidates are therefore $Q = 400$ at \$204.00, $Q = 500$ at \$198.10, and $Q = 1000$ at \$200.80. **Order 500.**

Level 3 is the instructive row. Its price is the lowest available, and evaluating its unconstrained candidate gives \$191.19, which is lower than any figure the table reports. However, 414 units cannot be bought at \$0.28, and buying the 1000 units required to reach that price costs more in holding than the price cut returns.

3. Deterministic Lot Sizing

Notice what the two tempting shortcuts would have done. Evaluating candidates in price order and stopping at the first feasible one would have chosen level 1 and \$204.00. Evaluating the lowest number in the table would have chosen a quantity that is not for sale. Every candidate must be evaluated, and the columns enforce that.

The incremental worksheet needs one more column, and the way it is built is neater than Equation 3.24. Rather than accumulating the filled-interval cost R_j and then subtracting $c_j q_j$, the sheet computes a single **fixed cost** column recursively.

- Let H_j represent that fixed cost column at level j .

That is,

$$H_1 = 0, \quad H_j = (c_{j-1} - c_j) q_j + H_{j-1} \quad (3.31)$$

Equation 3.31 is exactly $R_j - c_j q_j$ from Equation 3.27, obtained without ever forming R_j . Each step adds the amount saved on the q_j units already bought by dropping to the new price, the same accounting seen from the other side. The effective ordering cost is then $k_j = H_j + k$.

Figure 3.9 shows the sheet. The fixed cost column sits at the right of the schedule table, and the price and fixed cost applying at a given Q are retrieved from that table with a **VLOOKUP**. Notice what is absent. Unlike the all-units sheet, this one carries no candidate columns, so the evaluator at the left is the whole of it, and the comparison across levels has to be made by the reader.

Example 3.11 (Reading the incremental worksheet, and a warning). We keep the same inputs and the same break points as Example 3.10, now under an incremental schedule. The fixed cost column, from Equation 3.31, is

3. Deterministic Lot Sizing

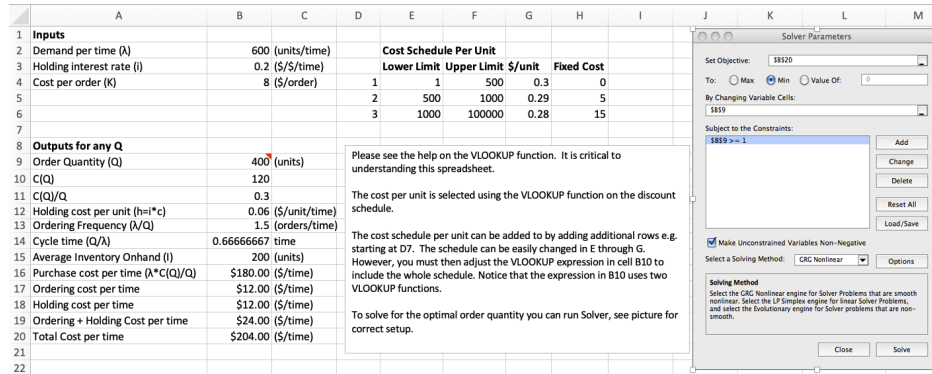


Figure 3.9.: The incremental worksheet. The fixed cost column at the right of the schedule accumulates $R_j - c_j q_j$ recursively. Unlike the all-units sheet, this one carries no candidate columns, so the evaluator at the left is the whole of it.

$$H_1 = 0, \quad H_2 = (0.30 - 0.29)(500) + 0 = 5, \quad H_3 = (0.29 - 0.28)(1000) + 5 = 15$$

Thus, the effective ordering costs are $k_1 = 8$, $k_2 = 13$, and $k_3 = 23$, and Equation 3.28 gives

Level	k_j	h_j	Q_j^*	Own inter-val	Feasi-ble?	TC
1	8	0.060	400.00	[0, 500)	Yes	204.00
2	13	0.058	518.62	[500, 1000)	Yes	204.58
3	23	0.056	702.04	[1000, ∞)	No	

Two candidates survive feasibility, and the smaller one wins. **Order 400 at a cost of \$204.00 per unit time.**

3. Deterministic Lot Sizing

An evaluator is not an optimizer

Either surviving candidate can be typed into the evaluator block, and both return a plausible number. That is, 400 returns \$204.00 and 518.62 returns \$204.58, because the block prices whatever quantity it is given.

Because the incremental worksheet carries no candidate columns, a solver run against the total cost cell is the only automated route on it, and that route is unsafe here, for a reason you should understand. The incremental cost function is continuous, so it has none of the jumps that defeat a solver in the all-units case. However, it is kinked at every break point, and each interval carries its own minimum. Both 400 and 518.62 are local minima. A gradient method started near either one reports that one and stops, so only enumeration distinguishes them.

The margin here is 58 cents per unit time against \$204, i.e. about a quarter of a percent, exactly the sort of difference that survives unnoticed. The check that catches it costs nothing. Level 2 is not the only feasible level, so its candidate cannot be accepted without evaluating level 1 as well.

3.10.5. What the Worksheets Do Not Do

Three limits need stating plainly, because each of them is a place where a correct spreadsheet still gives a wrong answer.

They evaluate, they do not choose. Every worksheet prices an entered quantity faithfully. Only the all-units sheet enumerates alternatives, and only because someone laid the enumeration out in columns. A solver can be pointed at any of them. However, the two discount models are precisely where a solver is least trustworthy, the case Example 3.11 makes.

They do not know their own assumptions. The production worksheet

3. Deterministic Lot Sizing

guards $p > \lambda$ and nothing else. Nothing checks that demand is actually constant, that the lead time is actually known, or that backorders are actually permitted. Recall that those judgments belong to Section 1.2.2, and you make them before the workbook is opened.

They round after the fact. The ROUNDUP column reports the integer that would be ordered, but the cost figures are computed at the unrounded quantity. For the classic EOQ item that gap is 1.4×10^{-7} of the relevant cost and can be ignored. For an item ordered in cases of 48, as in Exercise 3.5, it cannot. Thus, the evaluator block exists precisely so that the rounded quantity can be entered and priced.

3.11. Designing the Software

Recall that Section 3.10.5 ended with three things a worksheet does not do. It evaluates and does not choose, it does not know its own assumptions, and it rounds after the fact. Those are reasons to write a program. They are not a design.

A worksheet argues for itself. The blocks sit in an order you can see, and a formula is a cell you can click on. A program's structure is a set of decisions about which classes exist and what each class is responsible for, and a reader handed the finished code sees only the result. In this section we work through those decisions for the six models of this chapter, and Section 3.12 then shows what to type.

Four decisions carry most of the design, and each one is a question that returns in any model you write. We take them in turn.

3.11.1. What Changes, and What Must Not

A worksheet is a thing you edit. You change the ordering cost, the sheet recalculates, and you compare what you now see against the numbers from

3. Deterministic Lot Sizing

a minute ago. Both halves of that matter. The inputs have to be editable, and the answer you already took down has to stay where it was.

Those two requirements pull in opposite directions. We meet them with three types.

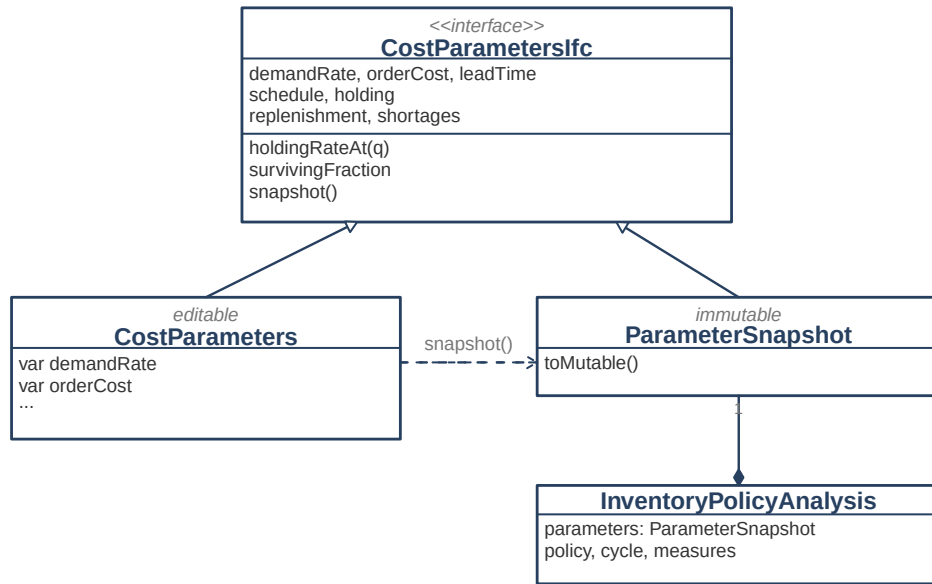


Figure 3.10.: One concept in three types. The interface is what everything downstream is allowed to see, the editable class is the sheet you change, and the snapshot is the printout. Notice that an analysis holds a snapshot by composition, so the record cannot outlive or drift from the result that carries it.

Figure 3.10 illustrates the three answers to one question. **CostParameters** is the sheet. Every property can be assigned, and every property validates on assignment with the same predicate the constructor applies, so no invalid parameter set ever exists. **ParameterSnapshot** is the printout, unchangeable once taken. **CostParametersIfc** is the read-only view, and

3. Deterministic Lot Sizing

it is what every model, every cycle and every cost calculation depends on.

The read-only view is the decision doing the work. A model receives the interface, so a model cannot change the parameters it was given. Thus, the mutability is confined to the one place a reader utilizes it, and nothing below the input layer has to be written defensively.

The snapshot makes the arrangement safe.

```
val analysis = EconomicOrderQuantity.optimize(sheet) //  
  ↪ sheet.orderCost is 20.0  
sheet.orderCost = 40.0  
analysis.parameters.orderCost //  
  ↪ still 20.0
```

Every result carries the values that produced it. Notice that the sweep in Section 3.12 assigns `sheet.leadTime = lead` inside a loop and keeps every analysis. That sweep is sound because each analysis froze its own inputs before the next assignment. Notice also that `ParameterSnapshot` is itself a `CostParametersIfc`, so a result can be re-analyzed from its own record.

You should take the general rule from this. When something must be editable and something must be stable, do not compromise between them on one type. Separate what can change from what has been recorded, and make the dependency run from the second to the first.

3.11.2. One Pipeline, Six Models

Section 3.4 derives one model and Section 3.5 obtains five more by imposing assumptions on it. The six therefore agree about almost everything. They agree about the geometry of Section 3.1, the cost assembly of Section 3.3,

3. Deterministic Lot Sizing

the reorder point of Section 3.6, and the service measures of Section 3.9. They disagree about one thing, the policy.

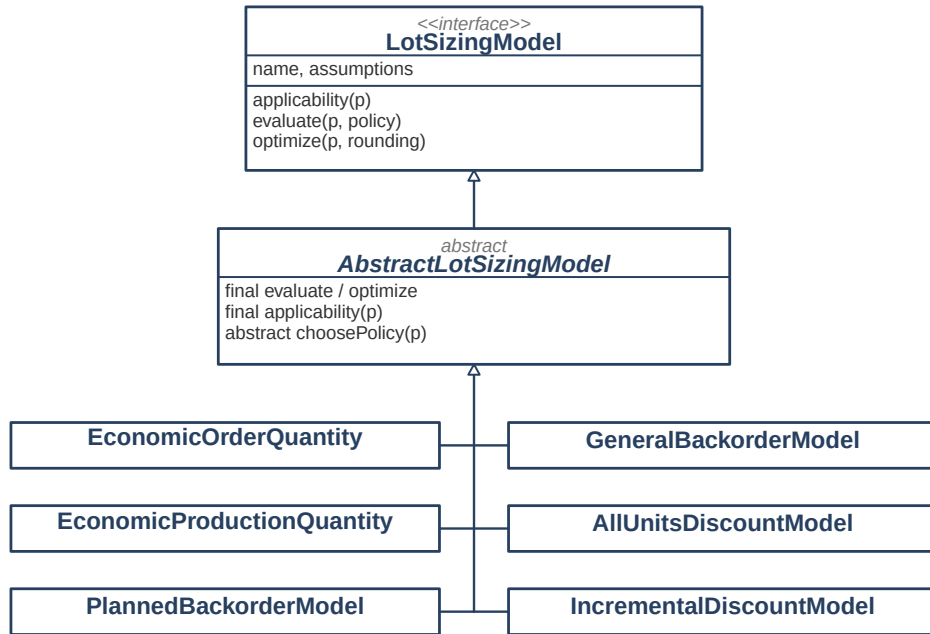


Figure 3.11.: One interface, one pipeline and six models. Everything the six share is written once in the abstract class, where **evaluate** and **optimize** are final, and each model supplies a single method that chooses a policy. Notice that no arrow joins the six to one another, which is the point of the next paragraph.

Figure 3.11 illustrates the arrangement. We write the shared part once and leave the varying part open.

```

abstract class AbstractLotSizingModel(override val name:
    ↪ String) : LotSizingModel {

    /** The policy this model chooses for these parameters,
    ↪ and how it chose it. */

```

3. Deterministic Lot Sizing

```
protected abstract fun choosePolicy(parameters:
    ↪ CostParametersIfc): ChosenPolicy

final override fun optimize(
    parameters: CostParametersIfc,
    rounding: QuantityRounding,
): InventoryPolicyAnalysisIfc {
```

`optimize` and `evaluate` are `final`. That is, a model may decide what to order, and it may not decide what a cycle is, what it costs, or when to reorder. Those belong to the chapter and not to any one model, so a model that overrode them would contradict Section 3.1 instead of specializing it.

Table 3.1 obtains the classical economic order quantity from the general model as the replenishment rate and the backorder cost both grow without bound. The obvious move is to make `EconomicOrderQuantity` a subclass of `GeneralBackorderModel`. The code does not do so.

A subtype must be usable wherever its parent is, and the classical model is not. It has no backorder cost to be given, and handing it one is the case Section 3.5.2 warns about. What is true is a statement about numbers, that one model's answer approaches another's in a limit, and a statement about numbers belongs where numbers can be checked. We record it instead in tests that take the limit and watch the quantities converge. Thus, the relation between the six models is real, is verified, and is not inheritance.

You should carry that distinction past this chapter. Before making B a subclass of A, ask whether every context that accepts an A would accept a B. In the mathematics, “B is a special case of A” is a different claim.

3.11.3. Sealed Types for Closed Sets

Four of the inputs are not numbers. Replenishment is instantaneous or at a rate. Shortages are forbidden or backordered. Holding cost is a charge on value or a rate. A schedule is flat, all-units or incremental. Each one is a small closed set of alternatives, and we write each as a sealed hierarchy.

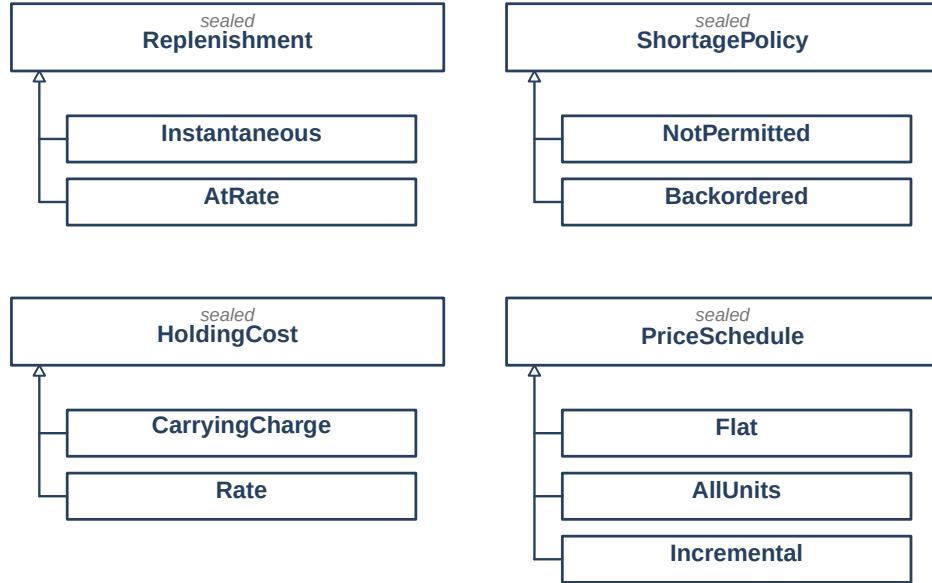


Figure 3.12.: The four inputs that are kinds rather than numbers. Each set is closed by the mathematics of this chapter, which is what makes sealing them honest: a seventh kind of price schedule would be a change to Section 3.8 before it was a change to any code.

Figure 3.12 illustrates the four. Sealing buys one thing. The compiler knows the whole list, so it can check a decision over the kinds for completeness. That pays when something dispatches on the kind, and here something does. Table 3.1 says which model matches which assumptions, and the

3. Deterministic Lot Sizing

code is that table.

```
fun forParameters(parameters: CostParametersIfc):
    ↪ LotSizingModel {
        val schedule = parameters.schedule
        if (schedule is AllUnits) return AllUnitsDiscountModel
        if (schedule is Incremental) return
            ↪ IncrementalDiscountModel
        return when (parameters.replenishment) {
            is Replenishment.Instantaneous -> when
                ↪ (parameters.shortages) {
                    is ShortagePolicy.NotPermitted ->
                        ↪ EconomicOrderQuantity
                    is ShortagePolicy.Backordered ->
                        ↪ PlannedBackorderModel
                }
            is Replenishment.AtRate -> when
                ↪ (parameters.shortages) {
                    is ShortagePolicy.NotPermitted ->
                        ↪ EconomicProductionQuantity
                    is ShortagePolicy.Backordered ->
                        ↪ GeneralBackorderModel
                }
        }
    }
}
```

There is no `else`, and the code will not compile if one is needed. For example, adding a third kind of replenishment stops the build at this function, the one place in the program that has to be revisited. The compiler enforces Table 3.1, and no comment asks anybody to remember it.

`ShortagePolicy` shows a second benefit. Recall from Section 3.5.2 that forbidding shortages is not the same as charging nothing for them, since a

3. Deterministic Lot Sizing

backorder cost of zero makes shortages free and sends the order quantity to infinity. A design with one nullable backorder cost would have to catch that at run time. Here `NotPermitted` and `Backordered(b)` are different things, and the constructor of the second requires a positive `b`. Thus, the mistake the callout warns about cannot be written down at all.

We apply one criterion, the closed set and not the tidiness. Section 4.8.5 faces the same choice for the aggregate measures of Chapter 4 and answers it the other way, because nothing there dispatches on which measure it is holding and an organization may always find a fifth thing to run short of. You should seal when the mathematics closes the list and something must visit every case, and leave the hierarchy open otherwise.

3.11.4. An Answer That Carries Its Reasoning

We come to the last decision. It concerns what comes back, and an order quantity is a number that cannot be questioned. Three types let the caller question it.

A model asked to work outside its assumptions has three replies and not two.

```
sealed interface Applicability {  
    data object Applicable : Applicability  
    data class NotApplicable(val reason: String) :  
        ↳ Applicability  
    data class Approximating(val ignored: List<String>) :  
        ↳ Applicability  
}
```

The third reply is the interesting one. Notice that applying the order quantity to an item with a finite production rate is a deliberate approximation, which Exercise 3.4 asks you to make, so the model allows it and the

3. Deterministic Lot Sizing

discarded assumption rides on the result. Asking the planned backorder model for a policy when shortages are forbidden is refused outright, since nothing is left for it to compute. A boolean would treat those two alike.

Each model declares its assumptions once, and the verdict is derived from that declaration in the one place all six share. Stating an assumption in a declaration and then testing for it again by hand would write the same rule twice, in two forms free to disagree.

QuantityChoice records how the quantity was reached, and the routes differ. A closed form produced an interior optimum; or the stationary point lay outside the feasible region, so the answer sits on a boundary and carries a reason; or a candidate list was ranked and retained; or an unrounded optimum was rounded and both neighbours evaluated. Section 3.8 is where the record pays, since a discount answer that does not show its candidates cannot be checked against the worksheet that produced the same number.

Section 3.1.2 says the four durations account for the whole cycle, and advises checking it because the check catches most algebra errors immediately. We write the advice into the constructor.

```
        checkClose(segments.total, length, "the segments sum
↪   to the cycle length")
        checkClose(maxOnHand + maxBackorder,
↪   policy.orderQuantity * survivingFraction,
        "the peaks account for the surviving part of the
↪   order")
```

Those checks run whenever a cycle is built, and not only in a test. The averages computed from the triangle areas are checked against the closed forms beside them. An identity that the chapter states and the program never verifies is an identity nobody holds to.

The same discipline applies to arguments. Every constructor validates, and so does every function that could divide by a quantity somebody passed

3. Deterministic Lot Sizing

as zero. The failure a guard prevents is usually not a crash. A rounding rule asked for an interval of two to the seventieth base periods computes two to the sixth, because the shift that raises two to a power keeps only the low six bits of the exponent. It returns a plausible small number. A guard at the point of construction refuses the exponent, and a test holds the guard in place.

We can now say what a seventh model would cost. It is one object with one `choosePolicy`, one `assumptions` declaration, and one branch that the compiler demands in `forParameters`. Nothing else in the package changes. Apply that test to any design and ask what the next requirement costs.

3.12. The Same Models in Code

The worksheets of Section 3.10 and the Kotlin code that accompanies this chapter compute the same quantities from the same inputs. Section 3.11 said why the code is shaped as it is; here we make the translation. Each part takes one piece of a worksheet, says what it is in code, and shows what to type in order to get the number the sheet shows. Appendix E covers obtaining and running the project.

Table 3.7 is the map, and you can find any equivalent in one lookup.

Table 3.7.: What each part of a worksheet is called in code.

In a worksheet	In code
The Inputs block	<code>CostParameters</code> , built once
The units column beside each input	<code>timeUnit</code> , required at construction
Intermediate calculations	Derived properties: <code>survivingFraction</code> , <code>holdingRateAt</code>
Outputs for any Q	<code>evaluate(parameters,</code> <code>orderQuantity)</code>

3. Deterministic Lot Sizing

In a worksheet	In code
Outputs for Q*	<code>optimize(parameters)</code>
Ratio, difference, relative error	<code>penaltyAgainst(reference)</code>
The ROUNDUP column	<code>roundedOrderQuantity,</code> <code>roundedReorderPoint</code>
<code>=MOD(, T) *</code>	<code>reorderPoint</code>
<code>=IF(>= P, "Infeasible",</code> <code>...)</code>	The constructor refuses the rate
VLOOKUP into the price schedule	<code>PriceSchedule.unitCostAt</code>
The Fixed Cost column	<code>Incremental.fixedChargeAt</code>
The candidate columns	<code>QuantityChoice.AmongCandidates</code>

3.12.1. The Four Blocks as Objects

The four-block layout of Section 3.10.1 survives the translation, with each block becoming something the language already has.

Inputs become an object. `CostParameters` holds what the Inputs block holds, and it takes the time unit as an argument instead of as a label in an adjacent cell. Notice the difference that makes. The worksheet's units column is a discipline the reader has to follow, while here it is a constructor argument. Thus, an annual carrying charge cannot be dropped silently beside a monthly demand rate.

Intermediate calculations become derived properties. They are computed on demand from the inputs and cannot go stale, the behavior a spreadsheet formula already has.

The two output blocks become two operations. `evaluate` prices a quantity that is handed to it, and `optimize` finds the best one. That is exactly the split Section 3.10.1 describes, and it is why the sensitivity rows have something to join.

3. Deterministic Lot Sizing

The joining rows become a method. `penaltyAgainst` compares two results and returns the ratio, the difference, and the relative error of Equation 3.22.

Example 3.12 (The classic EOQ sheet in code). Recall the item of Example 3.7, with $\lambda = 100$ units per month, $c = \$8$ per unit, an annual carrying charge of 25% entered as a monthly rate, $k = \$20$ per order, and $L = 0.25$ month. We build it once and ask it for the optimum.

```
val sheet = CostParameters(  
    demandRate = 100.0,  
    unitCost    = 8.0,  
    orderCost   = 20.0,  
    holding     = HoldingCost.carryingCharge(0.25 / 12.0),  
    leadTime    = 0.25,  
    timeUnit    = TimeUnit.MONTH,  
)  
  
val best = EconomicOrderQuantity.optimize(sheet)  
println(best.report())
```

Economic order quantity	
Order quantity	154.9193 units (155 rounded up)
Cycle length	1.5492 month
Order frequency	0.6455 orders per month
Average on hand	77.4597 units
Reorder point	25.0000 units (25 rounded up)
Ordering cost	12.91 per month
Holding cost	12.91 per month
Relevant cost	25.82 per month
Purchase cost	800.00 per month
Total cost	825.82 per month

3. Deterministic Lot Sizing

Every figure is the sheet's. Notice that the rounded companions in parentheses are the ROUNDUP column of Figure 3.4, and that the ordering and holding costs again agree at \$12.91, the balance check of Section 3.5.4.

The evaluator, and the three rows below it. Handing a quantity to `evaluate` and comparing it against the optimum reproduces the sheet's ratio, difference, and relative error:

```
val chosen = EconomicOrderQuantity.evaluate(sheet,  
    ↪ orderQuantity = 155.0)  
chosen.penaltyAgainst(best)
```

Table 3.8.: Typing a quantity into the evaluator, in code. The 150 and 160 rows carry the same penalty, Equation 3.22 depending only on the ratio.

Q	Ratio	Difference	Relative error
155	1.00000014	\$0.000003	0.000014%
150	1.00052070	\$0.013444	0.052%
160	1.00052070	\$0.013444	0.052%
200	1.03279556	\$0.846778	3.280%

3.12.2. The Reorder Point

Section 3.10.2 makes the case for `MOD( , T) *` , which handles both cases of Section 3.6 in one function with no branch. Kotlin has the same function on `Double`. Thus, the translation is direct and `reorderPoint` is one expression.

Example 3.13 (The reorder point against the lead time). We sweep the lead time on the item of Example 3.12, whose cycle is 1.5492 months.

3. Deterministic Lot Sizing

```
for (lead in listOf(0.25, 1.0, 1.6, 2.0, 3.0, 4.0)) {  
    sheet.leadTime = lead  
    println(EconomicOrderQuantity.optimize(sheet).  
↳ reorderPoint)  
}
```

Table 3.9.: The reorder point is not monotone in the lead time. It climbs toward the order quantity and drops back once per cycle length of lead time.

Lead time, months	0.25	1.00	1.60	2.00	3.00	4.00
Reorder point	25.00	100.00	5.08	45.08	145.08	90.16

Notice the drop between 1.00 and 1.60 months, the second order becoming outstanding. Nothing in the code tests for it, because `mod` has already done so.

Notice also what the loop does with the parameters. `sheet.leadTime = lead` edits the object in place, the way a worksheet is used. The analyses already produced are unaffected, because each one records the inputs that produced it.

3.12.3. The Same Substitution

Recall from Section 3.10.3 that the production worksheet does not implement Equation 3.14 at all. It computes $h' = h(1 - \lambda/p)$ and applies the ordinary formula to it. The code does the same thing, and it puts the surviving fraction on the replenishment type so that no formula anywhere has to ask which case applies.

The worksheet's feasibility guard has a counterpart in code, and it is stricter. `=IF( >= P, "Infeasible production rate", ...)` prints a

3. Deterministic Lot Sizing

message beside a sheet that has already computed a wrong answer. In code the rate is refused at construction. Thus, no parameter set with $p \leq \lambda$ ever exists.

Example 3.14 (The production sheet in code). Recall the item of Example 3.9, with $\lambda = 2500$ and $p = 10,000$ units per year, $c = \$2$ per unit, $i = 0.30$ per year, and $k = \$50$ per setup.

```
val production = CostParameters(  
  demandRate      = 2500.0,  
  unitCost         = 2.0,  
  orderCost        = 50.0,  
  holding          = HoldingCost.carryingCharge(0.30),  
  replenishment    = Replenishment.atRate(10_000.0, demandRate  
    ↪ = 2500.0),  
)  
  
production.baseHoldingRate           // 0.6000  
    ↪ h  
production.survivingFraction         // 0.7500  
    ↪ 1 - lambda/p  
production.baseHoldingRate * production.survivingFraction  
    ↪ // 0.4500    h'  
  
val made = EconomicProductionQuantity.optimize(production)  
made.orderQuantity                   // 745.3560  
made.cycle.maxOnHand                 //  
    ↪ 559.016994  
made.cycle.averageOnHand             // 279.5085  
made.cycle.replenishmentTime         // 0.0745356  
made.measures.ordering               // 167.71,  
    ↪ equal to holding
```

The machine runs for 0.0745 year of every 0.2981 year cycle, a quarter of

3. Deterministic Lot Sizing

the time, which is λ/p as it must be.

Passing an infeasible rate does not produce a message beside an answer:

```
The replenishment rate must exceed the demand rate,  
but 80.0 is not greater than 100.0
```

3.12.4. Discount Schedules as Candidates

Section 3.10.4 lays the discount procedure out as computed columns beside the price table. In code the price table is an object and the columns are a list of candidates. The result carries that list, so that the reasoning can be read and not only the answer.

Example 3.15 (The all-units sheet in code). Recall the schedule of Example 3.10, with $\lambda = 600$ units per time, $k = \$8$ per order, $i = 0.20$, and three price levels. The first break point is 0 here and not the 1 the worksheet uses, for the reason Section 3.10.4 gives.

```
val allUnits = CostParameters(  
    demandRate = 600.0,  
    orderCost   = 8.0,  
    holding     = HoldingCost.carryingCharge(0.20),  
    schedule    = AllUnits(listOf(  
        PriceLevel(breakPoint = 0.0, unitCost = 0.30),  
        PriceLevel(breakPoint = 500.0, unitCost = 0.29),  
        PriceLevel(breakPoint = 1000.0, unitCost = 0.28))),  
)  
val best = AllUnitsDiscountModel.optimize(allUnits)
```

3. Deterministic Lot Sizing

Table 3.10.: The candidate list the result carries, the worksheet’s computed columns.

Level	Break point	Price	Candidate	Total cost	
1	0	\$0.30	400.0000	204.0000	
2	500	\$0.29	500.0000	198.1000	chosen
3	1000	\$0.28	1000.0000	200.8000	

Order 500 at \$198.10. Notice that the candidates are ranked on the *total* cost and not on the relevant cost of Equation 1.16. Recall that Section 1.4.1 sets the purchase term aside because it is identical under every policy. Section 3.8 is where that premise stops holding, because once the price depends on the quantity the purchase term is exactly what the decision moves. For example, ranking these three candidates on ordering and holding alone chooses 400, wrong by \$5.90 a year.

Example 3.16 (The incremental sheet in code). We keep the same inputs under an incremental schedule. The worksheet’s recursive **Fixed Cost** column is the schedule’s own calculation:

```
val schedule = Incremental(listOf(
    PriceLevel(0.0, 0.30), PriceLevel(500.0, 0.29),
    ↪ PriceLevel(1000.0, 0.28)))

schedule.fixedChargeAt(0) // 0.00
schedule.fixedChargeAt(1) // 5.00
schedule.fixedChargeAt(2) // 15.00
```

With $k = \$8$ this gives the effective ordering costs 8, 13, and 23 of Equation 3.27. Optimizing then gives

3. Deterministic Lot Sizing

Table 3.11.: Two candidates fall inside their own intervals, and the smaller wins.

Level	Price	Candidate	Total cost	
1	\$0.30	400.0000	204.0000	chosen
2	\$0.29	518.6189	204.5799	

Order 400 at \$204.00. Recall from Section 3.10.4 that the worksheet has 518.62 entered in its evaluator, and that the sheet has no way to rank the two. Here the ranking is the operation itself. Thus, the 58 cents is not left to the reader to notice.

3.12.5. Doing the Three Things a Worksheet Does Not

Section 3.10.5 ends with three things the worksheets do not do. They are the three things the code is for, and we take them in the same order.

It chooses. A worksheet evaluates the quantity it is given. `optimize` instead returns the best one and carries the candidates it rejected, so the answer arrives with its reasoning attached. Example 3.16 is the case where that matters.

It refuses what it cannot model. A worksheet computes whatever is typed into it. Here the assumptions of Section 3.1.1 are checked where the value is made, and the message says what to do instead:

A backorder cost of zero makes shortages free rather than forbidden. Use `ShortagePolicy.NotPermitted`, which is the limit as the cost grows without bound. See Section 3.5.2.

That is the callout of Section 3.5.2 enforced, not printed. A model asked to work outside its assumptions says so too, and it distinguishes two cases.

3. Deterministic Lot Sizing

Applying the order quantity to an item with a finite production rate is a deliberate approximation and is allowed, with the discarded assumption recorded on the result. However, asking the planned backorder model for a policy when shortages are forbidden is refused outright.

It rounds as part of the decision. Section 3.7 observes that no warehouse orders 40.37 of anything, and Exercise 3.5 asks what a case quantity costs. Here rounding is an argument to `optimize`, which evaluates both admissible neighbours instead of rounding the answer afterwards:

```
EconomicOrderQuantity.optimize(item,  
    ↪ QuantityRounding.multipleOf(48.0))
```

Table 3.12.: For the item of Exercise 3.2, whose optimum is 202.26. Rounding down is the cheaper of the two, which rounding the answer up would have missed.

Case quantity	Relevant cost	Penalty
192	\$1,782.30	1.001355
240	\$1,806.00	1.014671

Thus, the reported cost is the cost of the quantity that will actually be ordered, the gap Section 3.10.5 identifies in the worksheets.

3.13. Summary

In this chapter we answered the two questions of Section 1.1, how much to order and when, under the strongest assumption available. That is, everything is known and nothing varies.

One model, specialized four ways. Section 3.1 derives the geometry of a single cycle once, and Table 3.1 obtains every classical result from it

3. Deterministic Lot Sizing

by switching an assumption off. The economic production quantity is the general model with shortages forbidden, the classical order quantity is that model with replenishment made instantaneous, and the planned backorder model is the general one with no charge per unit short.

Deriving the cycle once instead of four times is the reason the chapter is arranged this way. It is also why Section 3.9 gets the service measures for nothing, since they are ratios of segment lengths that had been computed already.

Forbidding a shortage is a limit, not a price of zero. A zero backorder cost makes shortages free, not forbidden, and it sends the order quantity to infinity. The no-shortage models are instead the limit as b grows without bound. That is the distinction Section 3.5.2 is careful about, and it is the one that most often goes wrong.

Cost balances at the optimum, and the total is flat around it. Ordering and holding cost are equal at Q^* , the fastest arithmetic check available. The penalty for departing from the optimum depends only on the ratio Q/Q^* and on nothing else about the item. Thus, being wrong by a fifth costs almost nothing and being wrong by half costs 25%. That tolerance is the reason it is reasonable to order in case quantities and to fit a delivery schedule, and it is what Chapter 4 exploits when it forces items to share a cycle.

Timing is a separate decision from quantity, and it reads the position. The reorder point covers only the demand not already covered by orders outstanding. It is therefore not monotone in the lead time, and Equation 3.21 is written in terms of inventory position and not stock on hand.

A price schedule changes the problem in kind. While the unit cost is constant the purchase term is inert and can be set aside. Once the supplier prices in tiers it is exactly what the decision moves. Thus, the cost rate is no longer smooth, and the answer is selected from a candidate list instead of obtained by differentiating. Section 3.8.1 and Section 3.8.2 differ

3. Deterministic Lot Sizing

in whether the break points are themselves candidates, and that difference follows from whether the cost function jumps.

Everything in this chapter rests on demand being known and constant. In the next chapter we keep that assumption and remove a different one, that items can be decided independently, the subject of Chapter 4. Chapter 5 then keeps independence and lets demand vary over time. From Chapter 7 onward demand stops being known at all, and the quantities this chapter computed exactly become quantities that must be estimated.

3.14. Exercises

Unless an exercise says otherwise, use these conventions so that your answer and the instructor's agree. Take the time unit to be one year, and convert any rate quoted on another basis before using it. Report order quantities and reorder points to two decimal places, cycle times to four, and costs to the nearest cent. Round an order quantity to an integer or to a case multiple only when the exercise asks for the quantity that would actually be ordered, and price the rounded quantity and not the unrounded one when you do.

Exercise 3.1. An item is demanded at $\lambda = 600$ units per year and replenished at $p = 1500$ units per year. The order quantity is $Q = 300$ and the policy permits a maximum backorder level of $\hat{B} = 40$ units.

Following Example 3.1, determine the peak on-hand level, the four segment lengths, and verify that they sum to Q/λ . Compute $\bar{I}$ and $\bar{B}$ both from the triangle areas and from the closed forms, and confirm they agree.

Then, following Example 3.6, report the order frequency, the fraction of time out of stock, the ready rate, and the average customer wait in days.

3. Deterministic Lot Sizing

Exercise 3.2. An item is demanded at $\lambda = 1500$ units per year and costs \$40 per unit. The ordering cost is \$120 per order, the carrying charge is 22% per year, no shortages are permitted, and replenishment is instantaneous.

Following Example 3.2, determine the economic order quantity, the optimal cycle length in days, the optimal total cost, and the reorder point for a 10 day lead time.

Then determine what the reorder point would be if the lead time were 70 days instead, using Equation 3.21, and explain in one sentence why the answer is not $\lambda(70/365)$.

Exercise 3.3. Show that at $Q = Q^*$ the ordering cost rate and the holding cost rate are equal, and that each equals $\frac{1}{2}\sqrt{2k\lambda h}$.

Then use this to construct a check that requires no square roots: given a candidate Q , state a one-line test that determines whether Q is too large or too small.

Exercise 3.4. A component is used at $\lambda = 4000$ units per year and produced in house at $p = 10,000$ units per year. The setup cost is \$250 and the holding cost is \$3 per unit per year. Shortages are not permitted.

Determine the economic production quantity from Equation 3.14 and the economic order quantity from Equation 3.17, and report the ratio of the two.

Show algebraically that the ratio depends only on λ/p , and determine what happens to the production quantity as the production rate approaches the demand rate. Explain what that limit means physically.

Exercise 3.5. The item of Exercise 3.2 is supplied in cases of 48 units, so the order quantity must be a multiple of 48.

Determine the two case quantities nearest the economic order quantity, use Equation 3.22 to compute the cost penalty of each, and recommend

3. Deterministic Lot Sizing

one. State the annual dollar cost of your recommendation relative to the unrounded optimum.

Exercise 3.6. An item has $\lambda = 5000$ units per year, an ordering cost of \$40, and a carrying charge of 25% per year. The supplier offers an all-units discount schedule:

Order quantity	Unit price
$0 \leq Q < 300$	\$20.00
$300 \leq Q < 700$	\$19.00
$Q \geq 700$	\$18.00

Following Example 3.4, determine the optimal order quantity. Report which candidate quantities you evaluated and why each was or was not feasible, and give the total cost at each.

Then state which of the three cases in Table 3.2 applies, and explain why evaluating only the first feasible candidate would have given the wrong answer.

Exercise 3.7. Repeat Exercise 3.6 with the same break points and prices, but under an **incremental** discount schedule rather than an all-units one.

Following Example 3.5, compute R_j , the effective ordering cost k_j , and the candidate quantity for each level. Determine the optimal order quantity and total cost.

Compare your answer with Exercise 3.6 and explain, in terms of what each scheme charges, why the incremental schedule produces the smaller order.

Exercise 3.8. An item has $\lambda = 1200$ units per year, an ordering cost of \$100, a holding cost of \$6 per unit per year, and a backorder cost of \$15 per unit per year. Replenishment is instantaneous and there is no per-unit stockout charge.

3. Deterministic Lot Sizing

Determine Q^* and $\hat{B}^*$ from Equation 3.15, and compute $\bar{I}$, $\bar{B}$, the ready rate, and the average customer wait.

Compare Q^* with the order quantity that would be used if backorders were forbidden, and explain why permitting backorders increases the order quantity. Then determine what backorder cost b would be needed for the two order quantities to differ by less than 1%.

Exercise 3.9. A planner argues that because the total cost curve is flat near its minimum, as Figure 3.2 shows, the economic order quantity is not worth computing and any reasonable quantity will do.

Using Equation 3.22, determine the range of order quantities, expressed as a multiple of Q^* , over which the relevant cost stays within 2% of its minimum. Then state the two circumstances from Section 1.4.8 under which the planner's argument fails, and explain which of them applies to the quantity discount problems of Section 3.8.

Exercise 3.10. The Classic EOQ worksheet of Example 3.7 is set up in months, with $\lambda = 100$ units per month, $c = \$8$, a carrying charge of 25% per year, $k = \$20$ per order, and $L = 0.25$ month.

Reproduce the optimizer block by hand: h , Q^* , T^* , the two cost components, the total cost rate, and the reorder point.

Then change the carrying charge to 30% per year and recompute. Report the percentage change in Q^* and show algebraically that it depends only on the ratio of the two carrying charges, not on λ or k .

Exercise 3.11. For the item of Exercise 3.10, whose optimal cycle is $T^* = 1.5492$ months, compute the reorder point from Equation 3.21 for lead times of $L = 0.25, 1.0, 1.6, 2.0, 3.0$, and 4.0 months.

Plot r against L and describe the shape. Explain why r can be smaller at $L = 1.6$ than at $L = 1.0$, and state the largest value r can take for this item.

3. Deterministic Lot Sizing

Then explain in one sentence why a worksheet that computed $r = \lambda L$ would be wrong for every lead time in the table beyond the first.

Exercise 3.12. The incremental discount worksheet of Example 3.11 evaluates a quantity that is entered instead of selecting among the candidates, so an analyst who computes 518.62 for level 2, finds it feasible, and enters it gets a plausible answer that is not the optimum.

Redesign it. Using Table 3.6 as the model, specify the computed columns that would turn the incremental schedule table into a self-selecting one: give the formula for each column, including the fixed cost recursion of Equation 3.31, and state the rule that picks the answer from the completed table.

Explain why your design needs no column corresponding to $TC(q_j)$, the break-point candidate that the all-units sheet requires.

Exercise 3.13. Take the item of Example 3.12 and, in code, reproduce every figure of Example 3.7: the holding cost rate, the order quantity, the cycle length, the two cost components, the total cost rate, and the reorder point.

Then set the order quantity to each of 150, 155, and 160 and report the ratio, difference, and relative error against the optimum. Explain in one sentence why two of the three carry the same penalty.

Exercise 3.14. The item of Example 3.14 is produced at $p = 10,000$ units per year against demand of $\lambda = 2500$.

In code, reduce the production rate toward the demand rate in steps and record the order quantity at each. Report what happens to it, and explain the limit in terms of Equation 3.14.

Then attempt to set the production rate to 2000 and report what happens. Contrast that with what the production worksheet of Example 3.9 does

3. *Deterministic Lot Sizing*

with the same value, and say which behavior you would rather have in a program that runs unattended.

Exercise 3.15. Example 3.15 ranks its three candidates on total cost. Rank them instead on the relevant cost of Equation 1.16, report the quantity that ranking selects, and compute what choosing it would cost per year.

Then state the condition under which the two rankings agree, and explain why the worked example of Example 3.4 does not reveal the difference while the schedule of Example 3.15 does.

4. Multi-Item and Constrained Systems

i Learning objectives

After reading this chapter you should be able to:

- explain how a shared resource couples items whose models are otherwise independent
- formulate a lot sizing problem with an aggregate constraint and solve it with a Lagrange multiplier
- interpret the multiplier as a shadow price, and recognize when it is instead an implied carrying charge
- show that a budget constraint scales every order quantity by the same factor
- construct an exchange curve and read from current practice the cost ratio it implies
- use the variety index to say how much of aggregate performance is driven by the spread across items
- evaluate a power-of-two policy and state what it costs against the best coordinated policy
- define echelon inventory and echelon holding cost, and explain why installation accounting double counts
- find the optimal reorder intervals for a serial system and for a distribution system

4. Multi-Item and Constrained Systems

Recall that Chapter 3 took one item at a time. Every model in it produced an order quantity from that item's own demand, prices, and replenishment rate, and nothing about any other item entered the calculation. That independence is what made the chapter tractable, and it is what this chapter removes.

In this chapter we develop the models that apply when items must be decided together. We begin with items that compete for something scarce, then turn to items that share the cost of an order, and end with stock that exists in order to replenish other stock. Each of the three couples the items differently, and each therefore needs its own machinery.

4.1. When Items Stop Being Independent

An item's lot sizing decision can be made on its own only if nothing it consumes is also consumed by something else. Two different things break that condition, and they lead to different mathematics.

A shared resource couples items through a constraint. Warehouse space, working capital tied up in stock, and the receiving department's capacity to process orders are all finite and shared. A larger order for one item leaves less for the others. The items' cost functions stay separate; what ties them together is a single inequality they must jointly satisfy. Four such constraints are common:

1. average total inventory investment, at most a budget
2. average number of replenishments per unit time, at most a fixed limit
3. maximum space used, at most what is available
4. maximum allowed backorder delay, at most a fixed limit

A shared setup couples items through a cost. For example, when several items come from one supplier on one truck, or run on one machine that is set up once for a family, part of the cost of ordering is incurred once regardless

4. Multi-Item and Constrained Systems

of how many items are on the order. The cost functions themselves are no longer separate.

Section 4.2 through Section 4.4 treat the first case, where the coupling is a constraint. Section 4.5 treats the second, where the coupling is a cost. The techniques have almost nothing in common, so they occupy separate halves of the chapter. Section 4.6 then takes up a third kind of coupling, in which one stock exists in order to replenish another, and shows that the power-of-two machinery carries over once the holding cost is accounted for correctly.

Throughout this chapter, demand is still known and constant, and every item is still governed by the classical model of Section 3.5.4. What changes is only that the items are decided together. To begin, we take up the case where the items compete for a single scarce resource.

4.2. A Constrained Lot Sizing Problem

Consider a group of items that draw on one scarce resource. We need notation for the items before we can write the problem down.

- Let n represent the number of items sharing the resource, indexed by j .
- Let λ_j represent the demand rate for item j , which is known and constant.
- Let c_j represent the unit cost of item j , and k_j its ordering cost.
- Let h_j represent the holding cost rate for item j , which is ic_j when holding cost is charged as a carrying charge on value.
- Let Q_j represent the order quantity for item j , the quantity we are choosing.

The decision is the vector $Q_1, \dots, Q_n$, and the objective is the sum of the relevant costs of Equation 1.16:

4. Multi-Item and Constrained Systems

$$\min C = \sum_{j=1}^n \left[\frac{k_j \lambda_j}{Q_j} + \frac{h_j Q_j}{2} \right] \quad (4.1)$$

Now consider the four constraints listed in Section 4.1. The first and third are **linear** in the order quantities.

- Let a_j represent what one unit of item j consumes of the shared resource.
- Let A represent how much of the resource there is.

Both take the form

$$\sum_{j=1}^n a_j Q_j \leq A \quad (4.2)$$

For example, under a **space** constraint a_j is the floor space one unit occupies and A is the space available, since immediately after a replenishment there are Q_j units on the floor. Under a **budget** constraint on average investment the average on-hand level is $Q_j/2$ and its value is $c_j Q_j/2$, so that $a_j = c_j/2$ and A is the budget. Both are Equation 4.2 with different coefficients, so one derivation covers them together.

The second constraint is not linear. An item ordered in quantity Q_j is replenished λ_j/Q_j times per unit time, so a limit on replenishments reads

$$\sum_{j=1}^n \frac{\lambda_j}{Q_j} \leq A \quad (4.3)$$

Consumption *falls* here as the quantities rise, instead of growing with them. Thus, this is a different problem, and Section 4.2.4 takes it up once the linear case is settled. The fourth constraint, on backorder delay, belongs with the models that permit shortages and is not treated in this chapter.

4. Multi-Item and Constrained Systems

4.2.1. First Check Whether the Constraint Binds

Equation 4.1 is **separable**: it is a sum of terms, each involving only one item's quantity. Recall from Section 3.5.4 that each such term is convex, and a sum of convex functions is convex. Thus, in the absence of the constraint each item can be optimized on its own, and the answer is the ordinary order quantity of Equation 3.17:

$$Q_j^* = \sqrt{\frac{2k_j\lambda_j}{h_j}}$$

That gives a three-step procedure whose first step is free:

1. Solve the unconstrained problem, item by item.
2. If $\sum_j a_j Q_j^* \leq A$, the constraint is **inactive** and the unconstrained answer is the answer. Otherwise it is **active** and step 3 applies.
3. Solve the problem with the constraint imposed as an equality.

Step 2 is easy to skip and expensive to skip. A constraint that does not bind changes nothing, and the machinery of the next two sections is needed only when the resource runs out. You should perform this check first every time.

Step 3 may treat the inequality as an equality because the objective is convex. That is, if the unconstrained optimum violates the limit, then the constrained optimum sits on the boundary of the feasible region, and on that boundary the inequality holds with equality. The problem we actually solve is an equality-constrained one.

4.2.2. Moving the Constraint into the Objective

The method of **Lagrange multipliers** moves a constraint into the objective in order that the unconstrained techniques of Chapter 3 apply to what is left.

4. Multi-Item and Constrained Systems

- Let θ represent the **multiplier**, a single scalar attached to the constraint, whatever the number of items.

With this notation we form the Lagrange function

$$L(\vec{Q}, \theta) = \sum_{j=1}^n \left[\frac{k_j \lambda_j}{Q_j} + \frac{h_j Q_j}{2} \right] + \theta \left[\sum_{j=1}^n a_j Q_j - A \right] \quad (4.4)$$

Notice that the bracketed term is zero whenever the constraint holds as an equality. At any feasible point on the boundary, L and C agree, so minimizing L is a legitimate substitute for minimizing C . Setting the partial derivatives to zero gives $n + 1$ equations in the n quantities and θ :

$$\frac{\partial L}{\partial Q_j} = -\frac{k_j \lambda_j}{Q_j^2} + \frac{h_j}{2} + \theta a_j = 0, \quad j = 1, 2, \dots, n$$

$$\frac{\partial L}{\partial \theta} = \sum_{j=1}^n a_j Q_j - A = 0$$

Solving the first set for each quantity in terms of θ , we obtain

$$Q_j(\theta) = \sqrt{\frac{2k_j \lambda_j}{h_j + 2\theta a_j}} \quad (4.5)$$

Notice that Equation 4.5 is Equation 3.17 with the holding cost rate h_j replaced by $h_j + 2\theta a_j$. **The multiplier acts as a surcharge on holding**, charged in proportion to how much of the scarce resource a unit consumes. An item that occupies a lot of space, or ties up a lot of capital, is penalized more and its quantity falls further. Doubling a_j for one item while leaving everything else alone doubles the surcharge that item pays.

4. Multi-Item and Constrained Systems

4.2.3. Finding the Multiplier

Substituting Equation 4.5 into the remaining equation, we are left with one equation in one unknown:

$$R(\theta) = \sum_{j=1}^n a_j \sqrt{\frac{2k_j \lambda_j}{h_j + 2\theta a_j}} - A \quad (4.6)$$

Let $R(\theta)$ represent the **resource function**, the amount of the shared resource the quantities consume at multiplier θ , less the amount available. Every term of the sum decreases as θ increases. R is strictly decreasing. At $\theta = 0$ it is positive, because that is precisely the case in which the unconstrained quantities violate the limit, and it tends to $-A$ as θ grows without bound. Thus, there is exactly one root $\theta^* > 0$, and any root-finding method will locate it. Bisection is enough: bracket the root, halve the interval, repeat.

The whole procedure is a one-dimensional search wrapped around a formula, so these problems are routinely solved in a spreadsheet with a nonlinear solver. The single root is also what makes the search dependable, a point we return to in Section 4.7.1. We now work a complete instance.

Example 4.1 (An investment limit on three items). A distributor stocks three items that are funded out of one working capital account. Each costs \$50 to order and the carrying charge is 30% per year. Let $i = 0.30$ represent that carrying charge, so that $h_j = ic_j$ for every item.

Table 4.1.: Three items and the quantities they would order if nothing were scarce.

Item	λ_j	c_j	$h_j = ic_j$	Q_j^* unconstrained
1	12,000	\$20	\$6.00	447.21
2	25,000	\$10	\$3.00	912.87

4. Multi-Item and Constrained Systems

Item	λ_j	c_j	$h_j = ic_j$	Q_j^* unconstrained
3	8,000	\$15	\$4.50	421.64

Step 1, the unconstrained solution. Applying Equation 3.17 to each item gives the quantities in Table 4.1, a relevant cost of \$7,319.26 a year, and an average investment of

$$\sum_j c_j \frac{Q_j^*}{2} = 20 \frac{447.21}{2} + 10 \frac{912.87}{2} + 15 \frac{421.64}{2} = \$12,198.77$$

Step 2, does the constraint bind? Management will fund \$10,000. Since \$12,198.77 exceeds \$10,000, the constraint is active. The unconstrained answer is not available, and we continue to step 3.

Step 3, solve for the multiplier. Recall that under a budget constraint $a_j = c_j/2$. Substituting into Equation 4.5, the surcharge $2\theta a_j$ becomes θc_j , and we have

$$Q_j(\theta) = \sqrt{\frac{2k_j\lambda_j}{h_j + \theta c_j}}$$

Bisection on Equation 4.6 gives $\theta^* = 0.14643$. Evaluating Equation 4.5 at that multiplier gives the quantities in Table 4.2, together with the cost each item then incurs.

Table 4.2.: The solution under a \$10,000 limit on average investment.

Item	Q_j constrained	Ordering cost	Holding cost
1	366.61	\$1,636.64	\$1,099.82
2	748.33	\$1,670.39	\$1,122.50
3	345.64	\$1,157.28	\$777.69

4. Multi-Item and Constrained Systems

Item	Q_j constrained	Ordering cost	Holding cost
Total		\$4,464.30	\$3,000.00

The arithmetic check. Notice that the holding cost total is exactly \$3,000, or $0.30 \times \$10,000$. The constraint fixed the investment, so it fixed the holding cost with it, and the two numbers must agree. You should perform that check every time a budget constraint is solved, because it catches an error in either the multiplier or the quantities immediately. The investment itself comes to exactly \$10,000, as an active constraint requires.

What the limit means for the distributor. The relevant cost is \$7,464.30, so the constraint raised the annual cost by \$145.04, or 1.98%. That is the price of holding \$2,199 less in stock. Thus, when someone proposes tightening or relaxing the working capital account, \$145.04 is the figure that belongs in the conversation, and you should be prepared to supply it instead of merely complying with the limit.

Notice also that the ordering and holding costs are no longer equal, at \$4,464.30 against \$3,000.00. Recall from Section 3.5.4 that at an unconstrained optimum the two are equal. That property fails here, as it must, because these quantities do not minimize the unconstrained cost. Thus, if you check a constrained solution by looking for balanced costs, you will conclude wrongly that something is broken.

4.2.4. When the Limit Is on Ordering, Not on Stock

Equation 4.3 limits how often the operation can act, not how much it can hold. For example, receiving docks, purchasing staff, and inspection capacity all impose limits of this kind. The mathematics turns out to run in the opposite direction from the linear case.

We form the Lagrange function as before, with Equation 4.3 in place of Equation 4.2:

4. Multi-Item and Constrained Systems

$$L(\vec{Q}, \theta) = \sum_{j=1}^n \left[\frac{k_j \lambda_j}{Q_j} + \frac{h_j Q_j}{2} \right] + \theta \left[\sum_{j=1}^n \frac{\lambda_j}{Q_j} - A \right]$$

The ordering term and the constraint both carry λ_j/Q_j . They combine when we differentiate:

$$\frac{\partial L}{\partial Q_j} = -\frac{(k_j + \theta)\lambda_j}{Q_j^2} + \frac{h_j}{2} = 0 \quad \Rightarrow \quad Q_j(\theta) = \sqrt{\frac{2(k_j + \theta)\lambda_j}{h_j}} \quad (4.7)$$

Compare Equation 4.7 with Equation 4.5. There the multiplier was added to the holding rate; here it is added to the **ordering cost**. That is the whole of the difference, and the three consequences below all follow from it.

The multiplier is a price per order. Its units are dollars per order, i.e. the units of k_j , because that is the quantity it is added to. Thus, a limit on ordering capacity and a supplier who charges more per order are the same thing seen from two sides, exactly as a budget limit and a higher cost of capital were in Section 4.3.

Quantities rise, they do not fall. Ordering less often means ordering more each time. Thus, every Q_j increases with θ and the replenishment count falls. The search still works for the reason given in Section 4.2.3, since $\sum_j \lambda_j/Q_j(\theta)$ is again strictly decreasing in θ with a single root. Bisection applies unchanged.

The proportions do not survive. Recall that Equation 4.9 held because the budget multiplier entered as a factor $(i + \theta)$ common to every item. Here it enters as a *sum*, $k_j + \theta$, and adding a constant to unequal ordering costs changes them by unequal proportions. Thus, a frequency limit reallocates between items where a budget does not. The one exception is the case in which every item already has the same ordering cost, when $k + \theta$ is again common to all items and the portfolio rescales uniformly.

4. Multi-Item and Constrained Systems

Example 4.2 (A limit on replenishments). Return to the distributor of Example 4.1, with the same demands and unit costs and the same carrying charge of 30% per year. This time the items differ in what they cost to order, and the limit falls on the receiving dock, not on the capital account.

Table 4.3.: Three items whose unconstrained policies ask for 85 replenishments a year.

Item	λ_j	c_j	k_j	h_j	Q_j^*	Orders per year
1	12,000	\$20	\$50	\$6.00	447.21	26.83
2	25,000	\$10	\$20	\$3.00	577.35	43.30
3	8,000	\$15	\$80	\$4.50	533.33	15.00
Total						85.13

Unconstrained, the three items cost \$6,815.33 a year and require 85.13 replenishments. The receiving dock can process 50. Since 85.13 exceeds 50, the constraint is active.

Solving. Bisection on $\sum_j \lambda_j / Q_j(\theta) - 50$, with $Q_j(\theta)$ taken from Equation 4.7, gives $\theta^* = \$64.33$ per order.

Table 4.4.: The solution under a limit of 50 replenishments a year.

Item	Effective $k_j + \theta^*$	Q_j	Orders per year	Q_j / Q_j^*
1	\$114.33	676.25	17.74	1.5121
2	\$84.33	1,185.53	21.09	2.0534
3	\$144.33	716.36	11.17	1.3432
Total			50.00	

The arithmetic check. The replenishments total exactly 50.00, as an active constraint requires. Notice also that each quantity in Table 4.4 is

4. Multi-Item and Constrained Systems

the ordinary economic order quantity of Equation 3.17 computed with the effective ordering cost in the second column instead of with k_j . Item 1 at $k_1 + \theta^* = \$114.33$ gives $\sqrt{2(114.33)(12,000)/6} = 676.25$. You should perform that check every time, because it confirms the multiplier and the quantities were computed from the same value.

What the limit means. The annual cost rises to \$7,621.27, so the limit costs \$805.93, or 11.83%. Every quantity has grown, as ordering less often requires. Notice in the last column that they have not grown together. Item 2, whose \$20 ordering cost is most changed by an addition of \$64.33, more than doubles, while item 3 grows by only a third. Thus, the dock's capacity has redistributed the quantities as well as lengthened them.

The shadow price reading is unchanged in form. One more replenishment per year would save about \$64. That is the figure you should carry to whoever is deciding whether to add capacity at the dock.

4.3. What the Multiplier Means

The multiplier θ^* is not merely a device for getting an answer. It carries the economics of the constraint, and it has three readings you should keep apart.

It is a shadow price. That is, θ^* is the rate at which the minimum cost falls as the limit is relaxed. For example, in Example 4.1 it is 0.14643, so one more dollar of working capital would save about 15 cents a year and a hundred more about \$14. Thus, it is the number you take to whoever controls the budget.

The rate is a local one. As the limit is relaxed the multiplier itself falls, reaching zero at \$12,198.77, where the constraint stops binding and the last of the \$145.04 has been recovered. Thus, you should not extrapolate the rate far from the current limit, and asking for more than \$12,198.77 buys nothing at all.

4. Multi-Item and Constrained Systems

Under a budget constraint it is an increment to the carrying charge. Substituting $a_j = c_j/2$ and $h_j = ic_j$ into Equation 4.5, we obtain

$$Q_j = \sqrt{\frac{2k_j\lambda_j}{ic_j + \theta c_j}} = \sqrt{\frac{2k_j\lambda_j}{(i + \theta)c_j}} \quad (4.8)$$

That is, Equation 4.8 is the ordinary order quantity computed with a carrying charge of $i + \theta$ in place of i . For example, in Example 4.1 the effective charge is $0.30 + 0.14643 = 0.44643$ per year. A budget constraint and a higher cost of capital are the same thing seen from two sides. That is no coincidence, because both say that money tied up in stock is scarcer than the accounting rate admits.

Every quantity scales by the same factor. Equation 4.8 depends on θ only through the factor $(i + \theta)$, which is common to all items. Taking the ratio to the unconstrained quantity,

$$\frac{Q_j(\theta)}{Q_j^*} = \sqrt{\frac{i}{i + \theta}} \quad \text{for every } j \quad (4.9)$$

For example, in Example 4.1 the factor is $\sqrt{0.30/0.44643} = 0.81975$. Notice that $366.61/447.21$, $748.33/912.87$, and $345.64/421.64$ are all equal to 0.81975 . Thus, **a budget constraint changes the scale of the whole portfolio and leaves the proportions between items untouched.** Nothing is reallocated; everything shrinks.

That is not what intuition suggests. Faced with a cut in working capital, a planner might expect to cut deepest on the items that consume the most of it. Equation 4.9 says otherwise. The unconstrained solution has already put each item in the right proportion, and the constraint asks only that everything be smaller.

4. Multi-Item and Constrained Systems

You should note what the result depends on. It requires $h_j = ic_j$, so it holds when holding cost is charged as a carrying charge on value and not when holding cost is quoted directly. It also fails under a space constraint, where $a_j = v_j$ is unrelated to h_j , and under the frequency constraint of Section 4.2.4. Thus, uniform rescaling is a property of the budget case, not of constrained lot sizing in general.

Having established what one constraint does to a group of items, we now step back and ask what the group is doing in aggregate whether or not any constraint binds.

4.4. The Exchange Curve

A constraint is one way to make items interact. Another is to stop looking at items at all and ask what the whole portfolio is doing. Aggregate performance is what management reports on, and the relationship between its components turns out to be sharper than one would expect.

4.4.1. One Item First

Take a single item and let Q vary. Its average investment and its order frequency are

$$\bar{I} = \frac{cQ}{2}, \quad \overline{OF} = \frac{\lambda}{Q}$$

Their product is

$$\bar{I} \cdot \overline{OF} = \frac{cQ}{2} \cdot \frac{\lambda}{Q} = \frac{c\lambda}{2} \quad (4.10)$$

4. Multi-Item and Constrained Systems

and the order quantity has cancelled. Whatever quantity is used, the product of investment and order frequency is the same. The achievable combinations therefore lie on a hyperbola fixed by the item's own data, and choosing Q chooses a point on it and nothing more. Ordering more often is not a policy that can be adopted on its own: it is the same policy as holding less stock.

Which point does the order quantity of Equation 3.17 pick? Through this section and the next the carrying charge is taken to be one value shared by every item, and it is written γ and not i to keep that assumption visible in every formula it enters. Substituting $Q^* = \sqrt{2k\lambda/(\gamma c)}$,

$$\frac{\bar{I}}{OF} = \frac{cQ^*}{2} \cdot \frac{Q^*}{\lambda} = \frac{c(Q^*)^2}{2\lambda} = \frac{c}{2\lambda} \cdot \frac{2k\lambda}{\gamma c} = \frac{k}{\gamma} \quad (4.11)$$

So the **product** of the two aggregates fixes the curve, and their **ratio** fixes the point on it. The ratio is k/γ , and nothing else about the item enters.

Example 4.3 (What a chosen operating point implies). A planner decides to order an item twice a year and, doing so, carries an average of \$4 of it in stock. From Equation 4.11,

$$\frac{\bar{I}}{OF} = \frac{4}{2} = 2 = \frac{k}{\gamma}$$

so the planner's choice implies $k = 2\gamma$, whether or not the planner knows it. If the firm's carrying charge is $\gamma = 0.20$ per year, the implied ordering cost is $k = \$0.40$.

That number can be checked. If ordering genuinely costs \$40, then the planner is ordering far too often for the stated carrying charge, and either the ordering practice or the belief about γ is wrong. Section 1.5.4 gives methods for estimating k directly. This gives a second reading of it, obtained from behavior and not from accounting.

4. Multi-Item and Constrained Systems

4.4.2. The Aggregate Curve

Now take n items. Define the **aggregate investment in cycle stock** and the **aggregate number of replenishments** per unit time:

$$\bar{I}^a = \sum_{j=1}^n c_j \frac{Q_j}{2}, \quad N^a = \sum_{j=1}^n \frac{\lambda_j}{Q_j} \quad (4.12)$$

The first is in dollars and appears on a balance sheet. The second is in orders per year and is what the receiving department feels. They trade against one another, and the question is exactly how.

Suppose that every item is ordered in its economic quantity, that the ordering cost is a common k for all items, and that the carrying charge is the common γ . Then $Q_j = \sqrt{2k\lambda_j/(\gamma c_j)}$, and substituting into Equation 4.12 we obtain

$$\bar{I}^a = \sqrt{\frac{k}{\gamma}} \cdot \frac{1}{\sqrt{2}} \sum_{j=1}^n \sqrt{\lambda_j c_j}, \quad N^a = \sqrt{\frac{\gamma}{k}} \cdot \frac{1}{\sqrt{2}} \sum_{j=1}^n \sqrt{\lambda_j c_j} \quad (4.13)$$

Two things follow at once. First, the product is

$$\bar{I}^a \cdot N^a = \frac{1}{2} \left(\sum_{j=1}^n \sqrt{\lambda_j c_j} \right)^2 \quad (4.14)$$

in which k and γ have both cancelled. Second, the ratio is

$$\frac{\bar{I}^a}{N^a} = \frac{k}{\gamma} \quad (4.15)$$

Equation 4.14 and Equation 4.15 are Equation 4.10 and Equation 4.11 again, one level up. That is, the product is fixed by the item data alone

4. Multi-Item and Constrained Systems

and defines a single hyperbola, called the **exchange curve**. The ratio k/γ is a ray from the origin, and where the ray crosses the exchange curve is where the portfolio operates.

Figure 4.1 draws both. The solid curve is the exchange curve for a portfolio whose data give a product of 120,000, so that every point on it satisfies $\bar{I}^a N^a = 120,000$. Following the curve from left to right, aggregate investment falls as the aggregate number of replenishments rises, the trade-off Equation 4.14 forces. Notice that the curve flattens toward the right, so that the stock given up for each additional order gets smaller as the portfolio orders more often.

The two dashed lines through the origin are cost ratios. The steeper one is $k/\gamma = 12$, and it meets the curve at 100 replenishments a year and \$1,200 of cycle stock. The shallower one is $k/\gamma = 3$, and it meets the curve at 200 replenishments and \$600. Notice that a firm does not choose a point on the curve directly. It chooses a cost ratio, and the ratio chooses the point.

The triangle marks 200 replenishments against \$1,200 of stock, which lies above the curve. Its product is 240,000 and not 120,000. No set of economic order quantities produces it, and a portfolio found at that point is ordering in quantities that are not economic. We return to what can be done about such a point in Section 4.4.3.

4.4.3. Reading a Cost Ratio from Practice

Figure 4.1 is more useful read backwards than forwards. Instead of choosing a ratio and finding the point, we observe a point and ask what it implies. Two readings matter.

A point off the curve means the quantities are not economic. The curve is the best that can be done. For any given number of orders no policy holds less stock, and for any given stock none orders less often. Thus, a portfolio operating above the curve can move onto it and improve

4. Multi-Item and Constrained Systems

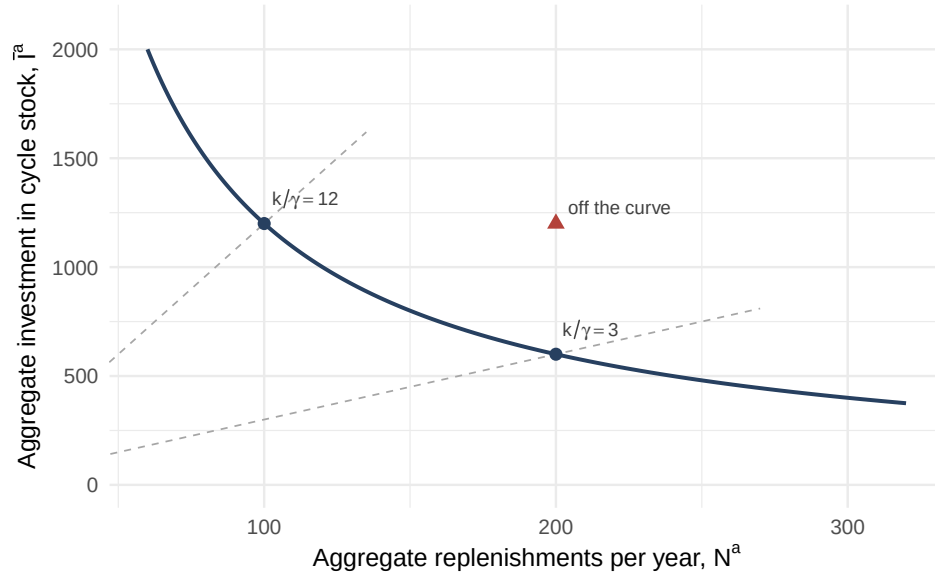


Figure 4.1.: The aggregate exchange curve, drawn for a portfolio whose data give a product of 120,000. Every combination of aggregate investment and aggregate order frequency reachable with economic order quantities lies on the curve. The two rays are cost ratios: $k/\gamma = 12$ meets the curve at 100 orders and \$1,200 of stock, and $k/\gamma = 3$ meets it at 200 orders and \$600. The marked point at 200 orders and \$1,200 of stock lies above the curve, which is only possible if the quantities in use are not economic ones.

4. Multi-Item and Constrained Systems

on both measures at once, with no trade-off to argue about and no new money. That is the rare case in which an inventory recommendation costs nothing.

A point on the curve reveals a cost ratio. Recall from Equation 4.15 that any operating point implies k/γ . If the firm's actual ordering cost and carrying charge are known, you can compare the implied ratio against them. A discrepancy says that current practice is optimal for costs the firm does not have.

Example 4.4 (What current practice implies). A distributor places 100 replenishment orders a year across a group of items and carries \$1,200 of cycle stock, a point that lies on the exchange curve for that group. From Equation 4.15,

$$\frac{\bar{I}^a}{N^a} = \frac{1200}{100} = 12 = \frac{k}{\gamma}$$

Case 1, the ordering cost is known. Cost estimation of the kind described in Section 1.5.4 puts $k = \$4$ per order. Then

$$\gamma = \frac{k}{12} = \frac{4}{12} = 0.333 \text{ per year}$$

A charge of 33% is what would make current practice optimal. If the firm's actual carrying charge is near 33%, then the practice is defensible. If the true figure is 20%, the ratio that should govern is $4/0.20 = 20$ rather than 12, so the firm is ordering too often for the stock it holds and should move along the curve toward fewer orders and more stock.

Case 2, neither is known. Recall from Section 1.5.5 that firms often cannot defend a number for either cost. The ratio is still informative, because it is what the firm's own behavior asserts. For example, a manager who cannot produce k or γ but who rejects $k/\gamma = 12$ as absurd has learned something about current practice.

4. Multi-Item and Constrained Systems

Case 3, the operating point is constrained. Suppose transportation arrangements fix the portfolio at 200 orders a year. The curve then gives $\bar{I}^a = \$600$ and a required ratio of $k/\gamma = 3$. Reaching that ratio cheaply means reducing k , since γ is a property of the firm's cost of capital and is not easily moved. Figure 4.1 gives guidance on changing the environment, not merely on operating within it. It says which cost to attack, and by how much.

4.4.4. When Ordering Costs Differ Across Items

Recall that Equation 4.14 assumed a common k and a common γ . That assumption is what let the curve be drawn without solving anything, and it is often too strong. For example, an item ordered from overseas and an item picked from a local supplier do not cost the same to order.

Dropping the assumption requires the machinery of Section 4.2.

- Let $\overline{OC}^a$ represent the aggregate ordering **cost** rate, in dollars per unit time, and not the count of orders.

That is,

$$\overline{OC}^a = \sum_{j=1}^n k_j \frac{\lambda_j}{Q_j}$$

We then pose the problem of minimizing it subject to a fixed total investment:

$$\min \sum_{j=1}^n \frac{k_j \lambda_j}{Q_j} \quad \text{s.t.} \quad \sum_{j=1}^n \frac{c_j Q_j}{2} = \bar{I}^a \quad (4.16)$$

Notice what is absent from Equation 4.16. There is no holding cost in the objective at all. The investment is a constraint and not a cost, the situation

4. Multi-Item and Constrained Systems

when management sets a stock target and asks for the cheapest way to live within it. Forming the Lagrange function and setting the derivatives to zero, we obtain

$$-\frac{k_j\lambda_j}{Q_j^2} + \frac{\theta c_j}{2} = 0 \quad \Rightarrow \quad Q_j = \sqrt{\frac{2k_j\lambda_j}{c_j\theta}} \quad (4.17)$$

Compare Equation 4.17 with the ordinary $Q^* = \sqrt{2k\lambda/(ic)}$. Notice that **the multiplier has taken the place of the carrying charge**. In Section 4.2 the multiplier was added to the holding rate because holding cost was in the objective. Here there is no holding cost to add to, and θ supplies it.

Thus, the same symbol has now carried three meanings, and the formulation decides which one applies: added to the holding rate under a limit on stock (Equation 4.5), added to the ordering cost under a limit on replenishments (Equation 4.7), and standing in for the carrying charge when there is no holding cost at all (Equation 4.17). In this last case it is precisely the implied carrying charge that Example 4.4 backed out of observed practice.

Substituting Equation 4.17 into the constraint and then into the objective, we obtain the general form of the curve:

$$\bar{I}^a \cdot \overline{OC}^a = \frac{1}{2} \left(\sum_{j=1}^n \sqrt{k_j c_j \lambda_j} \right)^2 \quad (4.18)$$

Equation 4.18 is Equation 4.14 with $\lambda_j c_j$ replaced by $k_j c_j \lambda_j$ and the order count replaced by the order cost. The same structure survives, and the curve is traced by varying θ .

The variety index. Zipkin (2000) relates the right-hand side of Equation 4.18 to a measure of how alike the items are.

4. Multi-Item and Constrained Systems

- Let J^* represent the **variety index**, defined as

$$J^* = \frac{\left(\sum_{j=1}^n \sqrt{k_j c_j \lambda_j}\right)^2}{\sum_{j=1}^n k_j c_j \lambda_j}, \quad 1 \leq J^* \leq n \quad (4.19)$$

The variety index measures dispersion among the products $k_j c_j \lambda_j$. If every item contributes equally, then $J^* = n$. If one item dominates, then J^* approaches 1. The variety index is the **effective** number of items, in the sense that a portfolio of n items with an index of J^* behaves in aggregate like a portfolio of J^* identical ones. With it, Equation 4.18 can be written

$$\bar{I}^a \cdot \overline{OC}^a = \frac{1}{2} J^* \sum_{j=1}^n k_j c_j \lambda_j \quad (4.20)$$

Reading the constant. The remaining sum decomposes into quantities a firm already reports.

- Let $C^a = \sum_j c_j \lambda_j$ represent the aggregate purchase cost rate.
- Let $\lambda^a = \sum_j \lambda_j$ represent the aggregate demand rate.

Then $c_j \lambda_j / C^a$ is item j 's share of purchasing, and λ_j / λ^a is its share of demand. Weighting by those shares, we obtain

$$K^w = \sum_{j=1}^n \left(\frac{c_j \lambda_j}{C^a} \right) k_j, \quad c^w = \frac{C^a}{\lambda^a} \quad (4.21)$$

where K^w is the purchase-weighted average ordering cost and c^w is the demand-weighted average unit cost. Since $\sum_j k_j c_j \lambda_j = K^w C^a = K^w \lambda^a c^w$, we have

$$\bar{I}^a \cdot \overline{OC}^a = \frac{1}{2} J^* K^w \lambda^a c^w \quad (4.22)$$

4. Multi-Item and Constrained Systems

Notice that four things move the whole curve, and that each is something a firm can act on: how varied the items are, what an order costs, how much is demanded, and what the goods are worth. Reducing the cost of placing an order shifts the entire frontier inward. Thus, Equation 4.22 is the quantitative version of the argument for simplifying purchasing. We now work a complete portfolio.

Example 4.5 (The exchange curve for a utility storeroom). A municipal electric utility runs a storeroom carrying nine items. Order handling costs $\kappa = \$55$ per hour and item j takes w_j hours per order regardless of the order size, so $k_j = \kappa w_j$ differs across items and the simple curve of Equation 4.14 does not apply. The carrying charge is $\gamma = 0.25$ per year.

Table 4.5.: Item data for the storeroom, with the ordering cost derived from the workload and the rate. The last column is the only aggregation the curve requires.

Item	λ_j	c_j	w_j	k_j	$\sqrt{k_j c_j \lambda_j}$
Pad-mount transformer	45	\$3,800.00	4	\$220.00	6,133.51
Smart meter	1,200	\$95.00	2	\$110.00	3,541.19
Fuse cutout	800	\$115.00	1.5	\$82.50	2,755.00
Meter socket	900	\$48.00	1.5	\$82.50	1,887.86
Riser conduit, 4 in	600	\$62.00	1.5	\$82.50	1,751.86
Splice kit	1,500	\$28.00	1	\$55.00	1,519.87

4. Multi-Item and Constrained Systems

Item	λ_j	c_j	w_j	k_j	$\sqrt{k_j c_j \lambda_j}$
Ground rod	2,400	\$14.00	0.75	\$41.25	1,177.29
Compression connector	6,000	\$3.40	0.5	\$27.50	749.00
Warning tape, roll	1,000	\$2.10	0.5	\$27.50	240.31
Total					19,755.87

The curve follows from the total of the last column:

$$\bar{I}^a \cdot \overline{OC}^a = \frac{1}{2}(19,755.87)^2 = 1.951 \times 10^8$$

Varying θ traces it. By Equation 4.17 and the constraint, $\bar{I}^a = \sum_j \sqrt{k_j c_j \lambda_j} / \sqrt{2\theta}$ and $\overline{OC}^a = \sqrt{\theta/2} \sum_j \sqrt{k_j c_j \lambda_j}$.

Table 4.6.: The curve traced by the multiplier. The last column is constant by Equation 4.18, and checking it is the arithmetic check for this section, in the way that the segment sum was for Section 3.1.

θ	$\bar{I}^a$	$\overline{OC}^a$	Order handling	Product
0.05	\$62,474	\$3,124	57 hours	1.951×10^8
0.10	\$44,175	\$4,418	80 hours	1.951×10^8
0.25	\$27,939	\$6,985	127 hours	1.951×10^8
0.50	\$19,756	\$9,878	180 hours	1.951×10^8
1.00	\$13,970	\$13,970	254 hours	1.951×10^8

4. Multi-Item and Constrained Systems

θ	$\bar{I}^a$	$\overline{OC}^a$	Order handling	Product
----------	-------------	-------------------	-------------------	---------

By Equation 4.17 the firm's own carrying charge is the multiplier, so $\theta = \gamma = 0.25$ puts the storeroom on the third row: \$27,939 of cycle stock against \$6,985 a year of order handling, or 127 hours of a storekeeper's time. Halving the carrying charge to 0.125 would move it between the second and third rows, raising the stock and cutting the workload, and Equation 4.18 fixes the exchange rate between the two exactly.

Look again at the ratio column. Since $\bar{I}^a / \overline{OC}^a = 1/\theta$ in this formulation, the storeroom at $\theta = 0.25$ holds exactly four dollars of stock for every dollar a year it spends on ordering, and that ratio is the reading Section 4.4.3 takes from observed practice.

The variety index. The denominator of Equation 4.19 is $\sum_j k_j c_j \lambda_j = 6.870 \times 10^7$, so

$$J^* = \frac{(19,755.87)^2}{6.870 \times 10^7} = 5.68$$

Nine items behave in aggregate like fewer than six identical ones. The transformer alone contributes 6,133 of the 19,756 in the last column of Table 4.5, and that concentration is what Section 2.2.2 measures a different way.

The decomposition. The storeroom buys $\lambda^a = 14,445$ units a year at an aggregate purchase cost rate of $C^a = \$555,500$, so the demand-weighted unit cost is $c^w = \$38.46$ and the purchase-weighted ordering cost is $K^w = \$123.67$. Then Equation 4.22 gives

$$\frac{1}{2}(5.6813)(123.67)(14,445)(38.46) = 1.951 \times 10^8$$

4. Multi-Item and Constrained Systems

the same constant reached from the item-by-item sum.

4.5. Joint Replenishment

The second kind of coupling is a shared cost, not a shared resource. Items that come from one supplier, travel on one truck, or run on one machine share part of the cost of replenishing, and that part is incurred once no matter how many items are on the order.

We split the ordering cost in two.

- Let K represent the **major** setup cost, incurred whenever an order is placed at all. For example, the truck, the paperwork, and the machine changeover to the family are all major setup costs.
- Let k_j represent the **minor** setup cost, incurred for each item included on that order. The line on the purchase order, the pick, and the receipt are all minor setup costs.

The minor cost keeps the symbol k_j that item j 's ordering cost has carried all along. What is new is K , sitting outside the sum.

Ordering each item independently pays K every time any item is replenished, which for n items means paying it n times over. Coordinating the orders so that items share them pays it once. The saving is the whole of what this section is about.

4.5.1. The Coordinated Problem

Coordination is expressed by a base period and a set of integer multipliers.

- Let T represent the **base period**, the time between successive order opportunities.

4. Multi-Item and Constrained Systems

- Let m_j represent the multiplier for item j , a positive integer, so that item j is replenished on every m_j -th order opportunity.

Item j 's own cycle is $m_j T$, and its order quantity is $\lambda_j m_j T$. The major cost is incurred once per base period, and item j 's minor cost once per m_j base periods. With average inventory $\lambda_j m_j T/2$, the relevant cost of Equation 1.16 is

$$C(T, \vec{m}) = \frac{K}{T} + \sum_{j=1}^n \left[\frac{k_j}{m_j T} + \frac{h_j \lambda_j m_j T}{2} \right] \quad (4.23)$$

The decisions are of two kinds. The base period is continuous and the multipliers are integers. The problem is harder than anything in Chapter 3, and the three policies below are ranked by how much structure they impose in order to make it tractable.

Independent ordering. Each item is ordered on its own schedule and pays the full setup. That is, item j faces an ordering cost of $K + k_j$, and its cost is $\sqrt{2(K + k_j)h_j \lambda_j}$, which is Equation 3.18 with the purchase term dropped. This is the baseline that coordination improves on.

A common cycle. Every item is replenished every base period, i.e. $m_j = 1$ for all j . Then Equation 4.23 reduces to a two-term function of T alone. By the argument of Section 3.5.4, its minimum is

$$T^* = \sqrt{\frac{2(K + \sum_j k_j)}{\sum_j h_j \lambda_j}}, \quad C^* = \sqrt{2 \left(K + \sum_j k_j \right) \sum_j h_j \lambda_j} \quad (4.24)$$

A common cycle is simple and it is often good. It is wasteful when the items differ widely in how often they want to be ordered, because a slow-moving expensive item is dragged onto the same frequency as a fast-moving cheap one.

4. Multi-Item and Constrained Systems

Integer ratios. Allowing each m_j to be any positive integer is the general problem. Notice in Equation 4.23 that once T is fixed the terms separate, so each multiplier can be chosen on its own. The best continuous value is

$$m_j = \frac{1}{T} \sqrt{\frac{2k_j}{h_j \lambda_j}} \quad (4.25)$$

Notice that the square root in Equation 4.25 is the cycle item j would choose if it paid only its minor cost. Rounding to an integer and searching over T solves the problem to any accuracy wanted. However, the answer is then a set of arbitrary integers, and that turns out to matter.

4.5.2. Power-of-Two Policies

A **power-of-two policy** restricts each multiplier to $m_j = 2^{\ell_j}$ for a non-negative integer ℓ_j . Items are ordered every base period, or every second, fourth, eighth, and so on.

The restriction is imposed for two reasons, and only one of them is mathematical.

Such schedules nest. For example, if one item is ordered every 2 base periods and another every 8, then every time the slow item is due the fast one is due as well, because 2 divides 8. With arbitrary integers this fails. Multipliers of 3 and 5 coincide only every 15 base periods, and in between there are orders that carry one item and not the other. Thus, a nested schedule is one a receiving dock can actually run, and nesting is the reason power-of-two policies are used in practice at all.

They cost almost nothing. That is the subject of Section 4.5.3, and it is the result that makes the restriction respectable instead of merely convenient.

4. Multi-Item and Constrained Systems

4.5.3. What a Power-of-Two Policy Costs

The bound is easiest to see one item at a time, so we fix the base period T and follow a single item, dropping its subscript on the item's own data. Its reorder interval is $T_j = m_j T$, which the power-of-two restriction writes as $2^\ell T$ for a non-negative integer ℓ .

We begin by putting the model of Section 3.5.4 in terms of the interval instead of the quantity.

- Let $C(T_j)$ represent the relevant cost rate written as a function of the reorder interval.
- Let $g = h\lambda/2$ represent the holding cost coefficient in that form.

Since $Q = \lambda T_j$, substituting into Equation 3.16 and dropping the purchase term gives

$$C(T_j) = \frac{k}{T_j} + \frac{h\lambda T_j}{2} = \frac{k}{T_j} + gT_j, \quad g \equiv \frac{h\lambda}{2} \quad (4.26)$$

Equation 4.26 is the same two-term function as before with the decision variable renamed. It therefore has the same solution in the new variable:

$$T_j^* = \sqrt{\frac{k}{g}} = \sqrt{\frac{2k}{h\lambda}}, \quad C(T_j^*) = 2\sqrt{kg} \quad (4.27)$$

Which power to use. Recall that C is convex. The best ℓ is the smallest one for which doubling again does not help. That is, we want the smallest ℓ satisfying $C(2^\ell T) \leq C(2^{\ell+1} T)$. Writing that out,

$$\frac{k}{2^\ell T} + g2^\ell T \leq \frac{k}{2^{\ell+1} T} + g2^{\ell+1} T$$

4. Multi-Item and Constrained Systems

Collecting the k terms on the left and the g terms on the right, we obtain $\frac{k}{2^\ell T} (1 - \frac{1}{2}) \leq g2^\ell T(2 - 1)$, hence $k/(2g) \leq (2^\ell T)^2$. Taking square roots,

$$\ell^* = \text{the smallest non-negative integer with } 2^\ell \geq \frac{T_j^*}{\sqrt{2}T} \quad (4.28)$$

Equation 4.28 is a formula, not a search. We take the base-2 logarithm of the right-hand side and round up.

Where the chosen interval sits. Notice that Equation 4.28 already says $T_j \geq T_j^*/\sqrt{2}$. The companion bound comes from running the same argument from the other side. Because ℓ^* is the *smallest* integer that works, the previous power was worse, i.e. $C(2^{\ell^*-1}T) > C(2^{\ell^*}T)$. Expanding that inequality exactly as above yields $2k/g > (2^{\ell^*}T)^2$, so $T_j < \sqrt{2}T_j^*$. Together,

$$\frac{T_j^*}{\sqrt{2}} \leq T_j < \sqrt{2}T_j^* \quad (4.29)$$

The worst case. The bracket in Equation 4.29 is the whole story, because C is convex and so takes its largest value over the bracket at one of the two ends. We evaluate it at the lower end, using $T_j^* = \sqrt{k/g}$:

$$C\left(\frac{T_j^*}{\sqrt{2}}\right) = \frac{k\sqrt{2}}{T_j^*} + \frac{gT_j^*}{\sqrt{2}} = \sqrt{2}\sqrt{kg} + \frac{1}{\sqrt{2}}\sqrt{kg} = \left(\sqrt{2} + \frac{1}{\sqrt{2}}\right) \sqrt{kg}$$

and at the upper end,

$$C(\sqrt{2}T_j^*) = \frac{k}{\sqrt{2}T_j^*} + g\sqrt{2}T_j^* = \frac{1}{\sqrt{2}}\sqrt{kg} + \sqrt{2}\sqrt{kg}$$

4. Multi-Item and Constrained Systems

Notice that the two are the same expression. Since $\sqrt{2} + 1/\sqrt{2} = 3/\sqrt{2}$ and $C(T_j^*) = 2\sqrt{kg}$, both equal

$$\frac{3}{2\sqrt{2}} C(T_j^*) = 1.0607 C(T_j^*) \quad (4.30)$$

Thus, **a power-of-two policy costs at most 6.07% more than the best interval, whatever the item's data and whatever the base period.** Rounding an interval up to the next power of two and rounding it down to the previous one cost exactly the same in the worst case, so the bracket in Equation 4.29 is symmetric in $\sqrt{2}$.

Two qualifications limit the statement. First, the derivation assumes ℓ^* is non-negative, which requires $T \leq \sqrt{2}T_j^*$. An item that wants to be ordered more often than the base period allows is stuck at $\ell = 0$, and for that item the base period, not the rounding, governs the cost.

Second, the base period may itself be a decision. Choosing it along with the multipliers improves the worst case from 6% to **2%** (Axsäter 2006). That sharper constant comes from a second route to the same bound. Rather than bracket one item's interval, it treats the rounding error as spread across the items and averages the resulting penalty, leaving the base period free to be optimized. It is also the constant behind Roundy's 98%-effective approximation for multi-stage systems (Roundy 1985). Muckstadt and Sapra (2010) develops power-of-two policies and their multi-stage extensions at length.

Recall from Section 3.7 that the total cost curve is flat near its minimum, and that this is why rounding to a case quantity is cheap. Equation 4.30 is the same fact applied to cycle lengths instead of order quantities. Thus, flatness is what licenses throwing away every multiplier that is not a power of two and keeping a schedule a warehouse can run.

4. Multi-Item and Constrained Systems

Example 4.6 (Rounding two items to a weekly base period). A plant can act on purchasing once a week, so the base period is $T = 1/52$ year. Two items are bought from the same supplier.

Table 4.7.: Two items facing a weekly base period.

Item	λ_j	h_j	k_j	$g_j = h_j \lambda_j / 2$
Drive unit	8,000	\$5.00	\$237	20,000
Control arm	6,000	\$4.00	\$300	12,000

The unconstrained intervals. From Equation 4.27,

$$T_{\text{drive}}^* = \sqrt{\frac{237}{20,000}} = 0.10886 \text{ yr} = 5.66 \text{ weeks}, \quad T_{\text{arm}}^* = \sqrt{\frac{300}{12,000}} = 0.15811 \text{ yr} = 8.22 \text{ weeks}$$

Neither is a whole number of weeks, let alone a power of two of them.

Rounding. Apply Equation 4.28 to the drive unit:

$$2^\ell \geq \frac{T_j^*}{\sqrt{2}T} = \frac{5.66}{\sqrt{2}} = 4.003 \quad \Rightarrow \quad \ell \geq \frac{\ln 4.003}{\ln 2} = 2.001 \quad \Rightarrow \quad \ell^* = 3$$

so the drive unit is ordered every $2^3 = 8$ weeks. For the control arm, $2^\ell \geq 8.22/\sqrt{2} = 5.81$, giving $\ell \geq 2.54$ and again $\ell^* = 3$, so it too is ordered every 8 weeks.

What the rounding cost. The drive unit was pushed from 5.66 weeks out to 8, a ratio of 1.4133, which is as far from 1 as Equation 4.29 permits. By Equation 3.22 its cost rises from \$4,354.31 to \$4,617.42, which is **6.04%**, essentially the worst case of Equation 4.30. The control arm was pulled from 8.22 weeks in to 8, a ratio of 0.9730, costing **0.04%**.

4. Multi-Item and Constrained Systems

Two readings. The penalty is bounded but not evenly spread: one item paid almost the entire worst case and the other paid nothing, and which one pays depends only on where its unconstrained interval happens to fall between two powers of two. And the two items ended up on the same eight-week cycle with no one coordinating them, which is the property Section 4.5.2 was after. Ordering them together is now free.

Example 4.7 (Coordinating four items from one supplier). Four items are bought from one supplier. Placing an order costs $K = \$400$ whatever is on it, and each item added to an order costs a further k_j . The carrying charge is 25% per year.

Table 4.8.: Four items sharing a supplier, with the cycle each would choose on its minor cost alone in the last column.

Item	λ_j	c_j	h_j	k_j	Annual value $c_j\lambda_j$	$\sqrt{2k_j/(h_j\lambda_j)}$
Drive unit	8,000	\$20	\$5.00	\$25	\$160,000	0.0354
Gear- box	800	\$20	\$5.00	\$20	\$16,000	0.1000
Seal kit	200	\$8	\$2.00	\$15	\$1,600	0.2739
Name plate	40	\$4	\$1.00	\$15	\$160	0.8660

The last column is the cycle each item would want if it paid only its minor cost, and it spans a factor of 24. The annual values span a factor of a thousand, the ABC spread of Section 2.2.2, and it is the reason the multipliers will not all be one.

Independent ordering. Each item paying $K + k_j$ on its own costs \$8,422.39 a year.

4. Multi-Item and Constrained Systems

A common cycle. From Equation 4.24, $T^* = 0.1462$ years, or 53 days, at a cost of \$6,497.54. Coordinating alone, with no differentiation between items, has already saved 23%.

The best integer-ratio policy. Searching over T with Equation 4.25 gives a base period of 0.14214 years, or 51.9 days, and multipliers (1, 1, 2, 6), costing \$6,402.06.

The power-of-two policy. Restricting to powers of two gives multipliers (1, 1, 2, 8) at a base period of 0.14192 years, or 51.8 days, costing \$6,403.34. Notice that the two policies do not share a base period. The search runs over T and the multipliers, so changing what multipliers are admissible moves the best T with them. That is **0.020%** above the best integer-ratio policy, against a worst case of 6.07%. The drive unit and the gearbox sit at $m_j = 1$ because the cycles in the last column of Table 4.8, 13 and 37 days, are shorter than the base period; nothing about them is rounded, and what sets their cost is the choice of base period.

The resulting schedule is

Table 4.9.: The power-of-two schedule on a base period of 51.8 days. Each interval is that base period times the multiplier, rounded to the nearest day, which is why the last row reads 414 and not eight times 52.

Item	Multiplier	Ordered every
Drive unit	1	52 days
Gearbox	1	52 days
Seal kit	2	104 days
Name plate	8	414 days

and it nests: every second order opportunity the seal kit joins an order the drive unit and gearbox were placing anyway, and every eighth the name plate joins one of those. No order is ever placed for one item alone.

4. Multi-Item and Constrained Systems

Two things can be read off. Coordination saved 24% against ordering separately, and almost all of that came from sharing orders and not from choosing clever multipliers: the common cycle captured 23 of the 24 points. And the power-of-two restriction, which makes the schedule runnable, cost two hundredths of one percent. The bound of Equation 4.30 is a worst case that a real instance rarely approaches.

4.6. Multi-Echelon Systems

Everything so far has coupled items that sit side by side. Items compete for a budget, or share a truck, but none of them feeds another. A **multi-echelon** system couples them end to end. That is, stock at one location exists in order to supply the next, and a downstream order is possible only because one was placed upstream.

The decision is still a set of reorder intervals, and the tool is still the power-of-two policy of Section 4.5.2. What changes is the accounting. Holding cost can no longer be charged location by location without double counting, because a unit sitting at a regional warehouse has already been paid for by the central warehouse that shipped it. Section 4.6.1 therefore fixes the accounting first, and the two sections after it apply it to the two structures that matter.

One point of notation before we start. The echelon sections have no major setup cost, because each location places its own orders and nothing is shared across a family. Thus, the K of Section 4.5 does not appear here, and the ordering cost at a location is written k_i , the same k the book has used for a fixed order cost since Section 1.4.5. The index changes from j to i as well, since what is counted is now a location, not an item.

Demand is still known and constant throughout. Chapter 10 returns to these same structures once demand is random.

4. Multi-Item and Constrained Systems

4.6.1. Echelon Inventory and Echelon Holding Cost

We number the stages so that stage i supplies stage $i - 1$, with stage 1 facing the customer:

$$n \longrightarrow n - 1 \longrightarrow \cdots \longrightarrow 2 \longrightarrow 1$$

Every stage sees the same demand rate λ , because a unit consumed at stage 1 must have passed through every stage above it.

The **echelon stock** of stage i is the total on-hand inventory at stages 1 through i . That is, it is everything at stage i together with everything that has already moved downstream of it. Table 4.10 gives an instance for a three stage chain.

Table 4.10.: On-hand and echelon stock for a three stage chain. Stage 3 owns six units in the sense that matters, even though only one is sitting at stage 3.

Stage	1	2	3
On hand	2	3	1
Echelon stock	2	5	6

Notice in Table 4.10 that stage 3 owns six units in the sense that matters, even though only one of them is sitting at stage 3. Echelon stock is the natural quantity because it is the one that behaves like the inventory in Chapter 3. Stage i 's echelon stock falls at the demand rate λ and jumps when stage i orders, which is the sawtooth the EOQ was built for. The on-hand stock at stage i does not behave that way at all, because it jumps down whenever the stage below it orders, in steps and not continuously.

Echelon holding cost. If echelon stock counts a unit at several stages at once, then charging each stage its full holding cost rate would count the same money several times.

4. Multi-Item and Constrained Systems

- Let h_i represent the conventional, or **installation**, holding rate at stage i , which measures the total value accumulated in a unit up to and including that stage.
- Let h'_i represent the **echelon holding cost** at stage i , which charges only the value that stage i itself adds.

$$h'_i = h_i - h_{i+1}, \quad h'_n = h_n \quad (4.31)$$

The definition telescopes. Nothing is lost and nothing is double counted:

$$\sum_{j=i}^n h'_j = (h_i - h_{i+1}) + (h_{i+1} - h_{i+2}) + \cdots + h_n = h_i \quad (4.32)$$

The total value-added rate carried up through stage i is recovered by adding the increments from stage i upward. The rate h_i decreases as i increases, because value accumulates on the way down the chain. h'_i is non-negative, and a stage that adds no value has $h'_i = 0$.

Example 4.8 (The two accountings agree). Take a two stage chain under a nested policy with $T_1 = \frac{1}{2}T_2$, so that stage 1 orders twice for every order stage 2 places. Lead times are zero. Installation holding rates are $h_1 = \$2.00$ and $h_2 = \$1.20$ per unit per year. By Equation 4.31, the echelon rates are $h'_1 = h_1 - h_2 = \$0.80$ and $h'_2 = h_2 = \$1.20$ per unit per year.

Figure 4.2 follows two full cycles, with $\lambda = 100$ units per year, $T_1 = 1/2$ year and $T_2 = 1$ year, so that $Q_1 = 50$ and $Q_2 = 100$ units. The three panels share one vertical scale, so you can add them by eye. The top panel is stage 1 on hand, an ordinary sawtooth falling from 50 to zero over each half year. The middle panel is stage 2 on hand, which is flat and not sloping, because stage 2 faces no external demand. Notice that it holds 50 units for the first half of the cycle and nothing for the second half. The bottom panel is stage 2 echelon stock, and it is the sum of the two panels

4. Multi-Item and Constrained Systems

above it at every instant. We now cost the same system twice, once from the top two panels and once from the bottom one.

Following the on-hand stock. At the start of a cycle stage 2 orders $Q_2 = \lambda T_2$ and, because the policy is nested, stage 1 simultaneously orders $Q_1 = \lambda T_1 = \frac{1}{2}Q_2$, which ships immediately. Stage 2 is left holding $Q_2 - Q_1 = \frac{1}{2}\lambda T_2$. It holds that amount for T_1 , until stage 1 reorders and takes the rest, after which stage 2 holds nothing for the remainder of the cycle. Its average on-hand cost rate is

$$h_2 \frac{(\frac{1}{2}\lambda T_2) T_1}{T_2} = \frac{1}{2}h_2\lambda T_1$$

It is a rectangle of height $\frac{1}{2}\lambda T_2$ and width T_1 , averaged over a cycle of length T_2 , the shaded block in the middle panel of Figure 4.2. Stage 1 is an ordinary sawtooth averaging $\bar{I}_1 = Q_1/2$.

- Let $\bar{I}_i$ represent the average on-hand stock at stage i .
- Let $\bar{I}_i^e$ represent the average echelon stock at stage i .

Writing the two on-hand averages the way Figure 4.2 labels them,

$$\bar{I}_1 = \frac{Q_1}{2} = \frac{\lambda T_1}{2}, \quad \bar{I}_2 = \frac{(Q_2 - Q_1)T_1}{T_2} = \frac{\lambda T_1}{2} \quad (4.33)$$

The system pays $h_1\bar{I}_1 + h_2\bar{I}_2$, which is

$$\frac{1}{2}h_1\lambda T_1 + \frac{1}{2}h_2\lambda T_1 = \frac{1}{2}(h_1 + h_2)\lambda T_1 \quad (4.34)$$

Following the echelon stock. Stage 2's echelon stock starts the cycle at $Q_2 = \lambda T_2$ and declines at rate λ to zero, i.e. a clean sawtooth over T_2 , because units shipped to stage 1 stay inside stage 2's echelon. Notice in Figure 4.2 that the red transfer at T_1 drops the middle panel to zero

4. Multi-Item and Constrained Systems

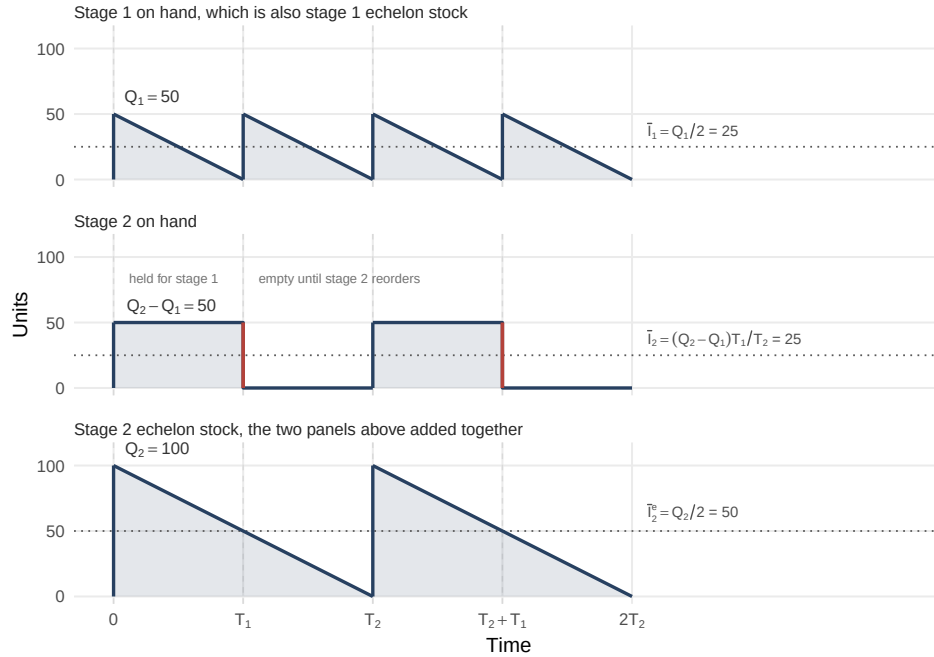


Figure 4.2.: On-hand and echelon stock for a two stage chain under a nested policy, drawn with $\lambda = 100$ units per year, $T_1 = 1/2$ year and $T_2 = 1$ year, so $Q_1 = 50$ and $Q_2 = 100$. Stage 1 is an ordinary sawtooth. Stage 2's on-hand stock is not: it receives Q_2 , ships Q_1 downstream at once, holds the remainder until stage 1 reorders at T_1 , and is then empty for the rest of the cycle. The red drops are transfers to stage 1, not demand. Adding the two upper panels gives the third, in which the transfers cancel, because a unit moved from stage 2 to stage 1 is still inside stage 2's echelon. What is left is a clean sawtooth of height Q_2 . The dotted lines mark the averages that enter the cost.

4. Multi-Item and Constrained Systems

and lifts the top panel by the same amount. Thus, the bottom panel passes through T_1 unbroken, and you should remember that mechanism. Its average is the third dotted line the figure marks:

$$\bar{I}_2^e = \frac{Q_2}{2} = \frac{\lambda T_2}{2} \quad (4.35)$$

Stage 1's echelon stock is its on-hand stock, so that $\bar{I}_1^e = \bar{I}_1$. The system pays $h'_1 \bar{I}_1^e + h'_2 \bar{I}_2^e$, and with $h'_2 = h_2$, $h'_1 = h_1 - h_2$, and $T_2 = 2T_1$, we obtain

$$\frac{1}{2}(h_1 - h_2)\lambda T_1 + \frac{1}{2}h_2\lambda T_2 = \frac{1}{2}(h_1 - h_2)\lambda T_1 + \frac{1}{2}h_2\lambda(2T_1) = \frac{1}{2}(h_1 + h_2)\lambda T_1 \quad (4.36)$$

the same total as Equation 4.34.

The arithmetic check. At the rates above, the three dotted lines in Figure 4.2 are 25, 25 and 50 units, and each cost term is a rate times one of them. Working both accountings side by side gives Table 4.11.

Table 4.11.: The same system costed both ways. Each average is a dotted line in Figure 4.2 and each rate comes from Equation 4.31.

Accounting	Term	Average	Rate	Cost rate
Installation	stage 1	$\bar{I}_1 = 25$	$h_1 = 2.00$	\$50.00
	stage 2	$\bar{I}_2 = 25$	$h_2 = 1.20$	\$30.00
	total			\$80.00
Echelon	stage 1	$\bar{I}_1^e = 25$	$h'_1 = 0.80$	\$20.00
	stage 2	$\bar{I}_2^e = 50$	$h'_2 = 1.20$	\$60.00
	total			\$80.00

Both totals are $\frac{1}{2}(h_1 + h_2)\lambda T_1 = \frac{1}{2}(3.20)(100)(0.5) = \80 per year, as Equation 4.34 and Equation 4.36 require. You should perform that check

4. Multi-Item and Constrained Systems

every time echelon rates are introduced, because it catches a sign error in Equation 4.31 immediately.

Notice what differs. Installation accounting charges the two stages \$50 and \$30, while echelon accounting charges them \$20 and \$60. Echelon accounting moves cost upstream, because stage 2's echelon stock includes what stage 1 is holding. Thus, **echelon accounting reassigns cost between stages and preserves the total**. That is the licence to use it. No money is invented or lost, and in exchange one awkward rectangle becomes a sawtooth, so that every stage looks like an EOQ.

4.6.2. Serial Systems

With the accounting settled, we can write the problem down.

- Let n represent the number of stages in the chain, indexed by i , so that stage 1 faces the customer and stage n orders from the outside supplier.
- Let k_i represent the fixed cost of placing an order at stage i .
- Let $g_i = \frac{1}{2}\lambda h'_i$ represent the holding cost coefficient at stage i , by analogy with Equation 4.26. Every stage sees the same λ , so no subscript is needed on it here; the distribution system of Section 4.6.3 is where that changes.
- Let T_i represent the reorder interval at stage i .

Stage i holding echelon stock on a cycle of T_i therefore costs $g_i T_i$ per unit time, and adding the ordering cost k_i/T_i at each stage gives

$$\min \sum_{i=1}^n \left[\frac{k_i}{T_i} + g_i T_i \right] \quad (4.37)$$

subject to two restrictions:

4. Multi-Item and Constrained Systems

$$T_i = 2^{\ell_i} T, \quad \ell_i = 0, 1, 2, \dots \quad (4.38)$$

$$T_i \geq T_{i-1} \geq 0 \quad (4.39)$$

Equation 4.38 is the power-of-two restriction of Section 4.5.2. Equation 4.39 is **nestedness**. That is, whenever a stage orders, every stage below it orders too, so no stage can replenish more often than the one it supplies. Nestedness keeps work in process from piling up between stages. Notice that with power-of-two intervals it comes for free, because any two powers of two divide one another.

The relaxation. We drop Equation 4.38 and keep Equation 4.39:

$$(RP) \quad \min \sum_{i=1}^n \left[\frac{k_i}{T_i} + g_i T_i \right] \quad \text{s.t.} \quad T_i \geq T_{i-1} \geq 0 \quad (4.40)$$

Without the nesting constraint each stage would separate and take $T_i = \sqrt{k_i/g_i}$, which is Equation 4.27 stage by stage. Thus, the constraint binds exactly when those intervals fail to increase up the chain. One feature of the answer makes the problem solvable: **stages that would violate the constraint are merged into a block and share one interval.**

Blocks. We partition the chain into consecutive blocks.

- Let G_r represent the r th block of consecutive stages, with G_1 holding the lowest-numbered stages, and let M represent the number of blocks.
- Let $T(r)$ represent the interval shared by every stage in G_r .
- Let $k(G_r) = \sum_{i \in G_r} k_i$ and $g(G_r) = \sum_{i \in G_r} g_i$ represent the block's totals.

4. Multi-Item and Constrained Systems

If every stage in block G_r shares the common interval $T(r)$, then the terms for that block collapse into a single two-term function of $T(r)$. Its solution is Equation 4.27 again with the block's totals in place of one stage's:

$$T(r) = \sqrt{\frac{k(G_r)}{g(G_r)}} \quad (4.41)$$

A set of blocks solves Equation 4.40 when three conditions hold.

1. Each block's interval is given by Equation 4.41.
2. The intervals increase, $T(1) \leq T(2) \leq \dots \leq T(M)$, so that the solution is feasible.
3. No block can be split into a lower part and an upper part whose ratio of block totals, $k(\cdot)/g(\cdot)$, is larger in the upper part.

We dwell on the third condition. If such a split existed, then the upper part could lengthen its interval on its own and save money. Thus, a partition admitting such a split cannot be optimal, and Algorithm 4.1 is built to prevent one.

The algorithm. We walk up the chain, adding stages to the current block while doing so is forced, and starting a new block when the next stage wants a longer interval than the current block has.

4. Multi-Item and Constrained Systems

Algorithm 4.1 Partitioning a serial chain into blocks that share a re-order interval. $\leftarrow$ is assignment and $+$ on blocks is set union. The ratio $k(G_r)/g(G_r)$ is the block total of Equation 4.41, so a block's own interval is always $\sqrt{k(G_r)/g(G_r)}$.

```
partition(stages 1 to n):
  r ← 1                                // index of the block being built
  G_1 ← {1}                            // stage 1 opens the first block

  for i = 2 to n:
    if k(G_r)/g(G_r) ≤ k_i/g_i:
      // stage i wants a longer interval: it stands alone
      r ← r + 1
      G_r ← {i}
    else:
      // stage i wants a shorter one, which is infeasible
      // upstream, so the current block absorbs it
      G_r ← G_r + {i}
      // absorbing i lowered this ratio, so it may now sit
      // below the block beneath and break condition 2
      r ← merge(r)

  return G_1 through G_r

merge(r):
  s ← r
  while s > 1 and k(G_{s-1})/g(G_{s-1}) > k(G_s)/g(G_s):
    // the lower block wants the longer interval, so the two
    // cannot stand apart: fold s into s-1 and retest
    G_{s-1} ← G_{s-1} + G_s
    s ← s - 1
  return s
```

4. Multi-Item and Constrained Systems

Now consider what **merge** is for. Absorbing a stage changes the block's totals, which can push its ratio below the block beneath it and break condition 2. **merge** walks back down the chain repairing that, folding adjacent blocks together until the ratios increase again. Notice that a single absorption can cascade, merging several blocks at once.

Example 4.9 (A five stage chain). A component passes through five stages, $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1$, at $\lambda = 500$ units per year. The plant can act on replenishment once a week, so $T = 1/52$ year.

Table 4.12.: A five stage chain. The echelon rates come from Equation 4.31, with $h'_5 = h_5$.

Stage i	1	2	3	4	5
k_i	\$10	\$12	\$25	\$7	\$2
h_i	\$0.60	\$0.50	\$0.35	\$0.10	\$0.05
$h'_i = h_i - h_{i+1}$	0.10	0.15	0.25	0.05	0.05
$g_i = \frac{1}{2}\lambda h'_i$	25	37.5	62.5	12.5	12.5
k_i/g_i	0.400	0.320	0.400	0.560	0.160

Why the constraint binds. Taking each stage on its own gives intervals $\sqrt{k_i/g_i}$ of 0.632, 0.566, 0.632, 0.748, and 0.400 years. These are not nondecreasing, so the schedule is infeasible: stage 2 would order less often than stage 1 supplies, and stage 5 less often than anything below it.

Running the algorithm.

4. Multi-Item and Constrained Systems

Table 4.13.: The forward pass.

i	Current block		k_i/g_i	Action
	ratio			
2	$\frac{10}{25} = .400$		$\frac{12}{37.5} = .320$	.400 > .320, absorb: $G_1 = \{1, 2\}$
3	$\frac{22}{62.5} = .352$		$\frac{25}{62.5} = .400$	.352 ≤ .400, start $G_2 = \{3\}$
4	$\frac{25}{62.5} = .400$		$\frac{7}{12.5} = .560$	.400 ≤ .560, start $G_3 = \{4\}$
5	$\frac{7}{12.5} = .560$		$\frac{2}{12.5} = .160$	.560 > .160, absorb: $G_3 = \{4, 5\}$, then merge

The merge: G_3 now has ratio $\frac{9}{25} = .360$, below G_2 's .400, so the two combine into $G_2 = \{3, 4, 5\}$ with ratio $\frac{34}{87.5} = .389$. That exceeds G_1 's .352, so the walk stops. The partition is

$$G_1 = \{1, 2\}, \quad G_2 = \{3, 4, 5\}$$

The relaxed intervals. From Equation 4.41,

$$T(1) = \sqrt{\frac{22}{62.5}} = 0.5933 \text{ yr} = 30.9 \text{ weeks}, \quad T(2) = \sqrt{\frac{34}{87.5}} = 0.6234 \text{ yr} = 32.4 \text{ weeks}$$

which are nondecreasing, so the solution is feasible, at a cost of \$183.25 a year. The infeasible stage-by-stage solution would have cost \$181.81, so nesting the chain costs 0.8%.

4. Multi-Item and Constrained Systems

Rounding to powers of two. Apply Equation 4.28 to each block. For G_1 , $2^\ell \geq 30.9/\sqrt{2} = 21.8$, so $\ell \geq 4.45$ and $\ell^* = 5$. For G_2 , $2^\ell \geq 32.4/\sqrt{2} = 22.9$, so $\ell \geq 4.52$ and again $\ell^* = 5$. Both blocks land on $2^5 = 32$ weeks, at a total of \$183.31, which is 0.03% above the relaxed solution.

The two blocks were distinguished by the relaxation and then rounded back together. That is not a failure of the method but a consequence of Equation 4.29: intervals within a factor of two of one another often round to the same power, and 30.9 and 32.4 weeks differ by 5%. When it happens the whole chain replenishes on one schedule, the strongest form of the coordination Section 4.5 was seeking.

4.6.3. Distribution Systems

Now we reverse the geometry. A central warehouse supplies several regional warehouses.

- Let index 0 denote the central warehouse, and let indices 1 through n denote the regional warehouses.
- Let λ_i represent the demand rate faced by regional warehouse i , so that $\lambda_0 = \sum_{i \geq 1} \lambda_i$ passes through the centre.

Recall from Section 4.6.1 that the echelon stock of a location is everything at that location together with everything downstream of it. The echelon stock of the central warehouse is therefore everything in the system, i.e. its own on-hand plus all of the regional on-hand. Its echelon rate is its installation rate, $h'_0 = h_0$, and each regional warehouse is charged only what it adds:

$$h'_i = h_i - h_0, \quad i = 1, 2, \dots, n \quad (4.42)$$

With $g_i = \frac{1}{2}\lambda_i h'_i$, which carries each location's own demand rate instead of the single λ a serial chain shares, the problem looks like Equation 4.37 with the index starting at 0. However, the nesting constraint runs the other way:

4. Multi-Item and Constrained Systems

$$T_0 \geq T_i \geq 0, \quad i = 1, 2, \dots, n \quad (4.43)$$

Recall that in a chain a stage cannot order more often than the stage it supplies. Here the central warehouse supplies everyone, so nothing downstream may wait longer than the centre. That is, a regional warehouse ordering less often than the centre would leave stock sitting at the centre with nowhere to go. Thus, Equation 4.43 caps each regional interval at T_0 instead of ordering the regions among themselves.

The shape of the answer. Relaxing the power-of-two restriction leaves a problem whose solution splits the regional warehouses into two groups. Those whose unconstrained interval $\sqrt{k_i/g_i}$ is short enough keep it and order on their own schedule. Those that would have wanted a longer interval than the centre can offer are pinned to the centre's interval and nested with it.

- Let $\mathcal{C}^0$ represent the **pinned set**, consisting of the central warehouse together with every region sharing its interval.

Building $\mathcal{C}^0$. We relabel the regional warehouses in ascending order of k_i/g_i , so that (1) wants the shortest interval and (n) the longest. We then work down from (n), adding a warehouse to $\mathcal{C}^0$ only when leaving it out would be infeasible. Adding warehouse i changes the centre's interval to the pooled value

$$T_0 = \sqrt{\frac{\sum_{j \in \mathcal{C}^0} k_j}{\sum_{j \in \mathcal{C}^0} g_j}} \quad (4.44)$$

The test at each step compares warehouse i 's own ratio against the pooled ratio of everything already in $\mathcal{C}^0$. If k_i/g_i is larger, then warehouse i standing alone would want a longer interval than the centre has, violating Equation 4.43, and it must join. We stop at the first warehouse for which k_i/g_i is smaller, since that warehouse and everything below it can be left

4. Multi-Item and Constrained Systems

alone. If even (n) fails the test at the start, then $\mathcal{C}^0$ is empty and every location takes $\sqrt{k_i/g_i}$.

Notice that this is a scan, not a search. One pass down the list suffices only because the regions were sorted first, so an implementation of this algorithm sorts once and then never looks backward.

Example 4.10 (A central warehouse and six regions). Six regional warehouses each face $\lambda_i = 200$ units per year, so the center sees $\lambda_0 = 1,200$. Replenishment decisions are made weekly, $T = 1/52$ year.

Table 4.14.: The distribution instance, with $h'_i = h_i - h_0$ and $g_i = \frac{1}{2}\lambda_i h'_i$.

i	0	1	2	3	4	5	6
k_i	\$500	\$100	\$125	\$110	\$150	\$75	\$90
h_i	\$10	\$15	\$12	\$20	\$18	\$20	\$17
h'_i	10	5	2	10	8	10	7
g_i	6,000	500	200	1,000	800	1,000	700
k_i/g_i	.0833	.200	.625	.110	.1875	.075	.1286

Relabel. Sorting the regions by k_i/g_i puts them in the order 5, 3, 6, 4, 1, 2, which become (1) through (6):

Table 4.15.: The regions relabeled in ascending order of k_i/g_i .

New label	(1)	(2)	(3)	(4)	(5)	(6)
Was	5	3	6	4	1	2
k_i	\$75	\$110	\$90	\$150	\$100	\$125
g	1,000	1,000	700	800	500	200
k_i/g_i	.075	.110	.1286	.1875	.200	.625

Build $\mathcal{C}^0$, starting from $\mathcal{C}^0 = \{0\}$ with pooled ratio $500/6,000 = .0833$:

4. Multi-Item and Constrained Systems

Table 4.16.: Constructing the pinned set.

Step	Candidate	Its k_i/g_i	Pooled $k(\mathcal{C}^0)/g(\mathcal{C}^0)$	Action
1	(6)	.625	$\frac{500}{6000} = .0833$	larger, add
2	(5)	.200	$\frac{625}{6200} = .1008$	larger, add
3	(4)	.1875	$\frac{725}{6700} = .1082$	larger, add
4	(3)	.1286	$\frac{875}{7500} = .1167$	larger, add
5	(2)	.110	$\frac{965}{8200} = .1177$	smaller, stop

So $\mathcal{C}^0 = \{0, (3), (4), (5), (6)\}$ and regions (1) and (2) are left alone. From Equation 4.44,

$$T_0 = \sqrt{\frac{965}{8,200}} = 0.3430 \text{ yr} = 17.8 \text{ weeks}$$

shared by the center and four regions, while (2) takes $\sqrt{.110} = 0.3317 \text{ yr} = 17.2 \text{ weeks}$ and (1) takes $\sqrt{.075} = 0.2739 \text{ yr} = 14.2 \text{ weeks}$.

Rounding. For the center, $2^\ell \geq 17.8/\sqrt{2} = 12.6$, so $\ell \geq 3.66$ and $\ell^* = 4$, giving $2^4 = 16$ weeks. Region (2) needs $2^\ell \geq 12.2$ and region (1) needs $2^\ell \geq 10.1$; both give $\ell^* = 4$ as well.

Every location ends up on a sixteen-week cycle. As in Example 4.9, the relaxation distinguished three different intervals and the power-of-two rounding put them back together, because all three fell in the same rounding band. By Equation 4.28 the choice $\ell^* = 4$ covers every interval from $8\sqrt{2} = 11.3$ weeks to $16\sqrt{2} = 22.6$ weeks, and 14.2, 17.2 and 17.8 all sit inside it. The system is fully synchronized, and the construction of $\mathcal{C}^0$ was still what proved the schedule optimal rather than merely convenient.

4.7. Building These Models in a Worksheet

Recall that Section 3.10 built a worksheet for a single item, and that its layout followed the shape of the mathematics: inputs in one block, quantities derived from them in a second, and two output blocks evaluating the same rows first at a quantity someone typed and then at Q^* . That layout worked because the decision was one number.

Every model in this chapter breaks that assumption. The decision is a vector, the items are tied together by something that belongs to none of them, and two of the models are algorithms, not formulas. Each of those changes forces the implementation into a different shape.

In the sections that follow we work through the models one at a time, in the order the chapter introduced them, showing how each piece of the mathematics becomes a piece of a worksheet. We build a worksheet first because everything in it is visible: a formula is a cell you can click on, and a layout is an argument about the model that you can see. Section 4.8 then asks the same question of a program, where nothing is visible and the structure has to be designed instead of looked at.

4.7.1. The Constrained Problem

Recall what we have to compute. Each item has an unconstrained quantity from Equation 3.17. The resource those quantities consume is compared against the limit. If the limit binds, a multiplier is found by bisection on Equation 4.6, and every quantity is recomputed from Equation 4.5 at that multiplier.

The blocks. We take them in the order the sheet reads, which Table 4.17 gives.

4. Multi-Item and Constrained Systems

Table 4.17.: The blocks of the constrained lot sizing worksheet, top to bottom. The first three are Section 4.2.1 and Section 4.2.3, and the item block is Equation 4.5 applied row by row.

Rows	Block	What it holds
5 to 7	Inputs	the carrying charge, the limit A , and a selector reading Budget or Space
10 to 11	Does the limit bind?	the resource the unconstrained quantities would consume, and the verdict
14 to 18	Solve	the multiplier, $R(\theta)$, and whether it has been driven to zero
22 to 27	Results	the same six quantities unconstrained and constrained
30 to 32	What the limit cost	a difference, a percentage, and the investment released
35 to 38	Check	what the shipped data should produce
43 onward	Item data	one row per item

Read Figure 4.3 from the top down and it follows the order of Section 4.2. The verdict sits above the multiplier and not below it, because Section 4.2.1 says an unconstrained solution already inside the limit is the answer and nothing further needs solving. Notice also that four columns hold the whole of it. Everything that is one number about the problem, and not one number about an item, lives here, and there are not many such numbers.

4. Multi-Item and Constrained Systems

	A	B	C	D	E	F	G	H
1	Constrained Lot Sizing							
2	Chapter 4, sections 4.2 and 4.3. One resource is shared by every item. Goal Seek prices it.							
3								
4	Inputs							
5	Carrying charge (i)	30.0%	per year					
6	Resource limit (A)	\$10,000.00	dollars					
7	Constraint type	Budget	Budget or Space					
8								
9	Does the limit bind?							
10	Resource the free qu	12,198.77						
11	Binds?	yes						
12								
13	Solve for the multiplier							
14	Multiplier (theta)	0.146429841	Goal Seek changes this cell					
15	R(theta), equation 4.	0.00000291						
16	Status	solved						
17	How	Data > What-If Analysis > Goal Seek. Set cell B15 to value 0 by changing cell B14.						
18	Why it is safe	Section 4.2.3 proves R decreases strictly and has one root, which is what a numerical search needs and cannot verify for itself.						
19								
20	Results							
21		Unconstrained	Constrained					
22	Ordering cost	\$3,659.63	\$4,464.30					
23	Holding cost	\$3,659.63	\$3,000.00					
24	Relevant cost	\$7,319.26	\$7,464.30					
25	Average investment	\$12,198.77	\$10,000.00					
26	Replenishments per	73.19	89.29					
27	Resource used	12,198.77	10,000.00					
28								
29	What the limit cost							
30	Extra cost per year	\$145.04						
31	As a fraction of the f	1.98%						
32	Investment released	\$2,198.77						
33								
34	Check							
35	Unconstrained releva	\$7,319.26, either constraint type						
36	Budget, limit 10,000	theta = 0.146429841, relevant cost \$7,464.30						
37	Budget arithmetic ch	holding cost = i times the limit = \$3,000.00						
38	Space, limit 800	theta = 2.953823172, relevant cost \$7,723.58						

Figure 4.3.: The constrained lot sizing worksheet, rows 1 to 38. Inputs come first, then the question of whether the limit binds, then the multiplier with a cell saying whether it has been solved, then the results as two columns, unconstrained beside constrained. The last two blocks price the limit and record what the shipped data should produce.

4. Multi-Item and Constrained Systems

The cells that carry the mathematics. We translate them cell by cell in Table 4.18. Read the last column first: every row of the table is one line of Section 4.2 rewritten in a language Excel evaluates.

Table 4.18.: The six cells of the constrained worksheet that implement a result from Section 4.2. Every one of them reaches the item block through a named range rather than a cell address.

Cell	Holds	Formula	Which result it is
B10	resource the free quantities consume	=SUMPRODUCT(Con_a, Con_Qheld)	the left side of Equation 4.2 at Q_j^*
B11	does the limit bind	=IF(B10>B6, "yes", "no")	the test of Section 4.2.1
B15	$R(\theta)$	=IF(B11="no", 0, SUMPRODUCT(Con_a, Con_Qheld) - B6)	driven to zero
C22	constrained ordering cost	=SUMPRODUCT(Con_a, Con_Qheld)	$\sum_j \text{Con}_j \lambda_j M_j(d)$
C23	constrained holding cost	=SUMPRODUCT(Con_a, Con_Qheld)/2	$\sum_j \text{Con}_j Q_j(\theta)/2$
C25	constrained investment	=SUMPRODUCT(Con_a, Con_Qheld)/2	the constraint budget limits

Notice what every formula in that table has in common. None of them names a cell in the item block. **Con_a**, **Con_Qheld** and the rest are named ranges spanning a hundred rows, so the summaries do not know, and do not need to know, how many items there are. Figure 4.4 shows the same block with its formulas displayed, the quickest way to see that the pattern holds everywhere and not only in the six cells quoted above.

One discipline that is not obvious. We make every summary either a SUM of one column or a SUMPRODUCT of several. It never divides one

4. Multi-Item and Constrained Systems

	A	B	C
1	Constrained Lot Sizing		
2	Chapter 4, sections 4.2 and 4.3. One resource is shown		
3			
4	Inputs		
5	Carrying charge (i)	0.3	per year
6	Resource limit (A)	10000	=IF(\$B5>"Budget","dollars","cubic feet")
7	Constraint type	Budget	Budget or Space
8			
9	Does the limit bind?		
10	Resource the free quantities would consume	=SUMPRODUCT(Con_a,Con_Qfree)	
11	Binds?	=IF(B10>B6,"yes","no")	
12			
13	Solve for the multiplier		
14	Multiplier (theta)	0.146429841	Goal Seek changes this cell
15	R(theta), equation 4.6, drive to zero	=IF(B11="no",0,SUMPRODUCT(Con_a,Con_Qheld)-B6)	
16	Status	=IF(B11="no","not needed, the limit does not bind",IF(ABS(B15)<0.000001*B6,"solved","STALE: re-run Goal Seek"))	
17	How	Data > What-If Analysis > Goal Seek. Set cell B15 to value 0 by changing cell B14.	
18	Why it is safe	Section 4.2.3 proves R decreases strictly and has one root, which is what a numerical search needs and cannot verify for itself.	
19			
20	Results		
21		Unconstrained	Constrained
22	Ordering cost	=SUMPRODUCT(Con_k,Con_NFree)	=SUMPRODUCT(Con_k,Con_NHeld)
23	Holding cost	=SUMPRODUCT(Con_h,Con_Qfree)/2	=SUMPRODUCT(Con_h,Con_Qheld)/2
24	Relevant cost	=B22+B23	=C22+C23
25	Average investment	=SUMPRODUCT(Con_c,Con_Qfree)/2	=SUMPRODUCT(Con_c,Con_Qheld)/2
26	Replenishments per year	=SUM(Con_NFree)	=SUM(Con_NHeld)
27	Resource used	=SUMPRODUCT(Con_a,Con_Qfree)	=SUMPRODUCT(Con_a,Con_Qheld)
28			
29	What the limit cost		
30	Extra cost per year	=C24-B24	
31	As a fraction of the free cost	=C24/B24-1	
32	Investment released	=B25-C25	
33			
34	Check		
35	Unconstrained relevant cost	\$7,319.26, either constraint type	
36	Budget, limit 10,000	theta = 0.146429841, relevant cost \$7,464.30	
37	Budget arithmetic check	holding cost = i times the limit = \$3,000.00	
38	Space, limit 800	theta = 2.953823172, relevant cost \$7,723.58	

Figure 4.4.: The same block with formulas displayed rather than values. Notice that no formula names a cell inside the item block. Every one of them refers to a named range, and that is what confines an added item to the item block.

4. Multi-Item and Constrained Systems

range by another. Dividing would ask Excel to evaluate zero over zero on every empty row inside a hundred-row range, and the cell returns an error instead of a number. Thus, where a ratio per item is genuinely needed, and replenishments per year is the only such case here, the ratio gets its own column and the summary sums it. Thus, C22 multiplies k_j by a column of frequencies instead of dividing $k_j\lambda_j$ by a column of quantities.

The search is a button, and you should know why that is safe.

Rows 14 to 18 hold the multiplier, $R(\theta)$, and a status cell. Goal Seek drives the second to zero by changing the first.

Goal Seek is a numerical search that carries no guarantee of finding a root, or of finding the one you wanted. What makes it dependable here is not the software. Recall from Section 4.2.3 that $R(\theta)$ decreases strictly and therefore has exactly one root. A search cannot establish that for itself, and a reader who reaches for the button without the proof behind it is trusting a tool to be right about a function it has not examined.

One property of the design deserves the caution. Goal Seek does not re-run by itself, so changing a demand rate leaves a stale multiplier with every result below it confidently wrong. The status cell exists for that, and you should watch it and not the numbers.

The item data goes last. We come to the layout decision that matters most, and it is not about tidiness.

The decision in this model is a vector, $Q_1, \dots, Q_n$, so the natural representation is a table whose rows are items. A table that rows are added to can only grow downward, and it can only grow downward if nothing sits beneath it. Put the item block in the middle, between the multiplier and the results, and every added item pushes two other blocks down and silently widens or breaks the ranges that point at it. Put it last and adding an item is typing in the first empty row.

Table 4.19 gives the columns.

4. Multi-Item and Constrained Systems

Table 4.19.: The item columns. Every summary above the block is a SUM or a SUMPRODUCT over these, reached by name.

Column	Holds	Formula in row 43
A to E	name, λ_j , c_j , k_j , v_j	typed
F	$h_j = ic_j$	=B\$5*C43
G	a_j	=IF(\$B\$7="Budget",C43/2,E43)
H	Q_j^*	=SQRT(2*D43*B43/F43)
I	$Q_j(\theta)$	=SQRT(2*D43*B43/(F43+2*\$B\$14*G43))
J, K	replenishments per year at each quantity	=B43/H43, =B43/I43
L	$Q_j(\theta)/Q_j^*$	=I43/H43

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
40	Item data													
41	Everything above is computed from this block. To add an item, type it in the first empty row and copy the computed cells down one row. No formula above needs changing: the named ranges already cover 100 rows.													
42	Item	Demand (lambda)	Unit cost (c)	Order cost (k)	Space per unit (v)	Holding (h = I*c)	Resource per unit (a)	Q free	Q held	Orders/yr free	Orders/yr held	Q held / Q free		
43	Item 1	12,000.	20.	50.	0.8	\$6.00	10,000	447.21	366.61	26.83	32.73	0.8198		
44	Item 2	25,000.	10.	50.	0.3	\$3.00	5,000	912.87	748.33	27.39	33.41	0.8198		
45	Item 3	8,000.	15.	50.	1.2	\$4.50	7,500	421.64	345.64	18.97	23.15	0.8198		

Figure 4.5.: The item block of the constrained worksheet. Columns A to E are typed and the rest are computed. Column I is equation 4.5 written out, and the reference to the multiplier in it is absolute while the references to the item's own data advance with the row.

Read column I of Figure 4.5 against Equation 4.5 and the correspondence is exact. D43 and B43 are k_j and λ_j , F43 is h_j , G43 is a_j , and \$B\$14 is θ^* . Notice that only the last of those is written with dollar signs. Every formula in the column is the same formula copied down, and the dollar signs are what decide whether copying it changes it. Dollar signs are the spreadsheet's only mechanism for saying what belongs to an item and what belongs to the problem, and you should learn to read them as a statement about the model.

4. Multi-Item and Constrained Systems

The multiplier belongs to no item. It appears in every row of column I and is not a property of any of them. It is one number describing the whole problem, and the layout says so by putting it in a single cell above the table.

You could instead give it a column of its own, repeated once per item. The sheet would still calculate correctly, and it would have stopped telling the truth about the model.

The coefficient is data, not structure. Column G is the only thing that distinguishes a budget constraint from a space constraint. Recall that Equation 4.2 says both are $\sum_j a_j Q_j \leq A$ with different coefficients. The worksheet holds a_j in a column and lets one cell choose what goes in it. The alternative, two worksheets differing in one formula, would have to be corrected twice every time anything else changed.

The evaluator and the optimizer, again. Rows 22 to 32 are the last two blocks of Section 3.10 in new clothing. There the sheet reported one quantity's cost at a typed Q and at Q^* , and the rows between them measured the penalty. Here the two columns are the portfolio unconstrained and constrained, and the same rows measure the price of the limit. The pattern survived because the question survived. What changed is only what is being compared.

4.7.2. The Exchange Curve

The constrained problem had one multiplier, fixed by the limit. The exchange curve of Section 4.4 has no limit at all, and sweeps the multiplier to trace what is reachable. That single change moves the multiplier from a cell to a column, and we take the why before the how.

What a sheet puts in a cell, and what it puts in a column. In Section 4.7.1 the multiplier described the problem, so it lived in one cell and every formula pointed at it absolutely. Here the multiplier is the thing being varied, so it runs down a column and every other quantity becomes

4. Multi-Item and Constrained Systems

a column beside it. Thus, the layout is decided by a question about the model and not about convenience: is this quantity one fact about the problem, or is it the axis?

Get this backwards and the sheet still computes. For example, a multiplier repeated in a column in Section 4.7.1 would produce identical numbers while asserting, falsely, that each item has its own shadow price. A single multiplier cell here would produce one point and no curve.

	A	B	C	D	E
1	Exchange Curve for a Group of Items				
2	<i>Chapter 4, section 4.4. Aggregate investment and aggregate ordering cost cannot be chosen separately; their product is fixed by the item data.</i>				
3					
4	Inputs				
5	Order handling cost (κ)	\$55.00	dollars per hour		
6	Carrying charge (γ)	25.0%	per year		
7	Operating point (θ)	0.25	per year	set equal to γ to price the firm's own position	
8					
9	Results				
10	Aggregate demand rate	14,445			
11	Aggregate purchase cost rate	\$555,500.00			
12	Demand-weighted unit cost ($\bar{c}$)	\$38.46			
13	Purchase-weighted ordering cost ($\bar{K}$)	\$123.67			
14	Variety index (J^*)	5.6813			
15	Curve constant, equation 4.15	195,147,265.65			
16	The same constant from J^* , $\bar{K}$ -bar, λ -bar, $\bar{c}$ -bar	195,147,265.65			
17	Aggregate investment at θ	\$27,939.02			
18	Aggregate ordering cost at θ	\$6,984.76			
19	Their product	195,147,265.65			
20	Replenishments per year at θ	81.22			
21	Order handling hours per year	127.0			
22	Investment over ordering cost	4.000			
23					
24	Check				
25	Sum of the SQRT column	19,755.87			
26	Curve constant	195,147,266, by either route			
27	Variety index J^*	5.6813, so nine items behave like fewer than six			
28	At $\theta = \gamma = 0.25$	\$27,939.02 of stock against \$6,984.76 a year, or 127 hours			
29	Investment over ordering cost	exactly 4.000, because the ratio is $1/\theta$			

Figure 4.6.: The exchange curve worksheet, rows 1 to 29. Every figure in the results block is an aggregate over the item data, and the check block below records what the shipped items should produce.

Figure 4.6 shows the results. Notice that the constant of Equation 4.18 appears twice, at rows 15 and 16, computed two different ways. The first squares the sum of the item column, and the second multiplies the variety index by the three averages. They agree to the cent, and that agreement

4. Multi-Item and Constrained Systems

is the arithmetic check for this sheet.

Table 4.20.: The six cells of the exchange curve worksheet that implement a result from Section 4.4. Rows 15 and 16 compute one quantity by two routes on purpose.

Cell	Holds	Formula	Which result it is
B13	purchase-weighted ordering cost	$\text{=SUMPRODUCT}(\text{Exc_k}, \text{Exc_c}, \text{Exc_lambda})/\text{B11}$	Equation 4.18
B14	variety index	$\text{=SUMPRODUCT}(\text{Exc_Equation 4.18})/\text{SUMPRODUCT}(\text{Exc_k}, \text{Exc_c}, \text{Exc_lambda})$	Equation 4.18
B15	the curve constant	$\text{=SUMPRODUCT}(\text{Exc_Equation 4.18})/\text{B14}$	Equation 4.18
B16	the same constant, decomposed	=B14*B13*B10*B12	Equation 4.22
B17	aggregate investment	$\text{=SUMPRODUCT}(\text{Exc_}\bar{E}^a, \text{Exc_Q})/2$	
B18	aggregate ordering cost	$\text{=SUMPRODUCT}(\text{Exc_}\bar{D}^a, \text{Exc_N})$	

Figure 4.7 is the curve itself. Notice that it is not the output of a search. Equation 4.14 says the product of the two aggregates is a constant fixed by the item data, so each row evaluates a closed form and the shape is known before any row is filled in. Column D exists to show that: it is the same value, twelve times.

Notice also that we never divide one item column by another here. Each is a `SUMPRODUCT` of square roots, Equation 4.17 rearranged so the multiplier comes out as a scalar. The fifth column, replenishments per year, could not be written that way, since an item's order frequency divides by its own ordering cost. Thus, it takes the same route the constrained sheet took:

4. Multi-Item and Constrained Systems

	A	B	C	D	E
31					
32	The curve				
33	<i>One row per operating point. Investment times ordering cost is the same on every row, which is the content of equation 4.15.</i>				
34	Multiplier (theta)	Aggregate investment	Aggregate ordering cost	Their product	Replenishments per year
35	0.05	\$62,473.56	\$3,123.68	195,147,265.65	36.32
36	0.08	\$49,389.68	\$3,951.17	195,147,265.65	45.95
37	0.10	\$44,175.48	\$4,417.55	195,147,265.65	51.37
38	0.15	\$36,069.12	\$5,410.37	195,147,265.65	62.91
39	0.20	\$31,236.78	\$6,247.36	195,147,265.65	72.65
40	0.25	\$27,939.02	\$6,984.76	195,147,265.65	81.22
41	0.30	\$25,504.72	\$7,651.42	195,147,265.65	88.98
42	0.40	\$22,087.74	\$8,835.10	195,147,265.65	102.74
43	0.50	\$19,755.87	\$9,877.94	195,147,265.65	114.87
44	0.65	\$17,327.05	\$11,262.58	195,147,265.65	130.97
45	0.80	\$15,618.39	\$12,494.71	195,147,265.65	145.30
46	1.00	\$13,969.51	\$13,969.51	195,147,265.65	162.45

Figure 4.7.: The curve, one row per operating point. The multiplier runs down column A, the two aggregates are computed beside it, and their product is the same number on every row.

a per-item column doing the division, and a summary that multiplies a scalar by its sum.

Figure 4.8 carries nine items, against three on the constrained sheet, and nothing above it changed to accommodate them. The named ranges are doing that work.

4.7.3. Intervals and Rounding

Joint replenishment is the first model in this chapter whose answer is not a vector of quantities. Recall from Section 4.5.1 that it is a base period T , which is continuous, together with a multiplier m_j for each item, an integer. A worksheet has to carry both, and they behave differently: the base period is one cell an optimizer changes, and the multipliers are columns that recompute themselves every time that cell moves.

One continuous decision. The base period is the only continuous decision, because Equation 4.25 says the multipliers separate once T is

4. Multi-Item and Constrained Systems

	A	B	C	D	E	F	G	H	I	J
51	Item data									
52	Everything above is computed from this block. To add an item, type it in the first empty row and copy the computed cells down one row. No formula above needs changing; the named ranges already cover 100 rows.									
53	Item	Demand (lambda)	Unit cost (c)	Workload per order (w, hr)	Order cost (k = kappa*w)	Holding cost (h = gamma*c)	$\text{SQRT}(k*c*\text{lambda})$	Q at theta	Orders/yr at theta	$\text{SQRT}(\text{lambda}*c/(2k))$
54	Pad-mount transformer	45.	3,800.	4.	\$220.00	\$950.00	6,133.51	4.57	9.86	19.7139
55	Smart meter	1,200.	95.	2.	\$110.00	\$23.75	3,541.19	105.43	11.38	22.7636
56	Fuse cutout	800.	115.	1.5	\$82.50	\$28.75	2,755.00	67.76	11.81	23.6130
57	Meter socket	900.	48.	1.5	\$82.50	\$12.00	1,887.86	111.24	8.09	16.1808
58	Riser conduit, 4 in	600.	62.	1.5	\$82.50	\$15.50	1,751.86	79.92	7.51	15.0151
59	Splice kit	1,500.	28.	1.	\$55.00	\$7.00	1,519.87	153.53	9.77	19.5402
60	Ground rod	2,400.	14.	0.75	\$41.25	\$3.50	1,177.29	237.85	10.09	20.1810
61	Compression connector	6,000.	3.4	0.5	\$27.50	\$0.85	749.00	623.09	9.63	19.2590
62	Warning tape, roll	1,000.	2.1	0.5	\$27.50	\$0.53	240.31	323.67	3.09	6.1791

Figure 4.8.: The exchange curve item block. Columns A to D are typed and the rest are computed. Column G is the only aggregation Equation 4.18 needs, and column J exists so that the replenishment column of the curve has something to sum.

fixed. That is, the whole problem is one-dimensional, so Solver has exactly one cell to change under each rule.

Notice in Figure 4.9 that the two rules have separate base period cells. They reach different optima, 51.882 days against 51.800, so one cell could not serve both.

Table 4.21.: The six cells of the joint replenishment worksheet that implement a result from Section 4.5. Notice that C17 adds the major setup once and sums a per-item column, rather than dividing anything by a column of multipliers.

Cell	Holds	Formula	Which result it is
B10	independent ordering	$=\text{SUMPRODUCT}(\text{SQRT}((\$B\$5 + \text{Jnt_k}) * \text{Jnt_h} * \text{Jnt_lambda}))$	paying $K + k_j$ alone
B11	common cycle T^*	$=\text{SQRT}((\$B\$5 + \text{SUM}(\text{Jnt_k})) / \text{SUM}(\text{Jnt_g}))$	
C17	cost at the integer-ratio base period	$=\$B\$5 / (\text{B17} / \$B\$7) + \text{SUM}(\text{Jnt_cost_int})$	

4. Multi-Item and Constrained Systems

Cell	Holds	Formula	Which result it is
H56	an item's integer multiplier	=MAX(1,CEILING((the integer 1+SQRT(1+4*(G56/((\$B\$17/\$B\$7))^2))/2,1))	Equation 4.25
J56	an item's power-of-two multiplier	=2^MAX(0,CEILING(LOG(G56/(SQRT(2)*(\$B\$18/\$B\$7))),2),1)	Equation 4.25
D18	what nesting costs	=C18/C17-1	the bound of Equation 4.30, measured

The formula in H56 looks worse than it is. The integer minimizing Equation 4.23 for one item is the m with $m(m-1) \leq (T_j^*/T)^2 \leq m(m+1)$, and solving that pair of inequalities for m gives exactly the expression in the cell. Thus, no cell has to evaluate the cost at two candidate multipliers and keep the smaller.

Notice in Figure 4.10 that the curve is not how the answer is found. Solver finds it, by changing one cell. The curve is there because the objective is *piecewise* smooth: it kinks wherever a multiplier jumps, so a hill climber can halt at a kink instead of at the minimum. Thus, plotting the objective before optimizing it is not decoration here, it is the only evidence available that Solver stopped in the right basin.

We carry one contrast away from these two sections. In Section 4.7.1 the search is a button and nothing is plotted, because Section 4.2.3 proves the function has one root. Here the search is also a button, and the plot is mandatory, because no such proof exists. What decides whether you may trust an optimizer is a property of the function, and it is established in the chapter, not in the sheet.

Notice in Figure 4.11 that the drive unit and the gearbox both sit at a multiplier of one under both rules. Their preferred intervals, 13 and 37

4. Multi-Item and Constrained Systems

	A	B	C	D	E	F	G	H	I
1	Joint Replenishment: a base period and integer multipliers								
2	<i>Chapter 4, section 4.5. The multipliers separate once the base period is fixed, so the only decision Solver has to make is the base period itself.</i>								
3									
4	Inputs								
5	Major setup cost (K)	\$400.00	dollars per order						
6	Carrying charge (i)	25.0%	per year						
7	Days per year	365	days						
8									
9	Two baselines, before any search								
10	Independent ordering	\$8,422.39							
11	Common cycle T (year)	0.146209							
12	Common cycle T (day)	53.4							
13	Common cycle cost	\$6,497.54							
14									
15	Solve for the base period								
16	Rule	Base period (days)	Total cost	Excess over integer ratio					
17	Integer ratio	51.882	\$6,402.06						
18	Powers of two	51.800	\$6,403.34	0.020%					
19	How	<i>Data > Solver. Set objective C17 to Min by changing B17, GRG Nonlinear, B17 >= 0.001. Then the same for C18 by changing B18.</i>							
20	Read the curve first	<i>The cost is piecewise smooth: it kinks wherever a multiplier jumps, so Solver can stop at a kink. The block below plots it, which is how yo</i>							
21									
22	Check								
23	Independent ordering	\$8,422.39							
24	Common cycle	0.146209 years, 53.4 days, \$6,497.54							
25	Integer ratio	51.882 days, \$6,402.06, multipliers 1, 1, 2, 6							
26	Powers of two	51.800 days, \$6,403.34, multipliers 1, 1, 2, 8, excess 0.020%							

Figure 4.9.: The joint replenishment worksheet, rows 1 to 26. Inputs, then the two baselines coordination is measured against, then the two rules with the base period Solver changes and what the power-of-two restriction costs against the best integer ratio.

4. Multi-Item and Constrained Systems

	A	B	C	D	E	F	G	H	I	J	K
28	The cost against the base period										
29	One row per candidate base period, so the shape can be seen before Solver is trusted. The multiplier columns are what kink the cost: watch one of them step up and the cost turn a corner in the same row										
30	T (days)	m1, integer	m2, integer	m3, integer	m4, integer	Cost, integer ratio	m1, pow 2	m2, pow 2	m3, pow 2	m4, pow 2	Cost, powers of two
31	26.7	1	1	4	12	\$7,837.08	1	1	4	16	\$7,838.66
32	29.4	1	1	3	11	\$7,441.77	1	1	4	8	\$7,443.87
33	32.0	1	1	3	10	\$7,148.83	1	1	4	8	\$7,152.87
34	34.7	1	1	3	9	\$6,916.62	1	1	4	8	\$6,922.78
35	37.4	1	1	3	8	\$6,742.13	1	1	2	8	\$6,746.04
36	40.0	1	1	3	8	\$6,617.61	1	1	2	8	\$6,618.50
37	42.7	1	1	2	7	\$6,523.16	1	1	2	8	\$6,523.21
38	45.4	1	1	2	7	\$6,458.77	1	1	2	8	\$6,459.11
39	48.0	1	1	2	7	\$6,421.34	1	1	2	8	\$6,421.94
40	50.7	1	1	2	6	\$6,403.76	1	1	2	8	\$6,404.82
41	53.4	1	1	2	6	\$6,404.73	1	1	2	8	\$6,406.31
42	56.0	1	1	2	6	\$6,420.75	1	1	2	4	\$6,422.76
43	58.7	1	1	2	5	\$6,450.82	1	1	2	4	\$6,452.27
44	61.4	1	1	2	5	\$6,492.71	1	1	2	4	\$6,493.80
45	64.0	1	1	2	5	\$6,542.98	1	1	2	4	\$6,543.75
46	66.7	1	1	2	5	\$6,604.26	1	1	2	4	\$6,604.71
47	69.4	1	1	2	5	\$6,673.72	1	1	2	4	\$6,673.87
48	72.0	1	1	1	4	\$6,745.91	1	1	1	4	\$6,745.91
49	74.7	1	1	1	4	\$6,825.75	1	1	1	4	\$6,825.75
50	77.4	1	1	1	4	\$6,911.52	1	1	1	4	\$6,911.52
51	80.0	1	1	1	4	\$6,999.15	1	1	1	4	\$6,999.15

Figure 4.10.: The cost against the base period, one row per candidate. The multiplier columns are the point: watch the last one step down from 6 to 5 and the cost column turn a corner in the same row.

	A	B	C	D	E	F	G	H	I	J	K
53	Item data										
54	Everything above is computed from this block. To add an item, type it in the first empty row and copy the computed cells down one row. No formula above needs changing: the named ranges already co										
55	Item	Demand (lambda)	Unit cost (c)	Minor setup (k)	Holding rate (h = I*c)	$g = h * \text{lambda} / 2$	Preferred interval (yr)	m, integer ratio	Cost at that m	m, powers of two	Cost at that m
56	Drive unit	8,000.	20.	25.	\$5.00	20,000.0	0.035355	1	\$3,018.73	1	\$3,014.51
57	Gearbox	800.	20.	20.	\$5.00	2,000.0	0.100000	1	\$424.99	1	\$424.76
58	Seal kit	200.	8.	15.	\$2.00	200.0	0.273861	2	\$109.62	2	\$109.61
59	Name plate	40.	4.	15.	\$1.00	20.0	0.866025	6	\$34.65	8	\$35.92

Figure 4.11.: The joint replenishment item block. The preferred interval in column G is what each item would choose paying only its minor setup, and the two multiplier columns move it to the nearest multiple of the base period under each rule.

4. Multi-Item and Constrained Systems

days, are shorter than the base period, so nothing about them is rounded and what sets their cost is the choice of base period alone. It is the first qualification of Section 4.5.3, visible in two cells.

Where the item block cannot carry the work. Every summary above the item block on this sheet reaches it through a named range, so it does not care how many items there are. The curve cannot. Each row needs the multipliers *at that row's base period*, and a multiplier is a rounding, which no arithmetic over a whole column will produce.

Two ways out present themselves, and we state the choice between them because the rest of this section keeps making it. One is to compute the rounding over the whole column inside a single formula, compact and independent of the item count. The other is to give each item its own column and compute the rounding one item at a time, longer but showing the reader something the first hides.

The second is what the sheet does, and the multiplier columns of Figure 4.9 are the reason. Watch the last one step down from 6 to 5 as the base period lengthens, and the cost column turns a corner in the same row. The kink is made visible there, and it justifies plotting the curve at all. A formula that hid the multipliers would have hidden the very thing the block exists to show.

The cost is a column per item, so a fifth item means a column in each of the two groups. The distribution scan in Section 4.7.4 makes the same admission, and we state it plainly: the item block grows by a row, and a calculation that works item by item grows with it.

4.7.4. The Two Echelon Structures

The echelon models of Section 4.6 differ from everything before them in that we must fix the accounting before any optimization is possible. Recall from Equation 4.31 that an echelon rate is a difference between neighbours.

4. Multi-Item and Constrained Systems

That is, no location can compute its own, and the sheet has to know the structure before it can charge anybody.

The distribution algorithm suits a worksheet and the serial one does not. The scan of Section 4.6.3 has a fixed number of steps known in advance, one per region, and each step depends only on the one before it. That is a table.

	A	B	C	D	E	F
1	Distribution System: the pinned set					
2	Chapter 4, section 4.6.3. Sorting the regions turns the construction of CO into running totals. No Solver, no iteration, no macros.					
3						
4	Inputs					
5	Base period	1 weeks				
6	Weeks per year	52 weeks per year				
7	Base period in years (T)	0.019231 years				
8						
9	Results					
10	First pinned rank ($\bar{i}$ -bar)	3 ranks at or above this share the central interval				
11	Central warehouse interval (years)	0.343049				
12	Central warehouse interval (weeks)	17.84				
13	Rounded to a power of two (weeks)	16.0				
14	Pinned set CO	the central warehouse plus the regions at ranks 3 through 6				
15						
16	Check					
17	Ranks, shortest interval first	nodes 5, 3, 6, 4, 1, 2				
18	First pinned rank ($\bar{i}$ -bar)	3				
19	Pinned set CO	the central warehouse plus nodes 6, 4, 1 and 2				
20	Left alone	nodes 5 and 3, at 14.24 and 17.25 weeks				
21	Central interval	0.343049 years, or 17.84 weeks				
22	Every power-of-two interval	16 weeks				

Figure 4.12.: The distribution worksheet, rows 1 to 22. The results come first and the check block records what the shipped locations should produce. Nothing here is iterative.

4. Multi-Item and Constrained Systems

Table 4.22.: The six cells of the distribution worksheet that implement a result from Section 4.6.3. Notice that the central warehouse is found with INDEX(...,1) rather than by a row number, so reordering the regions does not break the sheet.

Cell	Holds	Formula	Which result it is
E42	echelon rate h'_i	=IF(A42=INDEX(Dis_node,1,D42,D42-INDEX(Dis_h,1)))	Equation 1.4.24
G42	the ratio k_i/g_i	=C42/F42	what the regions are sorted on
F32	pooled k of the set so far	=INDEX(Dis_k,1)+SUMIF(\$A\$32:\$A\$37,">"&A32,\$C\$32:\$C\$37)	total of the scan
I32	the verdict at this step	=IF(E32>H32,"add","stop")	Section 4.6.3
B10	first pinned rank	=MAX(J32:J37)+1	where the scan stopped
B13	the rounded central interval	=2^MAX(0,CEILING(LOG(B11/2),2),1))*\$B\$7	Equation 1.4.25

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
	Rank	Node	k	g	k / g	Pooled k (C0 so far)	Pooled g (C0 so far)	Pooled k/g	Test	Stop index	Interval T (years)	T / (sqrt2*base) I*	Power-of-two T (years)		T (weeks)
31															
32	1	5	\$75.00	1,000.0	0.075000	\$1,075.00	9,200.0	0.116848	stop	1	0.273861	10.07	4	0.307692	16.0
33	2	3	\$110.00	1,000.0	0.110000	\$965.00	8,200.0	0.117683	stop	2	0.331662	12.20	4	0.307692	16.0
34	3	6	\$90.00	700.0	0.128571	\$875.00	7,500.0	0.116667	add	0	0.343049	12.61	4	0.307692	16.0
35	4	4	\$150.00	800.0	0.187500	\$725.00	6,700.0	0.108209	add	0	0.343049	12.61	4	0.307692	16.0
36	5	1	\$100.00	500.0	0.200000	\$625.00	6,200.0	0.100806	add	0	0.343049	12.61	4	0.307692	16.0
37	6	2	\$125.00	200.0	0.625000	\$500.00	6,000.0	0.083333	add	0	0.343049	12.61	4	0.307692	16.0

Figure 4.13.: The scan, one row per region. The candidate's own ratio sits beside the pooled ratio of everything already pinned, and the verdict is the comparison between them.

Figure 4.13 reproduces Table 4.16 row for row, and that is the point of the block. The pooled columns are running totals, so the sheet needs no iteration, no Solver and no macro. The serial partition has none of those

4. Multi-Item and Constrained Systems

properties: the number of blocks is not known until the walk is finished, and a merge changes blocks already built. There is therefore no serial worksheet in the accompanying workbook, and the absence is an answer, not an omission.

One row per region is also one row to add. Every summary above the block reaches it by name and does not care how many locations there are, but the scan cannot be written that way, because each step needs the pooled totals of the step before it. Thus a seventh region means a row in the location block and a row in the scan, and the sheet says so instead of hiding it.

	A	B	C	D	E	F	G	H
39	Item data							
40	Everything above is computed from this block. To add an item, type it in the first empty row and copy the co							
41	Node	Demand (lambda) (k)	order rate (h)	Installation rate (h')	Echelon rate (h')	g = lambda*h'/2	k / g	Rank
42	0 centr	1,200.	500.	10.	\$10.00	6,000.0	0.083333	
43	1	200.	100.	15.	\$5.00	500.0	0.200000	5
44	2	200.	125.	12.	\$2.00	200.0	0.625000	6
45	3	200.	110.	20.	\$10.00	1,000.0	0.110000	2
46	4	200.	150.	18.	\$8.00	800.0	0.187500	4
47	5	200.	75.	20.	\$10.00	1,000.0	0.075000	1
48	6	200.	90.	17.	\$7.00	700.0	0.128571	3

Figure 4.14.: The distribution item block. The central warehouse is the first row, and every region is charged only the value it adds, which is column E.

Notice in Figure 4.14 that the rank column is blank on the central warehouse row. Ranking is over the regions alone, because the scan works down from the region wanting the longest interval and the centre is what they are being compared against, not a candidate itself.

4.7.5. What a Worksheet Can and Cannot Do

Three questions were answered four times each in the preceding sections. Having seen every instance, we can state them.

4. Multi-Item and Constrained Systems

Is this a property of an item, or of the problem? The multiplier is one cell. The volume v_j is a column. Getting this wrong costs nothing arithmetically, and that is exactly what makes it dangerous: a multiplier repeated down a column computes the same numbers while asserting that each item has its own shadow price. Thus, the test is not whether the numbers come out, but whether the layout still describes the model.

Is this difference data, or a different sheet? A budget limit and a space limit differ in one column, so Equation 4.2 is implemented once and column G holds a_j . Two sheets differing in one formula would have to be corrected twice forever after. Notice that the same question has a harder answer for the coordination rules of Section 4.5.1, where the alternatives differ in what has to be done and not in a number, so the joint sheet carries two base period cells and two sets of multiplier columns instead of one of each with a switch.

When may a button be trusted? Twice in these sheets a search is a button, and the two cases are not alike. Goal Seek is safe in Section 4.7.1 because Section 4.2.3 proves $R(\theta)$ strictly decreasing with exactly one root. For example, Solver is not safe in Section 4.7.3, because the objective kinks wherever a multiplier steps, so that sheet plots the curve and the other does not. What decides whether an optimizer may be trusted is a property of the function, and it is established in the chapter, not on the sheet.

Where a worksheet stops. Three limits appeared, each in its place. It cannot bracket its own root or widen its own search, so it asks the reader to have established in advance what a program arranges in a loop. It cannot hold a structure whose size is not known until the computation finishes, so Algorithm 4.1 has no sheet. And it cannot be audited at three thousand items: every argument in this section about what the layout asserts depends on a person being able to see the layout.

None of that makes the worksheets inferior. They are better at the thing they are for, making a method visible. Figure 4.10 shows what a one-dimensional search over a base period is, and no loop shows that. What a worksheet cannot do is what Section 4.8 is for.

4.8. Designing the Software

In Section 4.7 we built each model as a worksheet. The layout of a worksheet is visible. A formula is a cell you can click on, a summary is a range you can highlight, and putting the multiplier in a single cell says that it belongs to the problem and not to any item.

A program shows none of that. Its structure is a set of decisions about which classes exist and what each class is responsible for, and those decisions leave no trace in the finished code. We therefore write them down first.

In this section we work through an object-oriented analysis and design of the five models of this chapter, in the order such an analysis proceeds: what the software must do, what the domain calls things, what each thing knows and does, how the things are related, and how they collaborate to produce an answer. Code appears at the end of each step, by which point little is left to decide.

The effort pays off because the five models share more than they appear to. A programmer who writes the constrained problem, then the exchange curve, then joint replenishment, without pausing between them, will write Equation 4.5 three times. Thus, when the third one is wrong there are three places to correct.

4.8.1. What the Software Must Do

We begin with a list of requirements. Each entry names something a user would ask the software for, together with the section of this chapter that supplies the answer.

4. Multi-Item and Constrained Systems

Table 4.23.: What the software has to do, and where in this chapter each requirement is worked out. Read the right-hand column as the specification: a requirement whose mathematics is not in the chapter is one nobody can check.

#	The software must	Established in
R1	compute each SKU's order quantity ignoring any coupling	Chapter 3
R2	report an aggregate measure of a set of decisions	Section 4.1
R3	decide whether a stated limit on a measure binds	Section 4.2.1
R4	find the cheapest decisions meeting a limit, and report its shadow price	Section 4.2.2
R5	price a limit against having none	Section 4.3
R6	trace the frontier of achievable aggregate positions	Section 4.4
R7	schedule SKUs sharing one setup, under a stated ordering practice	Section 4.5
R8	restrict intervals to what an operation can run, and price the restriction	Section 4.5.3

4. Multi-Item and Constrained Systems

#	The software must	Established in
R9	account for holding cost across locations without double counting	Section 4.6.1
R10	find reorder intervals across a chain or a distribution network	Section 4.6.2

Two features of Table 4.23 matter more than the list itself.

Notice that R3, R4 and R5 do not mention a budget. They name a limit on a *measure*. Recall from Section 4.1 that an organization runs short of four different things, and that Section 4.2 then solved for one of them. A design that treats the budget as the problem, and the other three as variations on it, carries that assumption into every signature it writes.

A requirement is also not a class. R7 says to schedule SKUs sharing one setup. That sentence names two candidate things, a group of SKUs and a schedule, and it does not say whether either one is an object. For example, turning requirements straight into classes is how software acquires a `ScheduleManager`.

A requirement says what the software does. A scenario says who asks, and in what order.

4. Multi-Item and Constrained Systems

Table 4.24.: The five scenarios the software is built to serve, each phrased as somebody’s question. A scenario that no requirement supports, or a requirement no scenario ever uses, is a defect in one of the two lists.

#	Scenario	The question behind it
S1	solve under a limit	“Management will fund \$10,000 of stock. What do I order?”
S2	price the limit	“What is that ceiling costing me?”
S3	trace the frontier	“If I want fewer orders, how much more stock is that?”
S4	coordinate a supplier’s line	“These four come on one truck. How often do I order each?”
S5	plan a network	“The depot feeds six branches. What interval does each run?”

S1 and S3 of Table 4.24 repay attention. “What do I order under a ten thousand dollar limit” and “if I want fewer orders, how much more stock is that” sound like two programs. Section 4.4 showed that they are one piece of mathematics asked from two ends. If the analysis is sound, the two will use the same objects and nearly the same messages, and we check that in Section 4.8.8.

4.8.2. Finding the Nouns

The candidate classes are the nouns of the domain, and the test for keeping one is whether somebody who runs a stockroom would use the word. That test is not a matter of taste. A name taken from the domain survives changes in the mathematics, because the domain is what the mathematics describes. A name invented to suit an implementation has to be renamed every time the implementation moves.

Table 4.25.: The vocabulary the design is built from. Every term is one that appears in this chapter and would be understood in a warehouse, which is the only test a candidate class has to pass at this stage.

Term	What it means
SKU	a stock keeping unit: one item held at one location, and the thing that gets replenished
Portfolio	the SKUs that are decided together
Aggregate measure	a number computed over a whole plan: investment, space occupied, replenishment workload, relevant cost
Constraint	an aggregate measure together with the ceiling management has put on it
Replenishment plan	how much and how often, for every SKU in a portfolio
Order family	SKUs sharing a supplier, a truck or a machine setup, so that one order serves several
Interval rule	what intervals the operation can actually run

4. Multi-Item and Constrained Systems

Term	What it means
Supply network	SKUs that replenish one another, whether in a chain or from a centre
Echelon holding rate	the value a location adds, as against the value accumulated through it

Four other nouns suggest themselves. None of them survives the test.

The **shadow price** is a number that a solution reports. A class for it would know nothing beyond its own value and would do nothing, so it becomes a property of a replenishment plan. Recall from Section 4.3 that the price falls to zero exactly where the limit stops binding. Thus, the mathematics has no missing case and the property need not be nullable.

Echelon stock is defined in Section 4.6.1, and what it defines is a measure of a network. It belongs with investment and space, not with SKUs and plans.

Item and **stage** are one noun. A SKU already is an item at a location. Section 4.2 holds many SKUs at one location and Section 4.6 holds one item across many locations. That is, one entity under two topologies. Two classes would duplicate every field and force a conversion between the sections.

Shared resource is the instructive rejection, because Section 4.1 uses the phrase. A budget, a floor and a receiving dock are all things that several SKUs compete for. As a class name it fails the test. Nobody in a warehouse has a shared resource. They have a working capital account, a building with pallet positions, and a purchasing department that processes so many orders a week. The word that does pass is *measure*, since each of those is a number we can compute about a plan and put a ceiling on.

Figure 4.15 illustrates the concepts and how they relate. Read it as sentences. A portfolio is one or more SKUs. A constraint limits a portfolio, an order family coordinates one, and a supply network sequences one. The

4. Multi-Item and Constrained Systems

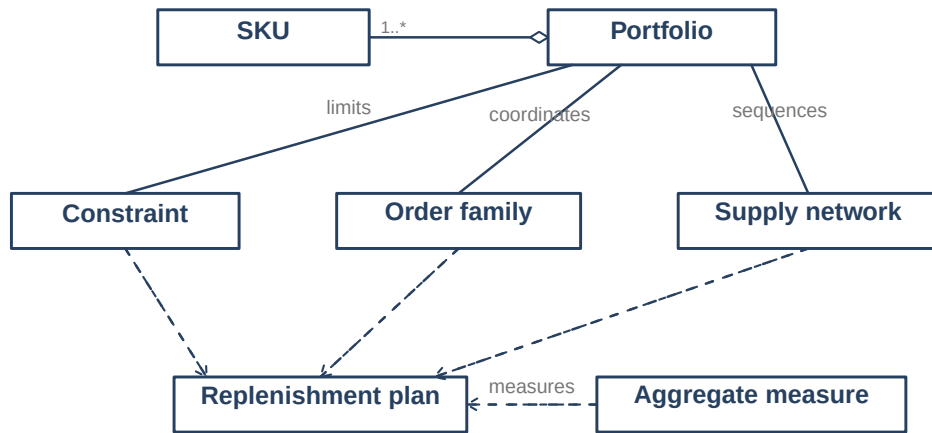


Figure 4.15.: The concepts of this chapter and how they relate, before any thought about implementation. A portfolio is one or more SKUs. A constraint limits a portfolio, an order family coordinates one, and a supply network sequences one. The dashed arrows show that all three produce a replenishment plan, and an aggregate measure measures that plan.

4. Multi-Item and Constrained Systems

dashed arrows say that all three produce a replenishment plan, and an aggregate measure measures that plan.

Figure 4.15 carries no operations, no types and no direction of dependency. It is a conceptual model and not yet a design. Every argument in the rest of this section asks whether the code still says what this picture says.

4.8.3. What Each Thing Knows and Does

We describe each candidate class before drawing it. The traditional form is a class-responsibility-collaborator card, one index card per candidate, with three questions on it. What does it know? Who does it know? What does it do?

The size of the card does the work. A responsibility that will not fit on one card is usually two responsibilities, and a card with no collaborators is either a value or a mistake.

Table 4.26.: The responsibilities, one row per candidate class. The middle column is the more important one: a class with no collaborators does nothing for anybody, and a class that knows about most of the others has probably absorbed a responsibility that belongs elsewhere.

Class	Knows	Knows about	Does
SKU	label, demand rate, unit value, order cost, holding rate	nobody	its economic order quantity; its cost at a quantity; the interval it would prefer

4. Multi-Item and Constrained Systems

Class	Knows	Knows about	Does
Portfolio	its SKUs, in a fixed order	SKU	hands out its SKUs; builds a plan from one quantity each
AggregateMeasure	its name and unit	ReplenishmentPlan	measure a plan
ConstrainableMeasure	whatever its kind knows	SKU	its marginal at a SKU; the quantity a SKU takes at a price
Constraint	a measure, a ceiling	AggregateMeasure	whether a plan meets it; the slack
ReplenishmentPlan	a quantity per SKU, and the price that produced it	SKU, AggregateMeasure	report any measure of itself; compare itself with another plan
ConstrainedLotSizing	a portfolio, a constraint, a method	all three	the cheapest plan meeting the limit; whether it binds
LotSizingMethod	whatever its kind needs	Portfolio, Constraint	produce that plan
ExchangeCurve	a portfolio	Portfolio, AggregateMeasure	the plan at a price; the curve over prices; the variety index

4. Multi-Item and Constrained Systems

Class	Knows	Knows about	Does
OrderFamily	a portfolio, the major setup	Portfolio , IntervalRule	a plan under a rule; what independent ordering would cost
IntervalRule	its base period	nobody	turn a preferred interval into one that can be run
SupplyNetwork	its SKUs, and who supplies whom	SKU, IntervalRule	the echelon rate of a location; the plan

Three rows of Table 4.26 repay a second look.

SKU knows nobody. A SKU holding a reference to its portfolio could not be evaluated inside a sweep without the portfolio coming with it, and the exchange curve of Section 4.4 evaluates the same SKUs at a thousand prices. Notice also what a SKU does not know: its volume per unit. A SKU carrying a volume because a space limit exists would have to carry a weight when a weight limit arrives, so the class would grow once per constraint anybody writes.

IntervalRule knows nobody either. Given the interval a location would prefer, it returns one the operation can run, and it never asks which location. Thus, the same rule applies per SKU in an order family, per block in a serial chain, and per location in a distribution network.

The measure knows the plan and the plan knows the measure. Two rows pointing at each other should make us nervous. That is, they are one message seen from two sides. The measure owns the formula and the plan owns the data, so the measure is the receiver and `plan.measuredBy(m)` delegates to `m.measure(plan)`. If both carried real implementations we

4. Multi-Item and Constrained Systems

would be starting on double dispatch, where adding a measure means editing every plan.

Reading the cards a second time, and asking who sends each message, finishes the analysis. One answer stands out. `marginalAt` is called by one kind of object, a lot sizing method, and by nothing that reports or displays. Thus, it belongs to the solving collaboration and not to the measure's public face. We act on that in Section 4.8.5.

4.8.4. The Classes

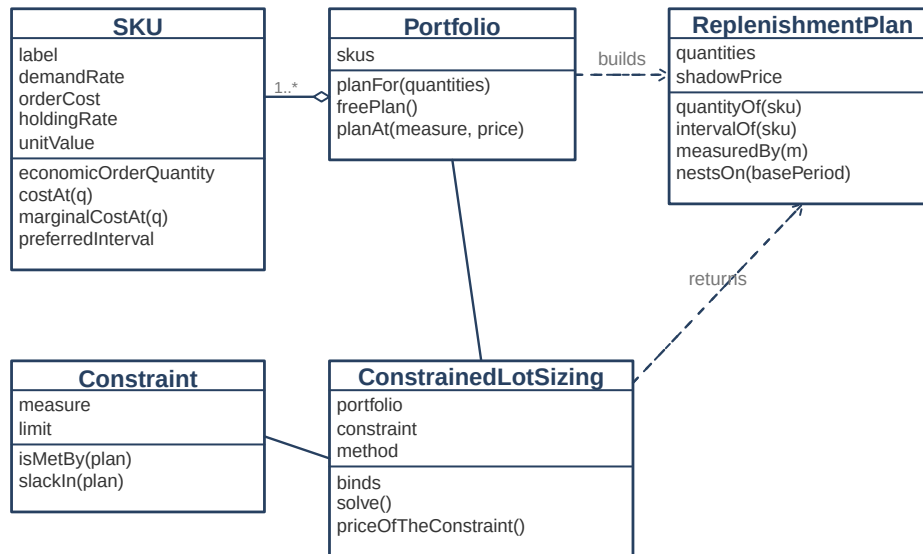


Figure 4.16.: The core classes and what they hold. A portfolio aggregates one or more SKUs and builds replenishment plans from them, and a constrained lot sizing study holds a portfolio, a constraint and a method. Notice that every arrow leaving the study points at something the study was handed, so nothing here constructs its own collaborators.

4. Multi-Item and Constrained Systems

Figure 4.16 illustrates the classes that carry an answer, and we take them in turn.

The SKU holds its own data and computes its own mathematics.

```
class SKU(  
    val label: String,  
    val demandRate: Double,  
    val orderCost: Double,  
    val holdingRate: Double,  
    val unitValue: Double = 0.0,  
) {  
    /** Equation 4.1, this SKU alone, ignoring anything it  
        ↪ shares. */  
    val economicOrderQuantity: Double  
        get() {  
            require(holdingRate > 0.0) {  
                "$label has no holding cost, so it has no  
                ↪ unconstrained order quantity"  
            }  
            return sqrt(2.0 * orderCost * demandRate /  
                ↪ holdingRate)  
        }  
  
    /** Ordering plus holding, per unit time, at a quantity  
        ↪ someone else chose. */  
    fun costAt(orderQuantity: Double): Double {  
        require(orderQuantity > 0.0) { "An order quantity  
        ↪ must be positive" }  
        return orderCost * demandRate / orderQuantity +  
            ↪ holdingRate * orderQuantity / 2.0  
    }  
  
    /** The derivative of costAt, which is what a  
        ↪ stationarity condition needs. */
```

4. Multi-Item and Constrained Systems

```
fun marginalCostAt(orderQuantity: Double): Double =  
    -orderCost * demandRate / (orderQuantity *  
        ↪ orderQuantity) + holdingRate / 2.0  
}
```

Notice that `costAt` takes a quantity instead of computing one. Every model in this chapter evaluates a SKU at a quantity chosen elsewhere, so a class that knew only its own optimum would be useless in four of the five. For example, the exchange curve evaluates every SKU at the quantity a price implies.

`marginalCostAt` looks like an implementation detail of `economicOrderQuantity`. It is in fact the piece every constrained model needs, because a stationarity condition is where a derivative is used.

One answer type serves all five models. The constrained problem and the exchange curve solve for quantities, and joint replenishment and both echelon structures solve for intervals. Recall that $Q_j = \lambda_j T_j$, so an object holding one can report the other.

```
class ReplenishmentPlan(  
    val skus: List<SKU>,  
    private val quantities: List<Double>,  
    val shadowPrice: Double = 0.0,  
) {  
    fun quantityOf(sku: SKU): Double =  
        ↪ quantities[indexOf(sku)]  
    fun intervalOf(sku: SKU): Double =  
        ↪ sku.intervalFor(quantityOf(sku))  
  
    /** Delegates to the measure, which owns the formula  
        ↪ while this owns the data. */  
    fun measuredBy(measure: AggregateMeasure): Double =  
        ↪ measure.measure(this)
```

4. Multi-Item and Constrained Systems

```
}
```

This is not unification for its own sake. The chapter claims that every model in it answers how much and how often. If that claim is true, then one answer type is the right number, and a sixth model whose answer did not fit here would be evidence against the claim.

A constraint is a measure together with a ceiling. “At most ten thousand dollars of average investment” is one sentence in the domain, so we make it one object. Keeping the measure separate from the ceiling lets the same measure serve the constrained problem, the exchange curve, and a management report that constrains nothing at all.

```
class Constraint(val measure: ConstrainableMeasure, val
  ↪ limit: Double) {
  fun isMetBy(plan: ReplenishmentPlan): Boolean =
    ↪ measure.measure(plan) <= limit

  /** Positive when there is room left, negative when the
    ↪ plan overshoots. */
  fun slackIn(plan: ReplenishmentPlan): Double = limit -
    ↪ measure.measure(plan)
}
```

Recall that the worksheet of Section 4.7.1 put the bind test in B11 and $R(\theta)$ in B15. Those two cells are `isMetBy` and `slackIn`. Both implementations put them on the constraint, because the comparison needs the measure and the limit together and needs nothing else.

4.8.5. Measuring, and Limiting a Measure

This is where the analysis pays for itself, so we go slowly.

4. Multi-Item and Constrained Systems

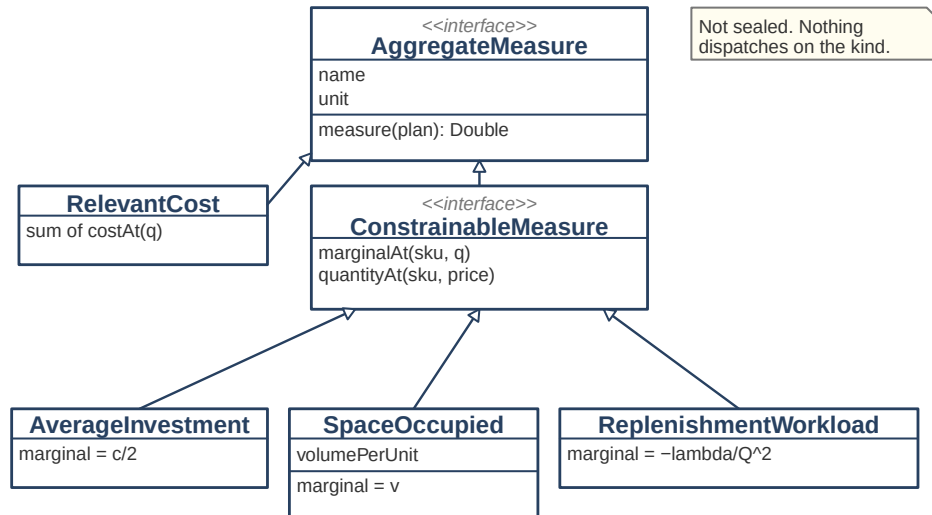


Figure 4.17.: The measures. What can be measured is one interface, and what a limit can be put on is a second interface extending it. Relevant cost implements only the first, since it is the objective and is never constrained. Notice that SpaceOccupied carries a volume per unit while the other two kinds carry nothing of their own.

4. Multi-Item and Constrained Systems

Figure 4.17 illustrates two interfaces and four kinds. We take the split first.

Relevant cost is measured constantly and constrained never. Nobody puts a ceiling on the objective, because the objective is what everything else is traded against. Putting `marginalAt` on one interface with `measure` would force `RelevantCost` to implement a method that has no meaning for it, and a method implemented only to satisfy a compiler is a method somebody will eventually call. Thus, `AggregateMeasure` is what can be measured, and `ConstrainableMeasure` adds what a limit needs.

The marginal is the primitive and the quantity is derived from it. Recall from Section 4.2.2 that the stationarity condition for one SKU sets the derivative of the Lagrangian to zero.

- Let M represent any aggregate measure computed over a plan, so that $\partial M / \partial Q_j$ is how much the measure changes per unit of order quantity at SKU j .

With this notation the condition is

$$-\frac{k_j \lambda_j}{Q_j^2} + \frac{h_j}{2} + \theta \frac{\partial M}{\partial Q_j} = 0$$

which is Equation 4.5 before the algebra. Notice what the condition needs. It needs the SKU's own cost derivative, supplied by `SKU.marginalCostAt`, and the measure's marginal. Nothing else. Thus, any measure that can report a marginal can be solved numerically, and we write that once as a default implementation.

```
interface ConstrainableMeasure : AggregateMeasure {  
  
    /** How much this measure changes per unit of order  
     * ↪ quantity at one SKU. */  
    fun marginalAt(sku: SKU, orderQuantity: Double): Double
```

4. Multi-Item and Constrained Systems

```
fun quantityAt(sku: SKU, shadowPrice: Double): Double {  
    val f = { q: Double -> sku.marginalCostAt(q) +  
        ↪ shadowPrice * marginalAt(sku, q) }  
    // bisect f to zero, widening the bracket until it  
    ↪ changes sign  
}  
}
```

A measure whose stationarity condition has a closed form overrides `quantityAt` and skips the search. Both of the measures that the chapter solves by hand do so.

```
object AverageInvestment : ConstrainableMeasure {  
    override fun measure(plan: ReplenishmentPlan): Double =  
        plan.skus.sumOf { it.unitValue * plan.quantityOf(it)  
    ↪ / 2.0 }  
    override fun marginalAt(sku: SKU, orderQuantity: Double):  
        ↪ Double = sku.unitValue / 2.0  
    override fun quantityAt(sku: SKU, shadowPrice: Double):  
        ↪ Double =  
        sqrt(2.0 * sku.orderCost * sku.demandRate /  
    ↪ (sku.holdingRate + shadowPrice * sku.unitValue))  
}
```

That is Equation 4.5 with $a_j = c_j/2$, and it is the only place in the program where the formula appears. The worksheet wrote it in column I of one sheet and again in the curve block of another, because a worksheet has no way to say the same thing once.

The third kind is the one that no coefficient can describe. A simpler interface suggests itself. Equation 4.2 writes every limit as $\sum_j a_j Q_j \leq A$,

4. Multi-Item and Constrained Systems

so a measure could report the single coefficient a_j and leave the rest to the solver. That interface serves a budget, where $a_j = c_j/2$, and it serves a floor, where $a_j = v_j$. Recall from Section 4.2.4 that a limit on replenishments is $\sum_j \lambda_j/Q_j \leq A$. Consumption falls as the quantities rise, so the marginal is negative and depends on the quantity, and the multiplier joins the ordering cost.

```
object ReplenishmentWorkload : ConstrainableMeasure {
    override fun measure(plan: ReplenishmentPlan): Double =
        plan.skus.sumOf {
            ↪ it.orderFrequencyAt(plan.quantityOf(it)) }
    override fun marginalAt(sku: SKU, orderQuantity: Double):
        ↪ Double =
            -sku.demandRate / (orderQuantity * orderQuantity)
    override fun quantityAt(sku: SKU, shadowPrice: Double):
        ↪ Double =
            sqrt(2.0 * (sku.orderCost + shadowPrice) *
            ↪ sku.demandRate / sku.holdingRate)
}
```

No coefficient expresses that. An interface reporting only a coefficient would force every caller to ask which kind of resource it was holding, and that is the thing an interface exists to prevent. Thus, an abstraction drawn from two examples fits the third by luck or not at all, and the mathematics is what tells you which.

Why a hierarchy, and not one class holding two functions? In Kotlin we could write a single `AggregateMeasure` class with `measure` and `marginalAt` as function-valued properties, and the four kinds as four instances. That design would be better if the kinds differed only in a formula. They do not. `SpaceOccupied` knows a volume per unit, and a weight limit would know a weight per unit, while `AverageInvestment` reads the SKU's own value and knows nothing extra. That is, the kinds

4. Multi-Item and Constrained Systems

differ in what they know, and holding state that varies by kind is what a subtype is for.

Notice also that this hierarchy is not sealed, while Section 3.11.3 seals four hierarchies in Chapter 3. Sealing buys exhaustive checking, and exhaustive checking pays only when something dispatches on the kind and the mathematics closes the list. Both conditions hold in Chapter 3, where Table 3.1 is a table of four rows. Neither holds here. A solver asks a measure for a number and never asks which measure it is holding, and the list of scarce things in Section 4.1 is open. Thus, a new measure needs two methods and no edit anywhere else.

4.8.6. Alternatives Are Types, Settings Are Values

Figure 4.18 illustrates two more hierarchies, and both exist because the domain offers alternatives.

Section 4.2.3 searches the multiplier until the measure meets its ceiling, which Goal Seek does on the worksheet and `LagrangianSearch` does here. Solver offers a second way, optimizing the quantities directly, which the code shown here does not implement. And Section 4.3 gives a third, a closed form under a budget when every SKU holds at $h_j = ic_j$. Every quantity then shrinks by the same factor, so we can invert Equation 4.9.

```
class ProportionalScaling(private val carryingCharge: Double)
↳ : LotSizingMethod {
    override fun solve(portfolio: Portfolio, constraint:
↳ Constraint): ReplenishmentPlan {
        val free =
↳ AverageInvestment.measure(portfolio.freePlan())
        val ratio = free / constraint.limit
        val price = carryingCharge * (ratio * ratio - 1.0)
        return portfolio.planAt(AverageInvestment, price)
    }
}
```

4. Multi-Item and Constrained Systems

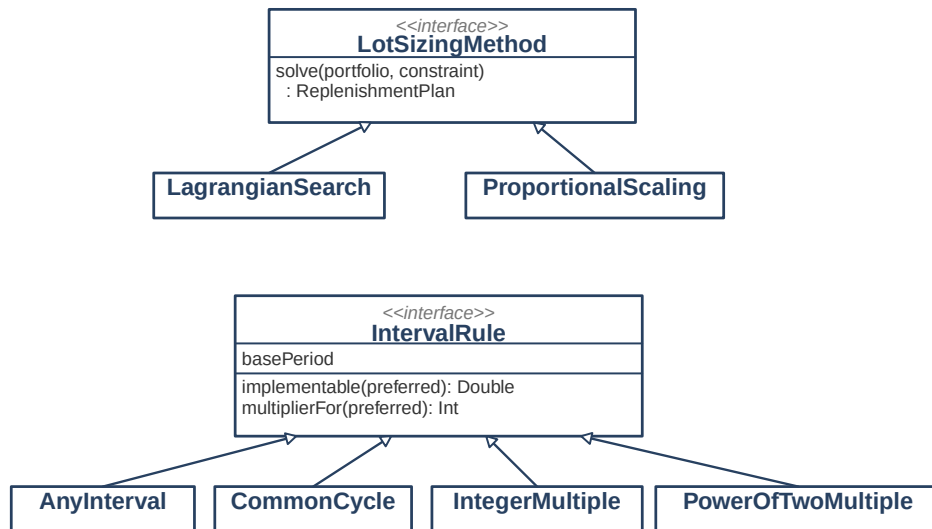


Figure 4.18.: The two strategy hierarchies. Each interface exists because the domain offers alternatives. Section 4.2 shows three ways to solve one constrained problem, of which two are implemented here, and Section 4.5.1 names four ordering practices. Notice that neither interface carries a setting that distinguishes its kinds.

4. Multi-Item and Constrained Systems

```
}
```

Notice that no search appears in it. It reproduces the multiplier of Section 4.2 to every digit the search finds, and it refuses to run when its preconditions fail. An interface with one implementation is speculation. The second implementation is what justifies this one.

Section 4.5.1 does not present one model with a setting. It presents four practices: order whenever you like, order everything on every cycle, order on whole multiples of a cycle, and order on powers of two. Each one answers the same question. Given the interval a location would prefer, what interval can the operation run?

```
interface IntervalRule {
    val name: String
    val basePeriod: Double

    /** The runnable interval nearest the one this SKU or
     *  ↪ block would prefer. */
    fun implementable(preferredInterval: Double): Double

    fun multiplierFor(preferredInterval: Double): Int =
        max(1, Math.round(implementable(preferredInterval) /
     ↪ basePeriod).toInt())
}

class PowerOfTwoMultiple(override val basePeriod: Double) :
    ↪ IntervalRule {
    override fun implementable(preferredInterval: Double):
        ↪ Double =
        (1 shl exponentFor(preferredInterval)) * basePeriod
}
```

4. Multi-Item and Constrained Systems

That question subsumes what look like two concepts. Coordinating a family and rounding an interval to a power of two appear to be different jobs, and they are one job with different answers. Thus, the arithmetic of Equation 4.28 is a function inside `PowerOfTwoMultiple` and not a class of its own, since it holds no state and there is only one way to do it. `multiplierFor` is written once on the interface, because under every rule the multiplier is the implementable interval divided by the base period.

We state the criterion behind both decisions once, since you will face the same question in every design you write.

Make a class when it holds state that varies independently of its arguments, or when it is one of several interchangeable ways of doing the same thing. A computation with neither property is a function.

Measured against that criterion, `LotSizingMethod` and `IntervalRule` are interfaces because they name interchangeable alternatives, `SpaceOccupied` is a class because it carries a volume map, and the power-of-two exponent is a function because it is neither.

4.8.7. Three Couplings and No Common Base

The temptation here is a `Model` interface with a `solve()` on it, so that all three couplings can be held in one list. Recall that Section 4.1 says these are three different kinds of interaction, and the mathematics agrees. One is a Lagrangian relaxation, one is a one-dimensional search over a base period, and one is a combinatorial partition. Figure 4.19 shows what they do share, the SKU and the plan. A common supertype would assert a kinship the chapter denies, and every caller would then have to ask what it was really holding.

The two network topologies do share a supertype. They differ in the shape of one map, from a location to the location that supplies it. A chain points

4. Multi-Item and Constrained Systems

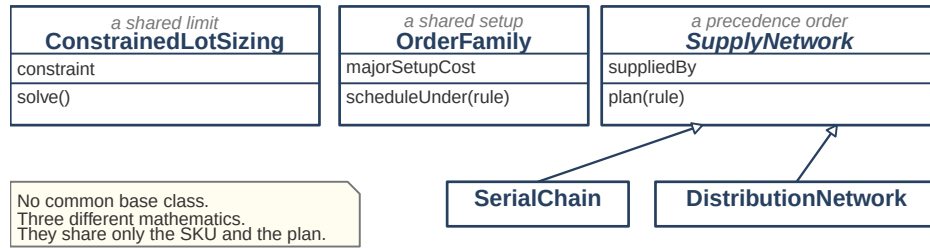


Figure 4.19.: The three couplings of this chapter as three unrelated classes. A shared limit, a shared setup and a precedence order are different mathematics, so they have no common supertype. The two network topologies do share one, since they differ only in the shape of a single map.

each location at the one above it, and a distribution network points every region at the centre.

```

abstract class SupplyNetwork(
    val skus: List<SKU>,
    private val suppliedBy: Map<SKU, SKU>,
) {

    /** Equation 4.35: the value this location adds, not the
     ↪ value accumulated through it. */
    fun echelonRate(sku: SKU): Double =
        sku.holdingRate - (suppliedBy[sku]?.holdingRate ?:
    ↪ 0.0)

    /** The intervals that solve the relaxed problem, before
     ↪ any rounding. */
    abstract fun relaxedIntervals(): List<Double>

    /** The relaxed intervals made runnable, one rule applied
     ↪ to every location. */
}

```

4. Multi-Item and Constrained Systems

```
fun plan(rule: IntervalRule): List<Double> =  
    ↪ relaxedIntervals().map { rule.implementable(it) }  
}
```

One expression serves both structures. It gives $h_i - h_{i+1}$ down a chain and $h_i - h_0$ across a distribution network, and it gives a location with no supplier its own rate, which Equation 4.31 states as a separate case. Notice that `echelonRate` sits on the network and not on the SKU. An echelon rate is a difference between neighbours, and a SKU that knew its neighbour would record the topology twice.

The two subclasses override the one thing that differs, the algorithm. `SerialChain` implements Algorithm 4.1 and `DistributionNetwork` implements the scan of Section 4.6.3. Section 4.7 built a worksheet for the second and not for the first, and the reason survives the change of language. The scan has a step count known in advance and the partition does not.

4.8.8. How the Objects Solve It

A class model says what exists. It does not say what happens, so we trace S1 of Table 4.24 from the caller to the answer.

The four middle messages of Figure 4.20 repeat until the bracket closes.

```
class LagrangianSearch(  
    private val precision: Double = 1.0E-12,  
    private val maximumIterations: Int = 400,  
) : LotSizingMethod {  
  
    override fun solve(portfolio: Portfolio, constraint:  
        ↪ Constraint): ReplenishmentPlan {  
        val measure = constraint.measure  
        var lo = 0.0
```

4. Multi-Item and Constrained Systems

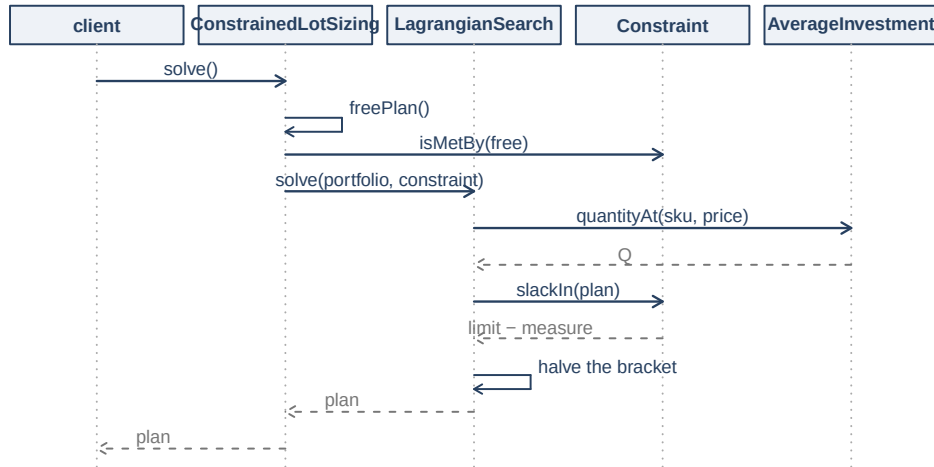


Figure 4.20.: Solving under a limit. The study computes the free plan and asks the constraint whether the limit binds, and the method takes over if it does. Notice that the search never computes an order quantity itself. It asks the measure, hands the plan back to the constraint for a slack, and halves its bracket on the sign of the answer. The dashed arrows are returns.

4. Multi-Item and Constrained Systems

```
var hi = 1.0
while (constraint.slackIn(portfolio.planAt(measure,
↪ hi)) < 0.0) {
  hi *= 2.0
  check(hi < 1.0E12) {
    "No price high enough to meet the
    ↪ ${measure.name} limit of
    ↪ ${constraint.limit}"
  }
}
repeat(maximumIterations) {
  val mid = 0.5 * (lo + hi)
  if (hi - lo < precision) return@repeat
  if (constraint.slackIn(portfolio.planAt(measure,
    ↪ mid)) < 0.0) lo = mid else hi = mid
}
return portfolio.planAt(measure, 0.5 * (lo + hi))
}
```

Notice what the search does not know. It never writes Equation 4.5, never names a budget, and never asks what the measure is. It knows that raising the price lowers consumption, which Section 4.2.3 proves, and it relies on nothing else. Thus, one search serves a working capital account, a warehouse floor and a purchasing department, and `quantityAt` belongs on the measure.

Notice also the first two lines of the loop. The search widens its bracket until the slack changes sign. Section 4.7.1 observed that Goal Seek does not take that step, since it starts from whatever is already in the cell. Neither tool is safe on its own, and both rest on the same proof. They differ in how much of the search they arrange for you.

4. Multi-Item and Constrained Systems

Section 4.8.1 said that solving under a limit and tracing the frontier would turn out to be one thing. Compare the two.

```
fun at(shadowPrice: Double): ReplenishmentPlan {
    val quantities = priced.skus.map {
        ↪ AverageInvestment.quantityAt(it, shadowPrice) }
    return portfolio.planFor(quantities, shadowPrice)
}
```

That is the body of the search's inner step with the search removed, because the exchange curve is handed the price instead of finding it. Same objects, same message, one fewer participant. Had these two restated Equation 4.5 separately, the design would have failed at the exact place where Section 4.4 says the two problems meet.

One subtlety deserves a line. `priced` is the portfolio with every holding rate set to zero, because Section 4.4 has the multiplier replace the carrying charge instead of adding to it. Expressing that as a property of the SKUs keeps `AverageInvestment.quantityAt` the only place a quantity is computed.

A point on the curve is not a new kind of thing. `at` returns a `ReplenishmentPlan`, the type the constrained solve returns, because a point on the exchange curve is a portfolio evaluated at a price. It already has an investment and an ordering cost, the two axes of Figure 4.1. A `CurvePoint` class would add a noun the chapter never uses and force a conversion at every boundary. Before adding a type for a result, check whether the result is a familiar object under new conditions.

Two smaller decisions follow. The curve returns a `Sequence` and not a `List`, so a thousand-point sweep evaluates one point at a time and holds one. And the variety index J^* of Equation 4.19 is computed once on the curve, since it depends only on the items. Putting it on each point would compute the same number a thousand times and suggest that it varies along the curve.

4.8.9. Solving the Chapter's Examples

A design is judged by what it is like to use. We take the five scenarios of Table 4.24 in order and solve the chapter's own examples with it. Every figure below is produced by running the code, in the way Section 3.12 produces the figures of Chapter 3.

Table 4.27.: What to build for each model of this chapter, and what to ask it. Notice that the middle column never names a measure or a rule, because those are handed in, and are not chosen by the type.

To answer	Build	Then ask for
Section 4.2, a limit	Portfolio, Constraint, ConstrainedLotSizing	binds, solve(), priceOfTheConstraint()
Section 4.3, what it cost	the same study	priceOfTheConstraint(), quantityRatios()
Section 4.4, the frontier	ExchangeCurve	constant, varietyIndex, at(price)
Section 4.5, a shared setup	OrderFamily, an IntervalRule	costOf(rule), bestBasePeriod, scheduleUnder(rule)
Section 4.6.2, a chain	SerialChain	blocks(), relaxedIntervals(), plan(rule)
Section 4.6.3, a network	DistributionNetwork	ranked(), pinnedSet(), centralInterval(), plan(rule)

Example 4.1 is three items, a carrying charge of 30% per year, and \$10,000

4. Multi-Item and Constrained Systems

of working capital. We build the portfolio and the constraint, then hand both to a study.

```
val skus = listOf(  
    SKU.pricedAt("Item 1", demandRate = 12000.0, unitValue =  
    ↪ 20.0,  
        orderCost = 50.0, carryingCharge = 0.30),  
    SKU.pricedAt("Item 2", demandRate = 25000.0, unitValue =  
    ↪ 10.0,  
        orderCost = 50.0, carryingCharge = 0.30),  
    SKU.pricedAt("Item 3", demandRate = 8000.0, unitValue =  
    ↪ 15.0,  
        orderCost = 50.0, carryingCharge = 0.30),  
)  
val storeroom = Portfolio(skus)  
val budget = Constraint(AverageInvestment, limit = 10000.0)  
val study = ConstrainedLotSizing(storeroom, budget)
```

Three lines of construction, and nothing has been computed yet. We now ask the study what it was built for.

```
study.binds // true  
val held = study.solve()  
held.shadowPrice // 0.146430  
held.measuredBy(AverageInvestment) // 10000.0000  
held.measuredBy(RelevantCost) // 7464.2984  
study.priceOfTheConstraint() // difference  
    ↪ 145.0375, 1.9816%  
study.quantityRatios() // 0.819755  
    ↪ three times
```

Every figure is Example 4.1's. Notice that we ask `measuredBy` for two different measures of one plan. One of them is the limited quantity and

4. Multi-Item and Constrained Systems

the other is the objective, and the plan does not know which one was constrained.

Changing the constraint changes one line. Example 4.2 is the same distributor with ordering costs that differ across items and a dock that can take 50 receipts a year, so `dockSkus` holds the data of Table 4.3.

```
val dock = ConstrainedLotSizing(  
    Portfolio(dockSkus),  
    Constraint(ReplenishmentWorkload, limit = 50.0),  
)
```

free orders per year	85.1341				
free relevant cost	6815.3324				
shadow price	64.3293	per order			
Item 1	k+theta 114.3293	held	676.2523	ratio	1.5121
Item 2	k+theta 84.3293	held	1185.5328	ratio	2.0534
Item 3	k+theta 144.3293	held	716.3594	ratio	1.3432
orders per year	50.0000				
relevant cost	7621.2652				
the limit cost	805.9328	(11.8253%)			

Section 4.1 called a budget and a dock two of the four things an organization runs short of. Here that claim is discharged by one word in one argument. `ConstrainedLotSizing`, `LagrangianSearch` and `Portfolio` are untouched. The measure changed, and with it the sign of the marginal, the term the multiplier joins, and the fact that the quantities grow instead of shrinking.

Look next at the ratios. Under the budget every item shrank by the same 0.819755, and here they grow by 1.51, 2.05 and 1.34. Thus, the code reproduces the distinction of Section 4.3, that a budget rescales while a dock reallocates, and no type in the program knows that the distinction exists.

4. Multi-Item and Constrained Systems

Where a closed form exists, we pass it as a third argument.

```
ConstrainedLotSizing(storeroom, budget,
  ↪ ProportionalScaling(carryingCharge = 0.30))
```

The search returns 0.14642984 and the closed form returns 0.14642984. For example, asking `ProportionalScaling` for a space limit raises an error instead of returning a wrong number, since Section 4.3 holds only under a budget with $h_j = ic_j$.

Example 4.5 is nine items in a utility storeroom, held in a `Portfolio` named `utility`.

```
val curve = ExchangeCurve(utility)
curve.constant //
  ↪ 195,147,265.65
curve.decomposedConstant //
  ↪ 195,147,265.65
curve.varietyIndex //
  ↪ 5.6813
curve.investmentAt(0.25) // 27,939.02
curve.orderingCostAt(0.25) // 6,984.76
```

The two routes to the constant, Equation 4.18 and Equation 4.22, agree to the cent. That is the arithmetic check the worksheet performs in rows 15 and 16. Sweeping the price illustrates the claim of Equation 4.14.

price 0.10	investment	44175.48	ordering	4417.55	product	195147265.65
price 0.25	investment	27939.02	ordering	6984.76	product	195147265.65
price 0.40	investment	22087.74	ordering	8835.10	product	195147265.65

Example 4.7 is four items on one truck with a \$400 major setup. Here the rule is what varies, so the rule is what we pass.

4. Multi-Item and Constrained Systems

```
val family = OrderFamily(Portfolio(supplierSkus),
    ↪ majorSetupCost = 400.0)
family.independentCost // 8422.39
family.costOf(CommonCycle(family.commonCycleInterval))
val best = PowerOfTwoMultiple(family.bestBasePeriod {
    ↪ PowerOfTwoMultiple(it) })
family.costOf(best)
family.scheduleUnder(best).nestsOn(best.basePeriod)
```

independent	8422.39		
common cycle	6497.54	T = 0.14621 year (53.4 days)	
integer multiple	6402.06	T = 0.14214 year (51.882 days)	m = [1, 1, 2,
power of two	6403.34	T = 0.14192 year (51.800 days)	m = [1, 1, 2,
nesting costs	0.0200%		
the schedule nests	true		

Notice that `bestBasePeriod` takes a function from a base period to a rule, and not a rule itself. The search runs over base periods and every rule carries one, so the caller has to say how to build a rule at each candidate. That is the one place where the shape of the interface imposes something on a caller.

The last line checks the chapter's argument instead of accepting it. Section 4.5.2 justifies powers of two by nesting, and `nestsOn` asks the answer whether it nests.

Example 4.9 is five stages and Example 4.10 is a warehouse feeding six regions. We build them differently and ask them the same questions.

```
val chain = SerialChain(stages)
chain.blocks() // {1,2} at
    ↪ 0.5933, {3,4,5} at 0.6234
chain.costOf(chain.relaxedIntervals()) // 183.25
```

4. Multi-Item and Constrained Systems

```
val weekly = PowerOfTwoMultiple(1.0 / 52.0)
chain.costOf(chain.plan(weekly))           // 183.31

val network = DistributionNetwork(warehouse, regions)
network.ranked()                           // 5, 3, 6, 4,
  ↪ 1, 2
network.pinnedSet()                        // 6, 4, 1, 2
network.centralInterval()                  // 0.343049
network.plan(weekly)                       // every
  ↪ location at 0.307692
```

The same `weekly` object goes to a chain and to a network, and neither asks what it is. An interval rule turns a preferred interval into a runnable one, and a chain block and a warehouse both have a preferred interval.

No listing in this section branches on which constraint, which method, which ordering practice or which topology is in play. The `if` that a first implementation would write in each of those places is instead the choice of which object we constructed, made once, at the top.

4.8.10. Checking the Design Against the Mathematics

A design that cannot be checked is an opinion. Two things make this one checkable.

The first is Table 4.28, in which every result of this chapter has one home in the code.

4. Multi-Item and Constrained Systems

Table 4.28.: Every result of this chapter and the one place that implements it. A row with no home, or a class with no row, is a defect in one of the two.

The mathematics	Where it lives
Equation 3.17	<code>SKU.economicOrderQuantity</code>
Equation 4.2	<code>AggregateMeasure.measure</code> , <code>Constraint.isMetBy</code>
Equation 4.5	<code>ConstrainableMeasure.quantityAt</code>
Equation 4.6	<code>LagrangianSearch</code> , over <code>Constraint.slackIn</code>
Equation 4.9	<code>ProportionalScaling</code>
Equation 4.14, Equation 4.19	<code>ExchangeCurve</code>
Equation 4.23, Equation 4.25	<code>OrderFamily</code> , <code>IntegerMultiple</code>
Equation 4.28, Equation 4.30	<code>PowerOfTwoMultiple</code>
Equation 4.31	<code>SupplyNetwork.echelonRate</code>
Algorithm 4.1	<code>SerialChain.blocks</code>

Read Table 4.28 in both directions. Downward it says where to look when a number is wrong. Upward it tests the design, because every class in the package appears in the table or collaborates with something that does. A class in neither would be doing something this chapter never asked for.

The second is a test suite. Three of its habits transfer to any model you build.

Every figure the chapter prints is pinned by a test, so that a change which moves an answer fails instead of going quiet. Every figure, and not most of them. For example, a test of a limit on replenishments written against a portfolio whose ordering costs are all equal will pass, and it will still check nothing that Example 4.2 demonstrates. Such a portfolio cannot exhibit the reallocation, because a constant added to equal ordering costs changes them equally. Notice that the test would look correct.

4. Multi-Item and Constrained Systems

The tests reproduce the workbook and not only the examples. They work the shipped workbook of Section 4.7 as well as the chapter's own instances. Recall from Section 3.12 that this caught a real defect in the previous chapter, where a ranking came out identical under two different wrong objectives and only the workbook's data told them apart.

The tests check the contract and not only the answers. Each `ConstrainableMeasure` is checked by comparing `marginalAt` against a numerical derivative of its own `measure`. A sign slip there still produces quantities and still converges, so nothing else would catch it. Each closed-form `quantityAt` is checked against the general default of the interface, which demonstrates that a new measure really does need only two methods.

One question separates a design from an arrangement. What does the next requirement cost? A fifth measure, say a limit on handling hours, is a new class with two methods and no edit anywhere. A fifth ordering practice is a new `IntervalRule`. A third solution method is a new `LotSizingMethod`, and a third network topology is a subclass supplying one map and one algorithm. None of those changes an existing class.

4.9. Summary

Recall that Chapter 3 decided one item at a time. This chapter kept every assumption of that chapter except the one that made it easy.

Three kinds of coupling, three kinds of mathematics. A shared resource couples items through a constraint and leaves their cost functions separate. A shared setup couples their cost functions and leaves them free of constraints. An echelon structure couples them end to end, so that one stock exists in order to replenish another. Thus, the first is solved by moving the constraint into the objective, the second by imposing structure on the schedule, and the third by fixing the accounting before either technique can be applied.

4. Multi-Item and Constrained Systems

The multiplier is the economics, not the arithmetic. The multiplier θ^* is the rate at which relaxing the limit reduces cost. Thus, it says what another dollar of budget or another square foot is worth, and when it stops being worth asking for.

Where it enters depends on the constraint, and the three cases should be kept separate. Under a limit on stock the multiplier is added to the holding rate, as a surcharge on whatever consumes the scarce resource. Under a limit on replenishments it is added to the ordering cost instead, and carries units of dollars per order. Where holding cost does not appear in the objective at all, as when management fixes an investment target, the multiplier takes its place and *is* the implied carrying charge.

A budget constraint rescales and does not reallocate. When holding cost is a carrying charge on value, every order quantity shrinks by the same factor $\sqrt{i/(i + \theta)}$. That is, the unconstrained solution already had the items in the right proportion, and the constraint asks only that everything be smaller. You should treat this as specific to a budget. A space constraint, whose coefficients have nothing to do with value, reallocates, and so does a limit on replenishments, because a constant added to unequal ordering costs changes them unequally.

Aggregate performance lies on a curve fixed by the item data. Investment and replenishment frequency cannot be chosen separately. Their product is determined by the item data and their ratio is k/γ . Thus, a portfolio above the curve can improve on both measures at once, and a portfolio on the curve is asserting a cost ratio. Comparing that assertion against estimated costs is a check on practice that needs no new data.

When ordering costs differ across items the same curve survives, in the form $\frac{1}{2}J^*K^w\lambda^ac^w$. Four things therefore set the frontier: how varied the items are, what an order costs, how much is demanded, and what the goods are worth.

Coordination is worth more than cleverness. In Example 4.7, sharing orders at a single common cycle captured 23 of the 24 percentage points

4. Multi-Item and Constrained Systems

available, and differentiating the multipliers added one. Restricting those multipliers to powers of two, which is what makes a schedule nest and therefore what makes it runnable, cost two hundredths of a percent against a worst case of 6.07%. Recall that this worst case is Equation 3.22 applied to cycle lengths. Thus, the flatness that made case quantities affordable in Section 3.7 is the same fact that makes power-of-two schedules affordable here.

Echelons need their own accounting, and then behave like everything else. A unit held at a downstream stage has already been paid for upstream, so charging every stage its installation rate counts the same money twice. Echelon stock and the value-added rate $h'_i = h_i - h_{i+1}$ fix that. Once fixed, every stage looks like an EOQ in the interval form of Equation 4.26.

What remains is the ordering constraint. Stages in a chain must not replenish faster than what they supply, and locations in a distribution system must not wait longer than the warehouse that feeds them. Both are solved the same way, by merging the locations that would violate the constraint into blocks that share an interval and then rounding each block to a power of two. Notice that in Example 4.9 and Example 4.10 the rounding put every location on one schedule, because intervals within a factor of two often collapse to the same power.

Two implementations, and what each one makes visible. Section 4.7 built these models as worksheets and Section 4.8 built them as a program, and the two were kept apart because they teach different things. A worksheet makes a method visible: a formula is a cell and a layout is a claim about the model that anybody can see. A program makes a structure visible instead, and only to whoever wrote down the analysis behind it. Notice that the same four questions were answered on both sides. What belongs to an item and what belongs to the problem, whether a difference is a value or a kind, whether a result needs a new name, and when a search may be trusted.

4. Multi-Item and Constrained Systems

Throughout this chapter demand has remained known and constant. In the next chapter we let it vary over time while keeping it known, which is the subject of Chapter 5. From Chapter 7 onward it stops being known at all, and Chapter 10 returns to the serial and distribution structures of this chapter once it does.

4.10. Exercises

Unless an exercise says otherwise, use these conventions so that your answer and the instructor's agree. Take the time unit to be one year. Report order quantities and reorder intervals to four decimal places, multipliers and reorder intervals in weeks to two, costs to the nearest cent, and multipliers, ratios and indices to six significant figures. Where an exercise asks for a bisection, iterate until the constraint is met to within one part in a million of the limit. Round an order quantity to an integer only when the exercise asks for the quantity that would actually be ordered.

Exercise 4.1. Four items are stocked, each costing \$40 to order, with a carrying charge of 25% per year.

Item	λ_j	c_j
1	6,000	\$30
2	15,000	\$8
3	2,400	\$50
4	40,000	\$2

Determine the unconstrained order quantities and the average investment they require. Then impose a budget of \$6,000 on average investment, find θ^* by bisection on Equation 4.6, and report the constrained quantities and the annual cost of the constraint.

4. Multi-Item and Constrained Systems

Verify Equation 4.9 by computing the ratio of each constrained quantity to its unconstrained value, and confirm that all four ratios agree.

Exercise 4.2. Repeat Exercise 4.1 with a **space** constraint instead of a budget. Item j occupies v_j cubic feet per unit, with $v_1 = 0.5$, $v_2 = 0.2$, $v_3 = 1.5$, and $v_4 = 0.05$, and 400 cubic feet are available.

Determine the constrained quantities, then compute the ratio of each to its unconstrained value as you did before.

Explain why the ratios are no longer all equal, and identify the property of the budget constraint that Equation 4.9 depends on.

Exercise 4.3. The multiplier is a shadow price. For the item set of Exercise 4.1, compute the minimum relevant cost at budgets of \$5,000, \$6,000, \$7,000, \$8,000, and at the unconstrained investment.

Plot the minimum cost against the budget, and estimate the slope at \$6,000 numerically. Compare it with $-\theta^*$ from Exercise 4.1 and explain any difference.

State the budget at which the multiplier reaches zero, and say what that budget is in terms of the unconstrained solution.

Exercise 4.4. A firm places 250 replenishment orders a year across a group of items and carries \$1,000,000 of cycle stock. Its accountants report a carrying charge of 22% per year, and a study of the purchasing department estimates the ordering cost at \$120 per order.

Assuming the firm is on its exchange curve, determine the cost ratio its practice implies and compare it with the ratio its stated costs imply.

State which direction along the curve the firm should move, whether that means ordering more often or less often, and what it would do to the stock it carries. Then state the one circumstance under which the comparison would tell you nothing.

4. Multi-Item and Constrained Systems

Exercise 4.5. For the nine items of Example 4.5, verify by direct computation that $\bar{I}^a \cdot \overline{OC}^a$ is the same at $\theta = 0.10$ and at $\theta = 0.50$.

Then compute the variety index from Equation 4.19, and recompute it after deleting the pad-mount transformer from the portfolio. Report both values and explain what the change says about that item's contribution to aggregate performance.

Finally, compute λ^a , C^a , c^w , and K^w for the full portfolio and confirm that Equation 4.22 reproduces the constant you verified above.

Exercise 4.6. For the four items of Example 4.7, suppose the major setup cost falls from \$400 to \$50 because the supplier introduces electronic ordering.

Recompute the independent, common-cycle, and best integer-ratio policies. Report the new multipliers and say what has happened to the value of coordinating.

Explain, in terms of Equation 4.23, why a small major setup cost pushes the solution back toward independent ordering.

Exercise 4.7. Show that for a cost of the form $C(T) = k/T + hT/2$, choosing T to be $\sqrt{2}$ times its optimal value costs exactly the same as choosing it to be $1/\sqrt{2}$ times that value, and that both cost the 6.07% of Equation 4.30.

Then explain why this is the worst case for a power-of-two policy, and why a policy whose base period may itself be chosen does better than 6.07%.

Exercise 4.8. The items of Example 4.7 are reordered under a common cycle policy, which Example 4.7 shows captures most of the available saving.

Compute, for each of the four items separately, the penalty its own cost incurs under the common cycle relative to the cycle it would choose

4. Multi-Item and Constrained Systems

under the power-of-two policy. Report which item bears the cost of the simplification and by how much.

Then state the management argument for adopting the common cycle anyway, and the argument against.

Exercise 4.9. A three stage chain has $\lambda = 1,000$ units per year and installation holding rates $h_1 = \$0.80$, $h_2 = \$0.55$, $h_3 = \$0.20$ per unit per year.

Compute the echelon holding rates h'_i and verify Equation 4.32 by showing that $h'_1 + h'_2 + h'_3$ recovers h_1 .

Then, at an instant when the on-hand quantities are 40, 90, and 25 units at stages 1, 2, and 3, report the echelon stock at each stage and the total value-added holding cost rate being incurred, and confirm that it equals what installation accounting would charge.

Exercise 4.10. A four stage chain, $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$, runs at $\lambda = 800$ units per year with a weekly base period.

Stage i	1	2	3	4
k_i	\$18	\$40	\$9	\$30
h_i	\$1.00	\$0.70	\$0.30	\$0.10

Compute h'_i and g_i , then show that the stage-by-stage intervals $\sqrt{k_i/g_i}$ violate Equation 4.39.

Run the algorithm of Section 4.6.2 to find the blocks, report $T(r)$ for each, and round each to a power of two. State the cost of the relaxed solution, the cost of the power-of-two solution, and the penalty between them.

4. Multi-Item and Constrained Systems

Exercise 4.11. In Example 4.10 the ordering cost at the central warehouse is \$500. Suppose it falls to \$150.

Rebuild $\mathcal{C}^0$ and report which regions are now pinned to the central warehouse and which are left alone.

Explain, in terms of the test in Section 4.6.3, why lowering k_0 pulls regions into $\mathcal{C}^0$ rather than releasing them, and say what that means physically about a central warehouse that becomes cheap to replenish.

Then work the comparison in the other direction and identify the value of k_0 at or above which no region is pinned at all.

Exercise 4.12. Both Example 4.9 and Example 4.10 end with every location on a single power-of-two interval, even though the relaxed solution gave each block or region a different one.

Explain why this happens, and state exactly when two relaxed intervals round to the same power of two. Then show that a ratio below two is not sufficient, using your own answer to Exercise 4.10 as the counterexample.

Finally, modify the data of Example 4.10 so that the pinned set and region (1) round to different powers of two, and report the intervals that result.

5. Dynamic Lot Sizing

i Learning objectives

After reading this chapter you should be able to:

- explain why the economic order quantity argument fails when the demand rate changes over time
- formulate the lot sizing problem for time-varying deterministic requirements
- state the zero-inventory ordering property and use it to reduce the problem to a choice of order periods
- derive the window cost of a replenishment and compute it for every pair of periods
- solve the problem to optimality with the Wagner-Whitin recursion and read the plan off the traceback
- formulate the same problem as a mixed integer program
- apply the Silver-Meal, least unit cost, and part-period balancing heuristics
- evaluate the cost penalty of a heuristic and say when a heuristic pays at all
- explain what a rolling horizon does to all of these methods

Chapter 3 decided one order quantity and used it forever. That was possible because the demand rate never changed. The stock fell at the same slope, the cycles were identical, and a cost per unit time could be written for a single cycle and minimized. Chapter 4 removed the assumption that items could be decided independently, but it left the constant rate in place.

5. Dynamic Lot Sizing

In this chapter we remove the constant rate. Demand is still known in advance and still deterministic, but it now varies from period to period. We take up how to state the problem, how to solve it exactly, how to solve it quickly, and what happens to all of that when the horizon slides forward a period at a time.

5.1. When the Demand Rate Changes

A distributor of turf-equipment parts stocks a mower deck belt. The belt sells almost nowhere in the winter, very heavily in the spring, and then again in the autumn as dealers build up for the following season. The forecast for the coming twelve months is given in Table 5.1.

Table 5.1.: The monthly requirement for the mower deck belt.

t	1	2	3	4	5	6	7	8	9	10	11	12
d_t	40	60	120	300	420	260	120	40	0	80	180	220

The total is 1,840 belts, so the average requirement is 153.33 belts a month. It costs \$300 to place an order, the belt costs \$50, and the carrying charge is 2% per month, which by Section 1.4.4 makes $h = ic = \$1.00$ per belt per month.

Nothing stops us from applying Equation 3.17 to the average, and we get

$$Q^* = \sqrt{\frac{2(300)(153.33)}{1.00}} = 303.3 \text{ belts,}$$

That is a supply of about two months. The trouble is that the answer means very little. Two months of supply is nine months of stock in February and less than one month of stock in May. The quantity was derived from a picture of inventory that this item never has: a sawtooth of identical triangles.

5. Dynamic Lot Sizing

Let's look at where the classical argument actually breaks. It minimizes cost per unit time over one cycle, and it can do that because every cycle is the same. Here there is no repeating cycle. An order placed in month 4 covers a different amount of demand, over a different length of time, at a different carrying cost, than an order placed in month 9. There is no single cycle whose cost can stand for all of them, so there is nothing for the calculus to minimize.

What replaces it is a finite horizon and a decision in every period of it. We are no longer looking for a quantity. We are looking for a schedule.

5.1.1. The Requirements Schedule

The horizon runs over N periods, indexed $t = 1, 2, \dots, N$. In each period we know

- d_t , the requirement in period t ,
- k_t , the cost of placing an order in period t ,
- c_t , the unit purchase cost in period t , and
- h_t , the cost of carrying one unit from the end of period t into period $t + 1$.

Together these are the *requirements schedule* for the item. The subscripts are carried through the derivations because nothing in them requires the costs to hold still. In every example of this chapter $k_t = k$, $c_t = c$ and $h_t = h$, and the subscript is dropped whenever that is so.

It is convenient to name the total requirement over a stretch of periods. Write

$$d[t, u] = \sum_{s=t}^u d_s$$

for the requirement of periods t through u inclusive, with $d[t, u] = 0$ when $u < t$. For the belt, $d[1, 3] = 220$ and $d[5, 9] = 840$.

5. Dynamic Lot Sizing

5.1.2. The Assumptions

The model of this chapter makes the following assumptions.

1. A single item is planned, independently of every other item.
2. The requirements d_t are known for all N periods and are not random.
3. The horizon is finite and ends at period N .
4. Demand within a period is satisfied from stock on hand at the start of that period, so a replenishment arriving in a period can cover that period.
5. Replenishment is instantaneous, and the lead time is zero or a known constant.
6. No shortages are allowed. Every requirement is met in the period it falls.
7. There is no limit on the order quantity and no limit on storage.
8. The starting inventory is zero, and nothing is left over at the end of the horizon.
9. The order cost k_t does not depend on the size of the order.
10. Carrying cost is charged on the inventory left at the end of a period.

Assumptions 1 through 3 place the chapter. Assumption 1 is what Chapter 4 removed and this chapter puts back, so that the time dimension can be studied on its own. Assumption 2 is the strong one, and Section 5.7 is about what to do because it is false.

Assumption 4 matters more than it looks. A unit that arrives in period t and is consumed in period t is never held, so it is charged nothing. Some treatments charge holding on the average of the beginning and ending balances instead, which gives a different answer to the same problem. We charge the ending balance, and that is the convention an MRP record uses and the one Chapter 6 will inherit.

Assumptions 8 and 10 are what make the horizon self-contained. Starting and ending empty means the plan neither inherits nor bequeaths a position,

5. Dynamic Lot Sizing

so the cost we compute is the cost of the horizon and not a share of something longer.

5.1.3. Where This Chapter Sits

Chapter 3 produced one quantity that applied for all time. This chapter produces a quantity for each of N periods, and most of them are zero. Chapter 6 then runs the methods of this chapter once at every level of a bill of material, where the requirements are not forecast at all but computed from the level above. That is the setting in which these methods are most often used in practice, and it is the reason a chapter on deterministic time-varying demand is not a curiosity.

5.2. What a Plan Costs

Recall that a plan is a quantity $Q_t \geq 0$ for each period. Given the plan, the inventory follows from the balance

$$I_t = I_{t-1} + Q_t - d_t, \quad t = 1, \dots, N, \quad (5.1)$$

with $I_0 = 0$. The no-shortage assumption is the requirement that $I_t \geq 0$ for every t , and starting and ending empty makes $I_0 = I_N = 0$.

The cost of the plan has three parts. Let $y_t = 1$ if an order is placed in period t and $y_t = 0$ otherwise. Then

$$TC = \underbrace{\sum_{t=1}^N k_t y_t}_{\text{ordering}} + \underbrace{\sum_{t=1}^N c_t Q_t}_{\text{purchase}} + \underbrace{\sum_{t=1}^N h_t I_t}_{\text{carrying}}. \quad (5.2)$$

5. Dynamic Lot Sizing

5.2.1. The Purchase Term

Section 1.4.1 argued that the purchase term can be dropped, because every feasible policy buys the same total amount and so pays the same for it. That argument is exactly right, and it depends on something this chapter can remove.

Suppose $c_t = c$ for all t . Assumptions 6 and 8 force $\sum_t Q_t = \sum_t d_t$, so the purchase term is $c \sum_t d_t$ no matter how the orders are arranged. It is a constant, it cannot affect which plan is best, and we may as well leave it out. What is left,

$$TRC = \sum_{t=1}^N k_t y_t + \sum_{t=1}^N h_t I_t, \quad (5.3)$$

is the *relevant cost*, and every figure quoted in this chapter is a relevant cost unless it says otherwise.

Now suppose the price changes. If the belt costs \$50 through June and \$54 afterwards, then buying in June for a July requirement is worth \$4 a belt, and the purchase term is no longer the same under every plan. It has become part of the decision. This is the one place in the book where the argument of Section 1.4.1 does not hold, and the derivations below keep c_t so that the case is available when it is needed. Exercise 5.15 works an instance with period-varying costs, where the answer does depend on the purchase term.

5.2.2. Three Plans Without Any Theory

Before any method, let's ask what a planner would do with Table 5.1 and no inventory theory at all. Three answers suggest themselves, and all three are used in practice.

Lot-for-lot orders exactly what each period needs: $Q_t = d_t$. Nothing is ever carried, because every unit arrives in the period it is consumed. It

5. Dynamic Lot Sizing

is the plan an MRP system produces by default, and it is the right plan when ordering is nearly free.

An adjusted economic order quantity computes Q^* from the average requirement, as we did above, and then covers the whole number of periods whose requirement comes closest to Q^* . It keeps the quantity roughly steady and lets the timing float.

The adjustment is the whole of the difference between this rule and a stricter one that orders exactly Q^* every time, whatever the period boundaries are. That stricter rule is the one usually called a fixed economic order quantity, and it does not appear in this chapter for a reason worth stating. On the belt it orders 303.3150 belts seven times over, which is 2,123 belts against a requirement of 1,840, and it finishes the year holding 283 of them. Assumption 8 says the horizon ends empty, and no rule that orders in a quantity unrelated to the requirements can promise that. Exercise 5.11 works it out.

A periodic order quantity does the reverse. It converts Q^* into a time supply, $T_{EOQ} = Q^*/\bar{d}$, rounds it to a whole number of periods, and then orders that many periods of requirement every time. Here $T_{EOQ} = 303.3/153.33 = 1.98$, which rounds to two months, so the rule orders in months 1, 3, 5, 7, 9 and 11.

Example 5.1 (Costing a plan by hand). Let's begin with lot-for-lot on the belt. Eleven of the twelve months have a requirement, so eleven orders are placed, at \$300 each. Every order arrives in the month it is consumed, so $I_t = 0$ in every month and nothing is carried. The relevant cost is \$3,300, all of it setup.

Now we take the periodic rule with $T = 2$. The order in month 1 covers months 1 and 2, so $Q_1 = 40 + 60 = 100$. At the end of month 1 there are $100 - 40 = 60$ belts left, and at the end of month 2 there are none. The ledger runs

5. Dynamic Lot Sizing

Table 5.2.: The periodic order quantity plan at a two month time supply.

t	1	2	3	4	5	6	7	8	9	10	11	12
d_t	40	60	120	300	420	260	120	40	0	80	180	220
Q_t	100	0	420	0	680	0	160	0	80	0	400	0
I_t	60	0	300	0	260	0	40	0	80	0	220	0

Six orders cost \$1,800. The ending balances sum to 960 belt-months, which at \$1.00 a belt-month costs \$960. The relevant cost is \$2,760.

Notice that the periodic rule pays \$960 to save \$1,500 of setup against lot-for-lot. That is a good trade on this item. Look also at where the money goes. Month 5 needs 420 belts and month 6 needs 260, so the order in month 5 carries 260 belts for a month, which alone costs \$260. An order in month 6 would have avoided that and cost \$300. The two are close, and that hints that the best plan lies near this one.

Thus, we have three plans and no reason yet to prefer any of them. Their costs are collected in Table 5.3.

Table 5.3.: Three plans a planner would reach for without any theory.

Plan	Orders	Setup	Carrying	Relevant cost
Lot-for-lot	11	\$3,300	\$0	\$3,300
Adjusted economic order quantity	6	\$1,800	\$800	\$2,600
Periodic order quantity, $T = 2$	6	\$1,800	\$960	\$2,760

None of the three is optimal, and Section 5.4 will show that the best plan costs \$2,480. Before we can look for it we need to know how many plans there are.

5.3. Two Properties That Make the Problem Small

The plan is a vector of N non-negative numbers, so the search space is uncountable. Two properties cut it down to something a computer, and sometimes a person, can enumerate.

5.3.1. Zero-Inventory Ordering

An optimal plan never orders in a period that starts with stock on hand. That is, $I_{t-1}Q_t = 0$ for every t .

The argument is an exchange. Suppose a plan orders $Q_t > 0$ in a period that begins with $I_{t-1} > 0$ units on hand. Let $\delta = \min(I_{t-1}, Q_t)$, which is positive. Those δ units were bought in some earlier period $r < t$ and carried from r to t . We now consider the plan that buys them in t instead.

Nothing else changes. The same units are available in the same periods, so the plan is still feasible, and it still orders in t , so the setup in t is unchanged. What does change is that the δ units are no longer carried from r to t , which saves $\delta \sum_{s=r}^{t-1} h_s \geq 0$, and they are bought at c_t instead of c_r . With constant purchase costs the second change costs nothing and the first saves something whenever any h_s is positive, so the new plan costs no more. If the shift empties period r entirely, a setup is saved as well.

Repeating the exchange removes every violation, and we arrive at a plan that costs no more and orders only when empty. Thus, there is an optimal plan of that kind, and we may look only at plans of that kind.

The consequence is what matters. If a plan orders only when empty, then whenever it orders in period t it must order enough to cover periods t through the period before its next order, and no more. That is, the quantity is determined by the timing:

$$Q_t = d[t, u] \quad \text{where } u + 1 \text{ is the next period with an order.}$$

5. Dynamic Lot Sizing

Thus, a plan *is* a set of order periods. Period 1 always orders, because inventory starts at zero and d_1 must be met. Each of periods 2 through N either orders or does not. There are 2^{N-1} plans, which for the belt is 2,048.

That is a finite number, and it is also 2,048 only because $N = 12$. At $N = 52$ it is about 2×10^{15} . Enumeration is not the answer, but the fact that the plans can be counted at all is what makes the rest of the chapter possible.

5.3.2. The Window Cost

Since a plan is a set of order periods, its cost decomposes over the stretches between them. We define the *window cost* $w(t, u)$ as the cost incurred by a replenishment placed in period t that covers the requirements of periods t through u .

The order costs k_t . It buys $d[t, u]$ units at c_t each. The units for period t arrive and leave in period t , so they are never carried. The units for period $t + 1$ sit in stock at the end of period t , so they are charged h_t each. The units for period $t + 2$ are charged $h_t + h_{t+1}$, and so on. Collecting,

$$w(t, u) = k_t + \sum_{s=t}^u d_s \left(c_t + \sum_{r=t}^{s-1} h_r \right). \quad (5.4)$$

Thus, the bracket deserves a name of its own. Write

$$\hat{c}(t, s) = c_t + \sum_{r=t}^{s-1} h_r \quad (5.5)$$

for the cost of one unit bought in period t and held until it is used in period s , with $\hat{c}(t, t) = c_t$. Then Equation 5.4 is

$$w(t, u) = k_t + \sum_{s=t}^u d_s \hat{c}(t, s). \quad (5.6)$$

5. Dynamic Lot Sizing

This is the step that makes everything afterwards cheap. A sum of carrying charges spread over many periods has been folded into a single price per unit, which depends only on where the unit was bought and where it is used. The cost of a window is then an ordinary inner product of requirements and prices.

Notice two things about Equation 5.6. First, $w(t, u)$ does not depend on any decision. It is a property of the data, so all of the window costs can be computed once, before any search begins. Second, $w(t, u)$ is increasing in u and the increments grow, because a unit added at the far end of the window has been held longer than any unit already in it.

With constant costs the relevant part of Equation 5.6 simplifies to

$$w(t, u) = k + h \sum_{s=t}^u (s - t) d_s, \quad (5.7)$$

where the purchase term has been dropped as Section 5.2.1 allows. The coefficient $s - t$ counts the periods a unit for period s waits. Every window cost quoted in this chapter is computed from Equation 5.7.

Example 5.2 (The window costs of the first half year). Take the first six months of the belt, so that the table fits on a page. With $k = \$300$ and $h = \$1.00$,

$$w(1, 3) = 300 + 1.00 [0(40) + 1(60) + 2(120)] = 300 + 300 = \$600,$$

and the full table of windows is Table 5.4.

Table 5.4.: Window costs for the first six months of the belt. A blank cell is a window that would end before it began.

$w(t, u)$	$u = 1$	2	3	4	5	6
$t = 1$	300	360	600	1,500	3,180	4,480
2		300	420	1,020	2,280	3,320

5. Dynamic Lot Sizing

$w(t, u)$	$u = 1$	2	3	4	5	6
3			300	600	1,440	2,220
4				300	720	1,240
5					300	560
6						300

Every diagonal entry is \$300, because a window covering one period carries nothing. Read across row 1 and the increments are 60, 240, 900, 1,680 and 1,300, which grow until month 6, whose requirement is smaller than month 5's. The increment from u to $u + 1$ is $h(u + 1 - t)d_{u+1}$, so it grows with the distance and with the requirement, and either can dominate.

5.3.3. Property 2: A Requirement That Forces an Order

There is a second property that is often useful and costs nothing to check.

Suppose the requirement in period t satisfies $d_t > k_t/h_{t-1}$. Carrying those d_t units in from period $t - 1$ would cost $h_{t-1}d_t > k_t$, which is more than a fresh order in period t costs. Thus, no optimal plan carries anything into period t , which means an optimal plan orders in period t .

For the belt, $k/h = 300$ belt-months. Month 5 needs 420 belts, and $420 > 300$, so every optimal plan orders in month 5. Nothing else in the year is that large.

The value of the property is that it splits the horizon. If period t must hold an order, then no window spans period $t - 1$ to period t , and the problem over periods 1 through $t - 1$ can be solved without reference to the problem over periods t through N . That is, one problem of size 12 becomes two of size 4 and 8. Section 5.7 shows that this is also how a long rolling horizon can be cut short without losing anything, since the periods beyond a forced order have no influence on the decision being made now.

5.4. Choosing the Order Periods

Section 5.3 leaves us with a clean problem. A plan is a set of order periods, the periods between orders partition the horizon into windows, the cost of a window is $w(t, u)$, and the cost of a plan is the sum of its windows' costs. What remains is to choose the partition.

5.4.1. The Problem Is a Shortest Path

Draw $N + 1$ nodes, numbered 1 through $N + 1$. Node t means “the plan arrives at period t holding nothing.” Node $N + 1$ means the horizon is finished. For every pair $t \leq u$, draw an arc from node t to node $u + 1$ and price it $w(t, u)$. The arc says: order in period t , cover through period u , and arrive at period $u + 1$ empty.

A path from node 1 to node $N + 1$ is a sequence of windows that tile the horizon, which by Section 5.3.1 is a plan. Its length is the sum of its arc costs, and that sum is the plan's cost. Thus, the lot sizing problem is a shortest path problem on an acyclic network of $N + 1$ nodes and $N(N + 1)/2$ arcs.

Figure 5.1 illustrates the network for the first four months of the belt.

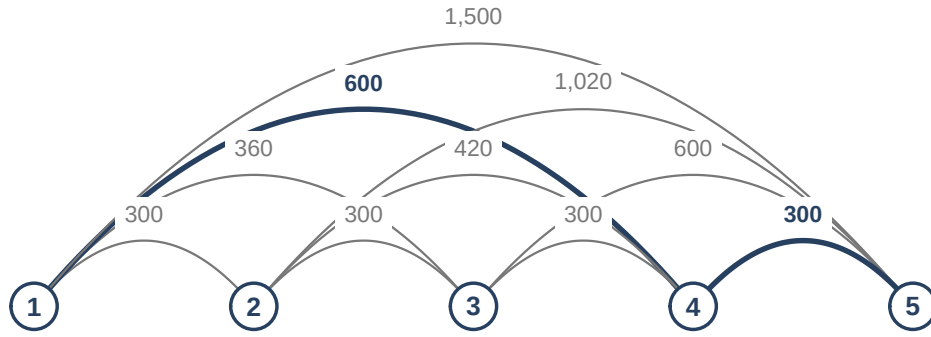
Notice what the shortest path formulation buys. The arc costs were computed in Section 5.3.2 without reference to any decision, so the entire network is known before the search begins. A shortest path on an acyclic network with the nodes already in topological order is a single sweep.

5.4.2. The Recursion

Let $V(u)$ be the least cost of covering periods 1 through u and arriving at period $u + 1$ with nothing on hand. Then $V(0) = 0$, and for $u = 1, \dots, N$,

$$V(u) = \min_{1 \leq t \leq u} \{V(t - 1) + w(t, u)\}. \quad (5.8)$$

5. Dynamic Lot Sizing



node t : the plan reaches period t with nothing on hand

Figure 5.1.: The first four months of the belt as a shortest path problem. Node t means the plan reaches period t holding nothing, and the arc from t to $u + 1$ is a replenishment in period t covering through period u , priced at $w(t, u)$. Every path from node 1 to node 5 is a plan. The heavy path, ordering in month 1 to cover months 1 through 3 and again in month 4, is the cheapest at \$900.

5. *Dynamic Lot Sizing*

Read it as a question about the last order. Whatever the optimal plan for periods 1 through u is, it has a last order, and that order falls in some period t and covers periods t through u . Everything before period t is a plan for periods 1 through $t - 1$ that arrives at t empty, and it had better be the cheapest such plan, or we could substitute the cheapest and do better. Thus, the cost is $V(t - 1) + w(t, u)$, and we take the best t .

Let $S(u)$ record the minimizing t . Then $V(N)$ is the optimal cost and S holds enough information to recover the plan. The last order starts in period $S(N)$ and covers through period N . The order before it covers through period $S(N) - 1$ and starts in period $S(S(N) - 1)$. Walking backward this way ends when an order starts in period 1.

This is the algorithm of Wagner and Whitin (1958). It is stated here as a recursion over the window costs because that is what it is, and because Section 5.5 then needs only to change how far a window reaches.

5. Dynamic Lot Sizing

Algorithm 5.1 The Wagner-Whitin algorithm. The three blocks are independent: the windows are a property of the data, the recursion is a shortest path, and the traceback reads the plan off the pointers.

```
wagnerWhitin(d, k, c, h over periods 1..N):

    // the windows, which depend on no decision
    for t = 1 to N:
        for u = t to N:
            w(t,u) <- k(t) + sum over s in t..u of d(s) * chat(t,s)

    // the recursion
    V(0) <- 0
    for u = 1 to N:
        V(u) <- infinity
        for t = 1 to u:
            if V(t-1) + w(t,u) < V(u):
                V(u) <- V(t-1) + w(t,u)
                S(u) <- t

    // the traceback
    orders <- empty list
    u <- N
    while u >= 1:
        t <- S(u)
        add (order in period t, quantity d[t,u]) to orders
        u <- t - 1

    return orders, V(N)
```

Example 5.3 (The recursion on the first half year). Take the six-month instance of Example 5.2, whose window costs are Table 5.4. The recursion

5. Dynamic Lot Sizing

fills one column at a time, and each column has one entry for each period that could hold the last order.

Table 5.5.: The recursion on the six month instance. Each entry is $V(t-1)$ from an earlier column plus a window cost from Table 5.4. The least entry in each column is in bold.

$V(t-1) + w(t, u)$	$u = 1$	2	3	4	5	6
last order in period 1	300	360	600	1,500	3,180	4,480
2		600	720	1,320	2,580	3,620
3			660	960	1,800	2,580
4				900	1,320	1,840
5					1,200	1,460
6						1,500
$V(u)$	300	360	600	900	1,200	1,460
$S(u)$	1	1	1	4	5	5

Consider column 4. Covering periods 1 through 4 with one order costs $w(1, 4) = 1,500$. Ordering in period 2 costs $V(1) + w(2, 4) = 300 + 1,020 = 1,320$. Ordering in period 3 costs $V(2) + w(3, 4) = 360 + 600 = 960$. Ordering in period 4 costs $V(3) + w(4, 4) = 600 + 300 = 900$, the least of the four, so $V(4) = 900$ and $S(4) = 4$.

The optimum is $V(6) = \$1,460$. The traceback starts at $u = 6$: $S(6) = 5$, so the last order falls in period 5 and covers periods 5 and 6, or $d[5, 6] = 420 + 260 = 680$ belts. Now $u = 4$. Since $S(4) = 4$, the next order back falls in period 4 and covers period 4 alone, 300 belts. Now $u = 3$, and

5. Dynamic Lot Sizing

$S(3) = 1$, so the first order falls in period 1 and covers periods 1 through 3, 220 belts.

Table 5.6.: The optimal plan for the first six months.

t	1	2	3	4	5	6
d_t	40	60	120	300	420	260
Q_t	220	0	0	300	680	0
I_t	180	120	0	0	260	0

Three orders cost \$900 and the ending balances of 560 belt-months cost \$560, for \$1,460. Notice that the two checks agree, which they must: the traceback and the recursion are two readings of the same numbers, and if the ledger does not reproduce $V(N)$ then one of them was done wrong.

Example 5.4 (The full twelve months). The same recursion over all twelve months of Table 5.1 gives

Table 5.7.: The recursion over the full horizon.

u	1	2	3	4	5	6	7	8	9	10	11	12
$V(u)$	300	360	600	900	1,200	1,460	1,620	1,700	1,700	2,000	2,180	2,480
$S(u)$	1	1	1	4	5	5	6	6	6	10	10	12

The optimum is \$2,480. Tracing back from $u = 12$ gives orders in months 12, 10, 6, 5, 4 and 1, so the plan is

Table 5.8.: The optimal plan for the year.

t	1	2	3	4	5	6	7	8	9	10	11	12
d_t	40	60	120	300	420	260	120	40	0	80	180	220

5. Dynamic Lot Sizing

t	1	2	3	4	5	6	7	8	9	10	11	12
Q_t	220	0	0	300	420	420	0	0	0	260	0	220
I_t	180	120	0	0	0	160	40	0	0	180	0	0

Six orders cost \$1,800 and 680 belt-months of stock cost \$680. Notice that $V(9) = V(8)$. Month 9 has no requirement, so covering through month 9 costs exactly what covering through month 8 costs, and the recursion is indifferent to where the next order falls. Notice also that month 5 holds an order, as 1 promised it would.

Against the three plans of Table 5.3, the optimum saves \$820 on lot-for-lot and \$120 on the adjusted economic order quantity, which was within 4.84% of optimal. Thus, there is a real question about how much the rest of this chapter buys, and Section 5.6 takes it up directly.

5.4.3. The Same Problem for Solver

The recursion is the right way to solve this problem. It is not the only way, and we should see the alternative, because it generalizes where the recursion does not.

Written as a mathematical program, we have

$$\begin{aligned}
 \min \quad & \sum_{t=1}^N k_t y_t + \sum_{t=1}^N c_t Q_t + \sum_{t=1}^N h_t I_t \\
 \text{subject to} \quad & I_t = I_{t-1} + Q_t - d_t, & t = 1, \dots, N \\
 & Q_t \leq M y_t, & t = 1, \dots, N \\
 & I_0 = 0, \quad I_t \geq 0, \quad Q_t \geq 0, & t = 1, \dots, N \\
 & y_t \in \{0, 1\}, & t = 1, \dots, N.
 \end{aligned} \tag{5.9}$$

Everything here is linear. That is not obvious, because the cost of ordering is not a linear function of the order quantity. It is zero at zero and jumps

5. Dynamic Lot Sizing

to k_t the moment anything is ordered. A fixed charge of that kind cannot be written as a linear function of Q_t alone.

The device is the binary y_t and the constraint $Q_t \leq My_t$. If $y_t = 0$ then Q_t is forced to zero, and if $y_t = 1$ then Q_t may be anything up to M . Thus, the setup is charged exactly when something is ordered, and the objective is linear in the pair (Q_t, y_t) . The constant M needs only to be large enough never to bind, and $M = d[1, N]$ serves, since no order in an optimal plan covers more than the whole horizon.

Take care with M , and keep it as small as is safe. A needlessly large M gives a weaker linear relaxation, and the branch and bound search that a solver runs will take longer for no benefit.

The formulation is in the **Solver** sheet of the workbook, where the problem is small enough for Excel's Solver to reach \$2,480 from any starting point.

Two reasons to know the formulation. The first is that it accepts constraints the recursion cannot. A limit on the order quantity, a shared capacity across several items, a minimum order size: none of these fit Equation 5.8, and all of them are one more row in Equation 5.9. The second is that Equation 5.9 is how the problem is actually solved inside commercial planning systems that must handle the extra constraints.

5.4.4. How Much Work It Is

The recursion computes $N(N+1)/2$ window costs and then takes $N(N+1)/2$ minima, so it runs in $O(N^2)$ time. For a twelve-month horizon that is 78 windows, about a page of arithmetic. For a 52-week horizon it is 1,378, nothing at all for a computer and out of the question by hand.

$O(N^2)$ was long thought to be the best possible. It is not. Federgruen and Tzur (1991) showed that the problem can be solved in $O(N \log N)$ time, and in $O(N)$ time in the constant-cost case, by exploiting the structure of the window costs so that most of the candidate minima never have to

5. *Dynamic Lot Sizing*

be evaluated. The practical consequence is small, since N is rarely large enough for the difference to matter, but it settles a question that stood for thirty years.

The reason lot sizing is still done with heuristics has nothing to do with running time, and Section 5.5 takes it up next.

5.5. Three Heuristics and One Shape

An optimal algorithm exists, it is not slow, and it fits on half a page. So why is almost every planning system in the field running something else?

The answer is Section 5.1.2, assumption 2. The recursion needs d_1 through d_N before it can compute $V(N)$, because $V(N)$ depends on every window cost, and every window cost depends on the requirements out to period N . Those requirements are a forecast. The decision that actually gets made is the one for period 1, and it has been made to depend on a number twelve periods out that nobody believes.

A heuristic that looks only a few periods ahead makes the period 1 decision from data the planner can believe. That is the case for heuristics here, and it is a different case from the usual one. It is not an argument about running time.

5.5.1. One Procedure

The three heuristics in common use are one procedure. We start an order in period 1 and extend the window one period at a time. At each step we compute a figure of merit for the window so far, and we stop when it stops improving. Whatever period the window reached, the next order starts in the one after it, and we begin again.

The three rules differ only in the figure of merit.

5. Dynamic Lot Sizing

Table 5.9.: Three stopping rules on one procedure.

Rule	Figure of merit for the window $[t, u]$	Stop when
Silver-Meal	$w(t, u)/(u - t + 1)$, cost per period	it rises
Least unit cost	$w(t, u)/d[t, u]$, cost per unit	it rises
Part-period balancing	$w(t, u) - k_t$, what the window carries	it passes k_t

All three use the window costs of Section 5.3.2, which were computed once and are the same numbers the recursion used. That is what makes the comparison in Section 5.6 a comparison and not an accident of arithmetic.

Silver-Meal minimizes cost per period. The reasoning is an appeal to Section 3.5.4, where the quantity that minimizes cost per unit time is optimal because every cycle is identical. Here the cycles are not identical, so minimizing cost per period over one window is a local imitation of that argument and not a consequence of it.

Least unit cost minimizes cost per unit instead. It is the older rule and the more intuitive one, since a buyer thinks in dollars per piece. It tends to order larger quantities than Silver-Meal, because adding a period to the window always adds units to the denominator. That is, a long window is flattered by the arithmetic.

Part-period balancing looks different and is not. In Section 3.5.4 the optimum sets the setup cost equal to the carrying cost, so this rule extends the window until the carrying it has accumulated is as close as it can get to k_t . Two versions are in circulation. The original, due to DeMatteis and Mendoza and given in Silver et al. (2016), takes the largest window whose carrying is at or under k_t and so never overshoots. The modification in Nahmias and Olsen (2015), which is the one used here and in Table 5.9,

5. Dynamic Lot Sizing

compares the two windows either side of k_t and keeps whichever comes closer. 2 works both. Dividing through by h turns the accumulated carrying into a count of unit-periods, or *part-periods*, and that is where the name comes from. For the belt, $k/h = 300$, so the rule extends until the window carries about 300 belt-months.

One difference in the last rule matters when it is implemented. Silver-Meal and least unit cost walk a number to a minimum and keep the period before the first rise. Part-period balancing walks a number past a target, and then keeps whichever of the last two periods came closer. It can step back.

Example 5.5 (The three rules on the belt). Start all three in month 1 and extend. The window costs come from Equation 5.7.

Table 5.10.: The first window under each of the three rules.

u	$w(1, u)$	Silver-Meal, $w/(u)$	Least unit cost, $w/d[1, u]$	Part-period, $w - k$
1	300	300.00	7.500	0
2	360	180.00	3.600	60
3	600	200.00	2.727	300
4	1,500	375.00	2.885	1,200

Silver-Meal stops at $u = 2$, because the cost per period rises from \$180.00 to \$200.00 when month 3 is added. It orders 100 belts.

Least unit cost keeps going to $u = 3$, where \$2.727 a belt is the least, and rises to \$2.885 at $u = 4$. It orders 220 belts.

Part-period balancing extends until the carrying passes $k = \$300$. At $u = 3$ it carries exactly \$300. Nothing can come closer, so it stops there and also orders 220 belts.

5. Dynamic Lot Sizing

Continuing each rule to the end of the horizon gives the three plans in Table 5.11.

Table 5.11.: The three heuristic plans and the optimum.

t	1	2	3	4	5	6	7	8	9	10	11	12
d_t	40	60	120	300	420	260	120	40	0	80	180	220
Silver-Meal	100	0	420	0	840	0	0	0	0	260	0	220
Least unit cost	220	0	0	720	0	380	0	300	0	0	0	220
Part-period balancing	220	0	0	720	0	420	0	0	0	260	0	220
Optimal	220	0	0	300	420	420	0	0	0	260	0	220

All three place five orders and the optimum places six. That is the first surprise. The heuristics are not ordering too often. They are ordering too seldom, and the carrying that buys is what costs them.

The 840 in month 5 is where Silver-Meal loses its money. Extending from month 5, the cost per period runs \$300.00, \$280.00, \$266.67, \$230.00, \$184.00, and only then rises to \$220.00 at month 10. It never rises through the whole run, so the rule never stops. Months 6 through 8 have falling requirements and month 9 has none at all, so each additional period adds almost nothing to the numerator while adding a full period to the denominator. The average keeps falling because the window is getting longer, not because the plan is getting better. That is the failure mode of

5. Dynamic Lot Sizing

a cost-per-period rule. Notice that it is triggered by a declining tail and a dead period, exactly the shape of a seasonal item going into its off season.

5.6. What a Heuristic Costs

Table 5.12 collects every method in this chapter on the belt.

Table 5.12.: Every method of this chapter on the mower deck belt.

Method	Orders	Setup	Carrying	Relevant cost	Penalty
Lot-for-lot	11	\$3,300	\$0	\$3,300	+33.06%
Least unit cost	5	\$1,500	\$1,540	\$3,040	+22.58%
Periodic order quantity, $T = 2$	6	\$1,800	\$960	\$2,760	+11.29%
Silver-Meal	5	\$1,500	\$1,160	\$2,660	+7.26%
Adjusted economic order quantity	6	\$1,800	\$800	\$2,600	+4.84%

5. Dynamic Lot Sizing

Method	Orders	Setup	Carrying	Relevant cost	Penalty
Part-period balancing	5	\$1,500	\$1,100	\$2,600	+4.84%
Wagner-Whitin	6	\$1,800	\$680	\$2,480	optimal

Read the table carefully, because it does not say what a table like this is usually taken to say. Silver-Meal, which the literature reports as the best of the three heuristics on average, is beaten here by part-period balancing and also by an adjusted economic order quantity computed from the average requirement. One instance settles nothing. What it does show is how far apart the methods can land on a single item: part-period balancing gives away 4.84% and least unit cost 22.58%, so the choice among the three heuristics is worth almost eighteen percentage points here, and lot-for-lot is worse than any of them at 33.06%.

5.6.1. When None of This Is Worth Doing

If the requirements barely vary, then Chapter 3 already has the answer and this chapter is an elaborate way of reproducing it. The question we need answered is how much variation is enough, and Silver et al. (2016) gives a test.

The *variability coefficient* is the variance of the requirements divided by the square of their mean:

$$VC = \frac{\frac{1}{N} \sum_{t=1}^N (d_t - \bar{d})^2}{\bar{d}^2} \quad \text{where} \quad \bar{d} = \frac{1}{N} \sum_{t=1}^N d_t. \quad (5.10)$$

5. Dynamic Lot Sizing

Dividing by $\bar{d}^2$ makes it dimensionless, so it does not matter whether the item is counted in belts or in pallets. A perfectly level requirement gives $VC = 0$.

Thus, the rule of thumb is that $VC < 0.2$ calls for an economic order quantity and $VC \geq 0.2$ calls for one of the methods of this chapter. For the belt, $\bar{d} = 153.33$ and the variance is 14,555.6, so

$$VC = \frac{14,555.6}{153.33^2} = 0.619,$$

well past the threshold. Notice that the threshold is a rule of thumb and not a theorem. Table 5.12 shows the adjusted economic order quantity within 4.84% of optimal at $VC = 0.619$, which is better than the rule would lead you to expect.

5.6.2. Sensitivity to the Cost Ratio

Section 3.7 showed that the economic order quantity is forgiving: getting k/h wrong by a factor of two costs about 6% of the relevant cost. We can ask the same question here, and the answer has the same shape.

Solve the belt with a ratio k/h that is wrong by a factor m , then charge the resulting plan at the true costs. Table 5.13 gives the result.

Table 5.13.: The cost of solving the belt with the wrong ratio k/h .

m	0.25	0.5	0.75	1	1.5	2	4
Or-	9	9	7	6	4	4	3
ders							
Cost	\$2,800	\$2,800	\$2,540	\$2,480	\$2,660	\$2,660	\$3,280
at							
true							
costs							

5. Dynamic Lot Sizing

m	0.25	0.5	0.75	1	1.5	2	4
Penalty	12.90%	12.90%	2.42%	0%	7.26%	7.26%	32.26%

A factor of two in either direction costs less than 13%. Thus, the flatness Section 3.7 found survives the move to discrete periods. The curve is not smooth here, because the number of orders is an integer and the plan does not change at all over a range of m . Doubling and halving are not symmetric either: at $m = 2$ the plan has four orders and costs 7.26% extra, and at $m = 0.5$ it has nine and costs 12.90%. Ordering too often is the more expensive mistake on this item, since every extra order costs a full \$300 while the carrying it saves is spread thin.

Put Table 5.13 beside Table 5.12 and something follows. The penalty for using Silver-Meal instead of the optimum is 7.26%. The penalty for getting the cost ratio wrong by a factor of two is also 7.26%. An organization that has not measured k to within a factor of two, and most have not, is not in a position to care which method it uses.

5.6.3. What the Aggregate Studies Say

One instance is one instance. Silver et al. (2016) reports comparisons over many generated problems, and the findings that hold up are these. All three heuristics land within a few percent of the optimum on average. Silver-Meal is usually the best of them. Least unit cost is usually the worst, for the reason Section 5.5 gave: the denominator grows with the window whether or not the window is a good one. Lot-for-lot is not competitive unless ordering is nearly free, and an economic order quantity adjusted to whole periods does better than its reputation whenever VC is small.

The averages hide a wide spread. Silver-Meal is best on average and can lose seven percent on a single item, as it does here. A heuristic is a claim about a population of problems, not about the problem in front of you.

5.7. The Rolling Horizon

Everything so far has assumed a horizon that ends. The belt's year is planned, the plan is executed, and the problem is over. No item is planned that way.

What actually happens is that the planner solves a problem over the next N periods, places only the order for the current period, waits a period, and solves again over a horizon that has moved forward one period and acquired a new last period. The plan for periods 2 through N was never executed. It existed to inform the decision for period 1.

Two things follow immediately. The first is that only the first decision matters. Thus, the quality of a method is the quality of the sequence of first decisions it produces, and not the quality of the plans it produces. The second is that Wagner-Whitin is no longer optimal. It is optimal for the problem it was handed, and the problem it was handed is a truncation of the real one.

5.7.1. What the Truncation Costs

Let's run the belt through a rolling horizon. At each month the planner solves over the next W months, places that month's order, and moves on. The realized cost of the twelve months of orders that actually get placed is Table 5.14.

Table 5.14.: What the belt actually costs when the horizon rolls, one month committed at a time. The full-horizon optimum is \$2,480.

Planning window W	2	3	4	5	6
Silver-Meal	\$2,640	\$2,840	\$2,660	\$2,660	\$2,660
Wagner-Whitin	\$2,640	\$2,480	\$2,480	\$2,480	\$2,480

5. Dynamic Lot Sizing

Wagner-Whitin reaches the full-horizon optimum at a window of three months and stays there. That is 1 doing its work. Month 5 forces an order, so the decision for month 4 cannot be improved by anything beyond month 5, and the three-month window already sees far enough.

Silver-Meal never reaches it, and its cost is not even monotone in the window. A three-month window costs it \$200 more than a two-month window does. Let's understand that instead of explaining it away. Lengthening the window gives the rule more periods to average over, and averaging over more periods is precisely what drives its cost-per-period down and its windows too long. A longer horizon is not always better information for a rule that was not built to use it.

5.7.2. Does the Heuristic Ever Win?

It is often said that under a rolling horizon a heuristic can beat the optimal algorithm, since neither is optimal for the real problem and the heuristic's myopia is no longer a handicap. The claim is true as stated. It is easy to read it as a claim about how the two typically compare. That is a different claim.

We can settle it by measurement. Generate 2,000 twelve-period schedules with requirements drawn at random from 0 to 300 units, $k = \$200$, $c = \$50$ and $i = 0.02$ per period. Then we roll each one with a window of W periods, committing one period at a time, under both rules.

Table 5.15.: Silver-Meal against Wagner-Whitin over 2,000 random schedules under a rolling horizon.

Window W	Silver- Meal cheaper	Wagner- Whitin cheaper	Tied	Mean, Silver- Meal	Mean, Wagner- Whitin
3	360	704	936	\$1,440.91	\$1,429.90
4	246	886	868	\$1,395.74	\$1,375.08

5. Dynamic Lot Sizing

Window W	Silver- Meal cheaper	Wagner- Whitin cheaper	Tied	Mean, Silver- Meal	Mean, Wagner- Whitin
6	71	1,062	867	\$1,383.20	\$1,350.14

Silver-Meal does win, and on a three-period window it wins 360 times out of 2,000. Thus, the claim is not empty. What the table adds is the rest of the picture. The two rules tie about 45% of the time, because on many schedules they make the same first decision every period. When they differ, the algorithm is ahead about two to one at a three-period window and about fifteen to one at six. On average the algorithm is cheaper at every window length, by 0.8% to 2.4%.

Thus, the rolling horizon does not rescue the heuristic. What it does is remove the guarantee, which turns a proof into an empirical question, and the empirical answer on these instances still favors the algorithm.

5.7.3. Nervousness

There is a cost the tables do not show. Each period the plan is recomputed, and the new plan may disagree with the old one about orders that have not been placed yet. Suppliers were told to expect 300 belts in month 7, and now the plan says 0. That instability is called *nervousness*, and it is expensive in ways that do not appear in Equation 5.3.

Part of it comes from the method. Table 5.14's first decision is instructive. Silver-Meal orders 100 belts in month 1 at every window length from two to six. Wagner-Whitin orders 100 at a two-month window and 220 at every longer one. The heuristic is steadier here, because it never looks far enough ahead for a distant period to change its mind.

Three devices are in common use against nervousness, and each of them buys stability at a price.

5. Dynamic Lot Sizing

A *freeze fence* fixes the plan for the first several periods and forbids the next solve from changing them. It converts instability into inflexibility, and that is sometimes the better problem. Committing F periods at a time instead of one is the same idea.

A *time fence* is softer. Inside it, changes need approval; outside it, the plan may move freely.

A *change penalty* adds a cost to the objective for every order that differs from the previous plan. That is, the previous plan becomes part of the data. It is the most direct treatment and the least used, for exactly that reason.

Chapter 6 returns to all three, because nervousness at one level of a bill of material propagates downward and multiplies.

5.8. Building These Models in a Worksheet

Section 4.7 built a worksheet for each model of the previous chapter, and the layouts followed the shape of the mathematics. Here they follow the shape of the horizon. Every quantity in this chapter is indexed by period, so the periods run down the rows of one block and every sheet reads that block by name.

The workbook is `Chapter5Models.xlsx`. It has five working sheets, and the sections below take them in the order the chapter introduced the material. As in Section 4.7, blue cells are inputs and grey cells are computed, every sheet carries a check block giving what the shipped data should produce, and every sheet carries a text box to the right of its widest block saying what the sheet is for and what to watch. A third colour appears here that Section 4.7 did not need. Yellow marks a cell a method picked: a period an order starts in, or a period a stopping rule chose.

5. Dynamic Lot Sizing

5.8.1. The Data Sheet

The requirements run down column B, one row per period, and the three cost columns beside them point at the input cells above. Writing them as columns instead of as three scalars costs nothing when the costs are constant and buys the period-varying case for free: type \$54 over one cell in the unit cost column and every window cost in the workbook accounts for the price rise.

Five named ranges carry the schedule to the rest of the workbook: `Dls_t`, `Dls_d`, `Dls_k`, `Dls_c` and `Dls_h`. A sub-range of one of them is written `INDEX(Dls_d,a):INDEX(Dls_d,b)`. That is the requirement of periods a through b , and it is Equation 5.7's inner sum in one expression.

Two helper columns sit to the right. One holds $(d_t - \bar{d})^2$, which totals to the numerator of Equation 5.10, and one holds the period number when $d_t h > k$ and nothing otherwise, so that the smallest number in it is 1's forced period. Both could be done with an array formula in a single cell. Both are done as columns because the reader can then see which period did it. They are tinted, and a rule down the right of the holding rate column says where the schedule stops and the figures derived from it start.

Figure 5.2 illustrates the top of the sheet. The three inputs in rows 5 through 7 are the only typed numbers on it, and everything below them follows. Notice that every figure the check block in rows 25 through 27 promises is on the screen above it: 1,840 belts, an average of 153.3333, a variability coefficient of 0.6191, $k/h = 300$, a first forced month of 5, and an economic order quantity of 303.3 belts that is a time supply of 1.98 months. Reading a check block against the sheet it sits on is the first thing to do with any workbook, and it takes ten seconds.

The requirements themselves are Figure 5.3. Column B is the only blue block on the sheet, and columns C, D and E hold one cost per period.

5. Dynamic Lot Sizing

	A	B	C
1	The item and its requirements		
2	<i>Chapter 5, section 5.1.</i>		
3			
4	INPUTS		
5	Order cost, k	\$300.00	dollars per order
6	Unit cost, c	\$50.00	dollars per belt
7	Carrying charge, i	0.020	per dollar per month
8	Holding rate, h = ic	\$1.00	
9	Periods in the horizon, N	12	
10			
11	WHAT THE PATTERN LOOKS LIKE		
12	Total requirement	1,840	
13	Average requirement per month	153.3333	
14	Variability coefficient	0.6191	
15	Use a heuristic?	yes, the pattern varies enough	
16	k / h, the carrying a single order buys	300	
17	First month whose requirement forces an order	5	
18	<i>Section 5.3: a requirement larger than k/h cannot be carried in from any earlier month.</i>		
19			
20	THE ECONOMIC ORDER QUANTITY, FOR COMPARISON		
21	EOQ on the average requirement	303.3	
22	As a time supply, months	1.98	
23	Rounded to whole months	2	
24			
25	CHECK with the data as shipped		
26	<i>Total 1,840 belts; average 153.3333; variability coefficient 0.6191; k/h = 300;</i>		
27	<i>the first forced month is 5; EOQ 303.3 belts, 1.98 months, rounding to 2.</i>		

Figure 5.2.: The Data sheet, rows 1 to 27. Three inputs, the figures derived from them, the economic order quantity of Chapter 3 for comparison, and the check block. Notice cell B15, which reads the variability coefficient against the threshold of Section 5.6.1 and answers the question in words rather than leaving the reader to compare two numbers.

5. Dynamic Lot Sizing

	A	B	C	D	E	F	G
29	REQUIREMENTS	type over column B to change the pattern					
30	Month t	Requirement d(t)	Order cost k(t)	Unit cost c(t)	Holding rate h(t)	(d(t) - mean)*2	Forced?
31	1	40	\$300.00	\$50.00	\$1.00	12,844.4	
32	2	60	\$300.00	\$50.00	\$1.00	8,711.1	
33	3	120	\$300.00	\$50.00	\$1.00	1,111.1	
34	4	300	\$300.00	\$50.00	\$1.00	21,511.1	
35	5	420	\$300.00	\$50.00	\$1.00	71,111.1	5
36	6	260	\$300.00	\$50.00	\$1.00	11,377.8	
37	7	120	\$300.00	\$50.00	\$1.00	1,111.1	
38	8	40	\$300.00	\$50.00	\$1.00	12,844.4	
39	9	0	\$300.00	\$50.00	\$1.00	23,511.1	
40	10	80	\$300.00	\$50.00	\$1.00	5,377.8	
41	11	180	\$300.00	\$50.00	\$1.00	711.1	
42	12	220	\$300.00	\$50.00	\$1.00	4,444.4	
43							
44	The cost columns point at the inputs above, so a constant-cost problem is typed once.						
45	Type a number over one of them to make that month's cost differ, which is the case of Section 5.2.						
46	Columns F and G feed the variability coefficient and the forced month. They are not data.						

Figure 5.3.: The Data sheet, rows 29 to 46, the requirements schedule. Columns C, D and E point at the input cells above, so a constant-cost problem is typed once and a period-varying one needs no second workbook. Columns F and G are tinted and stand to the right of a rule, because they are derived and not data. Notice that column G is empty except in month 5, where 420 belts exceed $k/h = 300$ and 1 forces an order.

5. Dynamic Lot Sizing

5.8.2. Costing a Plan

The **Plans** sheet is the ledger of Example 5.1, made general. Four plans sit side by side, each with two columns: an order column the reader types into and an ending-inventory column that computes Equation 5.1 as `= previous end + order - requirement`.

The results block above them counts the orders with **COUNTIF** on the order column, charges carrying with **SUMPRODUCT(Dls_h, end column)**, and reports the penalty against the optimum on the next sheet. Four plans are shipped: lot-for-lot, the fixed economic order quantity, the periodic rule at $T = 2$, and a fourth to type over.

A negative number in an ending-inventory column is a shortage, which Section 5.1.2 forbids. Nothing on the sheet prevents one from appearing. The sheet evaluates whatever plan is typed into it, including infeasible ones, and it is the reader's job to look.

Figure 5.4 is the whole sheet. Read a plan by reading one pair of columns downward and then the block above it.

5.8.3. The Recursion and the Traceback

The **WagnerWhitin** sheet has four blocks, and the first three match the three blocks of Algorithm 5.1. Nothing on it is typed.

Figure 5.5 is what the sheet answers, and it is the block to read first.

The window costs occupy an N by N upper triangle. Rather than write Equation 5.7 out in each cell, each cell adds one period to the cell on its left:

$$w(t, u) = w(t, u - 1) + d_u \sum_{r=t}^{u-1} h_r,$$

5. Dynamic Lot Sizing

	A	B	C	D	E	F	G	H	I	J
1	What a plan costs									
2	Chapter 5, section 5.2. Each plan owns an order column. Type over one and the results above it move.									
3										
4	RESULTS									
5			Lot-for-lot		Fixed EOQ		Periodic, T = 2		Your plan	
6	Orders placed		11		6		6		11	
7	Setup cost		\$3,300.00		\$1,800.00		\$1,800.00		\$3,300.00	
8	Carrying cost		\$0.00		\$800.00		\$960.00		\$0.00	
9	Purchase cost		\$92,000.00		\$92,000.00		\$92,000.00		\$92,000.00	
10	Relevant cost		\$3,300.00		\$2,600.00		\$2,760.00		\$3,300.00	
11	Total cost		\$95,300.00		\$94,600.00		\$94,760.00		\$95,300.00	
12	Penalty against the optimum		33.06%		4.84%		11.29%		33.06%	
13										
14	CHECK with the data as shipped									
15	Lot-for-lot: eleven orders, \$3,300 of setup, nothing carried, 33.06% above the optimum.									
16	Fixed EOQ: six orders, \$1,800 and \$800, for \$2,600 and 4.84%. Periodic: \$1,800 and \$960, for \$2,760 and 11.29%.									
17	THE LEDGER one row per month									
18	Month	Requirement	Order	End	Order	End	Order	End	Order	End
19	1	40	40	0	220	180	100	60	40	0
20	2	60	60	0	0	120	0	0	60	0
21	3	120	120	0	0	0	420	300	120	0
22	4	300	300	0	300	0	0	0	300	0
23	5	420	420	0	420	0	680	260	420	0
24	6	260	260	0	260	0	0	0	260	0
25	7	120	120	0	240	120	160	40	120	0
26	8	40	40	0	0	80	0	0	40	0
27	9	0	0	0	0	80	80	80	0	0
28	10	80	80	0	0	0	0	0	80	0
29	11	180	180	0	400	220	400	220	180	0
30	12	220	220	0	0	0	0	0	220	0
31	End = previous end + order - requirement. A negative end is a shortage, which section 5.1 forbids.									
32	Carrying cost charges the end-of-month balance, so a plan that orders only when empty carries nothing.									

Figure 5.4.: The Plans sheet, rows 1 to 32. Four plans sit side by side, each owning an order column and an ending-inventory column, with a rule between the pairs. The three shipped plans reproduce the three rows of Table 5.3 exactly, so the sheet can be checked against the chapter before it is trusted with a plan of your own. Notice the ending-inventory column under lot-for-lot: it is zero in every month, which is what “carries nothing” means when you look at it rather than read it. Notice also that the purchase cost is \$92,000 under all four plans, which is Section 5.2.1 made visible.

5. Dynamic Lot Sizing

	A	B	C
1	The optimal plan		
2	<i>Chapter 5, section 5.4. Nothing here is typed; everything reads the Data sheet.</i>		
3			
4	RESULTS		
5	Optimal relevant cost, $V(N)$	\$2,480.00	
6	Orders placed	6	
7	Setup cost	\$1,800.00	
8	Carrying cost	\$680.00	
9	Setup plus carrying, which must equal $V(N)$	\$2,480.00	
10			
11	CHECK with the data as shipped		
12	$V(12) = \$2,480$ in six orders: \$1,800 of setup and \$680 of carrying.		
13	<i>The orders fall in months 1, 4, 5, 6, 10 and 12, for 220, 300, 420, 420, 260 and 220 belts.</i>		

Figure 5.5.: The **WagnerWhitin** sheet, rows 1 to 13. Nothing here is typed. Row 9 exists to be compared with row 5: the setup and carrying of the plan the sheet traced out must add to $V(N)$, because the recursion and the ledger are two readings of the same numbers, and a sheet that has been edited will usually break that first.

with $w(t, t) = k_t$ on the diagonal. That is one short formula, filled right and down, and it is also the increment Example 5.2 read off row 1. The triangle is Figure 5.6.

The recursion occupies a second triangle of the same shape, one entry for each t and u , holding $V(t - 1) + w(t, u)$. Above it, $V(u)$ is the MIN down column u and $S(u)$ is MATCH of that minimum in the same column. Laying the candidates out instead of collapsing them into one formula per period is the point of the sheet. Table 5.5 is a table the reader is handed in the text, and Figure 5.7 is a range the reader can look at.

The traceback sits below them and is the only part that needs thought. It walks backward, so each step reads the pointer of the period before the one the previous step started in, and stops when a step starts in period 1. INDEX on the S row does the walk without a macro.

The fourth block converts the chain into an order quantity per period, with COUNTIF to ask whether this period starts an order and INDEX(D1s_d,a):INDEX(D1s_d,b) to total what it covers. Figure 5.8 is

5. Dynamic Lot Sizing

	A	B	C	D	E	F	G	H	I	J	K	L	M
14	WINDOW COSTS $w(t,u)$ order in month t , cover through month u												
15	$t \backslash u$	1	2	3	4	5	6	7	8	9	10	11	12
16	1	\$300	\$360	\$600	\$1,500	\$3,180	\$4,480	\$5,200	\$5,480	\$5,480	\$6,200	\$8,000	\$10,420
17	2		\$300	\$420	\$1,020	\$2,280	\$3,320	\$3,920	\$4,160	\$4,160	\$4,800	\$6,420	\$8,620
18	3			\$300	\$600	\$1,440	\$2,220	\$2,700	\$2,900	\$2,900	\$3,460	\$4,900	\$6,880
19	4				\$300	\$720	\$1,240	\$1,600	\$1,760	\$1,760	\$2,240	\$3,500	\$5,260
20	5					\$300	\$560	\$800	\$920	\$920	\$1,320	\$2,400	\$3,940
21	6						\$300	\$420	\$500	\$500	\$820	\$1,720	\$3,040
22	7							\$300	\$340	\$340	\$580	\$1,300	\$2,400
23	8								\$300	\$300	\$460	\$1,000	\$1,880
24	9									\$300	\$380	\$740	\$1,400
25	10										\$300	\$480	\$920
26	11											\$300	\$520
27	12												\$300
28	Each cell adds one month to the cell on its left: the new requirement, carried from the order month.												

Figure 5.6.: The window costs, rows 14 to 28. This is Table 5.4 extended to twelve periods. Read row 1 across and the first six entries are the table the text gives. Notice columns 8 and 9: they are equal in every row, because month 9 requires nothing, so a window that reaches it costs exactly what one ending at month 8 costs. Notice also how fast the upper right grows. Covering the whole year from month 1 costs \$10,420 against \$300 for a single month, and that ratio is the whole reason an optimal plan orders more than once.

5. Dynamic Lot Sizing

	A	B	C	D	E	F	G	H	I	J	K	L	M
29	THE RECURSION $V(u)$ = the least of $V(t-1) + w(t,u)$ over the months t that could start the last order												
30													
31	month u	1	2	3	4	5	6	7	8	9	10	11	12
32	$V(u)$, the least cost through month u	\$300	\$360	\$600	\$900	\$1,200	\$1,460	\$1,620	\$1,700	\$1,700	\$2,000	\$2,180	\$2,480
33	$S(u)$, the month that last order starts in	1	1	1	4	5	5	6	6	6	10	10	12
34	MATCH returns the first month achieving the least cost, which breaks a tie toward the earlier order.												
35													
36	candidate $V(t-1) + w(t,u)$	1	2	3	4	5	6	7	8	9	10	11	12
37	last order starts in month 1	\$300	\$360	\$600	\$1,500	\$3,180	\$4,480	\$5,200	\$5,480	\$5,480	\$6,200	\$8,000	\$10,420
38	last order starts in month 2		\$600	\$720	\$1,320	\$2,580	\$3,620	\$4,220	\$4,460	\$4,460	\$5,100	\$6,720	\$8,920
39	last order starts in month 3			\$660	\$960	\$1,800	\$2,580	\$3,060	\$3,260	\$3,260	\$3,820	\$5,260	\$7,240
40	last order starts in month 4				\$900	\$1,320	\$1,840	\$2,200	\$2,360	\$2,360	\$2,840	\$4,100	\$5,860
41	last order starts in month 5					\$1,200	\$1,460	\$1,700	\$1,820	\$1,820	\$2,220	\$3,300	\$4,840
42	last order starts in month 6						\$1,500	\$1,620	\$1,700	\$1,700	\$2,020	\$2,920	\$4,240
43	last order starts in month 7							\$1,760	\$1,800	\$1,800	\$2,040	\$2,760	\$3,860
44	last order starts in month 8								\$1,920	\$1,920	\$2,080	\$2,620	\$3,500
45	last order starts in month 9									\$2,000	\$2,080	\$2,440	\$3,100
46	last order starts in month 10										\$2,000	\$2,180	\$2,620
47	last order starts in month 11											\$2,300	\$2,520
48	last order starts in month 12												\$2,480
49	A blank means the window would have to end before it began.												
50	THE TRACEBACK walk the pointers backwards from month N												
51	Step	1	2	3	4	5	6	7	8	9	10	11	12
52	Order starts in month	12	10	6	5	4	1						
53	Covers through month	12	11	9	5	4	3						
54	Step 1 is the last order. Each step reads the pointer of the month before the one it starts.												

Figure 5.7.: The recursion and the traceback, rows 29 to 54. The V and S rows are Table 5.7 and sit above the candidate block they read. The candidate block is Table 5.5 with nothing collapsed: every entry is a $V(t-1)$ from an earlier column plus a window cost from the triangle above. Notice that $V(8)$ and $V(9)$ are both \$1,700 and that S is 6 at both, which is month 9 requiring nothing showing up a second time. The yellow row is the traceback, and reading it right to left gives orders in months 1, 4, 5, 6, 10 and 12.

5. Dynamic Lot Sizing

that block, and its two totals are the arithmetic check the sheet is built around.

	A	B	C	D	E	F
56	THE PLAN one row per month					
57	Month	Requirement	Order	End	Setup	Carrying
58		1	40	220	180	\$300
59		2	60	0	120	\$0
60		3	120	0	0	\$0
61		4	300	300	0	\$300
62		5	420	420	0	\$300
63		6	260	420	160	\$300
64		7	120	0	40	\$0
65		8	40	0	0	\$0
66		9	0	0	0	\$0
67		10	80	260	180	\$300
68		11	180	0	0	\$0
69		12	220	220	0	\$300
70	Total				\$1,800	\$680
71	These two totals are the setup and carrying reported at the top, and they add to V(N).					

Figure 5.8.: The plan, rows 56 to 71. The chain above has become an order quantity per month, and the two totals are the \$1,800 and \$680 of Table 5.12. Notice month 6: it receives 420 belts and ends the month holding 160, which is the one place in the optimal plan where a month both orders and carries something forward.

5.8.4. The Three Rules

The *Heuristics* sheet illustrates Section 5.5.1's claim. Every rule on it reads the same window costs from the sheet before it, so the three answers in Figure 5.9 differ because the rules differ and for no other reason.

Each rule gets a triangle of scores over the same (t, u) pairs, and beside it a triangle of zeros and ones answering "does the rule stop here?" The stop period for an order placed in period t is then $\text{MATCH}(1, \dots)$ along row t , the first period whose answer is one. Figure 5.10 is Silver-Meal's scores and Figure 5.11 is the test that reads them.

5. Dynamic Lot Sizing

	A	B	C	D	E
1	Three heuristics, one procedure				
2	<i>Chapter 5, section 5.5. Each rule extends a window from the order month until its own test says stop.</i>				
3					
4	RESULTS				
5		Silver-Meal	Least unit co	Part-period balancing	
6	Orders placed	5	5	5	
7	Relevant cost	\$2,660.00	\$3,040.00	\$2,600.00	
8	Penalty against the optimum	7.26%	22.58%	4.84%	
9					
10	CHECK with the data as shipped				
11	<i>Silver-Meal \$2,660, 7.26% above the optimum. Least unit cost \$3,040, 22.58%.</i>				
12	<i>Part-period balancing \$2,600, 4.84%. All three place five orders; the optimum places six.</i>				
13	<i>The optimum on the WagnerWhitin sheet is \$2,480.</i>				

Figure 5.9.: The Heuristics sheet, rows 1 to 13. The three columns are the three heuristic rows of Table 5.12, computed on this sheet from the window costs of the last one.

Figure 5.12 is the same pair of blocks for least unit cost, and only the divisor has changed. That one change is enough to make the rule order 220 belts in month 1 where Silver-Meal orders 100.

Part-period balancing needs a different test, and Figure 5.13 shows why. Its score is what the window carries, which only grows as the window grows, so the last period at or under k_t can be found with an ordinary ascending MATCH and no triangle of ones is needed. The two neighbors are then compared to see which comes closer. A score that can fall, as Silver-Meal's can, cannot be looked up that way.

Below the triangles, a chain row per rule starts at period 1 and reads each order's start from the previous order's stop, and a ledger per rule turns the chain into a plan and a cost. Figure 5.14 is both, and the three relevant costs it totals are the three heuristic rows of Table 5.12.

5.8.5. The Mixed Integer Program

The Solver sheet is Equation 5.9 laid out one row per period: the order quantity and the binary flag as changing cells, the balance and the two

5. Dynamic Lot Sizing

	A	B	C	D	E	F	G	H	I	J	K	L	M
14	SILVER-MEAL the relevant cost of the window, divided by its length												
15	<i>Extend while the cost per period falls. Stop at the first month that raises it.</i>												
16	t \ u	1	2	3	4	5	6	7	8	9	10	11	12
17	1	300.00	180.00	200.00	375.00	636.00	746.67	742.86	685.00	608.89	620.00	727.27	868.33
18	2		300.00	210.00	340.00	570.00	664.00	653.33	594.29	520.00	533.33	642.00	783.64
19	3			300.00	300.00	480.00	555.00	540.00	483.33	414.29	432.50	544.44	688.00
20	4				300.00	360.00	413.33	400.00	352.00	293.33	320.00	437.50	584.44
21	5					300.00	280.00	266.67	230.00	184.00	220.00	342.86	492.50
22	6						300.00	210.00	166.67	125.00	164.00	286.67	434.29
23	7							300.00	170.00	113.33	145.00	260.00	400.00
24	8								300.00	150.00	153.33	250.00	376.00
25	9									300.00	190.00	246.67	350.00
26	10										300.00	240.00	306.67
27	11											300.00	260.00
28	12												300.00

Figure 5.10.: The Silver-Meal scores, rows 14 to 28. Every entry is a window cost from Figure 5.6 divided by the length of its window. Read row 1 across: \$300.00, \$180.00, then \$200.00. The rise at $u = 3$ is where the rule stops, and it is the first column of Table 5.10. Now read row 5, which is where the rule loses its money. The score falls from \$300.00 through \$280.00, \$266.67 and \$230.00 to \$184.00 at $u = 9$, and only then rises to \$220.00. Nothing in that run tells the rule to stop, so one order covers five months.

5. Dynamic Lot Sizing

	O	P	Q	R	S	T	U	V	W	X	Y	Z	AA
stop here?													
1	2	3	4	5	6	7	8	9	10	11	12	stops at	
0	1	1	1	1	0	0	0	1	1	1	1	1	2
	0	1	1	1	0	0	0	1	1	1	1	1	3
		0	1	1	0	0	0	1	1	1	1	1	4
			1	1	0	0	0	1	1	1	1	1	4
				0	0	0	0	1	1	1	1	1	9
					0	0	0	1	1	1	1	1	9
						0	0	1	1	1	1	1	9
							0	1	1	1	1	1	9
								0	1	1	1	1	10
									0	1	1	1	11
										0	1	1	12
											0	1	12
												1	12

Figure 5.11.: The stopping test beside the Silver-Meal scores, columns O through AA of the same rows. Each entry answers one question: does the score rise at the next period? The stop for an order placed in month t is the first one in row t , which MATCH finds, and the yellow column collects them. Notice that the column reads 2, 3, 4, 4, then 9 four times over. A rule that reaches month 9 from anywhere in months 5 through 8 stops in the same place, which is what a period requiring nothing does to a rule that divides by the length of the window.

5. Dynamic Lot Sizing

	A	B	C	D	E	F	G	H	I	J	K	L	M
30	LEAST UNIT COST the relevant cost of the window, divided by the units it covers												
31	<i>A window covering no units has no cost per unit, so it is left blank and stepped over.</i>												
32	t \ u	1	2	3	4	5	6	7	8	9	10	11	12
33	1	7.50	3.60	2.73	2.88	3.38	3.73	3.94	4.03	4.03	4.31	4.94	5.66
34	2		5.00	2.33	2.13	2.53	2.86	3.06	3.15	3.15	3.43	4.06	4.79
35	3			2.50	1.43	1.71	2.02	2.21	2.30	2.30	2.58	3.22	3.95
36	4				1.00	1.00	1.27	1.45	1.54	1.54	1.84	2.50	3.25
37	5					0.71	0.82	1.00	1.10	1.10	1.43	2.18	2.98
38	6						1.15	1.11	1.19	1.19	1.64	2.53	3.38
39	7							2.50	2.13	2.13	2.42	3.10	3.75
40	8								7.50	7.50	3.83	3.33	3.62
41	9										4.75	2.85	2.92
42	10										3.75	1.85	1.92
43	11											1.67	1.30
44	12												1.36

Figure 5.12.: The least unit cost scores, rows 30 to 44. Every entry is the same window cost divided by the units the window covers instead of by its length. Row 1 runs 7.50, 3.60, 2.73, then 2.88, which is the second column of Table 5.10. Notice the blank on the diagonal at $t = u = 9$. That window covers no units at all, so it has no cost per unit, and the rule steps over it instead of dividing by zero. It is the only cell in the workbook that has to be guarded, and Section 5.9.5 shows the guard.

5. Dynamic Lot Sizing

	A	B	C	D	E	F	G	H	I	J	K	L	M
46	PART-PERIOD BALANCING what the window carries, against the order cost												
47	<i>Carrying only grows as the window grows, so the last month at or under k is found by a lookup.</i>												
48	t \ u	1	2	3	4	5	6	7	8	9	10	11	12
49	1	0.00	60.00	300.00	1,200.00	2,880.00	4,180.00	4,900.00	5,180.00	5,180.00	5,900.00	7,700.00	10,120.00
50	2		0.00	120.00	720.00	1,980.00	3,020.00	3,620.00	3,860.00	3,860.00	4,500.00	6,120.00	8,320.00
51	3			0.00	300.00	1,140.00	1,920.00	2,400.00	2,600.00	2,600.00	3,160.00	4,600.00	6,580.00
52	4				0.00	420.00	940.00	1,300.00	1,460.00	1,460.00	1,940.00	3,200.00	4,960.00
53	5					0.00	260.00	500.00	620.00	620.00	1,020.00	2,100.00	3,640.00
54	6						0.00	120.00	200.00	200.00	520.00	1,420.00	2,740.00
55	7							0.00	40.00	40.00	280.00	1,000.00	2,100.00
56	8								0.00	0.00	160.00	700.00	1,580.00
57	9									0.00	80.00	440.00	1,100.00
58	10										0.00	180.00	620.00
59	11											0.00	220.00
60	12												0.00

Figure 5.13.: The part-period balancing scores, rows 46 to 60. Here the entry is what the window carries, which is the window cost less the order cost. Row 1 runs 0.00, 60.00, 300.00, then 1,200.00, and the rule is looking for the entry closest to $k = \$300$. It finds it exactly at $u = 3$. Notice that every row increases from left to right without exception, which is the property that lets this rule use an ordinary ascending lookup and needs no stopping test beside it.

5. Dynamic Lot Sizing

	A	B	C	D	E	F	G	H	I	J	K	L	M	N
63	THE CHAINS the first order falls in month 1, and each order starts where the last one stopped													
64	Order	1	2	3	4	5	6	7	8	9	10	11	12	
65	Silver-Meal	1	3	5	10	12								
66	Least unit cost	1	4	6	8	12								
67	Part-period balancing	1	4	6	10	12								
68	A blank means the previous order already reached month N, so the plan needs no further order.													
69														
70	THE PLANS one row per month, one group of columns per rule													
71			Silver-Meal				Least unit cost				Part-period balancing			
72	Month	Requirement	Order	End	Setup	Carrying	Order	End	Setup	Carrying	Order	End	Setup	Carrying
73	1	40	100	60	\$300	\$60	220	180	\$300	\$180	220	180	\$300	\$180
74	2	60	0	0	\$0	\$0	0	120	\$0	\$120	0	120	\$0	\$120
75	3	120	420	300	\$300	\$300	0	0	\$0	\$0	0	0	\$0	\$0
76	4	300	0	0	\$0	\$0	720	420	\$300	\$420	720	420	\$300	\$420
77	5	420	840	420	\$300	\$420	0	0	\$0	\$0	0	0	\$0	\$0
78	6	260	0	160	\$0	\$160	380	120	\$300	\$120	420	160	\$300	\$160
79	7	120	0	40	\$0	\$40	0	0	\$0	\$0	0	40	\$0	\$40
80	8	40	0	0	\$0	\$0	300	260	\$300	\$260	0	0	\$0	\$0
81	9	0	0	0	\$0	\$0	0	260	\$0	\$260	0	0	\$0	\$0
82	10	80	260	180	\$300	\$180	0	180	\$0	\$180	260	180	\$300	\$180
83	11	180	0	0	\$0	\$0	0	0	\$0	\$0	0	0	\$0	\$0
84	12	220	220	0	\$300	\$0	220	0	\$300	\$0	220	0	\$300	\$0
85	Total				\$1,500	\$1,160			\$1,500	\$1,540			\$1,500	\$1,100
86	Setup plus carrying for each rule is the relevant cost reported at the top.													

Figure 5.14.: The chains and the three plans, rows 63 to 86. Each chain starts in month 1 and each order after it starts where the previous one stopped; a blank means the horizon is already covered. Reading the three chains down, the rules order in months 1, 3, 5, 10, 12; in 1, 4, 6, 8, 12; and in 1, 4, 6, 10, 12. All three place five orders and no two agree on where. The three ledgers below are Table 5.11, and their totals are \$1,500 of setup in every case against carrying of \$1,160, \$1,540 and \$1,100. Notice the 840 in month 5 of the Silver-Meal ledger, which Figure 5.10 explained.

5. *Dynamic Lot Sizing*

cost terms as formulas, and a column holding $Q_t - My_t$, which Solver constrains to be at most zero. The sheet ships holding the optimal plan, so the objective already reads \$2,480. Clear the two changing columns and Solver finds its way back. Figure 5.15 is the whole sheet, instructions and all.

5.8.6. What the Worksheet Cannot Do

The rolling horizon of Section 5.7 is not in the workbook, and it is not an oversight. Table 5.14 is the cost of a sequence of twelve plans, eleven of which were computed, discarded, and never executed. A worksheet holds one instance of a model. Producing Table 5.15 needs 2,000 schedules solved twelve times each under two rules, or 48,000 solves, and a worksheet that is asked to do that stops being a thing anyone can read.

That is the boundary. A worksheet is the right tool for a model you want to look at, and the wrong tool for a model you want to run many times. That is where Section 5.9 begins.

5.9. Designing the Software

Section 5.8 built the models as worksheets, where the structure is visible. A program has no such thing. Its structure is a set of decisions about which classes exist and what each one is responsible for, and none of those decisions leaves a mark in the finished code. So we make them first, in the order such an analysis proceeds: what the software must do, what the domain calls things, what each thing knows and does, and how they collaborate.

This chapter is a good one to design, because at first glance it is six unrelated recipes and at second glance it is one recipe with six settings. Getting that right in the code is what makes Table 5.12 a comparison.

5. Dynamic Lot Sizing

	A	B	C	D	E	F	G	H
1	The same problem for Solver							
2	Chapter 5, section 5.4.3.							
3								
4	OBJECTIVE							
5	Total relevant cost	\$2,480.00						
6	Largest shortage	0						
7								
8	HOW							
9	Data > Solver. Set objective B5 to a minimum by changing C14:D25.							
10	Subject to: D14:D25 = binary, C14:C25 >= 0, E14:E25 >= 0, H14:H25 <= 0							
11	Choose the Simplex LP engine. Section 5.4.3 explains why the model is linear in these variables.							
12								
13	Month	Requirement	Order Q(t)	Order? y(t)	End I(t)	Setup	Carrying	Q - My
14	1	40	220	1	180	\$300	\$180	-1,620
15	2	60	0	0	120	\$0	\$120	0
16	3	120	0	0	0	\$0	\$0	0
17	4	300	300	1	0	\$300	\$0	-1,540
18	5	420	420	1	0	\$300	\$0	-1,420
19	6	260	420	1	160	\$300	\$160	-1,420
20	7	120	0	0	40	\$0	\$40	0
21	8	40	0	0	0	\$0	\$0	0
22	9	0	0	0	0	\$0	\$0	0
23	10	80	260	1	180	\$300	\$180	-1,580
24	11	180	0	0	0	\$0	\$0	0
25	12	220	220	1	0	\$300	\$0	-1,620
26								
27	Column H is the linking constraint: an order can be positive only in a month whose flag is one.							
28	M is 1840, the whole horizon's requirement, which is the most any single order would ever carry.							
29								
30	CHECK with the data as shipped							
31	The sheet ships holding the optimal plan, so B5 already reads \$2,480 and B6 reads zero.							
32	Clear C14:D25 to zero and Solver should find its way back to \$2,480.							

Figure 5.15.: The Solver sheet, rows 1 to 32, with the constraints written out above the model. Column C is Q_t and column D is y_t , and the two together are the changing cells. Column H is $Q_t - My_t$, which Solver holds at or below zero, and that single column is the whole of what makes Equation 5.9 linear. The sheet ships holding the optimal plan, so the objective in B5 reads \$2,480 and the shortage in B6 reads zero. Notice column H in the months that order: it is a large negative number, meaning the linking constraint is slack and doing nothing. It binds only at zero, which is its job, and a reader who expects a constraint to be tight at the optimum should look at this column and ask why this one is not.

5.9.1. What the Software Must Do

Reading back over the chapter, we find that the software has to

1. hold a requirements schedule for one item, with costs that may vary by period,
2. price any window $w(t, u)$,
3. evaluate a plan someone else chose, reporting setup, purchase and carrying separately,
4. produce a plan under lot-for-lot, an adjusted economic order quantity, a periodic order quantity, Silver-Meal, least unit cost, part-period balancing, and Wagner-Whitin,
5. report the penalty of one plan against another, and
6. roll a horizon forward, committing part of each plan and discarding the rest.

Requirement 6 is the one Section 5.8 could not meet, and it is the reason for writing a program at all.

5.9.2. Finding the Nouns

Underline the nouns in the requirements, and three survive.

A **requirements schedule** is what the planner is handed. It is the item's requirements and its costs over a horizon, and it is the only thing in the chapter that is data and not decision.

A **plan** is what the planner produces. It is an order quantity per period, and it knows what it costs.

A **rule** is how one becomes the other. Seven of them appear in this chapter, and Section 5.6 ranks them, which means the software has to be able to hold one without knowing which.

Two candidate nouns are instructive rejections.

5. Dynamic Lot Sizing

Window looks like a class. It has a start, an end, a cost, and the chapter talks about it constantly. It fails because a window has no behavior of its own and no life of its own. It exists for the length of one comparison inside one rule, and $w(t, u)$ is a function of the schedule and two integers. Thus, making it a class would put $N^2/2$ objects on the heap to hold a number apiece.

Order also looks like a class, and it is a real thing in a warehouse. Here it is Q_t , a number in a position. Every question the chapter asks about an order is a question about a plan: how many are there, what do they cost, and when do they fall.

Thus, the schedule owns the window cost. That is the decision the rest of the design rests on. Every rule in Table 5.12 prices the same windows, so computing them in one place means the heuristics and the algorithm are compared on identical numbers and not on two implementations of Equation 5.7.

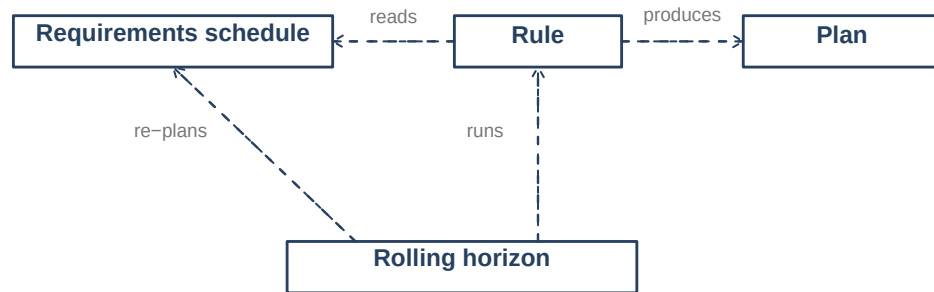


Figure 5.16.: The concepts of this chapter before any thought about implementation. A rule reads a requirements schedule and produces a plan. A rolling horizon runs a rule against one schedule repeatedly, keeping only part of each plan it gets back.

5. Dynamic Lot Sizing

5.9.3. What Each Thing Knows and Does

For each of the three we write a card, listing what it is responsible for and whom it needs in order to meet that responsibility.

Table 5.16.: Responsibilities for the four classes.

Class	Knows and does	Collaborates with
RequirementsSchedule	the requirements and the three cost streams; the requirement over a span; the unit cost held from one period to another; the window cost; the variability coefficient; the forced order period	LotSizingPlan
LotSizingPlan	the order quantities; ending inventory; setup, purchase and carrying; whether it orders only when empty; its penalty against another plan	RequirementsSchedule
LotSizingRule	how far a replenishment should reach	RequirementsSchedule, LotSizingPlan
RollingHorizon	the true schedule, the planning window, the frozen span; what the committed orders actually cost	all three

5. Dynamic Lot Sizing

Table 5.16 illustrates one thing by leaving it out. A plan does not know its ending inventory as stored data. Equation 5.1 determines the inventory from the orders, so storing both would allow a plan to exist whose inventory no sequence of orders produces. The plan stores the quantities and computes the rest.

Notice also that the plan validates itself. A plan that runs short in some period violates assumption 6, and the constructor refuses it. That is not defensive programming for its own sake. Thus, every plan anywhere in the program is feasible, and no method downstream has to ask.

5.9.4. The Schedule

```
class RequirementsSchedule(  
    val label: String,  
    requirements: List<Double>,  
    orderCosts: List<Double>,  
    unitCosts: List<Double>,  
    holdingRates: List<Double>,  
) {  
    /** The number of periods in the planning horizon,  
        ↪ written N in the text. */  
    val horizon: Int get() = d.size  
  
    /** The periods, 1 through N, as a range to iterate over.  
        ↪ */  
    val periods: IntRange get() = 1..horizon  
  
    /** The requirement of periods [from] through [through],  
        ↪ written d[t,u]. */  
    fun requirementOver(from: Int, through: Int): Double {  
        requireWindow(from, through)
```

5. Dynamic Lot Sizing

```
        return (from..through).sumOf { d[it - 1] }  
    }  
}
```

Periods are numbered 1 through N on the outside and from zero on the inside, which is the ordinary arrangement. The alternative, exposing zero-based periods, would make every call site translate and every example in this chapter read differently from the text.

The costs arrive as four parallel lists instead of four scalars, so a period-varying instance needs no second class. For example, the constant-cost case that every worked example in this chapter utilizes is a factory method:

```
companion object {  
    /**  
     * The common case: one order cost, one unit cost and one  
    ↪ carrying charge for the  
     * whole horizon, with h = ic as in Section 1.5.3.  
     */  
    fun constantCosts(  
        label: String,  
        requirements: List<Double>,  
        orderCost: Double,  
        unitCost: Double,  
        carryingCharge: Double,  
    ): RequirementsSchedule = RequirementsSchedule(  
        label = label,  
        requirements = requirements,  
        orderCosts = List(requirements.size) { orderCost },  
        unitCosts = List(requirements.size) { unitCost },  
        holdingRates = List(requirements.size) {  
    ↪ carryingCharge * unitCost },  
    )  
}
```

5. Dynamic Lot Sizing

```
}
```

The window cost is the schedule's central method, and it is memoized, since every rule asks for the same $N(N + 1)/2$ values and Section 5.4.1 requires the arc costs to be settled before any decision is taken.

```
/**
 * The window cost: order in [from] enough to cover the
 * ↪ requirements of periods
 * [from] through [through], and pay for the holding that
 * ↪ implies.
 */
fun windowCost(from: Int, through: Int): Double {
    requireWindow(from, through)
    val key = (from - 1) * horizon + (through - 1)
    cached[key]?.let { return it }
    val value = k[from - 1] + (from..through).sumOf {
        ↪ unitCostHeld(from, it) * d[it - 1] }
    cached[key] = value
    return value
}

/**
 * Setup plus carrying for a window. This is the cost the
 * ↪ heuristics compare. The
 * purchase term is left out on purpose: it is the same under
 * ↪ every plan while the
 * unit cost is constant, and a criterion that divides by the
 * ↪ length of the window
 * would otherwise be dominated by it.
 */
fun relevantWindowCost(from: Int, through: Int): Double =
    orderCostIn(from) + carryingCostOver(from, through)
```

5. Dynamic Lot Sizing

Two window costs, and the difference between them is not cosmetic. Equation 5.4 includes the purchase term and the recursion uses it, and that is right: adding the same constant to every plan changes no comparison. A heuristic that divides by the length of the window is a different matter. The purchase term grows with the window while the setup does not, so a cost-per-period built on Equation 5.4 falls simply because the window is longer, and Silver-Meal would extend to the end of the horizon on every instance. It has to divide the relevant cost, and `relevantWindowCost` supplies it.

5.9.5. Three Rules Are One Rule

Section 5.5.1 claims that Silver-Meal, least unit cost and part-period balancing are one procedure with three stopping rules. If that claim is true, the code should contain the procedure once.

```
interface LotSizingRule {
    val name: String
    fun plan(schedule: RequirementsSchedule): LotSizingPlan
}

abstract class WindowRule(override val name: String) :
    ↪ LotSizingRule {

    /** The figure of merit for covering [from] through
        ↪ [through], or null to skip. */
    protected abstract fun scoreOf(
        schedule: RequirementsSchedule,
        from: Int,
        through: Int,
    ): Double?

    final override fun plan(schedule: RequirementsSchedule):
        ↪ LotSizingPlan {
```

5. Dynamic Lot Sizing

```
val orders = mutableListOf<Int>()
var from = 1
while (from <= schedule.horizon) {
    orders.add(from)
    from = 1 + lastPeriodCovered(schedule, from)
}
return schedule.planFrom(orders)
}

/** How far a replenishment placed in [from] should
    ↪ reach. */
fun lastPeriodCovered(schedule: RequirementsSchedule,
    ↪ from: Int): Int {
    var best = from
    var previous: Double? = null
    for (through in from..schedule.horizon) {
        val score = scoreOf(schedule, from, through) ?:
            ↪ continue
        if (previous != null && score > previous +
            ↪ TOLERANCE) break
        previous = score
        best = through
    }
    return best
}
}
```

Then two of the three rules are three lines apiece.

```
/** Silver-Meal: the least cost per period. */
object SilverMeal : WindowRule("Silver-Meal") {
    override fun scoreOf(schedule: RequirementsSchedule,
        ↪ from: Int, through: Int): Double =
```

5. Dynamic Lot Sizing

```
        schedule.relevantWindowCost(from, through) / (through
↪ - from + 1)
    }

/** Least unit cost: the least cost per unit. */
object LeastUnitCost : WindowRule("Least unit cost") {
    override fun scoreOf(schedule: RequirementsSchedule,
↪ from: Int, through: Int): Double? {
        val units = schedule.requirementOver(from, through)
        return if (units <= 0.0) null else
            ↪ schedule.relevantWindowCost(from, through) /
            ↪ units
    }
}
```

That is Table 5.9, executable. The nullable return is the third column of that table's one irregularity: a window covering no units has no cost per unit, so the rule steps over it instead of dividing by zero. Month 9 of the belt is such a period.

The third rule does not fit, and forcing it in would be the mistake.

```
/**
 * Part-period balancing: extend until the carrying cost of
↪ the window is as close as
 * it can get to the order cost.
 *
 * It does not fit WindowRule because it does not walk a
↪ score to a minimum. It walks
 * a gap to its closest approach, and the period it keeps may
↪ be the one before the
 * test failed and not the one that failed.
 */
```

5. Dynamic Lot Sizing

```
object PartPeriodBalancing : LotSizingRule {
    override val name: String = "Part-period balancing"

    fun lastPeriodCovered(schedule: RequirementsSchedule,
        ↪ from: Int): Int {
        val setup = schedule.orderCostIn(from)
        var best = from
        for (through in from + 1..schedule.horizon) {
            val carried = schedule.carryingCostOver(from,
                ↪ through)
            if (carried > setup) {
                val below = schedule.carryingCostOver(from,
                    ↪ through - 1)
                return if (abs(carried - setup) < abs(below
                    ↪ - setup)) through else through - 1
            }
            best = through
        }
        return best
    }
}
```

A template method earns its place when three things share a shape, and should be abandoned the moment a fourth thing only appears to. Part-period balancing can step back, and a stopping rule that returns “the one before the one that failed” cannot be expressed as a figure of merit walked to a minimum. Section 5.8.4 found the same boundary in the worksheet, where the two walking rules need a triangle of stopping tests and part-period balancing needs an ascending lookup. That two independent implementations drew the line in the same place is a sign the line is in the domain and not in either implementation.

5. Dynamic Lot Sizing

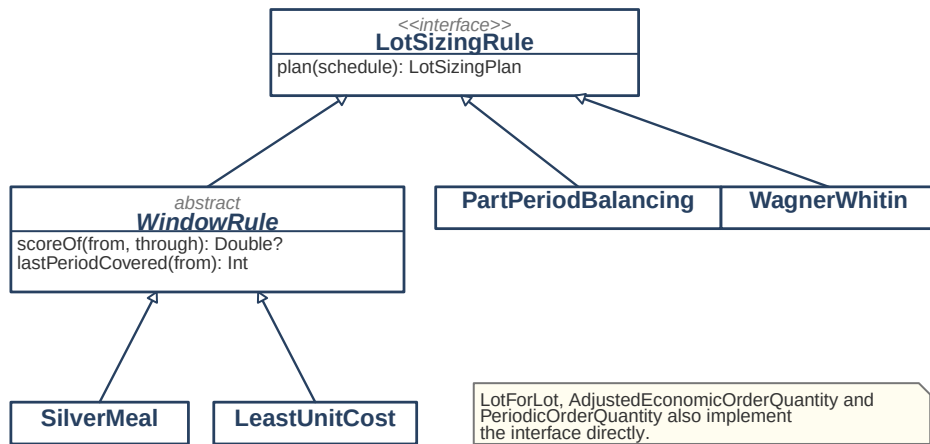


Figure 5.17.: The rule hierarchy. Silver-Meal and least unit cost share the window-walking procedure through WindowRule and differ only in the score. Part-period balancing and Wagner-Whitin implement the interface directly, because neither walks a score to a minimum.

5.9.6. The Algorithm Is Not a Heuristic

WagnerWhitin implements `LotSizingRule` and shares none of `WindowRule`. That is the arrangement Section 5.5 describes: an optimal algorithm and a stopping rule are different kinds of thing, and the interface says only that both answer the same question.

```
object WagnerWhitin : LotSizingRule {
    override val name: String = "Wagner-Whitin"

    /** The value function, the predecessor pointers, and the
     *  ↪ plan they trace out. */
    fun solve(schedule: RequirementsSchedule): Solution {
        val n = schedule.horizon
        val value = DoubleArray(n + 1)
        val from = IntArray(n + 1)
        for (t in 1..n) {
            var best = Double.MAX_VALUE
            var argument = 1
            for (s in 1..t) {
                val candidate = value[s - 1] +
                    ↪ schedule.windowCost(s, t)
                // A tie keeps the earliest period achieving
                ↪ it, the same one that
                // scanning s upward with a strict comparison
                ↪ keeps, and that MATCH
                // finds on the worksheet. Both artifacts
                ↪ must report the same S(t).
                if (candidate < best - TOLERANCE) {
                    best = candidate
                    argument = s
                }
            }
            value[t] = best
        }
    }
}
```

5. Dynamic Lot Sizing

```
        from[t] = argument
    }
    ...
}
}
```

`solve` returns more than a plan. It returns V and S , because Example 5.3 prints them and because a reader who wants to check the traceback needs the pointers. A method that returned only the plan would make Table 5.7 impossible to produce without solving the problem twice.

The two economic order quantity rules call into Chapter 3 instead of recomputing it.

```
class AdjustedEconomicOrderQuantity : LotSizingRule {
    override fun plan(schedule: RequirementsSchedule):
        ↪ LotSizingPlan {
        val quantity =
            ↪ EconomicOrderQuantity.orderQuantityFor(
                orderCost = schedule.orderCostIn(1),
                demandRate = schedule.averageRequirement,
                holdingRate = schedule.holdingRateIn(1),
            )
        ...
    }
}
```

This is the same `EconomicOrderQuantity` that Section 3.11 built, used unchanged. Equation 3.17 appears once in this book's code, so there is one place to correct when it is wrong.

5.9.7. How the Objects Solve It

Figure 5.18 illustrates one call. The caller hands a rule a schedule and gets a plan.

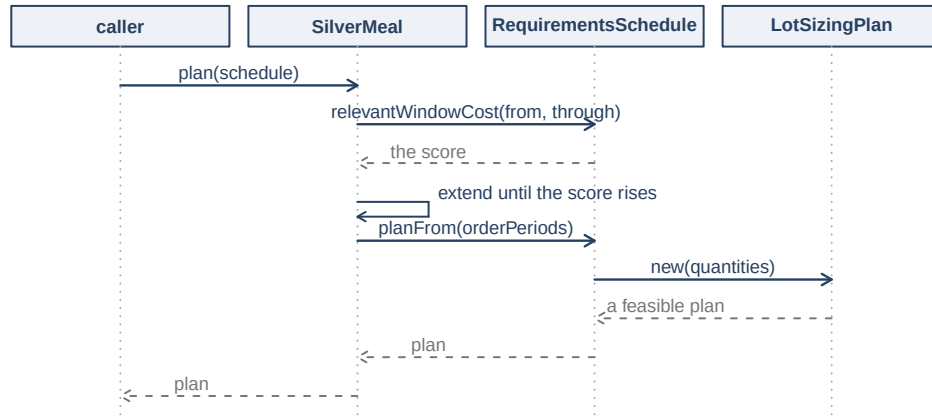


Figure 5.18.: Producing a Silver-Meal plan. The rule asks the schedule to price windows until its score rises, which fixes the order periods, and then asks the schedule to build the plan. Notice that the rule computes no cost of its own. It decides which costs to ask for. The dashed arrows are returns.

The schedule sits in the middle of every collaboration. That follows from giving it the window cost. A rule never computes a cost; it decides which costs to ask for. That is what lets Section 5.6 rank the rules: they are being compared on their decisions and not on their arithmetic.

5.9.8. Rolling the Horizon

`RollingHorizon` is the class that exists because a worksheet cannot do this.

5. Dynamic Lot Sizing

```
class RollingHorizon(  
    private val actual: RequirementsSchedule,  
    private val planningHorizon: Int,  
    private val freeze: Int = 1,  
) {  
    /**  
     * Run [rule] on a window that advances by [freeze]  
     ↪ periods at a time, implementing  
     * only the frozen orders, and report what the  
     ↪ implemented orders actually cost.  
     */  
    fun realizedPlan(rule: LotSizingRule): LotSizingPlan {  
        val n = actual.horizon  
        val placed = DoubleArray(n)  
        var period = 1  
        while (period <= n) {  
            val onHand = carriedInto(placed, period)  
            val committedTo = minOf(period + freeze - 1, n)  
            val net = netRequirements(period, minOf(period +  
            ↪ planningHorizon - 1, n), onHand)  
            if (net != null) {  
                val plan = rule.plan(net.window)  
                for (t in period..committedTo) {  
                    val offset = t - net.firstPlanned + 1  
                    if (offset in 1..net.window.horizon)  
                        ↪ placed[t - 1] = plan.orderIn(offset)  
                }  
            }  
            period = committedTo + 1  
        }  
        return LotSizingPlan(actual, placed.toList())  
    }  
}
```

5. Dynamic Lot Sizing

The subtle part is `netRequirements`. The planner at period t may be holding stock left over from an earlier order, and assumption 8 says a schedule starts empty. So the stock on hand is netted against the nearest requirements, exactly as a requirements record nets it, and planning begins at the first period the stock does not cover. Getting this wrong produces a plan that orders on top of stock it already has, which every rule in the chapter is supposed to forbid.

Notice that `realizedPlan` returns a `LotSizingPlan` built against the *true* schedule and not against any of the windows. The plans the rule produced along the way were mostly discarded. What is reported is the cost of the orders that were actually placed, the only thing Table 5.14 could mean.

5.9.9. Solving the Chapter's Examples

Every figure in this chapter comes from running this code. The item of Table 5.1 is

```
val belt = RequirementsSchedule.constantCosts(  
    label = "Mower deck belt",  
    requirements = listOf(40.0, 60.0, 120.0, 300.0, 420.0,  
↪ 260.0,  
                           120.0, 40.0, 0.0, 80.0, 180.0,  
                           ↪ 220.0),  
    orderCost = 300.0, unitCost = 50.0, carryingCharge =  
↪ 0.02,  
)  
  
println(belt.variabilityCoefficient) // 0.6191, Section 5.6  
println(belt.forcedOrderPeriod())   // 5, Section 5.3
```

Table 5.12 is a loop, and that is what the design was for.

5. Dynamic Lot Sizing

```
val optimum = WagnerWhitin.plan(belt)
val rules = listOf(
    LotForLot, AdjustedEconomicOrderQuantity(),
    ↪ PeriodicOrderQuantity(),
    ↪ LeastUnitCost, PartPeriodBalancing, SilverMeal,
    ↪ WagnerWhitin,
)
for (r in rules) {
    val p = r.plan(belt)
    println("%-32s %6d %9.0f %9.0f %9.0f %8.2f%%".format(
        r.name, p.orderCount, p.setupCost, p.carryingCost,
    ↪ p.relevantCost,
        p.penaltyAgainst(optimum).relativeError * 100.0))
}
```

Example 5.3 needs the recursion itself and not only its answer.

```
val quarter = RequirementsSchedule.constantCosts(
    label = "First half year",
    requirements = listOf(40.0, 60.0, 120.0, 300.0, 420.0,
    ↪ 260.0),
    ↪ orderCost = 300.0, unitCost = 50.0, carryingCharge =
    ↪ 0.02,
)
val solution = WagnerWhitin.solve(quarter)
for (t in 1..6) {
    val purchases = 50.0 * (1..t).sumOf {
        ↪ quarter.requirementIn(it) }
    println("V(%d) = %.0f, S(%d) = %d"
        .format(t, solution.valueAt(t) - purchases, t,
    ↪ solution.orderedFromAt(t)))
}
```

5. Dynamic Lot Sizing

The subtraction is Section 5.2.1. Equation 5.4 carries the purchase term, so `valueAt` carries it too, and what Example 5.3 reports is the relevant part. Leaving the term in shifts every V by the same amount and changes no decision, which is the argument of Section 5.2.1 seen from the other side.

And Table 5.14 is two lines per row.

```
for (window in 2..6) {  
    val sm = RollingHorizon(belt, window, freeze =  
        ↪ 1).realizedPlan(SilverMeal)  
    val ww = RollingHorizon(belt, window, freeze =  
        ↪ 1).realizedPlan(WagnerWhitin)  
    println("%d %.0f %.0f".format(window, sm.relevantCost,  
        ↪ ww.relevantCost))  
}
```

Those four fragments produce every number the chapter prints. Section 5.10 collects what it argued with them.

5.10. Summary

The chapter removed the constant demand rate and everything that depended on it. With no repeating cycle there is no average cost per cycle to minimize, so the economic order quantity has nothing to be derived from. What replaces it is a finite horizon with a decision in every period.

Two properties make the problem small enough to solve. Zero-inventory ordering says an optimal plan never orders into a shelf that is not empty, which means a plan is nothing but a choice of order periods. The window cost $w(t, u)$ prices any such choice, and it depends on no decision, so all of the window costs can be computed once.

5. Dynamic Lot Sizing

Those two together turn the problem into a shortest path on an acyclic network, and Equation 5.8 solves it in $O(N^2)$. The same problem is a mixed integer program, slower, but it accepts constraints the recursion cannot.

The heuristics exist because Equation 5.8 needs the whole horizon and the whole horizon is a forecast. Three of them are one procedure with three stopping rules. On the mower deck belt they land between 4.84% and 22.58% above the optimum, which is the same order of magnitude as the penalty for getting k/h wrong by a factor of two.

The rolling horizon is where all of this is actually used, and it removes Wagner-Whitin's guarantee. It does not, on the evidence of Table 5.15, hand the advantage to a heuristic. What it does is make the question empirical.

Chapter 6 takes these methods up again with the requirements computed from a bill of material instead of forecast. That is where they do most of their work.

Table 5.17.: Notation introduced in this chapter.

Symbol	Meaning
N	number of periods in the horizon
d_t	requirement in period t
$d[t, u]$	requirement of periods t through u
k_t, c_t, h_t	order cost, unit cost, holding rate in period t
Q_t, I_t, y_t	order quantity, ending inventory, order indicator
$\hat{c}(t, u)$	cost of a unit bought in t and used in u
$w(t, u)$	cost of a replenishment in t covering through u

5. Dynamic Lot Sizing

Symbol	Meaning
$V(u), S(u)$	least cost through period u , and the period its last order starts in
VC	variability coefficient

5.11. Exercises

Unless an exercise says otherwise, use these conventions so that your answer and the instructor's agree. Periods are numbered from one. Inventory starts at zero and ends at zero. Carrying cost is charged on the balance at the END of a period, so a unit that arrives and is consumed in the same period is charged nothing. Report costs to the nearest cent, penalties as percentages to two decimal places, and the variability coefficient to four. Report relevant cost, setup plus carrying, unless the exercise asks for the total. Where two plans tie, report the one whose last order starts in the earlier period. That is what Equation 5.8 returns when the minimum is scanned upward in t .

5.11.1. Terminology and Concepts

Exercise 5.1. Define each of the following in one sentence, and say which section of this chapter introduced it.

- requirements schedule
- window cost
- zero-inventory ordering
- relevant cost
- variability coefficient
- part-period
- rolling horizon
- nervousness

5. Dynamic Lot Sizing

Exercise 5.2. State whether each of the following is true or false, and give one sentence of justification. A one-word answer earns nothing.

- a. An optimal plan never places an order in a period that begins with stock on hand.
- b. Because the Wagner-Whitin algorithm is optimal, it is the method a planner should use when the horizon rolls forward one period at a time.
- c. The window cost $w(t, u)$ depends on which periods the plan chooses to order in.
- d. Two plans that place the same number of orders cost the same.
- e. Under the convention of this chapter, a unit that arrives in a period and is consumed in that period is charged no carrying cost.
- f. The variability coefficient is measured in units per period.
- g. Adding the same constant to every window cost cannot change which plan the recursion selects.
- h. When ordering is free, lot-for-lot is optimal.

Exercise 5.3. Choose the best answer.

1. The window costs are computed before any search begins because
 - (a) it saves memory; (b) they do not depend on any decision; (c) the heuristics need them before the algorithm does; (d) a worksheet cannot compute them on demand.
2. Silver-Meal's failure on the mower deck belt is triggered by
 - (a) a large requirement early in the horizon; (b) a run of declining requirements followed by a period with none; (c) an order cost that is too high; (d) a holding rate that is too low.
3. Put a ceiling on the size of any single order and Equation 5.8 no longer applies, because (a) the window costs become wrong; (b) zero-inventory ordering no longer holds; (c) the network acquires a cycle; (d) the horizon is no longer finite.

5. Dynamic Lot Sizing

4. The purchase term may be dropped from the objective when
 - (a) the horizon is short; (b) the unit cost is the same in every period;
 - (c) the holding rate is small relative to the order cost; (d) never, and the chapter is wrong to do it.
5. A heuristic is used instead of the Wagner-Whitin algorithm mainly because
 - (a) the algorithm is too slow; (b) the algorithm needs requirements out to period N and those are a forecast; (c) the algorithm cannot be done in a worksheet;
 - (b) the algorithm requires constant costs.

Exercise 5.4. An analyst argues that since the Wagner-Whitin algorithm is optimal and runs in $O(N^2)$, the heuristics of Section 5.5 are of historical interest only and should be dropped from the course.

Write a reply of at most 150 words. Your reply should name the assumption the argument depends on, say where in Section 5.1.2 it appears, and cite one number from this chapter.

Exercise 5.5. Section 5.3.1 proves that an optimal plan orders only into an empty shelf, and every method in this chapter rests on it.

- a. State where the proof would fail if backorders were permitted at a cost per unit per period.
- b. State where it would fail if the unit cost fell partway through the horizon.
- c. State where it would fail under a ceiling on the size of any single order.
- d. For each, say whether the problem is still a shortest path over some network, and if so what the nodes would have to be.

5. Dynamic Lot Sizing

5.11.2. Working the Methods by Hand

Exercise 5.6. An item has requirements of 30, 50, 10, 90, and 40 units over five periods. Ordering costs \$120, the item costs \$8, and the carrying charge is 2.5% per period.

- Compute h and the variability coefficient. Does Section 5.6.1 call for a heuristic?
- Cost the lot-for-lot plan.
- Cost the plan that orders in periods 1 and 4 only.
- Which is cheaper, and by how much?

Exercise 5.7. For the item of Exercise 5.6, compute the full 5×5 table of window costs $w(t, u)$ using Equation 5.7. Verify your entry for $w(1, 4)$ by costing the corresponding plan directly.

Exercise 5.8. Solve Exercise 5.6 to optimality with the recursion of Equation 5.8. Report $V(u)$ and $S(u)$ for every period, the traceback, the optimal plan, and its relevant cost.

Exercise 5.9. Apply Silver-Meal, least unit cost and part-period balancing to Exercise 5.6. Report each plan and its penalty against the optimum of Exercise 5.8. Explain why the answers come out the way they do.

Exercise 5.10. Take the last six months of the mower deck belt, months 7 through 12 of Table 5.1, and treat them as a horizon of their own numbered 1 through 6. The costs are unchanged: $k = \$300$, $c = \$50$, $i = 0.02$ per month.

t	1	2	3	4	5	6
d_t	120	40	0	80	180	220

5. Dynamic Lot Sizing

- a. Compute the variability coefficient and check 1. Does any period force an order?
- b. Build the 6×6 table of window costs.
- c. Solve the recursion, report V and S , and give the optimal plan and its cost.
- d. Apply all three heuristics. Report each plan and its penalty.
- e. Period 3 requires nothing. Say precisely what each of the three rules does when its window reaches that period, and which of them has to be told about it in code.

Exercise 5.11. Section 5.2.2 explains why the chapter compares an *adjusted* economic order quantity and not a fixed one. This exercise makes the reason concrete. Take the belt of Table 5.1 with $k = \$300$, $c = \$50$ and $i = 0.02$ per month, and apply the strict rule: order exactly $Q^* = 303.3150$ belts whenever the balance would otherwise go negative, and order twice in a month if once is not enough.

- a. Build the ledger and report the order quantities and the ending balances.
- b. How many belts are ordered in total, against a requirement of 1,840?
- c. What is left at the end of month 12?
- d. Which of Section 5.1.2 does the plan break, and why can no rule of this shape avoid breaking it?
- e. The relevant cost comes to \$3,792.49. Explain why comparing that with the \$2,600 of Table 5.12 is not a fair comparison, and say what you would have to do to the plan to make it one.

Exercise 5.12. Section 5.5 notes that part-period balancing comes in two versions. The original takes the largest window whose carrying is at or under k_t , and so never overshoots. The modification compares the two windows either side of k_t and keeps whichever comes closer.

- a. Using the carrying figures of Table 5.10 and the rest of the belt, work both versions over the full twelve months and report the stop period for each starting month.

5. Dynamic Lot Sizing

- Report each version's plan, its relevant cost and its penalty.
- Which is better here? Is the answer what you expected?
- Give the shape of a requirements pattern on which the original must do at least as well as the modification, and say why.

5.11.3. Modeling Problems

Exercise 5.13. The same distributor stocks a spindle assembly for the same line of mowers. It costs \$250 to place an order, the assembly costs \$25, and the carrying charge is 3% per month. The requirements for the coming year are

t	1	2	3	4	5	6	7	8	9	10	11	12
d_t	90	70	60	50	140	240	300	180	60	40	40	110

- Compute h , the average requirement, the variability coefficient, k/h , the economic order quantity on the average requirement, and its time supply in months.
- Does 1 force an order in any month?
- Cost the lot-for-lot plan and the periodic order quantity plan at the time supply from part (a).
- Solve the problem to optimality. Report V , S , the plan and its relevant cost.
- Apply all three heuristics and report the penalty of each.
- Compare your ranking of the three heuristics with Table 5.12. Does the belt's ranking survive?

Exercise 5.14. Table 5.12 ranks the methods on the belt at $k = \$300$. This exercise asks whether the ranking is a property of the methods or of that number.

Keep the requirements of Table 5.1, $c = \$50$ and $i = 0.02$ per month, and solve the problem again at $k = \$120$ and at $k = \$900$.

5. Dynamic Lot Sizing

- a. For each, report k/h and the periods 1 forces.
- b. For each, report the optimal plan and its relevant cost.
- c. For each, report the penalty of lot-for-lot, Silver-Meal, least unit cost and part-period balancing.
- d. Tabulate the three settings side by side and say what happens to the ranking of the heuristics as k rises.
- e. At $k = \$120$, use your answer to part (a) to say how many window costs the recursion actually has to compute, against the 78 it would need without

1.

Exercise 5.15. An item has requirements of 10, 2, 12, 4 and 14 units over five periods. Ordering costs \$40, the unit cost is \$2, and the holding rate is \$1 per unit per period.

- a. Compute the window costs including the purchase term, as in Equation 5.4.
- b. Solve the recursion and report the optimal plan.
- c. Separate the answer into relevant cost and purchase cost. Confirm that the purchase cost is the same under every feasible plan, and explain why.
- d. Now suppose the unit cost rises to \$3 beginning in period 3 and stays there. Recompute the window costs and re-solve. Does the plan change, and is the purchase cost still the same under every plan?

Exercise 5.16. The variability coefficient of Equation 5.10 is the test for whether the methods of this chapter are worth using.

- a. Construct a twelve-period schedule with VC just under 0.2 and one with VC near 1.0, both with the same total requirement.
- b. For each, compute the penalty of an adjusted economic order quantity against the Wagner-Whitin optimum.
- c. Does the 0.2 threshold do what it claims on your two instances?

5. Dynamic Lot Sizing

Exercise 5.17. Prove that if $d_t h_{t-1} > k_t$ then every optimal plan orders in period t . Then explain how the property can be used to split a twelve-period problem into two smaller ones, and say how much arithmetic that saves for the belt.

5.11.4. The Rolling Horizon and the Solver

Exercise 5.18. Take the mower deck belt of Table 5.1 and a planner who sees only three months ahead. Starting in month 1, solve the three-month problem, place that month's order, advance one month, and repeat. Report the twelve orders actually placed and their cost, and compare with Table 5.14.

Exercise 5.19. Open the Solver sheet of `Chapter5Models.xlsx`, clear the order and flag columns to zero, and run Solver. Confirm that it reaches \$2,480.

- a. Now add the constraint that no single order may exceed 400 belts. What does the plan cost?
- b. Explain why Equation 5.8 cannot answer part (a) without modification, and describe what would have to change.

Exercise 5.20. Section 5.7.3 describes three devices against nervousness. For each, say what it costs and under what circumstances you would recommend it. Use the first-month commitments reported in Section 5.7.3 to support your answer.

5.11.5. Using the Workbook

Exercise 5.21. Open `Chapter5Models.xlsx` and work entirely within it.

5. Dynamic Lot Sizing

- a. On the **Data** sheet, replace the requirements with those of Exercise 5.13 and set k , c and i to that item's values. Read the optimal cost off the **WagnerWhitin** sheet and the three heuristic costs off the **Heuristics** sheet, and check them against your answers to Exercise 5.13.
- b. Restore the belt. On the **Plans** sheet, type the Wagner-Whitin plan of Table 5.8 into the fourth order column and confirm that the penalty reads 0.00%.
- c. Now type a plan of your own into the same column that costs less than \$2,600 and is not the optimum. Report it, or explain how the **WagnerWhitin** sheet tells you in advance how many such plans there are.
- d. On the **Heuristics** sheet, find the row of the Silver-Meal block that produces the order of 840 belts. Read off the six scores along it from $u = 5$ to $u = 10$ and say which one the rule stopped at and why.
- e. Change the requirement in month 9 from 0 to 200. Report what happens to the six scores of part (d), to the Silver-Meal plan, to the optimum, and to the penalty. Two of those four move in a direction most students do not predict.

5.11.6. Using the Software

Exercise 5.22. The code of Section 5.9 is in `code/`, package `inventory.dynamiclotsizing`. Build it with `./gradlew build` and write a `main` of your own. Report the code you wrote alongside each answer.

- a. Build the spindle assembly of Exercise 5.13 with `RequirementsSchedule.constantCosts`. Print `averageRequirement`, `variabilityCoefficient` and `forcedOrderPeriod()`, and compute k/h from `orderCostIn(1)` and `holdingRateIn(1)`. Check all four against your answers to Exercise 5.13, and say what it means that `forcedOrderPeriod()` returns `null`.

5. Dynamic Lot Sizing

- b. Put the seven rules in a list and loop over it, printing each rule's name, orderCount, setupCost, carryingCost, relevantCost and penaltyAgainst(WagnerWhitin.plan(spindle)). Report the table. Which rule is worst, and does Section 5.6.3 predict it?
- c. Run `RollingHorizon(spindle, w, freeze = 1).realizedPlan(rule)` for $w = 2$ through 6, under both SilverMeal and WagnerWhitin, and report the two rows as Table 5.14 is reported. Two things happen on this item that do not happen to the belt. Identify both and say which claim of Section 5.7.2 they instantiate.
- d. You worked both versions of part-period balancing over the belt by hand in
 - 2. The package implements only the modified one. Implement the original as a `LotSizingRule`: extend the window while the carrying cost of the window is at or under k , and stop before the period that would carry it past k . Use `carryingCostOver(from, through)` and `planFrom(orderPeriods)`. Check it against your hand answer on the belt, then run both versions on the spindle assembly. The two items do not rank the two versions the same way, so say which is the better rule and defend the answer.
- e. The study of Table 5.15 draws its schedules as follows.

```
fun schedules(count: Int, seed: Long, orderCost:
    Double): List<RequirementsSchedule> {
    val draw = java.util.Random(seed)
    val choices = listOf(0.0, 10.0, 20.0, 40.0, 80.0,
        120.0, 200.0, 300.0)
    return (1..count).map { n ->
        var d: List<Double>
        do { d = (1..12).map {
            choices[draw.nextInt(choices.size)] } }
        while (d.first() <= 0.0 || d.sum() <= 0.0)
        RequirementsSchedule.constantCosts("Trial $n",
            d, orderCost, 50.0, 0.02)
```

5. Dynamic Lot Sizing

```
}  
}
```

Call it as `schedules(2000, 20240501L, 200.0)` and reproduce Table 5.15. Then raise the order cost to \$600 and run it again. Report both tables and state the conditions under which Section 5.7.2's conclusion holds.

5.11.7. From the Literature

Exercise 5.23. Wagner and Whitin (1958) is four pages long and is the origin of everything in Section 5.4. Read it.

- The paper states its result as a planning-horizon theorem rather than as a shortest path. State the theorem in the notation of this chapter.
- The paper's recursion and Equation 5.8 are the same recursion. Identify the correspondence between its symbols and ours.
- The paper was published in 1958 and the method was not widely adopted. Using Section 5.5 and Section 5.7, give two reasons that have nothing to do with the speed of computers.

6. Material and Distribution Requirements Planning

i Learning objectives

After reading this chapter you should be able to:

- distinguish dependent from independent demand, and explain why dependent demand should be calculated and not forecast
- read a bill of material, assign low-level codes, and say why an item must be planned at its deepest occurrence
- complete an MRP record: netting, lot sizing, and lead-time offsetting
- explode a master production schedule through a product structure, level by level
- say how long a planning horizon has to be before it can produce a plan at all
- explain how a parent's lot sizing rule reshapes the demand its components see, and why minimizing cost level by level does not minimize cost
- apply the same time-phased logic to a distribution network
- recognize how lot sizing, lead times, and schedule changes produce nervousness
- state what requirements planning assumes, and which of those assumptions the rest of this book removes

6. Material and Distribution Requirements Planning

Chapter 3 decided one order quantity at a constant demand rate. Chapter 5 let the rate vary and planned one item over a horizon whose requirements were given. This chapter is about where those requirements come from.

For most of the items in a manufacturing business they are not forecast at all. They are calculated, from the schedule for whatever is built out of them. In this chapter we do that calculation for every item at every level of a product structure, which is material requirements planning, and doing it across a network of stocking locations is distribution requirements planning. The two are the same arithmetic, and this chapter says so more than once.

6.1. Demand You Should Not Forecast

An item's demand is *independent* when it arises outside the firm and *dependent* when it arises from the firm's own plans. A distributor selling mower deck belts over the counter faces independent demand for them. The manufacturer who puts one belt into every deck it builds does not: the demand for belts is whatever the deck schedule says it is.

The distinction is not about the item. It is about the position. The same belt is independent demand at the distributor and dependent demand at the manufacturer, and the right way to plan it differs accordingly.

6.1.1. Why Forecasting It Is a Mistake

Suppose we forecast belt demand from belt history, as Chapter 5's distributor would. Two things go wrong.

The first is that the forecast throws away information. The manufacturer knows the deck schedule. Every belt it will need for the next year is implied by a document it wrote itself. Replacing that with an extrapolation from past belt usage replaces a known quantity with an estimate of it, and the

6. Material and Distribution Requirements Planning

estimate can only be worse. That is, the forecast is not merely unnecessary; it is a way of making the answer less certain than it already was.

The second is worse and less obvious. Dependent demand is *lumpy*. A deck assembly built in lots of 300 consumes belts in lots of 300, so the belt sees nothing for several periods and then 300 at once. A forecast fitted to that history produces an average, and an average is the one value the demand never takes. For example, the belt requirement in Table 6.5 is 420 in one month and nothing in the next, and its mean of 153 occurs in no month at all. Section 6.5 measures the lumpiness and shows where it comes from. That is not the item and not the customer but the lot sizing rule one level up.

Recall that Section 5.6.1 gave a test for when a pattern varies enough to need the methods of Chapter 5. Dependent demand almost always passes it, and it passes it for a reason that is manufactured and not observed.

6.1.2. Where This Chapter Sits

Chapter 5 planned one item, and this chapter plans an item for every node of a product structure. What it hands each of them is the requirements schedule of Section 5.1.1, and what it does with the answer is pass it down to the level below. The lot sizing of Section 5.5 is used here unchanged and is not restated.

The stochastic chapters that follow remove the assumption that any of this is known. 4 collects what requirements planning takes for granted, and it is the bridge to them.

6.2. The Bill of Material

A *bill of material* records what goes into what, and how many. Figure 6.1 illustrates the manufacturer's mower deck assembly.

6. Material and Distribution Requirements Planning

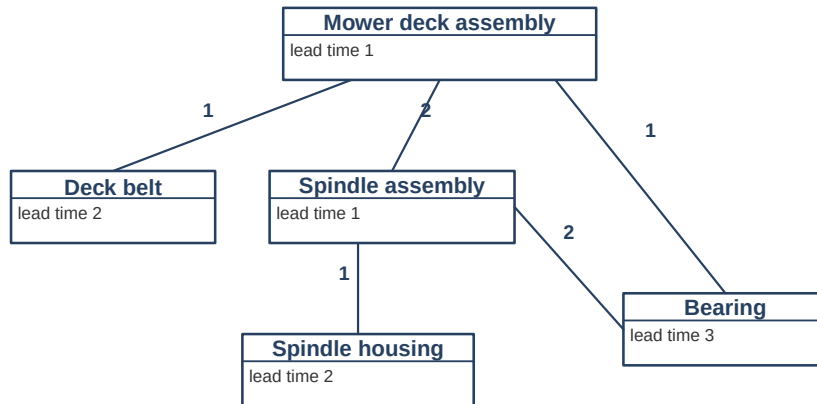


Figure 6.1.: The mower deck assembly. The number on an arc is the quantity per. Notice that the bearing hangs from two places: one is fitted directly to the deck and two more sit inside each spindle assembly. It therefore appears at level 1 and at level 2, and its low-level code is the deeper of them.

The same information is often written as an indented list, easier to type and harder to read:

```
Mower deck assembly (DA), lead time 1
  Deck belt (B), 1 each, lead time 2
  Spindle assembly (S), 2 each, lead time 1
    Spindle housing (H), 1 each, lead time 2
    Bearing (BR), 2 each, lead time 3
  Bearing (BR), 1 each, lead time 3
```

Each indentation is a *level*. The deck assembly is level 0, the belt and the spindle assembly are level 1, and the housing is level 2. The number in parentheses is the *quantity per*: one deck assembly takes two spindle assemblies, and each of those takes two bearings.

6. Material and Distribution Requirements Planning

Notice that the bearing appears twice. An idler bearing is fitted directly to the deck and two more sit inside each spindle assembly. That is the usual case, not an awkward one, and it is the reason for the next idea.

6.2.1. Low-Level Codes

An item must be planned once, and it must be planned after every parent that consumes it. The bearing appears at level 1 and again at level 2. If it were planned at level 1, its requirements would be netted against the stock on hand before the spindle assembly had asked for any, and the spindle assembly would then find the stock already spoken for.

Thus, each item carries a *low-level code*: the deepest level at which it appears anywhere in the structure. The bearing's is 2. Table 6.1 collects them.

Table 6.1.: Low-level codes for the deck assembly structure.

Item	Appears at level	Low-level code
Mower deck assembly, DA	0	0
Deck belt, B	1	1
Spindle assembly, S	1	1
Spindle housing, H	2	2
Bearing, BR	1 and 2	2

Thus, planning in low-level-code order, shallowest first, guarantees that every parent of an item has a plan before the item does. For this structure the order is DA, then B and S, then BR and H.

6. Material and Distribution Requirements Planning

6.2.2. The Explosion Calculation

Before any record, we need the arithmetic that connects two levels. Let item i be a parent of item j , and q_{ij} of j go into one i , then the requirement j inherits in period t is

$$G_{jt} = \sum_{i \in \text{parents}(j)} q_{ij} Rel_{it}, \quad (6.1)$$

where Rel_{it} is the quantity of i that the plan *releases* in period t .

Releases, and not receipts. A parent needs its components in hand when it starts building, and the release is when it starts. Using the receipt row would ask for the components on the day the parent is finished, a whole lead time too late. This is the single most common error in a hand-worked explosion.

For the bearing, Equation 6.1 has two terms, because it has two parents:

$$G_{BR,t} = 1 \cdot Rel_{DA,t} + 2 \cdot Rel_{S,t}.$$

The two arrive in different periods, because the two paths through the structure have different lead times.

6.3. The MRP Record

We give every item a record: six rows over a common horizon of periods. The periods run across the columns. That is how a requirements record is laid out everywhere it is used, and the reverse of Chapter 5's worksheets. Nothing in the arithmetic depends on it.

Let G_t represent the gross requirement in period t , SR_t the scheduled receipt, I_t the projected on hand at the **end** of period t , NR_t the net requirement, POR_t the planned order receipt, and Rel_t the planned order release. Let L be the lead time and SS the safety stock.

6. Material and Distribution Requirements Planning

Gross requirements are what is needed. For an end item they come from the master production schedule. For everything else they come from Equation 6.1.

Scheduled receipts are what is already on order. These are commitments made in some earlier planning cycle, and they arrive whether or not this plan wants them.

Projected on hand is what is left at the end of a period:

$$I_t = I_{t-1} + SR_t + POR_t - G_t. \quad (6.2)$$

That is Equation 5.1 with the scheduled receipts added, and it is an end-of-period figure for the same reason: Section 5.1.2 charges carrying on what crosses a period boundary.

Net requirements are what is short once the opening stock and the scheduled receipts have been counted:

$$NR_t = \max(G_t + SS - SR_t - I_{t-1}, 0). \quad (6.3)$$

Planned order receipts are what the lot sizing rule decides should arrive. Under lot-for-lot, $POR_t = NR_t$. Under anything else it is larger and less frequent, and Section 6.5 is about what that costs.

Planned order releases are the receipts moved back by the lead time:

$$Rel_{t-L} = POR_t. \quad (6.4)$$

This row is the record's output. That is, it is what Equation 6.1 reads at the next level down, and it is the only row that says anything the planner will act on.

6. Material and Distribution Requirements Planning

6.3.1. The Maximum Is the Whole Idea

Equation 6.3 has a max in it, and a reader meeting the record for the first time usually leaves it out. Let's see what happens.

Example 6.1 (A record without netting). Let's take the deck assembly's master schedule and carry the on-hand balance forward without netting anything, so that Equation 6.2 runs with $POR_t = 0$ throughout.

Table 6.2.: The deck assembly with nothing netted. Every negative entry is a shortage the plan has not dealt with.

Period	6	7	8	9	10	11	12	13	14	15	16	17
Gross requirements	40	60	120	300	420	260	120	40	0	80	180	220
Projected on hand	-40	-	-	-	-	-	-	-	-	-	-	-
	100	220	520	940	1,200	1,320	1,360	1,360	1,440	1,620	1,840	

The balance reaches $-1,840$, the whole year's requirement. That is correct arithmetic and a useless plan: it reports that if nothing is ordered then everything is short, which the reader knew.

Netting turns each of those shortages into an order. Because this item starts with nothing on hand and nothing on order, Equation 6.3 gives $NR_t = G_t$ in every period, lot-for-lot has $POR_t = NR_t$, and Equation 6.2 then holds I_t at zero throughout. The max is what stops the shortage propagating into the next period instead of being planned for.

6. Material and Distribution Requirements Planning

6.3.2. Safety Stock and Allocations

We meet two modifications to Equation 6.3 constantly, and neither needs new machinery.

Safety stock is the SS term. It raises every net requirement by the same amount, which has the effect of holding SS units permanently and planning around what is left. Whether it belongs in a system whose entire premise is that demand is known is a real question, and the field has never fully settled it. 4 returns to it.

Allocated stock is inventory that is physically present but already promised to something else. It is handled by reducing I_0 , the opening balance, in order to keep the stock that is promised elsewhere out of the calculation. Nothing else changes.

6.3.3. How Long the Horizon Has to Be

A record cannot release an order before period 1, and we have to allow for that. If a receipt is due in period t and the lead time is L , then the release falls in period $t - L$, and when that is less than 1 the order is *past due*: the plan has discovered a requirement it no longer has time to meet.

Past-due orders at the bottom of a structure are usually not a data problem. That is, they are a horizon problem. The lead times accumulate down every path, and the horizon must lead the first requirement by the longest of those accumulations.

For the deck assembly the deepest path runs assembly, spindle assembly, bearing, with lead times 1, 1 and 3, so the cumulative lead time is 5 months. Table 6.3 shows what happens on either side of that.

6. Material and Distribution Requirements Planning

Table 6.3.: What a horizon that is too short produces. Five months of lead-in is exactly enough, and that is the cumulative lead time through the deepest branch.

Empty months in front of the schedule	3	4	5	6
Units the plan cannot release in time	520	160	0	0

Thus, the horizon used throughout this chapter is 17 months: five empty months and then the twelve-month schedule.

6.4. Running the Explosion

With the record defined and the order of the items settled, we produce the plan by doing one and then the other, level by level.

6. Material and Distribution Requirements Planning

Algorithm 6.1 The explosion. The sort is what makes the sum over parents safe: by the time an item is reached, every parent of it already has a release row.

```
plan(bill of material, master production schedule):

    assign every item its low-level code
    sort the items by low-level code, shallowest first

    for each item in that order:
        if a master schedule speaks about this item:
            G <- the master schedule
        else:
            G(t) <- sum over parents i of q(i, item) * Rel(i, t)

            // the record of section 6.3
            NR(t) <- max(G(t) + SS - SR(t) - I(t-1), 0)
            POR <- the lot sizing rule applied to NR
            I(t) <- I(t-1) + SR(t) + POR(t) - G(t)
            Rel(t - L) <- POR(t)

    return every item's record
```

Example 6.2 (The deck assembly, all five items). We plan every item lot-for-lot, with nothing on hand and nothing on order. The master schedule carries the twelve-month pattern in months 6 through 17.

The deck assembly nets to its own schedule, and Table 6.4 reads its releases off as that schedule pulled back one month.

6. Material and Distribution Requirements Planning

Table 6.4.: The deck assembly's record, months 5 through 17.

Pe- riod	5	6	7	8	9	10	11	12	13	14	15	16	17
DA	0	40	60	120	300	420	260	120	40	0	80	180	220
gross re- quire- ments													
DA	40	60	120	300	420	260	120	40	0	80	180	220	0
planned or- der re- leases													

Now we apply Equation 6.1. The belt takes one per deck, so its gross requirements are the deck assembly's releases unchanged, and its own lead time of two months pulls them back again. The spindle assembly takes two per deck, so its gross requirements are twice that row.

Table 6.5.: The two level-one items.

Pe- riod	3	4	5	6	7	8	9	10	11	12	13	14	15	16
B	0	0	40	60	120	300	420	260	120	40	0	80	180	220
gross re- quire- ments														

6. Material and Distribution Requirements Planning

Pe- riod	3	4	5	6	7	8	9	10	11	12	13	14	15	16
B planned or- der re- leases	40	60	120	300	420	260	120	40	0	80	180	220	0	0
S gross re- quire- ments	0	0	80	120	240	600	840	520	240	80	0	160	360	440
S planned or- der re- leases	0	80	120	240	600	840	520	240	80	0	160	360	440	0

The bearing is the interesting one, because Equation 6.1 has two terms for it. One bearing per deck assembly and two per spindle assembly, and Table 6.6 keeps the two apart before adding them, because the two parents release in different periods.

6. Material and Distribution Requirements Planning

Table 6.6.: The bearing's gross requirements, which two levels feed.

Pe- riod	4	5	6	7	8	9	10	11	12	13	14	15	16
One per deck as- sem- bly	0	40	60	120	300	420	260	120	40	0	80	180	220
Two per spin- dle as- sem- bly	160	240	480	1,200	1,680	1,040	480	160	0	320	720	880	0
BR gross re- quire- ments	160	280	540	1,320	1,980	1,460	740	280	40	320	800	1,060	220

Notice the shape of the last row. It is not the shape of the master schedule, and it is not the shape of either contribution taken alone. Two streams arriving on different offsets add up to a pattern that looks nothing like the demand that caused it, and nobody forecast any of it.

Lot-for-lot throughout costs \$14,500, every dollar of it setup, because no item ever carries anything from one period to the next. That is the plan the next section improves on, and the way it improves on it is not the obvious one.

6.5. What Lot Sizing Does to the Level Below

Example 6.2 ordered lot-for-lot at every level. Thus, it is the plan an MRP system produces when nobody has told it otherwise. Every rule of Section 5.5 can be used instead, applied to the net requirement row of whichever record it is planning. None of them is restated here. What this section is about is something Chapter 5 could not see, because Chapter 5 had only one item.

A parent's lot sizing rule does not only decide what the parent costs. By Equation 6.1 it decides *when and in what sizes the components are asked for*, and so it decides what their problem looks like before they have made any choice at all.

6.5.1. One Rule Changes, and Everything Moves

We hold every component at lot-for-lot and change only the rule used on the deck assembly. Nothing below level 0 decides anything, so whatever we see move is the end item's doing.

Table 6.7.: What the end item's rule does to the whole structure. The variability coefficient is measured on what the deck assembly has arrive, which by Equation 6.1 is what the belt is asked for.

Rule on the deck as- sembly	Orders	VC at level 1	Cost at DA	Cost below	Total
Lot-for- lot	11	0.6191	\$5,500	\$9,000	\$14,500

6. Material and Distribution Requirements Planning

Rule on the deck as- sembly	Orders	VC at level 1	Cost at DA	Cost below	Total
Peri- odic order quan- tity	11	0.6191	\$5,500	\$9,000	\$14,500
Wagner- Whitin	9	0.6701	\$4,860	\$7,500	\$12,360
Silver- Meal	8	0.9650	\$4,936	\$6,700	\$11,636
Least unit cost	7	1.0274	\$5,588	\$6,100	\$11,688
Part- period balanc- ing	7	1.5463	\$5,444	\$6,100	\$11,544
Ad- justed eco- nomic order quan- tity	7	0.8459	\$5,588	\$5,900	\$11,488

We draw three things out of that table.

Lot-for-lot is the only rule that changes nothing. Its receipts are the master schedule itself, so the components see the schedule, and $VC = 0.6191$ is the schedule's own figure. Every rule that batches raises it,

6. Material and Distribution Requirements Planning

as far as 1.5463 under part-period balancing. The periodic order quantity happens to agree with lot-for-lot here, because the deck assembly's holding rate is high enough that Equation 3.17 gives a time supply of 1.35 months, which rounds to one.

The cost below a level rises with how often the level above orders.

Read the Orders column against the Cost below column and the relation is exact: seven orders cost between \$5,900 and \$6,100 below, eight cost \$6,700, nine cost \$7,500, and eleven cost \$9,000. The reason is Equation 6.1 and nothing else. Every release a parent makes is a requirement its components have to meet, and meeting a requirement costs a setup. For example, lot-for-lot releases eleven times and the components below it place their own orders eleven times each.

Thus, the rule that is best at a level is not the rule that is best for the product. Wagner-Whitin is optimal at the deck assembly, and it is optimal there by construction, since Section 5.4.2 proves no rule can beat it on that item's own requirements. It costs \$4,860 where the adjusted economic order quantity costs \$5,588, and it is \$872 worse over the structure as a whole. Choosing the rule by the level you can see costs 7.59% here.

6.5.2. What the Variability Coefficient Does Not Tell You

You will be tempted to read the VC column as a cost. It is not one. Part-period balancing produces much the lumpiest pattern in Table 6.7 and is not the most expensive plan; Wagner-Whitin produces nearly the smoothest and is the second most expensive. Notice that the order count explains the cost below and the variability coefficient does not.

The variability coefficient is still the right thing to look at when you are asking whether a component's own planner needs the methods of Chapter 5. Section 5.6.1 set the threshold at 0.2, and every row of Table 6.7 clears it comfortably. That is the point made in Section 6.1.1, now with a number

6. Material and Distribution Requirements Planning

attached: dependent demand does not merely happen to be variable, it is made variable, by a decision taken one level up.

6.5.3. What Is Actually Done

Most installed systems run lot-for-lot below the top level or two, and reading Table 6.7 it is easy to see that as laziness. It is not, quite.

Lot-for-lot at a component is the rule that leaves the level beneath it alone, and in a structure several levels deep that matters more at each level down. Thus, it also produces the smallest projected balances, and that is what a planner wants when the requirements above are going to be revised anyway, and Section 6.7 is about how often that happens. And it needs no cost parameters, which matters because a bill of material with ten thousand items has ten thousand order costs that nobody has ever measured.

The defensible position is that batching pays where the setup is large and genuinely known, usually the end item and the major subassemblies, and that below that the information needed to do it well does not exist.

6.6. Distribution Requirements Planning

The same arithmetic plans a distribution network. Let's be precise about how little has to change.

Chapter 5's distributor does not build anything. It holds belts at three regional warehouses, each serving its own dealers, and replenishes all three from a central warehouse. Figure 6.2 illustrates the network.

A regional warehouse's demand is independent: dealers buy what they buy. The central warehouse's demand is not. It is whatever the three regions order, a consequence of their own plans, exactly as a component's demand is a consequence of its parent's. Thus, the central warehouse is planned the same way a component is.

6. Material and Distribution Requirements Planning

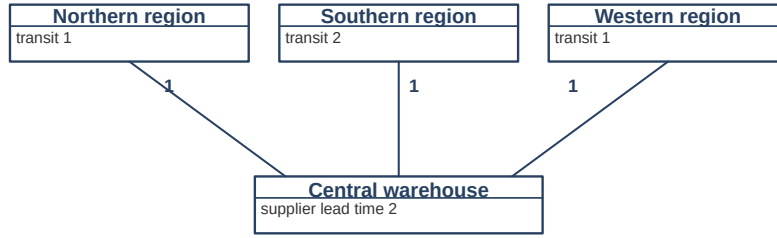


Figure 6.2.: The distributor's network. Three regions sell to dealers and one warehouse supplies all three. The arrows point the way a requirement travels, which is the same direction they point in Figure 6.1. Every quantity per is one, which is the whole of what separates this from a bill of material.

6.6.1. It Is the Same Calculation

Let's write Equation 6.1 for the central warehouse. Its parents are the three regions, and the quantity per is one belt per belt:

$$G_{CW,t} = \sum_{r \in \text{regions}} 1 \cdot Rel_{rt}.$$

That is Equation 6.1 with every $q_{ij} = 1$. Transit time plays the part of lead time, the regions are the level-0 items, the warehouse is their component, and Algorithm 6.1 runs unchanged. That is, nothing in the algorithm has to know which kind of graph it is walking. The software of Section 6.10 uses one class for both, and Section 6.10.6 explains why that is not a trick.

Two differences are real and neither is arithmetic.

A bill of material has a quantity per and a distribution network does not, or rather has one everywhere. A parent needs two spindle assemblies per deck; a region needs one belt per belt.

And the direction of the relationship reads backwards. In a bill of material the parent is what the child goes into. In a network the parent is the

6. Material and Distribution Requirements Planning

customer and the child is the *supplier*. The arrow points the way the requirement travels, upstream in both cases, and that is why the same code carries them.

Example 6.3 (The three regions and the centre). The three regions sell 1,840 belts between them over the twelve months, split 830 to the north, 640 to the south and 370 to the west. Transit is one month to the north and west and two to the south, and the central warehouse's own supplier takes two months. The cumulative lead time is therefore four months, so the horizon leads the first sale by four.

Every location orders lot-for-lot. In Table 6.8 the regions' releases are their own sales pulled back by their transit times, and the centre's gross requirements are the sum of the three.

Table 6.8.: The roll-up. The centre's row is the three above it added down the column.

Pe- riod	3	4	5	6	7	8	9	10	11	12	13	14	15
North re- leases	0	20	30	60	140	180	110	50	20	0	40	80	100
South re- leases	10	20	40	100	160	100	50	10	0	20	60	70	0
West re- leases	0	10	10	20	60	80	50	20	10	0	20	40	50

6. Material and Distribution Requirements Planning

Pe- riod	3	4	5	6	7	8	9	10	11	12	13	14	15
Cen-10 tre's gross re- quire- ments	50	80	180	360	360	210	80	30	20	120	190	150	

Notice that the south is a month out of step with the other two, because its transit time is longer. The centre's requirement in period 3 is the south's alone.

6.6.2. What the Regions Do to the Centre

Section 6.5's result has a counterpart here, and it is the more famous of the two. We change the rule the regions use and watch what the centre sees.

Table 6.9.: What the regions' own rule does to the demand the centre sees.
The regions sell a pattern whose variability coefficient is 0.6191.

Rule at every region	VC at the centre	Regions	Centre	Total
Lot-for-lot	0.6013	\$6,600	\$5,200	\$11,800
Part-period balancing	1.1588	\$3,860	\$3,200	\$7,060
Wagner-Whitin	1.4234	\$3,830	\$3,200	\$7,030
Silver-Meal	1.5706	\$3,980	\$3,600	\$7,580
Periodic order quantity	1.9584	\$4,310	\$3,200	\$7,510

The demand the centre faces is between two and three times as variable as the demand the regions actually sell, and the regions' customers did nothing

6. Material and Distribution Requirements Planning

to cause it. This is the *bullwhip effect*, and Table 6.9 is its mechanism in one table: batching at one stage is amplification at the next.

Notice two things before we let the conclusion run away.

Under lot-for-lot the centre sees $VC = 0.6013$, slightly *less* variable than the 0.6191 the regions sell. Three locations ordering on different transit times are slightly out of step with one another, and the staggering smooths what the centre sees. Amplification is caused by batching and not by the network.

And every batching row in Table 6.9 is cheaper than lot-for-lot, at the regions and at the centre both. In a world where demand is known, amplification costs nothing: the centre can see the lumpy pattern coming and plan for it, as it just did. The bullwhip effect is expensive when demand is uncertain, because then the centre cannot tell an amplified pattern from a change in the market and must hold stock against both. Chapter 10 takes that up, and it is the reason this chapter can report the amplification without reporting a cost for it.

6.7. Nervousness

Section 5.7.3 met nervousness with one item: the horizon rolls, the plan is recomputed, and the new plan disagrees with the old about orders not yet placed. In a product structure the disagreement propagates, and the propagation is the problem.

For example, a change to the master schedule in a distant period changes the deck assembly's releases. Those are the belt's requirements, so the belt replans. The belt's releases move, and with a two-month lead time they move into periods nearer than the change that caused them. Two levels down, a revision to a requirement eleven months out can move an order that was going to be placed next week.

Three things make it worse than the single-item case of Section 5.7.3.

6. Material and Distribution Requirements Planning

Lot sizing amplifies it. A batching rule has thresholds, and a small change in requirements moves a plan across one. Section 5.7.3 measured the first-period commitment for two rules and found that Silver-Meal held steady while Wagner-Whitin did not. Here that instability is multiplied by every level beneath.

Lead times concentrate it. A change far out in the schedule arrives at the bottom of the structure much sooner, because each level's lead time pulls it earlier.

Low-level codes spread it. An item with several parents is replanned when any of them moves.

We have the devices of Section 5.7.3, with one addition. A *firm planned order* is a planned order the system is forbidden to move. It is the freeze fence applied to a single order instead of to a span of periods, and it is how a planner overrides the calculation without turning it off. A *time fence* does the same for a span. A *change penalty* is rare here for the reason Section 5.7.3 gave, and rarer still because it would have to be defined for every item at every level.

Safety *lead time* is the device this chapter adds. Instead of holding extra stock, the item is planned to arrive a period early. It buffers against a late delivery in a way that safety stock does not, and it costs a period of carrying on everything instead of a permanent balance on one item. Which you should use depends on whether the uncertainty is in the quantity or in the timing, and requirements planning gives you no way to tell.

6.8. What Requirements Planning Assumes

The arithmetic of this chapter is exact, and every assumption we made to get it is questionable. We collect them here, and they are the bridge to the rest of the book.

6. Material and Distribution Requirements Planning

1. **Capacity is infinite.** Nothing in Algorithm 6.1 asks whether the shop can build 1,980 bearings in period 8. Thus, a plan that cannot be executed is not a plan, and capacity requirements planning takes the releases and compares them with what the work centres can do. When it finds a conflict the master schedule is changed and the explosion is run again. That loop, with capacity and the financial side attached, is what manufacturing resource planning means. It is outside this book, which takes up inventory and not scheduling.
2. **Lead times are fixed and known.** They are treated as properties of an item. In a real shop a lead time is mostly queueing, so it depends on how loaded the shop is, which depends on the lot sizes, which are what the plan is choosing. Planning with a fixed lead time and then releasing work that makes the true lead time longer is a well-documented way to make a plan worse by following it.
3. **Requirements are known.** The master production schedule is a forecast for the end items, and everything below it inherits that forecast's error with the amplification of Section 6.5 applied.
4. **The records are right.** Section 2.3 showed what record accuracy actually is in the field. An explosion nets against on-hand balances. Thus, an error in one balance becomes a wrong order for every item beneath it. Requirements planning is less tolerant of bad records than an order-point system, because it computes rather than reacts.
5. **Everything is deterministic.** No demand distribution appears anywhere in this chapter, and the only concession to uncertainty is a safety stock that Section 6.3.2 could not justify from inside the model.

Assumption 5 is the one the rest of this book removes. Chapter 7 begins with a single period and a random demand, and Chapter 8 through Chapter 10 build the machinery that assumption 3 and assumption 5 need.

6.9. Building These Models in a Worksheet

The workbook is `Chapter6Models.xlsx`. It has four working sheets, and one thing about it will look wrong to a reader coming from Section 5.8: the periods run across the columns and not down the rows.

That is deliberate and it is not this book's choice. A requirements record is laid out with periods across the columns everywhere it is used, because a planner reads one item's six rows at a time and compares them with each other. Chapter 5 ran the periods downward because there the thing that grows is the horizon and every calculation was indexed by it. Here the thing that repeats is the item. The layout follows the field.

6.9.1. The Record, Computed Twice

The **Record** sheet computes the same gross requirements twice. The upper block carries the balance forward without netting and produces Table 6.2; the lower block nets and produces a plan. The two blocks differ in one formula, and putting them one above the other is the point of the sheet.

Below them, three cells: the lowest balance each block reaches, and the units the lead time pushed off the front of the horizon. Notice what that last one measures. It is the difference between what the plan wants to receive and what it managed to release, and it is zero only when the horizon is long enough. Figure 6.3 is the sheet.

6.9.2. The Explosion

The **Explosion** sheet holds the structure in three columns at the top and then one record per item, in low-level-code order. A component's gross requirement row is Equation 6.1 written out: a sum of terms, one per parent, each a quantity per times a release.

6. Material and Distribution Requirements Planning

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	One item's record																	
2	<i>Chapter 6, section 6.3. The same gross requirements, computed twice.</i>																	
3																		
4	WITHOUT NETTING what a reader writes the first time																	
5	<i>Projected on hand is carried forward and allowed to go negative. Every negative entry is a shortage the plan has not dealt with.</i>																	
6	DA Mower deck assembly lead time 1	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
7	Gross requirements	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	
8	Scheduled receipts	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
9	Projected on hand	0	0	0	0	-40	-100	-220	-520	-940	-1,200	-1,320	-1,360	-1,360	-1,440	-1,620	-1,840	
10	Planned order receipts	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
11	Planned order releases	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
12																		
13	WITH NETTING the record of section 6.3																	
14	<i>A net requirement is what is short after the opening stock and the scheduled receipts have been counted, and it is never negative.</i>																	
15	DA Mower deck assembly lead time 1	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
16	Gross requirements	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	
17	Scheduled receipts	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
18	Projected on hand	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
19	Net requirements	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	
20	Planned order receipts	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	
21	Planned order releases	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	0
22																		
23	CHECK with the data as shipped																	
24	<i>Without netting the projected on hand reaches -1,840 by the last period.</i>																	
25	<i>With netting it never goes below zero, and every net requirement equals the gross requirement, because nothing is on hand and nothing is on order.</i>																	
26	Lowest projected on hand, without netting	-1,840																
27	Lowest projected on hand, with netting	0																
28	Units due before they could be ordered, which	0																

Figure 6.3.: The Record sheet. The upper block has no net requirement row at all, because a reader writing a record for the first time has not met netting; what it has instead is a projected-on-hand row running to $-1,840$. The lower block nets, and its projected-on-hand row never leaves zero. Notice that the two blocks share a gross requirement row to the unit, so the only thing that differs between them is the arithmetic.

6. Material and Distribution Requirements Planning

The quantities in the structure block are the cells those formulas read. Type a 3 over the 2 and every record below the spindle assembly moves. A workbook that showed the structure as a decoration while the formulas carried the numbers would be worse than one that did not show it at all, because it would look as though it worked. Figure 6.4 is the top of the sheet and Figure 6.5 the rest.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	The explosion																	
2	Chapter 6, section 6.4. Five items, planned in low-level-code order, every one on lot-for-lot.																	
3																		
4	THE STRUCTURE type over the quantities and every record below moves																	
5	Parent	Child	Quantity per															
6	DA	B	1															
7	DA	S	2															
8	DA	BR	1															
9	S	H	1															
10	S	BR	2															
11	The bearing appears twice, under the deck assembly and under the spindle assembly, which is why its low-level code is 2.																	
12																		
13	CHECK with the data as shipped																	
14	Lot-for-lot everywhere costs \$14,500 in setup and nothing in carrying.																	
15	Nothing is released before period 1, because the horizon leads the first requirement by the cumulative lead time of five months.																	
16	Period	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
17	DA Mower deck assembly	lead time 1																
18	Gross requirements	0	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220
19	Scheduled receipts	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
20	Projected on hand	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
21	Net requirements	0	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220
22	Planned order receipts	0	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220
23	Planned order releases	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	0
24																		
25	Period	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
26	B Deck belt	lead time 2																
27	Gross requirements	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	0
28	Scheduled receipts	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
29	Projected on hand	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
30	Net requirements	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	0
31	Planned order receipts	0	0	0	0	40	60	120	300	420	260	120	40	0	80	180	220	0
32	Planned order releases	0	0	40	60	120	300	420	260	120	40	0	80	180	220	0	0	0

Figure 6.4.: The top of the **Explosion** sheet: the structure, the check block, and the first two records. The five blue cells in column C are the quantities per, and they are the only typed numbers on the sheet. Notice the deck belt's gross requirement row against the deck assembly's release row above it. They are the same numbers, because Equation 6.1 multiplies by a quantity per of one.

6. Material and Distribution Requirements Planning

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
34	Period	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
35	S Spindle assembly lead time 1																	
36	Gross requirements	0	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0
37	Scheduled receipts	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
38	Projected on hand	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
39	Net requirements	0	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0
40	Planned order receipts	0	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0
41	Planned order releases	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0	0
42																		
43	Period	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
44	BR Bearing lead time 3																	
45	Gross requirements	0	0	0	160	280	540	1,320	1,980	1,460	740	280	40	320	800	1,060	220	0
46	Scheduled receipts	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
47	Projected on hand	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
48	Net requirements	0	0	0	160	280	540	1,320	1,980	1,460	740	280	40	320	800	1,060	220	0
49	Planned order receipts	0	0	0	160	280	540	1,320	1,980	1,460	740	280	40	320	800	1,060	220	0
50	Planned order releases	160	280	540	1,320	1,980	1,460	740	280	40	320	800	1,060	220	0	0	0	0
51																		
52	Period	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
53	H Spindle housing lead time 2																	
54	Gross requirements	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0	0
55	Scheduled receipts	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
56	Projected on hand	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
57	Net requirements	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0	0
58	Planned order receipts	0	0	0	80	120	240	600	840	520	240	80	0	160	360	440	0	0
59	Planned order releases	0	80	120	240	600	840	520	240	80	0	160	360	440	0	0	0	0
60																		
61																		
62																		
63	TOTALS																	
64	Orders placed, all five items	57																
65	Setup cost, all five items	\$14,500																
66	Carrying cost, all five items	\$0																
67	Lot-for-lot never carries anything from one period to the next, so the carrying cost is zero and every dollar of the total is setup.																	

Figure 6.5.: The rest of the Explosion sheet: the spindle assembly, the bearing, the housing, and the totals. The bearing's gross requirements are the largest numbers on the sheet, reaching 1,980 in period 8, although no deck assembly is ever built in a quantity approaching that. Five bearings per deck and two paths through the structure will do that. The three cells at the foot are the check block's claim made computable: 57 orders, \$14,500 of setup, and nothing carried.

6.9.3. The Lumpiness Sheet

Lumpiness illustrates Table 6.7 with the arithmetic exposed. Each row is what one rule has the deck assembly receive, and the columns to the right count the orders and compute the variability coefficient over the twelve scheduled months.

The variability coefficient is written as the mean of the squares less the square of the mean, over the square of the mean. The more obvious `SUMPRODUCT((range - mean)^2)` computes the same number and is an array expression, which a reader cannot follow by clicking on it.

The rows are blue, so a reader can type a pattern of their own. Anything that sums to 1,840 and covers the schedule is a plan, and the sheet will price the consequences. Figure 6.6 is Table 6.7 as a reader meets it.

6.9.4. The Network

DRP is the **Explosion** sheet with three level-0 locations instead of one and every quantity per equal to one. It is a separate sheet instead of a note on the first, because seeing the two side by side is what makes Section 6.6.1 believable. Figure 6.7 and Figure 6.8 are that sheet.

6.10. Designing the Software

Section 6.9 built the models as worksheets, and a worksheet shows its own structure. A program does not, so the structure has to be decided and written down. In Section 5.9 we did this for Chapter 5 and found that three heuristics were one procedure. This chapter's design has a different shape, because most of what it needs already exists.

6. Material and Distribution Requirements Planning

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q
1	What the end item's rule does to everything below it																
2	<i>Chapter 6, section 6.5. Every component stays on lot-for-lot. Only the deck assembly's rule changes.</i>																
3																	
4	WHAT EACH RULE HAS ARRIVE AT THE DECK ASSEMBLY																
5	<i>The twelve scheduled months only. The lead-in of the Explosion sheet is a property of the horizon and not of this co</i>																
6	Period	1	2	3	4	5	6	7	8	9	10	11	12	Orders placed	Largest receipt	Variability coefficient	Total belts
7	Lot-for-lot	40	60	120	300	420	260	120	40	0	80	180	220	11	420	0.6191	1,840
8	Adjusted EOQ	220	0	0	300	420	260	240	0	0	0	180	220	7	420	0.8459	1,840
9	Periodic order quantity	40	60	120	300	420	260	120	40	0	80	180	220	11	420	0.6191	1,840
10	Least unit cost	100	0	420	0	420	260	160	0	0	260	0	220	7	420	1.0274	1,840
11	Part-period balancing	100	0	120	300	680	0	160	0	0	260	0	220	7	680	1.5463	1,840
12	Silver-Meal	100	0	120	300	420	420	0	0	0	80	180	220	8	420	0.9650	1,840
13	Wagner-Whitin	100	0	120	300	420	260	160	0	0	80	180	220	9	420	0.6701	1,840
14																	
15	CHECK with the data as shipped																
16	<i>Lot-for-lot passes the schedule through unchanged, so its variability coefficient is the schedule's own, 0.6191.</i>																
17	<i>Every rule that batches raises it, as far as 1.5463 under part-period balancing. Every row totals 1,840 belts.</i>																
18	<i>Part-period balancing is also the only rule with a receipt larger than 420, and its single lot of 680 is why its coefficient is the highest on the sheet.</i>																

Figure 6.6.: The Lumpiness sheet. Seven rules, twelve months, and four columns of summary. The lead-in of the Explosion sheet is left out, because it is a property of the horizon and not of this comparison. Notice the Largest receipt column: six of the seven rules never exceed 420, and part-period balancing's single lot of 680 is why its variability coefficient is the highest on the sheet. Notice also the Total column, which reads 1,840 on every row and is there to be checked.

6. Material and Distribution Requirements Planning

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	The distribution network																	
2	<i>Chapter 6, section 6.6. The same arithmetic as the Explosion sheet, over a network instead of a bill of material.</i>																	
3																		
4	CHECK with the data as shipped																	
5	<i>The three regions sell 1,840 belts between them, the same total the deck assembly schedule carries.</i>																	
6	<i>Nothing is past due: the horizon leads the first sale by four months, which is the southern region's two plus the centre's two.</i>																	
7	Period	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
8	N Northern region transit time 1																	
9	Gross requirements	0	0	0	0	20	30	60	140	180	110	50	20	0	40	80	100	0
10	Scheduled receipts	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
11	Projected on hand	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
12	Net requirements	0	0	0	0	20	30	60	140	180	110	50	20	0	40	80	100	0
13	Planned order receipts	0	0	0	0	20	30	60	140	180	110	50	20	0	40	80	100	0
14	Planned order releases	0	0	0	20	30	60	140	180	110	50	20	0	40	80	100	0	0
15																		
16	Period	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
17	S Southern region transit time 2																	
18	Gross requirements	0	0	0	0	10	20	40	100	160	100	50	10	0	20	60	70	0
19	Scheduled receipts	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
20	Projected on hand	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
21	Net requirements	0	0	0	0	10	20	40	100	160	100	50	10	0	20	60	70	0
22	Planned order receipts	0	0	0	0	10	20	40	100	160	100	50	10	0	20	60	70	0
23	Planned order releases	0	0	10	20	40	100	160	100	50	10	0	20	60	70	0	0	0

Figure 6.7.: The two nearer regions of the DRP sheet. Each is planned exactly as an end item is planned on the **Explosion** sheet, and the only word that changes is the one naming the offset: a region waits a transit time where an assembly waits a lead time. Notice that the southern region's releases start a period earlier than the northern region's, although both regions' sales begin in the same month, because its transit time is a month longer.

6. Material and Distribution Requirements Planning

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
25	Period	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
26	W Western region transit time 1																	
27	Gross requirements	0	0	0	0	10	10	20	60	80	50	20	10	0	20	40	50	0
28	Scheduled receipts	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
29	Projected on hand	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
30	Net requirements	0	0	0	0	10	10	20	60	80	50	20	10	0	20	40	50	0
31	Planned order receipts	0	0	0	0	10	10	20	60	80	50	20	10	0	20	40	50	0
32	Planned order releases	0	0	0	10	10	20	60	80	50	20	10	0	20	40	50	0	0
33																		
34	Period	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
35	CW Central warehouse supplier lead time 2																	
36	Gross requirements	0	0	10	50	80	180	360	360	210	80	30	20	120	190	150	0	0
37	Scheduled receipts	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
38	Projected on hand	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
39	Net requirements	0	0	10	50	80	180	360	360	210	80	30	20	120	190	150	0	0
40	Planned order receipts	0	0	10	50	80	180	360	360	210	80	30	20	120	190	150	0	0
41	Planned order releases	10	50	80	180	360	360	210	80	30	20	120	190	150	0	0	0	0
42																		
43	The centre's gross requirements are the sum of what the regions release. That is the explosion of section 6.4 with every quantity per equal to one.																	

Figure 6.8.: The western region and the centre. The centre's gross requirement row is the three regions' release rows added down the column, and nothing else. Notice period 3, where the centre needs ten belts: the southern region alone is asking, because the other two are a month behind it.

6. Material and Distribution Requirements Planning

6.10.1. What the Software Must Do

1. Hold a product structure, with quantities per, and assign low-level codes.
2. Hold what the planner knows about an item that is not its requirements.
3. Compute one item's record: net, lot size, offset.
4. Explode a master production schedule through the structure, in the right order.
5. Do the same over a distribution network.
6. Report what a plan costs, by item and in total.

Requirement 5 is the one to watch. It looks like a second feature and Section 6.10.6 argues that it is not one.

6.10.2. Finding the Nouns

A **planned item** is what the item master holds: lead time, opening stock, what is on order, the costs, and the rule. Not its requirements, because for every item but an end item those are computed and change whenever the level above is replanned.

A **bill of material** is the structure. It owns the low-level codes, which are a property of the graph and not of any item in it. An item cannot answer “what is my low-level code” without seeing every path that reaches it, so the question belongs to the thing that holds the paths.

An **MRP record** is one item's six rows over a horizon.

A **requirements plan** is every item's record, and the order they were computed in.

A **receipt policy** is how much to have arrive, given what must arrive.

Level is the instructive rejection. It reads like a class and the chapter uses the word constantly. It fails because a level has no behaviour: it is an

6. Material and Distribution Requirements Planning

integer attached to an item, and the only thing anyone does with it is sort by it.

Figure 6.9 illustrates the five and how they stand to one another. Notice that the bill of material holds the low-level codes and the planned item does not.

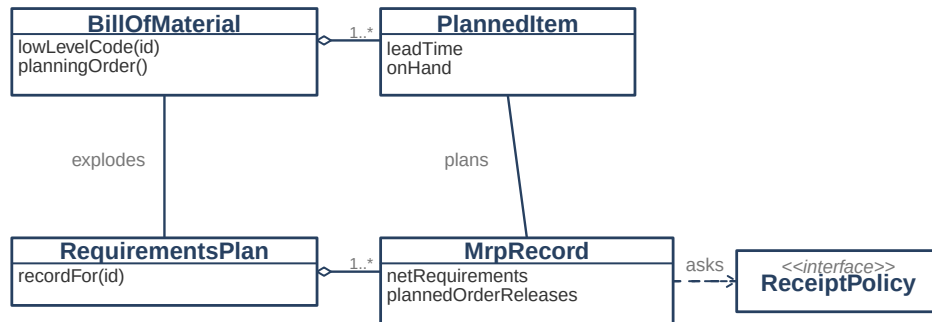


Figure 6.9.: The concepts of this chapter before any thought about implementation. A bill of material relates planned items to one another. A requirements plan holds one record per item, and each record asks a receipt policy how much should arrive. The low-level code sits on the bill of material and not on the item, because an item cannot answer what its own code is without seeing every path that reaches it.

6.10.3. The Join with Chapter 5

The design turns on one sentence: **the net requirement stream is a Section 5.1.1 requirements schedule**. Once that is said, none of Section 5.5 needs an MRP version.

```
/**
 * The net requirements of this item, as a Chapter 5
 * requirements schedule.
```

6. Material and Distribution Requirements Planning

```
*
* This is the join between the two chapters and it is worth
↪ naming. Section 5.1
* took a requirements schedule as given. Chapter 6 computes
↪ one, for every item
* at every level, and hands it back to Section 5.5
↪ unchanged.
*/
fun scheduleFor(netRequirements: List<Double>):
↪ RequirementsSchedule =
    RequirementsSchedule.constantCosts(
        label = label,
        requirements = netRequirements,
        orderCost = orderCost,
        unitCost = unitCost,
        carryingCharge = carryingCharge,
    )
```

The adapter that uses it is the whole of what Section 5.5 needed to be reused.

```
class PlanTheHorizon(private val rule: LotSizingRule) :
↪ ReceiptPolicy {
    override fun receiptsFor(netRequirements: List<Double>,
↪ item: PlannedItem): List<Double> {
        val first = netRequirements.indexOfFirst { it > 0.0 }
        if (first < 0) return netRequirements.map { 0.0 }
        val plan = rule.plan(item.scheduleFor(
↪ netRequirements.drop(first)))
        return netRequirements.indices.map {
            if (it < first) 0.0 else plan.orderIn(it - first
↪ + 1)
        }
    }
```

6. Material and Distribution Requirements Planning

```
    }  
}
```

The **drop** is part of the contract and not a convenience. Section 5.1.2 has inventory starting at zero and period 1 holding an order, so a rule handed a stream that opens with empty periods places a receipt in the first of them and carries it. The empty periods here are the lead-in of Section 6.3.3, and they are not part of the problem the rule is being asked to solve.

6.10.4. One Interface for Two Kinds of Rule

A fixed order quantity decides period by period and looks nowhere. Silver-Meal cannot decide anything until it has seen the whole horizon. They look as though they need two interfaces and they do not, because the net requirement stream is computed *before* any rule runs. That is what makes the whole of it available to a rule that needs the whole of it.

```
fun interface ReceiptPolicy {  
    /** Planned order receipts, one per period, covering  
     ↪ [netRequirements]. */  
    fun receiptsFor(netRequirements: List<Double>, item:  
     ↪ PlannedItem): List<Double>  
}
```

Figure 6.10 illustrates what implements it. Two of the three are new, and the third is an adapter onto Chapter 5.

The fixed order quantity is the rule Section 5.2.2 had to exclude, and it is admissible here.

6. Material and Distribution Requirements Planning

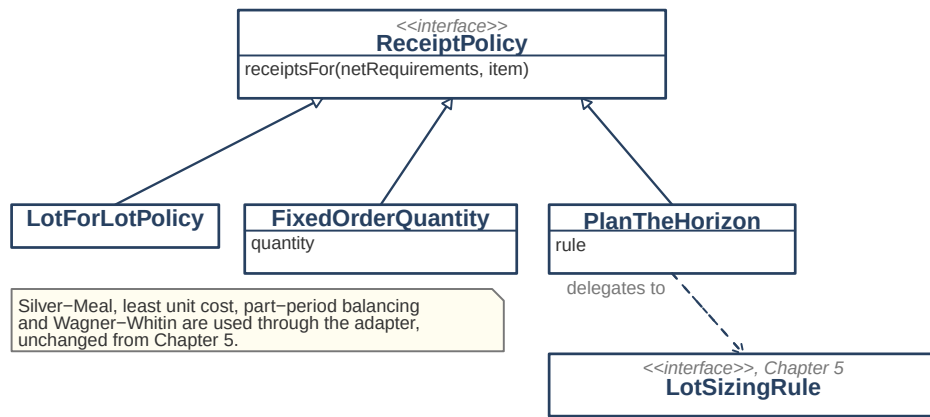


Figure 6.10.: The receipt policies. Lot-for-lot and the fixed order quantity are this chapter's own, and **PlanTheHorizon** is an adapter that hands the net requirement stream to any rule of Section 5.5. The dashed arrow is the whole of what Chapter 5 needed in order to be reused: no rule in that chapter has an MRP-specific version.

6. Material and Distribution Requirements Planning

```
/**
 * Order [quantity], or the period's net requirement when
 * ↪ that is larger.
 *
 * Section 5.2.3 excluded this rule, because a quantity
 * ↪ unrelated to the
 * requirements cannot finish the horizon empty. Inside an
 * ↪ MRP record it is
 * admissible, and widely used, precisely because the record
 * ↪ is not required to
 * finish empty: what is left over is on hand at the start of
 * ↪ the next planning
 * cycle. That is what the projected-on-hand row is for.
 */
class FixedOrderQuantity(private val quantity: Double) :
  ↪ ReceiptPolicy
```

6.10.5. Two Readings of “Net Requirement”

The record computes in two sweeps, and the reason matters, because this is the only place in the package where a name means two things.

The first sweep asks what would have to arrive in each period if nothing were carried beyond the opening stock and the scheduled receipts. That is the stream a policy is asked to cover, and it does not depend on the policy.

The second sweep works out what the policy’s receipts actually leave on hand, and recomputes the row the record *prints*. Those two are usually different. A fixed order quantity that covers three periods at once makes the printed net requirement zero in the two periods after it, and the stream it was given had positive numbers there.

6. Material and Distribution Requirements Planning

```
val netRequirementsBeforeLotSizing: List<Double> // what
  ↪ the policy is given
val netRequirements: List<Double>                // what
  ↪ the record shows
```

Thus, the two names. One property meaning whichever the caller happened to need would put the package into a quiet disagreement with every worked example in the literature, and nothing in the arithmetic would say where the disagreement came from.

6.10.6. A Network Is a Bill of Material

Section 6.6.1 claimed that distribution requirements planning is the same calculation. The design's job is to make that claim structural and not rhetorical, and the way it does so is by not writing a second class.

```
BillOfMaterial(
  locations,
  regions.map { BillOfMaterial.Usage(parent = it, child =
    ↪ "CW", quantity = 1.0) },
)
```

That is the whole of the network's definition. `RequirementsPlan` then assigns low-level codes, which puts the regions at 0 and the centre at 1, plans them in that order, and computes the centre's gross requirements as the sum over its parents. Figure 6.11 traces one item through that, and nothing in the trace would change if the graph were a network instead of a structure.

The temptation is a `DistributionNetwork` class with `regions` and `centre` on it, reading more naturally at the call site. It would duplicate the low-level coding, the planning order and the explosion, and it would then be

6. Material and Distribution Requirements Planning

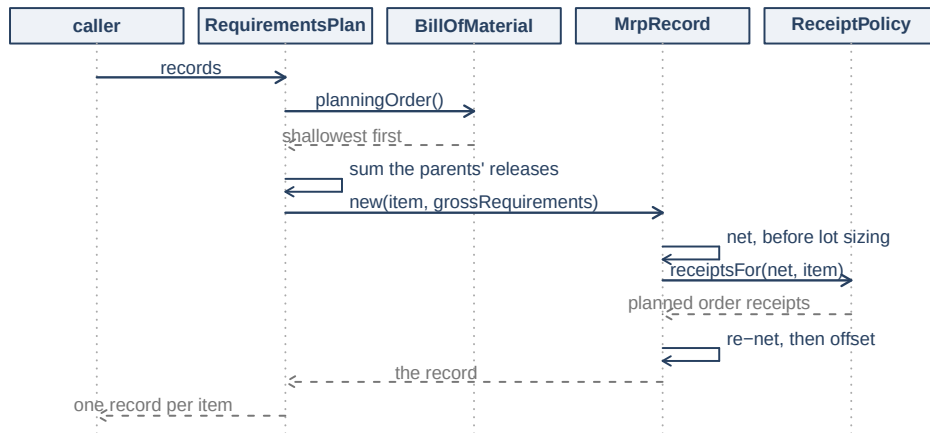


Figure 6.11.: Planning one component. The plan asks the bill of material for the order, then for each item asks the already-completed parents for their releases, builds the record, and the record asks the policy how much should arrive. Notice that the record computes its net requirements BEFORE the policy is consulted, which is what makes the whole stream available to a rule that needs all of it. The dashed arrows are returns.

6. Material and Distribution Requirements Planning

possible for the two to disagree about a network that is also a structure, and that is a real thing: a warehouse supplying both dealers and an assembly plant.

Section 4.6.3 *does* have a `DistributionNetwork`, and it is not this one. It solves a different problem, the power-of-two reorder intervals of Section 4.6.2, and it holds no time-phased anything. Two classes with the same name in one book is a fair criticism; two classes for two problems is not.

6.10.7. Running the Chapter's Examples

Every figure in this chapter comes from running this code. The structure of Figure 6.1 is a list of items and a list of usages, and nothing else.

```
val bom = BillOfMaterial(  
  listOf(  
    PlannedItem("DA", "Mower deck assembly", leadTime =  
↪ 1, policy = LotForLotPolicy,  
      orderCost = 500.0, unitCost = 180.0,  
↪ carryingCharge = 0.02),  
    PlannedItem("B", "Deck belt", leadTime = 2, policy =  
↪ LotForLotPolicy,  
      orderCost = 300.0, unitCost = 50.0,  
↪ carryingCharge = 0.02),  
    PlannedItem("S", "Spindle assembly", leadTime = 1,  
↪ policy = LotForLotPolicy,  
      orderCost = 250.0, unitCost = 25.0,  
↪ carryingCharge = 0.03),  
    PlannedItem("H", "Spindle housing", leadTime = 2,  
↪ policy = LotForLotPolicy,  
      orderCost = 150.0, unitCost = 12.0,  
↪ carryingCharge = 0.02),  
    PlannedItem("BR", "Bearing", leadTime = 3, policy =  
↪ LotForLotPolicy,
```

6. Material and Distribution Requirements Planning

```
        orderCost = 100.0, unitCost = 4.0, carryingCharge
↪ = 0.02),
    ),
    listOf(
        BillOfMaterial.Usage("DA", "B", 1.0),
        BillOfMaterial.Usage("DA", "S", 2.0),
        BillOfMaterial.Usage("DA", "BR", 1.0),
        BillOfMaterial.Usage("S", "H", 1.0),
        BillOfMaterial.Usage("S", "BR", 2.0),
    ),
)

println(bom.lowLevelCodes.toSortedMap())    // {B=1, BR=2,
↪ DA=0, H=2, S=1}
println(bom.planningOrder())                // [DA, B, S, BR,
↪ H]
```

Notice that no level was typed anywhere. `lowLevelCodes` is Section 6.2.1 and `planningOrder` is the order Section 6.4 plans in, and both are read off the usages, which is the argument of Section 6.10.2 made operational.

A master production schedule drives it. The five zeros in front are the lead-in of Section 6.3.3, and `RequirementsPlan` takes the schedule as a map because a structure may have more than one end item.

```
val schedule = List(5) { 0.0 } +
    listOf(40.0, 60.0, 120.0, 300.0, 420.0, 260.0, 120.0,
↪ 40.0, 0.0, 80.0, 180.0, 220.0)
val plan = RequirementsPlan(bom, mapOf("DA" to schedule))

val bearing = plan.recordFor("BR")
println(bearing.grossRequirements.map { it.toInt() })
println(bearing.plannedOrderReleases.map { it.toInt() })
```

6. Material and Distribution Requirements Planning

```
println("%d orders, %.0f setup, %.0f carrying"
        .format(bearing.orderCount, bearing.setupCost,
        ↪ bearing.carryingCost))
println("%.0f past due".format(plan.pastDueReleases))
```

```
[0, 0, 0, 160, 280, 540, 1320, 1980, 1460, 740, 280, 40, 320, 800, 1060, 220, 0]
[160, 280, 540, 1320, 1980, 1460, 740, 280, 40, 320, 800, 1060, 220, 0, 0, 0, 0]
13 orders, 1300 setup, 0 carrying
0 past due
```

That first row is Table 6.6 and the second is the bearing's release row in Figure 6.5. Notice `pastDueReleases`, which is how a horizon is checked without reading the record: it is zero here, and with four zeros in front instead of five it is 160, because the bearing's first release would fall before period 1.

Changing a rule means changing one item's policy, and `PlanTheHorizon` is what admits a rule of Section 5.5. Putting the deck assembly on Silver-Meal, with every component still on lot-for-lot, is one row of Table 6.7.

```
PlannedItem("DA", "Mower deck assembly", leadTime = 1,
            policy = PlanTheHorizon(SilverMeal),
            orderCost = 500.0, unitCost = 180.0, carryingCharge =
            ↪ 0.02)
```

```
println("%.0f at the deck assembly, %.0f in total"
        .format(plan.recordFor("DA").relevantCost,
        ↪ plan.relevantCost))
// 4936 at the deck assembly, 11636 in total
```

That is against \$14,500 with lot-for-lot everywhere, and Section 6.5.1 is that comparison run once per rule. Notice that `relevantCost` is available

6. Material and Distribution Requirements Planning

on the record and on the plan, which is what lets the experiment separate what the rule saved at the deck assembly from what it cost below.

The network of Section 6.6 uses the same two classes, with the regions as parents and every quantity per equal to one.

```
val demand = mapOf(
    "N" to listOf(20.0, 30.0, 60.0, 140.0, 180.0, 110.0,
        ↪ 50.0, 20.0, 0.0, 40.0, 80.0, 100.0),
    "S" to listOf(10.0, 20.0, 40.0, 100.0, 160.0, 100.0,
        ↪ 50.0, 10.0, 0.0, 20.0, 60.0, 70.0),
    "W" to listOf(10.0, 10.0, 20.0, 60.0, 80.0, 50.0, 20.0,
        ↪ 10.0, 0.0, 20.0, 40.0, 50.0),
)
val regions = listOf("N" to 1, "S" to 2, "W" to 1)
val network = BillOfMaterial(
    regions.map { (id, transit) ->
        PlannedItem(id, "$id region", leadTime = transit,
        ↪ policy = LotForLotPolicy,
        ↪ orderCost = 200.0, unitCost = 50.0,
        ↪ carryingCharge = 0.02)
    } + PlannedItem("CW", "Central warehouse", leadTime = 2,
        ↪ policy = LotForLotPolicy,
        ↪ orderCost = 400.0, unitCost = 50.0, carryingCharge =
        ↪ 0.02),
    regions.map { (id, _) -> BillOfMaterial.Usage(parent =
        ↪ id, child = "CW", quantity = 1.0) },
)
val drp = RequirementsPlan(network, demand.mapValues {
    ↪ List(4) { 0.0 } + it.value })

println(network.lowLevelCodes.toSortedMap()) // {CW=1, N=0,
    ↪ S=0, W=0}
println(network.planningOrder()) // [N, S, W,
    ↪ CW]
```

6. Material and Distribution Requirements Planning

```
println(drp.recordFor("CW").grossRequirements.map {  
  ↪ it.toInt() })  
// [0, 0, 10, 50, 80, 180, 360, 360, 210, 80, 30, 20, 120,  
  ↪ 190, 150, 0]
```

The last row is the centre's gross requirement row in Figure 6.8. Notice the lead-in is four here and five above, because Section 6.6 has a shallower graph. Notice also what is absent: no class in this package is told that the graph is a network, and the only thing that says so is the word **region** in a label.

That is the whole of the interface. Section 6.11 collects what the chapter argued with it.

6.11. Summary

Most items in a manufacturing business have demand that is implied by the firm's own plans. Calculating it from those plans beats forecasting it, and the calculation is a record per item: net what is needed against what is already there, lot size the result, and move it back by the lead time.

Two ideas make the calculation run over a whole structure. A low-level code says when an item may be planned, and that is after every parent of it. And a component's gross requirements are its parents' planned order *releases* times the quantity per, and that is Equation 6.1.

The horizon has to lead the first requirement by the cumulative lead time down the deepest path, or the plan reports orders it has no time to place.

Chapter 5's rules run inside a record unchanged, and using them raises a question Chapter 5 could not ask. A parent's rule decides what its components are asked for, so it decides their problem. The cost below a level rises with how often the level above orders, and the rule that is

6. Material and Distribution Requirements Planning

cheapest at a level is not the rule that is cheapest for the product. On the deck assembly that error costs 7.59%.

A distribution network is the same calculation with every quantity per equal to one, and it has the same result: batching at one stage amplifies what the next stage sees, by a factor of two to three here. That amplification costs nothing while demand is known. Thus, it belongs to Chapter 10 and not to this chapter.

Everything here assumes infinite capacity, fixed lead times, accurate records and known demand. The last of those is what the rest of the book removes.

Table 6.10 collects the notation, which Appendix A repeats alongside the rest of the book's.

Table 6.10.: Notation introduced in this chapter.

Symbol	Meaning
G_t	gross requirement in period t
SR_t	scheduled receipt in period t
I_t	projected on hand at the end of period t
NR_t	net requirement in period t
POR_t	planned order receipt in period t
Rel_t	planned order release in period t
L	lead time, in periods
SS	safety stock
q_{ij}	units of j in one i

6.12. Exercises

Unless an exercise says otherwise, use these conventions. Periods are numbered from one. Projected on hand is an end-of-period figure. A net

6. Material and Distribution Requirements Planning

requirement is never negative. A planned order release falls in the period the receipt is due less the lead time, and a release that would fall before period 1 is past due and is reported instead of placed. Report costs to the nearest cent and the variability coefficient to four decimal places.

6.12.1. Terminology and Concepts

Exercise 6.1. Define each of the following in one sentence, and say which section introduced it.

- a. dependent demand
- b. quantity per
- c. low-level code
- d. gross requirement
- e. net requirement
- f. planned order release
- g. past-due release
- h. cumulative lead time
- i. firm planned order
- j. safety lead time

Exercise 6.2. State whether each is true or false and give one sentence of justification.

- a. A component's gross requirements are its parents' planned order receipts times the quantity per.
- b. An item that appears at two levels of a bill of material is planned twice.
- c. Lot-for-lot at a parent leaves its components' demand pattern unchanged.
- d. The variability coefficient of a component's demand tells you what the component's plan will cost.
- e. If every item uses the rule that is cheapest for it, the plan is the cheapest plan for the product.

6. Material and Distribution Requirements Planning

- f. A past-due release means the inventory records are wrong.
- g. The bullwhip effect is caused by the network having several stages.
- h. Safety stock and safety lead time buffer against the same thing.

Exercise 6.3. Choose the best answer.

- 1. Low-level codes exist so that (a) items can be sorted for printing; (b) an item is planned only after every parent has asked for it; (c) the bill of material can be drawn as a tree; (d) lead times can be added up.
- 2. The max in Equation 6.3 is there to (a) keep the arithmetic positive; (b) stop a surplus in one period being treated as a requirement in the next; (c) round the order quantity; (d) prevent division by zero.
- 3. A planning horizon must lead the first requirement by at least (a) the longest single lead time; (b) the average lead time; (c) the cumulative lead time down the deepest path; (d) one period.
- 4. The cost below a level rises with (a) the variability coefficient of the demand the components see; (b) how often the level above orders; (c) the number of levels;
(d) the quantity per.
- 5. Distribution requirements planning differs from material requirements planning in that (a) it uses a different netting formula; (b) every quantity per is one and the parent is a customer; (c) it cannot use lot sizing; (d) it has no lead times.

6.12.2. Working the Records by Hand

Exercise 6.4. The deck belt has the twelve-month gross requirements of Table 5.1, a lead time of two months, 60 belts on hand, and a scheduled receipt of 200 belts due in month 3.

- a. Complete the record under lot-for-lot: net requirements, projected on hand, planned order receipts, and planned order releases.

6. Material and Distribution Requirements Planning

- b. How many belts are past due, and why?
- c. Now use a fixed order quantity of 250, meaning that every receipt is 250 belts or the net requirement if that is larger. Complete the record again.
- d. Compare the two on orders placed and on unit-months carried. Which would you use, and what would you need to know to decide?

Exercise 6.5. A product P is assembled from two A, one B and one W. Each A contains one C and three W. Each B contains two C. Each W contains two C. Lead times are P one period, A two, B one, W two and C one.

- a. Draw the product structure and the indented list.
- b. Assign every item its low-level code.
- c. Give the order the items must be planned in.
- d. C appears at three different levels. Say what would go wrong if it were planned at the shallowest of them.
- e. Compute the cumulative lead time, and say how long a horizon must be to plan a twelve-period schedule for P.

Exercise 6.6. Using the structure of Exercise 6.5, suppose the master production schedule calls for 100 P in period 10 and 150 P in period 13, with nothing on hand or on order anywhere and lot-for-lot at every item.

- a. Complete the record for P and read off its planned order releases.
- b. Explode to A, B and W, and complete their records.
- c. Explode to C. Show the three contributions separately before summing them.
- d. In which period is the earliest order released, and for which item?

Exercise 6.7. Table 6.3 reports what happens to the deck assembly structure when the horizon is too short.

6. Material and Distribution Requirements Planning

- a. Reproduce the 160 units that are past due when the schedule is given four empty months instead of five. Which item are they, and which path through the structure produced them?
- b. The bearing has a lead time of three months. If it were reduced to two, how many empty months would the horizon need?
- c. A planner proposes to solve past-due orders by starting the schedule later rather than by lengthening the horizon. Say what is wrong with that.

6.12.3. Modeling Problems

Exercise 6.8. Table 6.7 holds every component at lot-for-lot and varies only the deck assembly's rule.

- a. Explain in your own words why the Cost below column rises with the Orders column.
- b. The periodic order quantity and lot-for-lot give identical results. Compute the deck assembly's economic order quantity from Equation 3.17 and its time supply, and explain the coincidence.
- c. Wagner-Whitin is optimal at the deck assembly and third worst over the structure. State precisely what Section 5.4.2 does and does not guarantee.
- d. Propose a procedure that would find the rule minimizing total cost over the structure, and say why it is not what an MRP system does.

Exercise 6.9. Take the three regions of Example 6.3.

- a. Verify that the three regional demands sum to the 1,840 belts of Table 5.1.
- b. Reproduce the centre's gross requirements in periods 3 through 8.
- c. The southern region's transit time is two months and the others' are one. Change the south to one month and recompute the centre's gross requirements. Does the total change? Does the variability coefficient?

6. Material and Distribution Requirements Planning

- d. Table 6.9 reports that every batching rule at the regions is cheaper than lot-for-lot, at the regions and at the centre both. Explain why, and then say what would have to be true about the world for the amplification to cost something.

Exercise 6.10. The bearing has a lead time of three months and a supplier who is occasionally a month late.

- a. Compare holding one month's average requirement as safety stock against planning the bearing with a safety lead time of one month. Which protects against a late delivery, and which against a larger requirement than planned?
- b. Safety stock enters Equation 6.3 as an addition to the gross requirement. Write the corresponding modification for safety lead time and say which row it changes.
- c. A colleague proposes safety stock at every level of the structure. Say what that does to the total stock held, and why buffering at one level is usually preferred.

6.12.4. Using the Workbook

Exercise 6.11. Open `Chapter6Models.xlsx`.

- a. On the **Record** sheet, change the opening stock to 200 and describe what happens to each of the six rows.
- b. On the **Explosion** sheet, change the quantity per for spindle assemblies from 2 to
 3. Report the new gross requirements for the bearing and explain the change.
- c. On the **Lumpiness** sheet, type a plan of your own into the last row: any pattern that sums to 1,840 and covers the schedule. Can you find one with a lower variability coefficient than lot-for-lot's 0.6191? Explain your answer.

6. Material and Distribution Requirements Planning

- d. On the DRP sheet, give all three regions the same transit time. What happens to the centre's variability coefficient, and what does that tell you about 3?

6.12.5. Using the Software

Exercise 6.12. The code of Section 6.10 is in `code/`, package `inventory.requirementsplanning`. Section 6.10.7 is the starting point: it builds the structure, plans it, and reads a record. Report the code you wrote alongside each answer.

- a. Build the structure of Figure 6.1 and print `lowLevelCodes` and `planningOrder()`. Now suppose the belt runs over an idler pulley that carries one of the same bearing, which is `BillOfMaterial.Usage("B", "BR", 1.0)`. Add it and print both again, then print the bearing's gross requirements before and after. Finally, vary the number of empty months in front of the master schedule and report the shortest lead-in that leaves `pastDueReleases` at zero. It is not the five of Section 6.3.3. Explain why it moved when no low-level code did.
- b. Reproduce Table 6.7. Give the deck assembly `PlanTheHorizon(rule)`, leave every component on `LotForLotPolicy`, and loop over the seven rules, reporting `orderCount` and `relevantCost` at the deck assembly, the same total over every other item, and the variability coefficient of the deck assembly's receipts over the twelve scheduled months. Which rule makes the deck assembly's receipts lumpiest, and is it the rule that costs the most?
- c. Bearings are bought in boxes of 500. Implement a `ReceiptPolicy` whose receipt in each period is the smallest multiple of the box size that covers what is still short, where what is still short is that period's net requirement less anything left over from earlier receipts. Give it to the bearing, leave everything else on lot-for-lot, and report the bearing's receipts, `orderCount`, `setupCost`, `carryingCost`, and

6. Material and Distribution Requirements Planning

the last entry of `projectedOnHand`. Is the plan cheaper? Which assumption of Section 5.1.2 does that last entry break, and why is breaking it admissible inside a record?

- d. Section 6.5.1 holds every component on lot-for-lot. Put every component on `PlanTheHorizon(WagnerWhitin)` instead and run the comparison of part (b) again. Report the table. Does the cost below the deck assembly still rise with the deck assembly's order count? Does choosing the rule on the deck assembly alone still pick the wrong one, and what does the error cost now?
 - e. The distributor opens an eastern region whose transit time is three months and whose sales are those of the northern region, at the same costs as the other regions. Add it to the network of Section 6.6, report the centre's gross requirements, and compare the variability coefficient of what the centre sees against the 0.6013 of
3. Explain the direction it moved.

6.12.6. From the Literature

Exercise 6.13. Requirements planning was promoted in the 1970s as a replacement for order-point systems, and the claims made for it were larger than what it delivered.

- a. Using 4, give two assumptions whose failure would account for a disappointing implementation.
 - b. Section 2.3 reports what inventory record accuracy actually is in practice. Explain why an order-point system tolerates a bad record better than a requirements planning system does.
 - c. Find one account of an MRP implementation in the literature and report what its authors say went wrong. Classify their explanation against the five assumptions of
- 4.

Part II.

Stochastic Models

7. Single-Period Stochastic Inventory

i Learning objectives

After reading this chapter you should be able to:

- recognize a single-period problem and say why it is not a one-period version of a multi-period one
- work the ordering decision by hand as a marginal analysis
- characterize a demand history and fit a distribution to it
- formulate the newsvendor problem in both its cost and its profit forms
- derive and interpret the critical ratio
- solve the model for continuous and for discrete demand
- extend the model to a penalty cost and to stock already on hand
- estimate the answer by simulation, and say when that is worth doing

7.1. One Order, One Chance

Every model so far has known its demand. Chapter 3 and Chapter 4 took a constant rate, Chapter 5 let the rate vary by period and still knew it, and Chapter 6 computed it from a production plan. Assumption 5 of 4 is the one this chapter removes. From here on demand is a random variable, and the decision is made before it is observed.

7. Single-Period Stochastic Inventory

We begin with the simplest such problem, in which the decision is made **once**.

Example 7.1 (The last order for the mower deck belt). The distributor of Chapter 5 has stocked the mower deck belt for years. The deck it fits is now out of production, so the seasonal pattern of Table 5.1 is over and what remains is steady replacement demand from owners of machines already in the field. The supplier is discontinuing the belt and has offered one last order.

What is stocked? One item, the belt.

What is the demand process? Weekly demand is random and no longer seasonal. It will run for about fourteen more weeks, until a redesigned deck reaches the dealers and the belt becomes unsellable.

When is inventory reviewed? Once. There is one decision and no second chance.

What triggers replenishment, and how much? Nothing triggers it. The distributor names a quantity now.

What happens to unmet demand? It is lost. A dealer who cannot get the belt from this distributor gets it elsewhere.

What costs are incurred, and when? The belt costs \$50 and sells for \$95. Belts left when the deck is withdrawn go to a scrap dealer at \$12.

Three things about Example 7.1 distinguish it from everything before it.

There is no ordering cost in the model. Recall from Table 1.10 that k is the price of placing an order, and that the models of Chapter 3 through Chapter 6 trade it against holding. Here exactly one order is placed whatever its size, so k is the same under every decision. A constant added to every alternative cannot change which alternative is least, which is the argument of Section 1.4.1. Thus, k drops out, and with it the entire question those chapters were about.

7. Single-Period Stochastic Inventory

Leftover stock has no future value. Recall from Section 1.2.7 that a single-period problem is not a multi-period problem with the number of periods set to one. In a multi-period problem a unit left at the end of a period is available at the start of the next, so the cost of having too many is a holding charge for one period. Here it is the whole loss on the unit, \$50 paid against \$12 recovered. That is why the trade-off is overage against underage rather than holding against ordering.

The two errors are priced differently, and neither is free. Order too many and the distributor loses \$38 a belt. Order too few and it loses the \$45 margin on each belt it could have sold. Notice that both are opportunity costs measured against the decision that could have been made, and that neither appears in any ledger. Section 1.4.2 is why.

7.1.1. Demand Is Not Sales

One distinction has to be made before any data are used, because it is the commonest error in this material and it is invisible in the answer.

- Let **demand** represent what customers would have bought.
- Let **sales** represent what they did buy.

The two differ whenever the shelf was empty. A week that sold 40 belts because 40 were on the shelf and a week that sold 40 because 40 were wanted look identical in a sales record. That is, sales are demand **censored** at whatever was available, and a history of sales understates both the average and the spread. It understates them most in exactly the weeks when the item mattered, because those are the weeks it ran out.

Recall from Section 2.3 that a stock record is only as good as the transactions behind it. This is the same point about a different record. An organization that wants to model demand has to record the demand it could not fill, and most do not. Where that record does not exist, the fitted distribution of Section 7.3 is biased low and every quantity computed

7. Single-Period Stochastic Inventory

from it is too small. You should say so when you report the answer, rather than let the arithmetic imply a precision the data do not carry.

The next section makes the decision without any distribution theory at all.

7.2. Working the Decision by Hand

Before any calculus, the decision can be made with arithmetic a planner would do on a whiteboard. The method is worth more than the answer, because it produces the rule that the rest of the chapter derives formally.

Start with a sketch of demand rather than a fitted distribution. Suppose the distributor, asked for a judgment, offers five outcomes for total remaining demand and a probability for each.

Table 7.1.: A five-point sketch of remaining demand for the belt. The mean is 485 belts. Notice that the sketch is coarse on purpose: the argument below needs the shape of demand and not its detail.

Demand x , belts	300	400	500	600	700
$P\{X = x\}$	0.15	0.25	0.30	0.20	0.10
$P\{X \leq x\}$	0.15	0.40	0.70	0.90	1.00

The decision is how many belts to buy, in hundreds. Rather than evaluate every candidate, ask a smaller question. **Given that we have decided on some quantity, is one more hundred belts worth buying?**

Buying one more hundred pays off when demand turns out to exceed the quantity we already have, which happens with probability $P\{X \geq Q+100\}$, and each of those belts earns the \$45 margin. It costs when demand falls at or below the quantity we already have, which happens with probability

7. Single-Period Stochastic Inventory

$P\{X \leq Q\}$, and each unsold belt loses \$38. The extra hundred is worth buying when

$$45 \times 100 \times P\{X \geq Q + 100\} > 38 \times 100 \times P\{X \leq Q\} \quad (7.1)$$

Table 7.2 works Equation 7.1 one step at a time.

Step 1, from 300 to 400. Demand reaches 400 or more with probability 0.85, so the gain is $45(100)(0.85) = \$3,825$. Demand stops at 300 with probability 0.15, so the loss is $38(100)(0.15) = \$570$. The step is worth \$3,255. Take it.

Step 2, from 400 to 500. Demand reaches 500 or more with probability 0.60, giving $45(100)(0.60) = \$2,700$. Demand stops at 400 or below with probability 0.40, giving $38(100)(0.40) = \$1,520$. The step is worth \$1,180. Take it.

Step 3, from 500 to 600. Demand reaches 600 or more with probability 0.30, giving \$1,350. Demand stops at 500 or below with probability 0.70, giving \$2,660. The step **loses** \$1,310. Stop.

Table 7.2.: The marginal analysis, one hundred belts at a time. Notice that the net column changes sign once and never changes back, which is what makes stopping at the first negative step correct rather than merely convenient.

From	To	$P\{X \geq \text{to}\}$	Gain	$P\{X \leq \text{from}\}$	Loss	Net
300	400	0.85	\$3,825	0.15	\$570	+\$3,255
400	500	0.60	\$2,700	0.40	\$1,520	+\$1,180
500	600	0.30	\$1,350	0.70	\$2,660	-\$1,310
600	700	0.10	\$450	0.90	\$3,420	-\$2,970

7. Single-Period Stochastic Inventory

Order 500 belts. Notice that 500 is *more* than the mean of 485. The distributor should plan to be left with belts more often than it plans to run out, because running out costs more per belt than scrapping does.

That the net column changes sign exactly once is not luck. Rearranging Equation 7.1, the step is worth taking while

$$P\{X \leq Q\} < \frac{45}{45 + 38} = \frac{c_u}{c_u + c_o} = 0.5422 \quad (7.2)$$

and the left side is a cumulative distribution function, which never decreases. Thus, once the test fails it fails for every larger quantity, and the rule is to order the smallest quantity whose cumulative probability reaches 0.5422. From Table 7.1 that is 500 belts, since $F(400) = 0.40$ is short of it and $F(500) = 0.70$ clears it.

That ratio is the answer to the whole chapter, and Section 7.6 derives it properly.

A check, and a reason to trust the shortcut. Costing all five candidates directly should agree, and it does.

Table 7.3.: Expected profit at each candidate quantity, computed from Table 7.1 without any shortcut.

Q	300	400	500	600	700
Ex-pected profit	\$13,500	\$16,755	\$17,935	\$16,625	\$13,655

Table 7.3 peaks at 500, which is where Table 7.2 stopped and where Equation 7.2 pointed. Three routes, one answer. That check is worth performing the first time you use marginal analysis on a new problem,

7. Single-Period Stochastic Inventory

because the marginal argument is only valid when the expected profit has a single peak, and the direct calculation is what confirms it.

Notice what the hand method cost. Five outcomes, four steps, no distribution and no calculus. What it cannot do is tell the distributor anything between 400 and 500 belts, and the real decision is a number of belts rather than a number of hundreds. For that we need a distribution, and that is the next two sections.

7.3. Getting a Demand Distribution

Every model from here to the end of the book takes a demand distribution as an input, and the distribution carries more of the answer than the policy formula built on top of it does. This section fits one.

It is deliberately short, because the subject has a book of its own. Appendix B of *Simulation Modeling using the KSL* (Rossetti 2026) gives the procedure in full, and Chapter 2 of the same text shows how to carry it out with the library this book's code uses. What follows is that procedure applied to an inventory problem, with the one difference that matters here made explicit: demand for a spare part is a **count**, so every tool below is a discrete one.

The data. The distributor has 104 weeks of demand for the belt, recorded after the deck went out of production. They are in `data/ch7-weekly-demand.csv` in the course code project, two columns and a header:

```
week,demand
1,94
2,67
3,31
```

Read them the way Section C.4 assumes they arrive, as a file whose contents you did not choose:

7. Single-Period Stochastic Inventory

```
val df = DataFrame.readCSV(  
    File("data/ch7-weekly-demand.csv"),  
    colTypes = mapOf("week" to ColType.Int, "demand" to  
    ↪ ColType.Int)  
)  
val history: IntArray = df.getColumn("demand").toList().map  
    ↪ { it as Int }.toIntArray()
```

Notice that the column types are named rather than left to be inferred. That is the practice Section 2.4.4 of (Rossetti 2026) follows, and the reason is that a column of counts silently read as text, or as real numbers, fails later and somewhere else.

7.3.1. Look Before You Model

Appendix B of (Rossetti 2026) opens its fitting procedure by plotting the data, and it is worth saying why rather than treating it as a ritual. A summary statistic answers a question you already thought to ask. A picture shows you the thing you did not.

Three plots answer three different questions, and the KSL draws all three.

```
val h = Histogram(Histogram.createBreakPoints(0.0, 12, 10.0),  
    ↪ "Weekly demand")  
h.collect(history.toDouble())  
h.histogramPlot().showInBrowser()  
  
ObservationsPlot(history.toDouble()).showInBrowser()  
ACFPlot(history.toDouble()).showInBrowser()
```

What shape is it? Figure 7.1 is the histogram, on bins of ten belts. The break points are given rather than recommended, because ten belts to a

7. Single-Period Stochastic Inventory

bin is what the Data sheet of `Chapter7Models.xlsx` uses and a figure that disagrees with the spreadsheet beside it is worse than no figure.

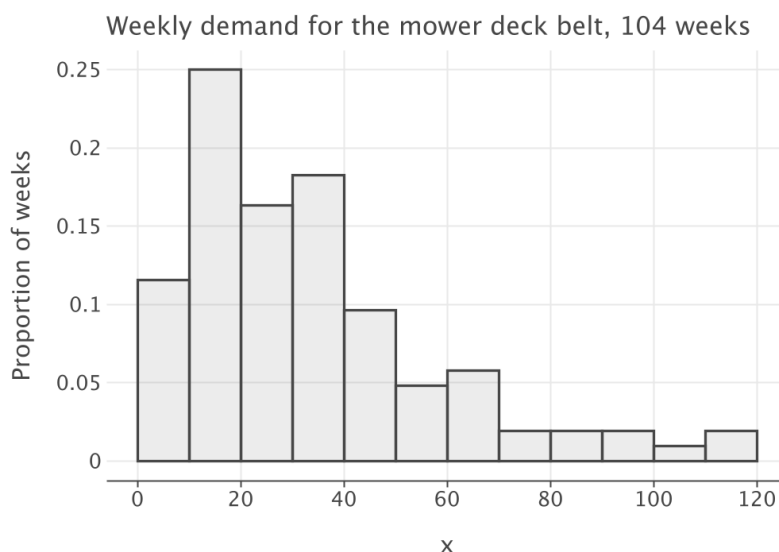


Figure 7.1.: Weekly demand for the belt over 104 weeks, in bins of ten. Each bin is closed below and open above, so the tallest bar counts the 26 weeks that sold 10 through 19 belts. Notice the shape: a peak well to the left, a long thin tail reaching past 110, and no suggestion of symmetry anywhere in it. That single picture has already ruled out the normal distribution.

Is it the same process throughout? Figure 7.2 plots the weeks in the order they occurred. What you are looking for is a trend, a level shift, or a season, and what is there is none of those: the series wanders about a constant level for two years. That matters because the season really is over, which is what Example 7.1 claimed and what this plot lets a reader check rather than accept.

Are the weeks independent? Figure 7.3 is the autocorrelation plot.

7. Single-Period Stochastic Inventory

Every lag falls inside the band, including the first, whose correlation is 0.1423 against a band of about $\pm 1.96/\sqrt{104} = \pm 0.19$. Recall from Section B.5.5 that autocorrelated demand breaks the variance of a sum. Section 7.7 is about to add fourteen of these weeks together and treat their variances as adding, so this is the plot that licenses the arithmetic two sections from now.

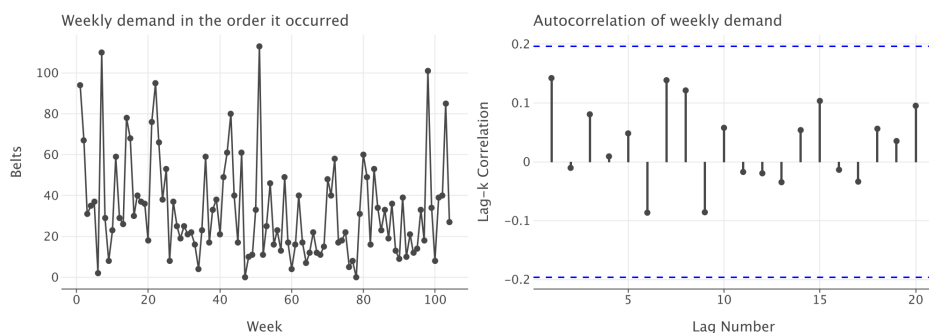


Figure 7.2.: The weeks in the order they occurred. Figure 7.3.: Autocorrelation through twenty lags.

Taken together the three plots say the weeks are independent, identically distributed, and strongly right-skewed. That is the finding that lets the rest of the section treat the 104 numbers as 104 draws from one distribution.

7.3.2. Summarize It

Now the statistics, which the KSL accumulates as the data are collected.

```
val s = Statistic("Weekly demand", history.toDoubles())
val b = BoxPlotSummary(history.toDoubles(), "Weekly demand")
println(s)
println(b)
```

7. Single-Period Stochastic Inventory

Table 7.4.: Summary of the 104 weeks. **Statistic** carries the first four moments and the lag one correlation; **BoxPlotSummary** carries the quartiles.

Statistic	Value
Weeks observed	104
Mean	33.2404
Variance	613.0387
Standard deviation	24.7596
Coefficient of variation	0.7449
Variance to mean ratio	18.4426
Skewness	1.2630
Smallest week	0
First quartile	16.0
Median	26.5
Third quartile	40.0
Largest week	113
Weeks with no demand	2

Read three things off Table 7.4 before going further.

The median sits well below the mean. A median of 26.5 against a mean of 33.24 says the distribution is right-skewed: most weeks are quiet and a few are very large. The skewness of 1.2630 is the same statement with a number attached, and Figure 7.1 is the same statement with a picture attached. Three routes, one finding.

Demand is a count. The weeks are whole belts, two of them are zero, and the quantity has no meaning between whole numbers. Thus, the families to consider are the discrete ones of Section C.1 and not the continuous ones of Section C.2.

For a count the diagnostic is the variance to mean ratio. Equation C.1 rather than the coefficient of variation, because the ratio is what

7. Single-Period Stochastic Inventory

each discrete family fixes or bounds. Here

$$VMR = \frac{613.0387}{33.2404} = 18.44$$

7.3.3. Hypothesize, Estimate, Test

Step 1, hypothesize. Equation C.1 does most of this step by itself. Recall from Section C.1.1 that the Poisson fixes $VMR = 1$, and from Section C.1.3 that the binomial forces $VMR < 1$. A ratio of 18.44 is near neither, and no choice of parameter repairs that, because in both families the variance is decided the moment the mean is estimated. What remains is Section C.1.2, the negative binomial, which admits any $VMR > 1$ and which is the Poisson with its constant-rate assumption relaxed. That is a plausible story for a spare part: the rate at which mower decks need belts is not the same in every week.

Step 2, estimate. The KSL estimates the parameters of every discrete family it knows in one call. `PMFModeler` is the discrete counterpart of the `PDFModeler` of Section 2.4.2 of (Rossetti 2026), and it is deliberately narrower: there are fewer discrete families, so it estimates parameters and leaves the choosing to you.

```
val modeler = PMFModeler(history)
for (result in
  ↪ modeler.estimateParameters(modeler.defaultEstimators)) {
  println(if (result.success) "${result.parameters}" else
  ↪ "FAILED: ${result.message}")
}
```

RV Type = NegativeBinomial

Double Parameters {probOfSuccess=0.05422232292974839, numSuccesses=1.905702484443969

RV Type = Poisson

7. Single-Period Stochastic Inventory

```
Double Parameters {mean=33.24038461538462}
FAILED: Cannot match moments when sample average <= estimated variance
RV Type = Binomial
Double Parameters {probOfSuccess=0.29133420825260514, numTrials=114.09708737864078}
```

Read that output carefully, because the most useful line in it is the failure. The binomial estimator that matches moments **refuses to return an answer**, and the reason it gives is Equation C.1 in words: a binomial cannot have a variance above its mean. That is the software performing Step 1 rather than the analyst. The binomial that did return an answer did so by matching the largest observation instead of the moments, and it reports 114.1 trials, a fractional number of trials for a quantity that has no trials in it. Notice that an estimate being returned is not the same thing as a family being appropriate.

The negative binomial estimate is the method of moments of Section C.4, and it is worth confirming by hand. Matching Equation C.3 to the sample mean $\bar{x}$ and sample variance s^2 gives

$$\hat{p} = \frac{\bar{x}}{s^2} = \frac{33.2404}{613.0387} = 0.054222, \quad \hat{r} = \frac{\bar{x}^2}{s^2 - \bar{x}} = \frac{33.2404^2}{613.0387 - 33.2404} = 1.9057 \quad (7.3)$$

which is what the KSL reported. Notice that $1/\hat{p} = 18.44$ is the variance to mean ratio we started with, so the fitted $\hat{p}$ carries no information the summary did not already have. That is true of the method of moments generally and is worth knowing: the fit is a restatement of two summary numbers in the parameters of a family.

The KSL uses the success convention of Equation C.3, so **probOfSuccess** is p and the mean is $r(1-p)/p$. That agrees with the check above. The warning in Section C.1.2 is not hypothetical; you should confirm the convention every time before passing a parameter to a routine.

7. Single-Period Stochastic Inventory

Step 3, test. For a discrete model the KSL performs a chi-squared goodness of fit test, forming bins of approximately equal probability.

```
val fitted = NegativeBinomial(0.054222, 1.9057)
val gof = DiscretePMFGoodnessOfFit(
    history.toDoubles(), fitted, numEstimatedParameters = 2,
    breakPoints =
    ↪ PMFModeler.makeZeroToInfinityBreakPoints(history.size,
    ↪ fitted)
)
println(gof.chiSquaredTestResults())
```

Table 7.5.: The chi-squared goodness of fit test on the 104 weeks, for the two families Equation C.1 left in contention. Notice that the Poisson is not rejected narrowly. Its test statistic is more than twenty times the negative binomial's.

Model	χ^2	Degrees of freedom	p -value	At the 0.05 level
Negative binomial, $\hat{p} = 0.0542$, $\hat{r} = 1.9057$	23.35	17	0.1382	do not reject
Poisson, $\hat{\lambda} = 33.2404$	571.98	16	0.0000	reject

Table 7.5 is the confirmation, not the decision. The decision was made by the variance to mean ratio, and a test that had contradicted it would have been telling us that something is wrong with the data rather than with Equation C.1.

7. Single-Period Stochastic Inventory

Figure 7.4 is the same conclusion drawn rather than tabulated. The KSL calls this a PMF comparison plot and Section 2.4.4 of (Rossetti 2026) uses one at the same point in the procedure.

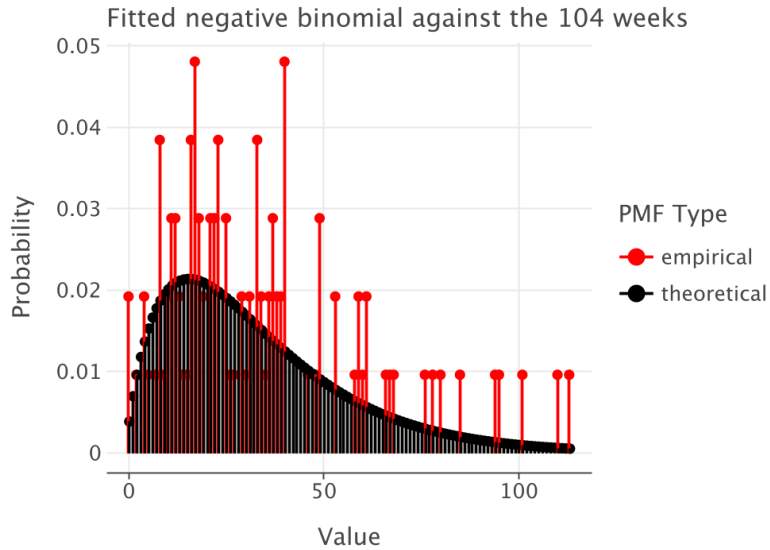


Figure 7.4.: The fitted negative binomial against the 104 weeks. The empirical mass function is spiky because it has to be: 104 observations spread over 54 distinct values put a mass of $1/104$ or $2/104$ on most of them. That is exactly why Table 7.5 bins before it tests. What to read here is whether the black curve runs through the middle of the red forest, which it does.

7.3.4. What the Test Does Not Settle

A goodness of fit test asks whether the data are *inconsistent* with a family. Failing to reject is not proof. With 104 observations spread over twenty bins, several of the bins carry an expected count below five, which is the condition under which the chi-squared approximation is least trustworthy,

7. Single-Period Stochastic Inventory

and the KSL flags each of them. The test and the ratio narrow the field, and the choice among what survives is yours to defend. Where two survivors disagree enough to change the decision, Section 7.7 shows what to do: compute the answer under both and report the spread.

What this section did not do. It did not deseasonalize, because Figure 7.2 shows nothing to remove and the season is over. It did not forecast, which is a different job from fitting a distribution and is outside this book. And it did not correct for censoring, because the distributor does not record lost sales, which is exactly the limitation Section 7.1.1 warned about and which no amount of care here repairs.

It also did not have to deal with an item whose history is mostly zeros. The belt sold in 102 of its 104 weeks. Section 7.4 takes up the items that do not.

7.4. Intermittent and Slow-Moving Demand

The belt is demanded in almost every week: 2 of its 104 weeks were empty. Many items are not like that, and a history in which most periods are empty needs a different treatment.

Recall from Example 2.4 that a demand history is classified by two statistics: the average interval between demands, ADI , and the squared coefficient of variation of the non-zero sizes, CV^2 . An item with $ADI > 1.32$ is **intermittent**, and one that is also variable in size, $CV^2 > 0.49$, is **lumpy**. Those two cut-offs are not conventions; Syntetos et al. (2005) derive them as the points at which one forecasting method overtakes another, which is why they are the oddly specific numbers they are. For the belt, $ADI = 104/102 = 1.02$, comfortably under the threshold, which is the check that licenses everything Section 7.3 did.

Two things go wrong when an intermittent series is handed to Section 7.3 unchanged.

7. Single-Period Stochastic Inventory

A single family has to explain two different things. The distribution of period demand for an intermittent item is a mixture of a point mass at zero and a distribution of sizes. Fitting one continuous family to it forces that family to account for both how often demand happens and how large it is, and no family in Section C.2 can do that well. Section C.4.3 gives the two repairs, a zero-modified family or a compound representation.

The coefficient of variation goes past where the normal is usable. Table C.1 gives a normal model at a coefficient of variation of 1.0 a sixth of its probability below zero. Intermittent items routinely run past 1.0, so the family that is most often reached for is the one most surely wrong. That is not a small error in a tail; it is a model asserting that demand is frequently negative.

Neither repair is needed for this chapter, because Section 7.8 handles the slow-moving case directly with a discrete distribution and no mixture. Both matter from Chapter 8 onward, where the quantity being modeled is demand over a lead time and the zeros no longer aggregate away.

One thing this book does **not** cover is worth naming so you know it exists, and worth naming precisely enough that you can go and read it.

Forecasting an intermittent series, as opposed to fitting a distribution to it, begins with Croston (1972). Croston's insight was that exponentially smoothing an intermittent series directly is wrong, because the smoothing is dragged toward zero by every empty period and the estimate is then lowest just after a demand occurs, which is precisely when the next one is least likely. His method smooths two series separately instead, the size of a demand when one happens and the interval between demands, and combines them into a demand rate.

Syntetos and Boylan (2005) then showed that the obvious way of combining them is biased. The ratio of two independent smoothed estimates does not estimate the ratio of their means, so Croston's estimator runs high, and they gave the correction that now carries their names. The same authors'

7. Single-Period Stochastic Inventory

categorization scheme (Syntetos et al. 2005) is the source of the *ADI* and CV^2 cut-offs above, and it exists to say which of these estimators to use.

Fitting asks what the distribution of demand is. Forecasting asks what next period's demand will be. They are different questions, both are needed in practice, and this chapter asks only the first.

7.5. The Newsvendor Model

We can now state the problem formally. It carries the name of a vendor buying newspapers for the day, and it applies to any resource committed before the need for it is known.

- Let X represent demand over the period, a random variable with cumulative distribution function $F(x) = P\{X \leq x\}$.
- Let Q represent the quantity ordered, the decision.
- Let c_o represent the **overage cost**, the loss on each unit left at the end of the period, in dollars per unit.
- Let c_u represent the **underage cost**, the loss on each unit of demand that could not be met, in dollars per unit.

Both are opportunity costs measured against the decision that could have been made, and both are strictly positive. Section 7.9 says what it means when one of them is not.

At the end of the period exactly one of two things has happened. Either $Q > X$ and $Q - X$ units are left over, or $Q < X$ and $X - Q$ units of demand went unmet. Using the notation $(y)^+ = \max(y, 0)$ of Section C.3, both cases are covered by one expression:

$$C(Q, X) = c_o(Q - X)^+ + c_u(X - Q)^+ \quad (7.4)$$

7. Single-Period Stochastic Inventory

Notice that exactly one of the two terms is non-zero for any realized demand. Equation 7.4 is a random variable because X is, and the decision is made to minimize its expectation, $E[C(Q, X)]$.

7.5.1. The Same Model as a Profit

Much of the literature and all of the practice writes this as a profit rather than a cost, in terms of quantities that appear on an invoice.

- Let c represent the purchase cost, s the selling price, and u the salvage value per unit, with $s > c > u$.

The vendor sells $\min(X, Q)$ units, salvages $(Q - X)^+$, and pays for Q :

$$G(Q, X) = s \min(X, Q) + u(Q - X)^+ - cQ \quad (7.5)$$

The two formulations are the same model. Matching terms,

$$c_o = c - u, \quad c_u = s - c \quad (7.6)$$

so the overage cost is what a unit loses between the counter and the scrap yard, and the underage cost is the margin forgone. For the belt, $c_o = 50 - 12 = \$38$ and $c_u = 95 - 50 = \$45$, which are the numbers Section 7.2 used.

Maximizing Equation 7.5 and minimizing Equation 7.4 give the same Q , and the reason is worth stating because it is the same argument as Section 1.4.1. The two expressions differ by $(s - c)E[X]$, which is the profit the vendor would make if every unit demanded were sold. That quantity does not depend on Q . It shifts the objective without moving its optimum, and

$$E[G(Q, X)] = (s - c)E[X] - E[C(Q, X)] \quad (7.7)$$

7. Single-Period Stochastic Inventory

Use whichever formulation the audience speaks. This chapter derives in costs, because Equation 7.4 is shorter, and reports in profits, because that is what the distributor will ask for.

7.6. The Critical Ratio

The expected cost has a closed form, and it is written in the first order loss function of Section C.3. Recall that $G^1(b) = E[(X - b)^+]$ is the expected amount by which demand exceeds b .

Take the expectation of Equation 7.4 term by term. The second term is $G^1(Q)$ by definition, so the expected shortage is

$$E[(X - Q)^+] = G^1(Q) \quad (7.8)$$

The first term needs one step. Notice the identity $(Q - X)^+ = (X - Q)^+ + Q - X$, which holds in both cases: when $X < Q$ the right side is $0 + Q - X$, and when $X \geq Q$ it is $X - Q + Q - X = 0$. Taking expectations,

$$E[(Q - X)^+] = G^1(Q) + Q - E[X] \quad (7.9)$$

which says the expected leftover is the expected shortage plus however far the order sits above the mean. Putting the two together,

$$E[C(Q)] = c_o (G^1(Q) + Q - E[X]) + c_u G^1(Q) = (c_u + c_o)G^1(Q) + c_o (Q - E[X]) \quad (7.10)$$

Equation 7.10 is the whole model in one line. The first term is the cost of being short, weighted by both prices because a unit short is both a sale lost and a unit not held. The second is the cost of the position itself.

7. Single-Period Stochastic Inventory

Differentiating. Section C.3 gives $dG^1(Q)/dQ = -G^0(Q)$, where $G^0(Q) = 1 - F(Q)$. Differentiating Equation 7.10 term by term,

$$\frac{d}{dQ}E[C(Q)] = -(c_u + c_o)G^0(Q) + c_o \quad (7.11)$$

Setting Equation 7.11 to zero gives $G^0(Q) = c_o/(c_u + c_o)$, and since $G^0 = 1 - F$,

$$F(Q^*) = \frac{c_u}{c_u + c_o}, \quad Q^* = F^{-1}\left(\frac{c_u}{c_u + c_o}\right) \quad (7.12)$$

The quantity $c_u/(c_u + c_o)$ is the **critical ratio**. It is the rule Equation 7.2 produced by hand, and Equation 7.12 is why that rule was right. Notice that the optimum is a minimum: the second derivative is $(c_u + c_o)f(Q)$, which is non-negative everywhere, so the expected cost is convex and the stationary point is the bottom.

7.6.1. Reading the Critical Ratio

Equation 7.12 says something sharper than “solve for Q ”. The left side is $P\{X \leq Q^*\}$, the probability that demand falls short of the order. That is the probability of **having too much**. Thus, the critical ratio is not an input to the answer; it *is* the answer, expressed as a service level the costs imply.

The ratio depends only on the relative size of the two costs, so it can be read as odds. For example, if being short costs three times as much as being long, then $c_u/(c_u + c_o) = 3/4$ and it is optimal to run a 75% chance of leftovers. At four times it is 80%, and at nine times it is 90%.

Two consequences follow, and both contradict what an organization usually rewards.

7. Single-Period Stochastic Inventory

Never having leftovers is bad. A buyer who ends every period with nothing left is not being careful. They are ordering at a critical ratio near zero, which is only correct when a lost sale costs nothing. For the belt the ratio is 0.5422, so the distributor should expect leftovers in about 54% of futures. A manager who treats scrapped belts as evidence of a mistake will push the buyer below the optimum, and the loss will never appear in any account because it is a sale that did not happen.

Never having unsatisfied customers is also bad. The mirror error costs just as much. At the belt's ratio the distributor should expect to run out about 46% of the time. A service target of 99% is a claim that c_u is ninety-nine times c_o , and if that is not true the stock is too large.

Thus, the useful question to ask of a stated service level is not whether it is high enough. It is what ratio of costs would make it optimal, and whether anyone believes those costs. Section 4.4.3 made the same move for the economic order quantity, reading a firm's behavior back into the parameters it implies.

7.7. Continuous Demand

With a continuous demand distribution, Equation 7.12 is a quantile and the work is arithmetic.

Example 7.2 (The final buy, solved). Take Example 7.1 with the demand distribution of Section 7.3.

Notation. Let X represent total demand over the fourteen remaining weeks. Let X_i represent demand in week i , negative binomial with mean 33.2404 and variance 613.0387 from the fit of Equation 7.3.

The distribution of the total. Weekly demands are independent, so by Section B.2 the mean and variance of a sum add:

7. Single-Period Stochastic Inventory

$$E[X] = 14(33.2404) = 465.37 \text{ belts}, \quad \text{Var}[X] = 14(613.0387) = 8582.54$$

so $\sigma = \sqrt{8582.54} = 92.64$ belts and the coefficient of variation is $92.64/465.37 = 0.1991$.

Which family. Section 7.3 fitted a **discrete** model to weekly demand, and demand over fourteen weeks is discrete too. Treating the total as continuous is an approximation, and it is a safe one here for the reason Section 7.8 gives at the end: one belt is $1/92.64$, or 1.1%, of the standard deviation of the total, so rounding cannot decide the answer. The weekly data, where one belt is 4% of the standard deviation, would not have earned the same treatment.

The family to approximate with is the normal. Section 7.3 rejected the normal for **weekly** demand, whose coefficient of variation is 0.7449. The total over fourteen weeks has a coefficient of variation of 0.1991, and Table C.1 says a normal at that coefficient puts essentially nothing below zero. The normal is safe here and was not safe there, and the reason is Section B.2: a sum of many independent terms tends toward the normal whatever the terms look like.

The critical ratio. From Equation 7.6, $c_o = \$38$ and $c_u = \$45$, so

$$\frac{c_u}{c_u + c_o} = \frac{45}{83} = 0.542169$$

The quantity. Standardizing, $Q^* = \mu + z\sigma$ where $z = \Phi^{-1}(0.542169) = 0.1059$. That is,

$$Q^* = 465.37 + 0.1059(92.64) = 475.18 \text{ belts}$$

so the distributor orders **475 belts**.

7. Single-Period Stochastic Inventory

The consequences. From Table C.2 the standard normal loss function at $z = 0.1059$ is $\Phi^1(0.1059) = 0.3482$, so by Equation C.20 the expected shortage is $92.64(0.3482) = 32.26$ belts. From Equation 7.9 the expected leftover is $32.26 + 475.18 - 465.37 = 42.07$ belts. The distributor expects to scrap about 42 belts and to turn away about 32 dealers.

The cost. From Equation 7.10,

$$E[C(Q^*)] = 38(42.07) + 45(32.26) = 1598.71 + 1451.72 = \$3,050.43$$

The check. The other form of Equation 7.10 must agree: $(45 + 38)(32.26) + 38(475.18 - 465.37) = 2677.62 + 372.81 = \$3,050.43$. The two agree, which confirms both the loss function and the quantity computed from it. That check is worth performing every time, because it catches a missing σ in the loss function immediately.

In business terms. By Equation 7.7 the expected profit is $(95 - 50)(465.37) - 3050.43 = \$17,891.01$. The distributor commits \$23,750 to buy 475 belts and expects to clear about \$17,900 on them.

Three things about Example 7.2 are worth more than the answer.

The optimum is flat. Ordering the mean rounded to whole belts, 465, costs \$3,068.88 against the optimum's \$3,050.43, which is 0.6% worse. The economic order quantity of Section 3.7 is flat in the same way and for the same reason: the objective is smooth and its derivative vanishes at the optimum. Thus, the value of this calculation is not the last belt. It is knowing that 475 is right and 700 is not, and 700 costs \$8,930, nearly three times the optimum.

The family matters less than the fit, and here that can be measured. Recall from Equation C.3 that a sum of independent negative binomials sharing p is negative binomial with the same p and the numbers of successes added. Thus, the fitted weekly model of Equation 7.3 gives the

7. Single-Period Stochastic Inventory

fourteen-week distribution **exactly**, as $NB(\hat{p} = 0.054222, 14\hat{r} = 26.6798)$, with the same mean and variance used above. Apply Equation 7.13 to it and the answer is 469 belts, because $F(468) = 0.5392$ is short of the ratio and $F(469) = 0.5434$ clears it. Now price the four candidate orders against that exact model.

Table 7.6.: What the choice of family costs, measured against the distribution the data were fitted to rather than against the other candidates. The gamma happens to land on the exact answer.

Model used to choose Q	Normal	Lognormal	Gamma	Exact negative binomial
Q , rounded to whole belts	475	466	469	469
Expected cost under the exact model	\$3,061.28	\$3,057.19	\$3,055.38	\$3,055.38
Penalty against the exact answer	0.19%	0.06%	0.00%	0.00%

Read Table 7.6 as a bound rather than as a ranking. The worst of the three continuous families costs 0.19% more than choosing correctly, on a decision worth about \$3,050. That spread is small because the coefficient of variation of the total is 0.1991. It would not be small at the weekly coefficient of 0.7449, which is another reason Section 7.3 is worth doing carefully.

7. Single-Period Stochastic Inventory

The normal's defect is real and did not bite. A normal with this mean and standard deviation puts $\Phi(-465.37/92.64) = \Phi(-5.02)$ below zero, which is about one in four million. Notice that this is a statement about *this* aggregation and not about the normal in general. The same family applied to the weekly data would have put about 9% of its mass below zero.

7.8. Discrete Demand

When demand is counted in small whole numbers, a continuous distribution is the wrong tool and the inverse in Equation 7.12 does not exist. A discrete cumulative distribution function is a staircase, and a critical ratio almost never lands exactly on a step.

The rule that replaces it follows from the same marginal argument as Equation 7.2. Keep adding a unit while the expected gain exceeds the expected loss, which means while $F(Q) < c_u/(c_u + c_o)$. That is,

$$Q^* = \min \left\{ Q : F(Q) \geq \frac{c_u}{c_u + c_o} \right\} \quad (7.13)$$

Equation 7.13 is not an inverse. It is the smallest quantity whose cumulative probability *reaches* the ratio, and at that quantity $F(Q^*)$ is usually strictly greater than the ratio. That gap is not an error. It is the staircase, and it means a discrete newsvendor generally achieves a service level a little better than the one its costs call for, because it cannot buy a fraction of a unit.

7. Single-Period Stochastic Inventory

⚠ Evaluate a discrete loss function only at whole units

The first order loss function of a discrete distribution is defined for every real argument, but the KSL's implementations of `firstOrderLossFunction` and `secondOrderLossFunction` are correct only when the argument is a whole number. At a fractional argument `Poisson` and `NegativeBinomial` return a value that is usually negative, `Binomial` returns the value at the whole number below, and `Geometric` returns an approximation. Version R1.6.2 is affected and none of these raises an error.

That restriction costs nothing here, because Equation 7.13 only ever asks about whole units. It matters when a quantity that came from a continuous model is priced against a discrete one, which is exactly what Table 7.6 does: round first, then evaluate.

Example 7.3 (A one-time buy of spare transformers). Recall the municipal utility storeroom of Example 4.5, which carries a pad-mount transformer at \$3,800. The utility is about to begin a capital project after which the specification changes, so transformers not used will be obsolete.

What is the demand process? Failures arrive independently at a low rate. Over the project window the utility expects 6 failures, and models the count as Poisson with $\lambda = 6$.

What costs are incurred? A transformer bought and not used scraps at \$900, so $c_o = 3800 - 900 = \$2,900$. A failure with no spare on the shelf is covered by an emergency purchase at \$9,500, so $c_u = 9500 - 3800 = \$5,700$.

The critical ratio. $5700/(5700 + 2900) = 5700/8600 = 0.662791$.

The quantity. Walk the Poisson distribution function until it reaches the ratio.

7. Single-Period Stochastic Inventory

Table 7.7.: The Poisson newsvendor, walked one transformer at a time. Notice that the cost column confirms the rule rather than being used by it: the rule reads the $F(Q)$ row and stops at 7 because 0.6063 is short of 0.6628 and 0.7440 clears it.

Q	4	5	6	7	8
$F(Q)$	0.2851	0.4457	0.6063	0.7440	0.8472
Ex-pected short-age $G^1(Q)$	2.2330	1.5181	0.9637	0.5700	0.3140
Ex-pected cost	\$13,404	\$10,155	\$8,288	\$7,802	\$8,501

The answer. **Buy 7 spare transformers**, at a cost of \$26,600, with an expected total of \$7,802 in obsolescence and emergency purchases.

The check. Table 7.7 costs each candidate directly from Equation 7.10, using the Poisson loss function of Table C.3. The minimum is at 7, where the rule put it.

Does rounding the continuous answer work? Usually, and not always. Treating demand as normal with mean 6 and variance 6 gives $6 + \Phi^{-1}(0.6628)\sqrt{6} = 6 + 0.4201(2.4495) = 7.029$, which rounds to 7 and agrees.

Now change one number. Suppose the emergency purchase costs \$8,150 rather than \$9,500, so that $c_u = \$4,350$ and the critical ratio is exactly 0.6000. The exact rule reads $F(5) = 0.4457$ and $F(6) = 0.6063$, and stops at **6 transformers**. The normal approximation gives $6 + 0.2533(2.4495) = 6.621$, which rounds to **7**. The approximation buys one transformer too many, at \$3,800, which is a real error on a real decision.

7. Single-Period Stochastic Inventory

The lesson is about when the approximation is safe rather than whether it is. A normal approximation to a discrete distribution is good when the mean is large enough that one unit is a small fraction of the standard deviation. At $\lambda = 6$ the standard deviation is 2.45 units, so a single transformer is 40% of it, and rounding decides the answer. At $\lambda = 600$ a single unit is 4% of the standard deviation and rounding decides nothing. Thus, use Equation 7.13 whenever the numbers are small, which is exactly when slow-moving items are involved and exactly when each unit is expensive.

7.9. Extensions

Four variations cover most of what a real single-period problem adds, and none of them changes Equation 7.12. They change what goes into it.

7.9.1. A Penalty Beyond the Lost Margin

Equation 7.6 sets $c_u = s - c$, the margin forgone. That is the whole cost only when a customer turned away this time returns next time as though nothing had happened. Where a stockout costs more than the sale, add it.

- Let π represent the stockout penalty of Section 1.4.6, in dollars per unit of unmet demand, so that $c_u = s - c + \pi$.

For the belt, suppose the distributor judges that a dealer refused a belt costs \$40 in future business beyond the margin lost today. Then $c_u = 45 + 40 = 85$ and the critical ratio rises from 0.5422 to $85/123 = 0.6911$. The quantity rises with it, from 475 belts to $465.37 + 0.4989(92.64) = 511.58$, so **512 belts**.

Notice the direction and the size. A penalty of \$40, which is under half the selling price, moved the order by 37 belts, which is 7.8%. Recall from Section 1.5.5 that π is the parameter no ledger contains and the one hardest

7. Single-Period Stochastic Inventory

to defend. The honest way to report this decision is not a single number. It is that 475 belts is right if a stockout costs only the margin and 512 if it costs \$40 more, and that the organization should say which it believes.

7.9.2. Stock Already on Hand

Suppose the distributor already holds I belts when the final order is placed. The decision is now how many to *add*, but the model is unchanged in a way worth seeing.

What the model determines is the quantity that should be **available** for the period, which is the stock on hand plus the order. Call that the order-up-to level S^* , and notice that Equation 7.12 computes it exactly as before, because the cost of ending the period with $S - X$ units left does not care how they got there. That is,

$$S^* = F^{-1}\left(\frac{c_u}{c_u + c_o}\right), \quad Q^* = (S^* - I)^+ \quad (7.14)$$

For the belt, $S^* = 475.18$, so with 120 belts already on hand the order is $475.18 - 120 = 355.18$, or **355 belts**. With 600 already on hand the order is zero, and the distributor is over-stocked before it begins.

Equation 7.14 is the first appearance of a structure the rest of the book uses constantly. Chapter 9 builds its whole policy on an order-up-to level, and the (s, S) policy of Chapter 8 is the same idea with a trigger attached. Notice that the truncation at zero matters: a newsvendor cannot sell stock back, so an excess position is simply carried.

7.9.3. When the Model Refuses

Equation 7.12 requires $c_o > 0$ and $c_u > 0$, and the two ways that fails are both informative.

7. Single-Period Stochastic Inventory

If $c_u \leq 0$ there is no penalty for being short, so the critical ratio is zero or negative and the answer is to order nothing. That is correct, and it usually means the problem was stated wrongly: an item nobody loses anything by running out of is an item nobody should stock.

If $c_o \leq 0$, which happens when the salvage value equals or exceeds the purchase cost, the critical ratio is one or more and the answer is to order without limit. That is also correct and also a sign of a mis-stated problem. A unit that can always be disposed of for at least what it cost carries no risk, so nothing stops you buying it. An unbounded answer is the model reporting that you have described a situation with no downside, and the right response is to find the cost you left out.

7.9.4. Is It Really a Single Period?

The strongest assumption in this chapter is the one in the title. Leftover stock must have no value after the period. That is exactly true for a perishable item, a newspaper or a flu vaccine, and exactly true for the final buy of Example 7.1 because the deck is withdrawn.

It is false for most inventory. A belt left over in an ordinary week is a belt available next week, worth its purchase cost less one week of holding, and that is a multi-period problem. Applying the newsvendor model there charges the whole value of the unit against a decision that costs only a week of carrying, so the order comes out far too small.

Thus, the test to apply before using this chapter is a single question. **What is a leftover unit worth at the end of the period?** If the answer is its scrap value, this is a newsvendor problem. If the answer is nearly what you paid, it is not, and Chapter 8 is where to go.

7.10. Estimating the Answer by Simulation

Everything so far has been a formula. This section computes the same answer by sampling, and it is the first simulation in this book.

The order matters. Equation 7.12 already gives the exact answer for Example 7.2, so simulating it proves nothing about the belt. What it proves is that the simulation is right, and that is worth having before the simulation is pointed at a problem where no formula exists.

The model is Equation 7.5 evaluated on sampled demand. Notice that the sampling is done from the **fitted** model of Equation 7.3 and not from the normal approximation to it. The simulation can sample counts as easily as it can sample real numbers, so there is no reason to hand it the approximation, and sampling counts removes the truncation at zero a normal would have needed.

```
val demand = NegativeBinomialRV(probOfSuccess = 0.054222,
  ↪ numSuccess = 26.6798, streamNum = 1)
val stat = Statistic("profit at Q")

demand.resetStartStream()
repeat(100_000) {
  val d = demand.value
  stat.collect(s * minOf(d, q) + u * maxOf(0.0, q - d) - c
  ↪ * q)
}
println("%.2f +/- %.2f".format(stat.average, stat.halfWidth))
```

Notice `resetStartStream`. Running it before each candidate makes every quantity face the **same** hundred thousand demands, which is what lets the rows of Table 7.8 be compared with one another rather than only with the exact answer. It also makes each row reproducible on its own, which a shared advancing stream would not.

7. Single-Period Stochastic Inventory

Table 7.8.: Expected profit estimated by simulation, 100,000 replications at each quantity, every quantity facing the same demands. The closed form is Equation 7.10 evaluated on the exact negative binomial of Table 7.6.

Q	440	460	469	490	510
Esti- mated profit	\$17,737.18	\$17,877.57	\$17,893.03	\$17,818.04	\$17,610.98
Half- width	\$21.02	\$24.68	\$26.31	\$29.95	\$33.15
Closed form	\$17,732.96	\$17,871.00	\$17,886.06	\$17,811.30	\$17,606.01

Read Table 7.8 one column at a time. The estimate at $Q^* = 469$ is \$17,893.03 with a half-width of \$26.31, so the interval is [\$17,866.72, \$17,919.34], and the exact \$17,886.06 lies inside it. So does the exact figure in every other column. Five agreements are a great deal more convincing than one, and they cost nothing extra to report. That agreement is the validation, and it is the reason to do the easy case first. Notice also that the simulation puts its peak at the same candidate the formula does, which the common demands make a fair comparison rather than a lucky one.

Notice what the simulation has that the formula does not. Every figure in Table 7.8 carries a half-width. A simulated answer is an estimate with a standard error, and reporting one without its interval is reporting a number that is not there. By Section B.1 the half-width shrinks with the square root of the replication count, so ten times the precision costs a hundred times the run.

And notice what it costs. The exact answer took one walk up a distribution function. The estimate took half a million samples and is still

7. Single-Period Stochastic Inventory

uncertain in the third significant figure. Simulation is not how you solve the newsvendor problem.

7.10.1. When It Is Worth It

Simulation earns its place the moment any assumption behind Equation 7.12 fails, and four failures are common.

A demand distribution with no convenient inverse. A mixture, from Section C.3.6, has no closed-form quantile. The formula needs a search; the simulation does not care.

A salvage rule that is not linear. For example, suppose the scrap dealer takes the first 100 belts at \$12 and the rest at \$4. Equation 7.4 no longer has constant c_o , the derivation of Equation 7.12 breaks, and three lines change in the simulation.

A constraint. A supplier minimum, a case quantity, a budget shared with another item. Chapter 4 solved constrained problems when everything was deterministic; under uncertainty the multiplier argument is harder and a search over simulated costs is often the shortest route.

More than one item that interact. Two products competing for one display, or a substitution rule sending refused demand for one to the other. There is no newsvendor formula for that, and there is an obvious simulation.

In all four the pattern is the same. The *policy* stays simple, a single quantity chosen once. What changes is the cost function, and a cost function is easier to sample than to integrate. Chapter 11 returns to this properly, with the machinery for building and running models of systems that no formula reaches.

7.11. Building These Models in a Worksheet

The worksheets of Section 5.8 and Section 6.9 each laid out a procedure across rows. This one is different, because the decision is a single number and the work is in producing the distribution it is a quantile of.

Chapter7Models.xlsx has five working sheets, and the division between them is the argument of this chapter made visible. A sixth, **Notes**, says what the other five are for.

Data holds the 104-week history, its summary statistics, and a histogram. The statistics block computes the mean, the variance, the standard deviation, the coefficient of variation, and the median, and puts the median next to the mean because Section 7.3 reads them together. Nothing on this sheet assumes a distribution.

Fit does Steps 1 to 3 of Section 7.3. It computes Equation C.1, states what each discrete family requires of it, and answers yes or no for each, which is the step the KSL's binomial estimator performs by refusing the data. It then fits the one family left standing by Equation 7.3, and checks the fit twice: $1/\hat{p}$ must return the ratio, and the fitted moments must return the moments they were fitted from. Both checks read zero, and both catch the wrong convention for p .

Horizon is the fitted model aggregated over the fourteen weeks and walked one belt at a time. It needs no distribution function Excel does not have, because the mass function is the recursion $g(x) = g(x-1)(x+r-1)(1-p)/x$ started from $g(0) = p^r$, and everything after it is a running sum. The walk stops at 700 belts, so the sheet reports the tail it did not reach rather than leaving a reader to wonder why the last cumulative probability is not one. This sheet is what Table 7.6 measures the other three against.

Newsvendor is the decision. Inputs are s , c , u and an optional penalty π ; the derived block computes c_o , c_u and the critical ratio by Equation 7.6; and the output block computes Q^* , the expected shortage, the expected leftover, the expected cost and the expected profit. Both forms of Equation 7.10

7. Single-Period Stochastic Inventory

are computed and their difference is displayed, which is the arithmetic check of Example 7.2 wired into the sheet. The first order loss function is a formula and not a macro: for the normal it is $\sigma[\phi(z) - z\Phi^0(z)]$, which is `sigma*(NORM.S.DIST(z,FALSE) - z*(1-NORM.S.DIST(z,TRUE)))`.

Discrete walks Equation 7.13. One row per candidate quantity, columns for $g(Q)$, $F(Q)$, the loss function by the recursion of Equation C.13, and the expected cost. The chosen row is highlighted by a formula rather than by hand, so a reader who changes a cost watches the highlight move.

Three things the worksheet does deliberately. It carries no macros, so every number on it can be traced by clicking on the cell that produced it. It computes the expected cost at every candidate on the **Discrete** sheet even though Equation 7.13 does not need them, because the point of Table 7.7 is that the rule and the direct calculation agree. And it rounds every quantity on the **Fit** sheet to whole belts before pricing it, for the reason Section 7.8 gives.

What the worksheet cannot do is Section 7.10, and that is the honest boundary. A spreadsheet can sample, but a sampled spreadsheet recalculates every time anything is touched and gives a different answer each time, which is the wrong tool for a number someone will be asked to defend.

7.12. Designing the Software

Section 5.9 and Section 6.10 each had an algorithm to organize. This chapter has a formula, so the design question is different and smaller: what are the pieces, and what keeps the two ways of computing the answer honest?

7. Single-Period Stochastic Inventory

7.12.1. Finding the Nouns

A **demand model** is a distribution with a quantile and a first order loss function. Notice that this is not a new class. It is `LossFunctionDistributionIfc` from Section C.3.7, which the KSL already provides for every family this chapter uses, and the design's first decision is not to reimplement it.

A **single-period cost structure** is c_o and c_u , together with the mapping from s , c , u and π that produces them. It owns Equation 7.6, it owns the critical ratio, and it owns the refusal of Section 7.9.3: a cost structure with a non-positive c_o or c_u is rejected at construction rather than allowed to return an absurd quantity later.

A **solution** pairs a quantity with what it implies: the expected shortage, the expected leftover, the expected cost, the expected profit. It is computed for any quantity and not only the optimal one, because Table 7.7 and Table 7.8 both need a grid.

Newsvendor is the instructive rejection. It reads like the central class and the chapter is named after it. It fails because there is nothing for it to do that the cost structure and the demand model do not already do between them: the optimal quantity is one line given both, and a class whose only method is one line is a function wearing a costume.

7.12.2. The Two Routes Must Agree

The design's real content is Section 7.10. The same cost structure is evaluated two ways, exactly and by sampling, and the two must agree.

That agreement is not a convenience; it is what the design is for. A solver and an estimator that share a cost function can be checked against each other, and one that does not share it cannot. The cost of a quantity is defined once, in the cost structure, and both routes call it. The exact

7. Single-Period Stochastic Inventory

route integrates it through the loss function; the simulation route samples demand and evaluates it directly.

Notice what that buys when the exact route stops applying. Under the nonlinear salvage rule of Section 7.10.1 the loss function integration is no longer valid, but the sampled evaluation still is, and it is still the same function of a realized demand that was validated against the formula when the formula held. That is the whole argument for building it this way.

7.13. Summary

The single-period problem is not a short multi-period problem. What distinguishes it is that leftover stock has no future value, so the cost of ordering one too many is the whole loss on the unit rather than a period of carrying. That is also the test for whether this chapter applies.

The ordering cost disappears. One order is placed whatever its size, so k is constant across every alternative and cannot affect which is best. The trade-off that remains is overage against underage, and both are opportunity costs that appear in no ledger.

The answer is a quantile, and the quantile is the answer. The critical ratio $c_u/(c_u + c_o)$ is the probability of having too much that the costs imply, and $Q^* = F^{-1}$ of it. Read as odds it says that if being short costs three times as much as being long, it is optimal to run a 75% chance of leftovers. Thus, never having leftovers is not careful buying, and never disappointing a customer is not good service. Each is a claim about a ratio of costs, and the useful question about a service target is what ratio would make it optimal.

Discrete demand needs the rule and not the inverse. A staircase distribution function rarely lands on the critical ratio, so the answer is the

7. Single-Period Stochastic Inventory

smallest quantity whose cumulative probability reaches it. Rounding a continuous approximation usually gives the same answer and sometimes does not, and it fails exactly where the items are slow-moving and expensive.

The demand distribution carries more of the answer than the formula does. Characterizing a history is cheap and settles more than a goodness of fit test does. Demand arrives in whole units, and for a count the variance to mean ratio of Equation C.1 decides the family before any of them is fitted: it is one for a Poisson and below one for a binomial, so a ratio of 18 leaves only the negative binomial standing. The test then confirms what the ratio chose. And a sales history is not a demand history, so a fitted distribution built from sales is biased low in exactly the periods that mattered.

Which family you pick costs less than you would guess, and it can be measured. Fitting a count model makes the aggregate exactly computable, because negative binomials sharing p add. Table 7.6 prices the normal, lognormal and gamma answers against that exact model and none of them costs more than a fifth of a percent. Thus, the argument for care in Section 7.3 is not that the family is delicate; it is that the moments are, and every family in the table was given the same two.

Simulation belongs after the closed form, not instead of it. The newsvendor is the one model in this book whose simulation can be checked against an exact answer, which makes it the right place to build the habit. What simulation buys is the cases the formula cannot reach: a distribution with no inverse, a salvage rule that is not linear, a constraint, or two items that interact.

Table 7.9 collects the notation, which Appendix A repeats alongside the rest of the book's.

7. Single-Period Stochastic Inventory

Table 7.9.: Notation introduced in this chapter.

Symbol	Meaning
X	Demand over the single period, a random variable
$F(x)$	Cumulative distribution function of demand, $P\{X \leq x\}$
Q	Order quantity, the decision
Q^*	The optimal order quantity
S^*	Order-up-to level, when stock is already on hand
c_o	Overage cost, the loss per unit left over
c_u	Underage cost, the loss per unit of unmet demand
c, s, u	Purchase cost, selling price, salvage value per unit
π	Stockout penalty beyond the lost margin, per unit
$G^1(Q)$	First order loss function, the expected shortage at Q

Chapter 8 removes the last assumption this chapter kept. There the period does not end, stock left over is worth what it cost, and the decision is not how much to buy once but when to buy and how much each time.

7.14. Exercises

Unless an exercise says otherwise, use the conventions of Section 1.7. Report quantities to the nearest whole unit and state the unrounded value alongside. Where a demand history is called for, generate it with the code of Section 7.3 and the seed the exercise gives, so that your answer and the instructor's agree.

7.14.1. Terminology and Concepts

Exercise 7.1. Define each of the following in one sentence, and say what unit it carries: overage cost, underage cost, critical ratio, order-up-to level, censored demand.

7. Single-Period Stochastic Inventory

Exercise 7.2. True or false, with one sentence of justification for each.

- Increasing the salvage value raises the optimal order quantity.
- A buyer who never has leftover stock is managing the item well.
- The ordering cost k does not appear in the newsvendor model because it is small.
- If the critical ratio is 0.5 then the optimal quantity is the mean of demand.
- Rounding the normal-approximation answer always gives the discrete optimum.

Exercise 7.3. Table 1.10 says that k is irrelevant with one order. Explain why, and then describe a single-period situation in which it would *not* be irrelevant. State what would have to be true of the supplier for your situation to arise.

Exercise 7.4. A distribution manager states a target of 98% service for an item. Using Equation 7.12, determine what ratio of underage to overage cost would make that target optimal. The item costs \$40, sells for \$60, and salvages for \$25. What stockout penalty π would be needed to justify the target, and is it credible?

7.14.2. Working the Model by Hand

Exercise 7.5. A caterer must decide how many trays of a dish to prepare for one event. Each tray costs \$18 to make and sells for \$45. Unsold trays are donated and carry no salvage value. Demand is

Trays x	10	20	30	40	50
$P\{X = x\}$	0.10	0.20	0.35	0.25	0.10

- Compute c_o , c_u and the critical ratio.

7. Single-Period Stochastic Inventory

- b. Build the marginal analysis table of Table 7.2, one step of ten trays at a time, and report where it stops.
- c. Compute the expected profit at all five candidates directly and confirm the answer of part (b).
- d. Report the expected number of trays donated and the expected number of customers turned away at the optimum.

Exercise 7.6. An airline holds one spare engine module at a maintenance base. Failures over the coming season are Poisson with a mean of 3. A module costs \$220,000 and is worth \$60,000 if the fleet is retired before it is used. A failure with no module on hand grounds an aircraft at a cost of \$500,000.

- a. Compute c_o , c_u and the critical ratio.
- b. Using Equation 7.13, determine how many modules to hold. Show the $F(Q)$ column you walked.
- c. Compute the expected cost at $Q^* - 1$, Q^* and $Q^* + 1$ and confirm the minimum.
- d. Compute the normal-approximation answer and compare. Explain the comparison in terms of the ratio of one unit to the standard deviation.

Exercise 7.7. For the final buy of Example 7.2, compute the expected shortage and expected leftover at $Q = 450$, $Q = 475.18$ and $Q = 510$, using Table C.2 and Equation C.20. Work with the normal approximation, $E[X] = 465.37$ and $\sigma = 92.64$.

- a. Verify Equation 7.9 at each quantity, i.e. that the expected leftover equals the expected shortage plus $Q - E[X]$.
- b. Compute the expected cost at each and confirm that 475.18 is the smallest.
- c. Express the cost at 450 and at 510 as a percentage above the optimum, and compare the shape of that penalty with Equation 3.22.

7. Single-Period Stochastic Inventory

- d. The two quantities in part (c) sit 15 belts below and 35 belts above the optimum. Explain why the smaller departure is the more expensive one per belt, in terms of c_o and c_u .

7.14.3. Modeling Problems

Exercise 7.8. A second item's history is in `data/ch7-exercise-demand.csv`, 104 weeks in the same two columns as the belt's. Work Section 7.3 on it from beginning to end, and report the code alongside each answer.

- a. Draw the three plots of Section 7.3.1 and say what each one settles. Include them with your answer.
- b. Report the summary statistics of Table 7.4 for this item. Which families does Equation C.1 rule out before anything is fitted, and why can no choice of parameter rescue them?
- c. Run `PMFModeler` on the history. Report every estimate it returns, including any that fail, and say what the failures mean.
- d. Compute $\hat{p}$ and $\hat{r}$ by hand from Equation 7.3 and confirm the KSL's estimates. Then compare $1/\hat{p}$ with the ratio from part (b) and explain why they must agree.
- e. Test the fitted negative binomial with `DiscretePMFGoodnessOfFit` and report the statistic, the degrees of freedom and the p -value. Do the same for a Poisson with the same mean, and compare the two.
- f. Suppose the item is being bought for a 20-week close-out with $c = \$30$, $s = \$55$ and $u = \$8$. Compute the mean and variance of 20-week demand, the critical ratio, and Q^* under a normal approximation. Then compute Q^* exactly from the aggregated negative binomial, as Table 7.6 does, and report what the approximation costs.

Exercise 7.9. Return to Example 7.2.

- a. Plot or tabulate Q^* against the stockout penalty π for $\pi = 0, 20, 40, 60, 80, 100$ dollars per belt.

7. Single-Period Stochastic Inventory

- b. At what π does the optimal order reach 550 belts?
- c. The distributor is unwilling to name π but is confident the order should not exceed 525 belts. What upper bound on π does that imply? This is the move of Section 4.4.3, applied here.
- d. Your answer to part (a) should show the quantity rising steeply at first and then flattening. Explain that shape from the critical ratio rather than from the plot.

Exercise 7.10. A hospital pharmacy must decide a final order for a drug being withdrawn from the formulary. Demand over the remaining 90 days is normal with a mean of 400 doses and a standard deviation of 70. A dose costs \$22, is billed at \$58, and expired doses are destroyed at a disposal cost of \$3 each.

- a. Note that the salvage value is negative. Compute c_o and explain the sign.
- b. Compute S^* and the order quantity if 85 doses are already on hand.
- c. Compute the order quantity if 450 doses are already on hand, and say what the pharmacy should do about the excess.

Exercise 7.11. For each of the following, determine whether the newsvendor model applies, and if it does not, identify the cost that has been left out or mis-stated.

- a. A retailer can return all unsold units to the supplier for a full refund.
- b. A publisher prints a commemorative program that has no value after the event and no shortage cost, because attendees who cannot get one simply do not.
- c. A grocer stocks a staple with a two-year shelf life, ordering weekly.
- d. A contractor buys a specialist part for one job; leftovers can be sold to another contractor at slightly above what was paid.

7. Single-Period Stochastic Inventory

7.14.4. Using the Software

Exercise 7.12. The code of Section 7.12 is in `code/`, package `inventory.newsvendor`. Report the code you wrote alongside each answer.

- Reproduce Table 7.7, computing $F(Q)$, the first order loss function and the expected cost for $Q = 4$ through 10 from a KSL `Poisson`.
- Reproduce Table 7.6. Take the three continuous quantities with `invCDF` on a KSL `Normal`, `Lognormal` and `Gamma`, take the fourth by walking the aggregated `NegativeBinomial`, and price all four against that same aggregated model. Round every quantity to whole belts before pricing it, and say why Section 7.8 requires you to.
- Reproduce Table 7.8 and confirm that the closed form lies inside the confidence interval in every column, not only at Q^* . Then increase the replications tenfold and report how the half-width changes; state whether it changed by the factor Section 7.10 predicts.
- Modify the simulation for the nonlinear salvage rule of Section 7.10.1: the first 100 belts scrap at \$12 and the rest at \$4. Search a grid of quantities and report the best. Explain why Equation 7.12 cannot be used, naming the step of the derivation that fails.

7.14.5. Using the Workbook

Exercise 7.13. Open `Chapter7Models.xlsx` and work entirely within it.

- On the `Newsvendor` sheet, reproduce Example 7.2 and confirm that the difference between the two forms of Equation 7.10 reads zero.
- Raise the salvage value from \$12 to \$30 and report what happens to c_o , the critical ratio, Q^* and the expected cost. Explain the direction.
- Raise the salvage value to \$50 and describe what the sheet does. Relate the result to Section 7.9.3.

7. Single-Period Stochastic Inventory

- d. The **Discrete** sheet ships with Example 7.3 already entered. Confirm that the highlighted row is the one Table 7.7 gives. Then lower the emergency purchase price until the highlight moves, and report the price at which it does, to the nearest dollar.

7.14.6. From the Literature

Exercise 7.14. The newsvendor problem is old and its applications are not obvious. Read the treatment in Porteus (2008) and answer the following.

- a. The chapter analyzes airline overbooking as a newsvendor problem. Identify X , Q , c_o and c_u in that setting, and say which of them is the hardest to estimate.
- b. It also treats a flexible medical spending account, in which an employee chooses an annual contribution before knowing their expenses. Identify the four quantities again, and explain why the critical ratio is not the same thing as a service level here.
- c. Give one application of your own from an industry you know, stating the four quantities. Say what makes it a genuine single period rather than a short multi-period problem, using the test of Section 7.9.4.

8. Continuous Review Systems

i Learning objectives

After reading this chapter you should be able to:

- describe the operation of the (r, Q) and (s, S) policies
- construct the lead-time demand distribution under various demand models
- relate safety stock to the safety factor and the service measure chosen
- distinguish Type 1 and Type 2 service and compute each
- develop approximate cost models for the (r, Q) policy
- analyze the (r, Q) policy exactly under Poisson demand
- account for variability in the lead time itself
- compute policy parameters that meet a stated service target

8.1. The (r, Q) Policy

TODO: write this section.

8.2. Modeling Lead-Time Demand

TODO: write this section.

8. Continuous Review Systems

The reorder point is a statement about the lead-time demand distribution, so everything that follows depends on getting this distribution right.

8.3. Safety Stock and the Safety Factor

TODO: write this section.

8.4. Type 1 Service: Cycle Service Level

TODO: write this section.

8.5. Type 2 Service: Fill Rate

TODO: write this section.

8.6. Setting Service Targets

TODO: write this section.

8.7. An Approximate Cost Model

TODO: write this section.

8.8. Exact Analysis under Poisson Demand

TODO: write this section.

8.9. Stochastic Lead Times

TODO: write this section.

8.10. The (s, S) Policy

TODO: write this section.

8.11. Computing Policy Parameters

TODO: write this section.

8.12. Exercises

Exercise 8.1. An item has Poisson demand at 40 units per year, a lead time of 0.25 years, an ordering cost of \$50, and a holding cost of \$6 per unit per year. Using the approximate model, determine Q and r so that the fill rate is at least 98%.

Exercise 8.2. For the system in the previous exercise, compute the exact expected number of backorders under your policy and compare it with the value predicted by the approximation.

Exercise 8.3. Show that if period demands are independent and identically distributed with mean μ and variance σ^2 , and the lead time is a constant L periods, then lead-time demand has mean $L\mu$ and variance $L\sigma^2$. State clearly where independence is used.

8. Continuous Review Systems

Exercise 8.4. Extend the previous exercise: derive the mean and variance of lead-time demand when both period demand and the lead time are random and mutually independent. State the assumptions your derivation requires.

Exercise 8.5. An item currently carries safety stock sufficient for a 95% cycle service level. Determine the percentage increase in safety stock required to move to 99%, and comment on whether the service improvement justifies the cost.

9. Periodic Review Systems

i Learning objectives

After reading this chapter you should be able to:

- describe the operation of the base-stock, (R, S) , and (R, s, S) policies
- determine the order-up-to level that meets a service target
- explain the role of the review interval in the protection interval
- compare continuous and periodic review on cost and service

9.1. The Base-Stock Policy

TODO: write this section.

9.2. The (R, S) Policy

TODO: write this section.

9. Periodic Review Systems

9.3. The (R, s, S) Policy

TODO: write this section.

9.4. Choosing the Review Interval

TODO: write this section.

9.5. Comparing Continuous and Periodic Review

TODO: write this section.

9.6. Exercises

Exercise 9.1. An item is reviewed every four weeks and has a two-week lead time. Weekly demand is normal with mean 50 and standard deviation 15. Determine the order-up-to level S that achieves a 97.5% cycle service level.

Exercise 9.2. Show how the required safety stock in the previous exercise changes as the review interval is varied from one to eight weeks, and explain the shape of the resulting curve.

10. Multi-Echelon Inventory Systems

i Learning objectives

After reading this chapter you should be able to:

- explain what random demand changes about the multi-echelon problem
- analyze a serial system with base-stock policies at each stage
- analyze a one-warehouse, multiple-retailer distribution system under uncertainty
- apply the METRIC model to a repairable item system
- explain the bullwhip effect and its causes

Section 4.6 introduced echelon inventory, echelon holding cost, and the reorder intervals that minimize cost when demand is known and constant. This chapter keeps the structures and removes that assumption. Echelon accounting carries over unchanged, which is most of the reason it was worth defining; what does not carry over is the idea that a location can be described by a reorder interval alone, since with random demand it needs stock held against the variability as well.

10.1. Stochastic Demand in a Multi-Echelon System

TODO: write this section. Recalls echelon stock and echelon holding cost from Section 4.6.1 without redefining them, then states what random

10. Multi-Echelon Inventory Systems

demand changes.

10.2. Serial Systems

TODO: write this section. The base-stock counterpart of Section 4.6.2.

10.3. Distribution Systems

TODO: write this section. The stochastic counterpart of Section 4.6.3.

10.4. The METRIC Model

TODO: write this section.

10.5. The Bullwhip Effect

TODO: write this section.

10.6. Exercises

Exercise 10.1. For a two-stage serial system with base-stock policies, show that the optimal echelon base-stock levels can be computed sequentially from the downstream stage upward.

Exercise 10.2. A depot supplies three bases with a repairable item. Using the METRIC approximation, determine the stock levels at each location that minimize expected total backorders subject to a budget of 20 units.

11. Simulating Inventory Systems

i Learning objectives

After reading this chapter you should be able to:

- recognize when an inventory system requires simulation rather than analysis
- model the inventory position and net inventory processes over time
- build a continuous review inventory model using the KSL
- apply appropriate output analysis to a non-terminating inventory simulation
- compare inventory policies using a designed simulation experiment

11.1. Why Simulate an Inventory System?

TODO: write this section.

11.2. Modeling the Inventory Position Process

TODO: write this section.

11.3. Building an (r, Q) Model with the KSL

TODO: write this section.

11.4. Output Analysis for Inventory Simulations

TODO: write this section.

11.5. Comparing Policies by Simulation

TODO: write this section.

11.6. Exercises

Exercise 11.1. Build a simulation model of the (r, Q) system analyzed in Chapter 8. Compare the simulated fill rate with the value predicted by the approximate model, and account for any discrepancy.

Exercise 11.2. Using common random numbers, compare four candidate (r, Q) policies on expected total cost. Report a confidence interval for each pairwise difference and state which policy you would recommend.

A. Notation

A single reference for the symbols used throughout the book. Each entry gives the symbol, what it denotes, its unit of measure, and where it is defined.

A.1. Conventions

Time. The models are written in a generic time unit. Whether that unit is a day, a week, or a year is a matter of convention; only two rules matter. Every rate in a model must use the same unit, and every price quoted per period must be converted to that unit before it enters the model. Carrying charges and positioning costs are quoted per year by universal accounting convention, so a model running in days divides them by 365 on entry.

Time averages. An overbar denotes a time average of a time-persistent variable:

$$\overline{X} = \frac{1}{T} \int_0^T X(t) dt$$

Under the conditions discussed in Section 1.3, these averages converge as $T \rightarrow \infty$ to the corresponding steady-state quantities. An overbar on a quantity indexed by demand transaction rather than by time denotes an average over transactions, and the definition is given where that quantity is introduced.

A. Notation

Functions of time. A symbol written $X(t)$ is a quantity that varies over time and can, in principle, be observed at any instant. A symbol without the argument denotes either a parameter or the corresponding average.

Positive and negative parts. For a real quantity x , $[x]^+ = \max(x, 0)$ and $[x]^- = \max(-x, 0)$.

A.2. State Variables

Table A.1.: State variables of an inventory system.

Symbol	Meaning	Unit	Defined in
$I(t)$	Inventory on hand	units	Section 1.3
$IO(t)$	Inventory on order	units	Section 1.3
$B(t)$	Units backordered	units	Section 1.3
$I_a(t)$	Inventory actually on hand	units	Section 2.3.2
$I_r(t)$	Inventory on hand according to the record	units	Section 2.3.2
$D(t)$	Record discrepancy, $I_a(t) - I_r(t)$	units	Section 2.3.2
$IN(t)$	Net inventory, $I(t) - B(t)$	units	Section 1.3
$IP(t)$	Inventory position, $I(t) + IO(t) - B(t)$	units	Section 1.3

A. Notation

Symbol	Meaning	Unit	Defined in
$SO(t)$	Stockout indicator, 1 when $I(t) \leq 0$	dimensionless	Section 1.3
$RR(t)$	Ready indicator, $1 - SO(t)$	dimensionless	Section 1.3

Table A.2.: Time averages of the state variables.

Symbol	Meaning	Unit	Defined in
$\bar{I}$	Average inventory on hand	units	Section 1.3
$\bar{B}$	Average units backordered	units	Section 1.3
$\bar{N}$	Average order rate	orders per unit time	Section 1.6.1
$\overline{IO}$	Average inventory on order	units	Section 1.3
$\overline{IN}$	Average net inventory	units	Section 1.3
$\overline{IP}$	Average inventory position	units	Section 1.3
$\overline{SO}$	Proportion of time out of stock	dimensionless	Section 1.3

A.3. Demand and Time

A. Notation

Table A.3.: Demand and timing quantities.

Symbol	Meaning	Unit	Defined in
λ	Demand rate	units per unit time	Section 1.3
D_i	Size of the i th demand transaction	units	Section 1.3
n	Number of demand transactions observed	transactions	Section 1.3
L	Replenishment lead time	time	Section 1.3
p	Production or replenishment rate	units per unit time	Chapter 3
T_p	Time to replenish, Q/p	time	Chapter 3
$\bar{L}$	Average lead time	time	Section 1.3
$\bar{W}$	Average time a customer waits	time	Section 1.3
$\bar{T}_s$	Average time a unit spends on hand	time	Section 1.3
TO	Inventory turnover ratio, $\lambda/\bar{I}$	per unit time	Section 1.3
λ_ℓ	Rate of lost demand	units per unit time	Section 1.3

A. Notation

Symbol	Meaning	Unit	Defined in
T	Length of the observation interval, or a review interval	time	Section 1.3

A.4. Cost Parameters

Table A.4.: Cost parameters. Note that h and b are themselves rates and therefore multiply a level, while k and π multiply rates.

Symbol	Meaning	Unit	Defined in
c	Unit cost	dollars per unit	Section 1.4.3
i	Carrying charge	dollars per dollar of value per unit time	Section 1.4.4
h	Holding cost, $h = ic$	dollars per unit per unit time	Section 1.4.4
k	Fixed order cost	dollars per order	Section 1.4.5
b	Backorder cost	dollars per unit per unit time	Section 1.4.6
π	Stockout cost	dollars per unit	Section 1.4.6
f	Stocking position cost	dollars per item per location per unit time	Section 1.4.7
γ	Carrying charge taken common to every item in an aggregate development	dollars per dollar of value per unit time	Section 4.4

A. Notation

Symbol	Meaning	Unit	Defined in
K	Major setup cost, incurred once per order regardless of its contents	dollars per order	Section 4.5

A.5. Performance Measures

Table A.5.: Service and performance measures.

Symbol	Meaning	Unit	Defined in
$\overline{RR}$	Ready rate, the proportion of time stock is on hand	dimensionless	Section 1.6
$\overline{FR}$	Transaction fill rate	dimensionless	Section 1.6
$\overline{FR}_u$	Unit fill rate	dimensionless	Section 1.6
Y_i	Indicator that the i th demand is filled immediately	dimensionless	Section 1.6
CSL	Cycle service level	dimensionless	Section 1.6
TC	Total cost rate, Equation 1.15	dollars per unit time	Section 1.6.1

A. Notation

Symbol	Meaning	Unit	Defined in
C	Relevant cost rate, the part of TC that varies with the policy, Equation 1.16	dollars per unit time	Section 1.6.1
β	Stated service target	dimensionless	Section 1.6.3

A.6. Policy Parameters

These are the decision variables. Each is introduced with the policy that uses it; they are collected here for reference.

Table A.6.: Policy parameters.

Symbol	Meaning	Unit	Defined in
Q	Order quantity	units	Chapter 3
r	Reorder point	units	Chapter 8
s	Reorder point in an (s, S) policy	units	Chapter 8
S	Order-up-to level	units	Chapter 9
R	Review interval	time	Chapter 9

A.7. Multi-Item Quantities

Items in a multi-item problem are indexed by j , and any symbol of the previous tables carries that subscript when it refers to one item. In

A. Notation

Chapter 4 the letter n counts items rather than demand transactions.

Table A.7.: Quantities that appear only when items are decided together.

Symbol	Meaning	Unit	Defined in
n	Number of items sharing the resource or the setup; in Section 4.6 it counts stages or regional warehouses instead	items, or locations	Chapter 4
a_j	Amount of the shared resource one unit of item j consumes	resource units per unit	Section 4.2
A	Amount of the shared resource available	resource units	Section 4.2
θ	Lagrange multiplier on the shared-resource constraint	varies with the resource	Section 4.2.2
$R(\theta)$	Resource function, the amount consumed at θ less the amount available	resource units	Section 4.2.3

A. Notation

Symbol	Meaning	Unit	Defined in
v_j	Volume one unit of item j occupies, the space-constraint case of a_j	cubic units per unit	Section 4.2
$\overline{OF}$	Order frequency for one item, λ/Q	orders per unit time	Section 4.4.1
$\bar{I}^a$	Aggregate investment in cycle stock	dollars	Section 4.4.2
N^a	Aggregate replenishments across all items	orders per unit time	Section 4.4.2
$\overline{OC}^a$	Aggregate ordering cost rate	dollars per unit time	Section 4.4.4
J^*	Variety index, the effective number of items	items	Section 4.4.4
C^a	Aggregate purchase cost rate, $\sum_j c_j \lambda_j$	dollars per unit time	Section 4.4.4
λ^a	Aggregate demand rate, $\sum_j \lambda_j$	units per unit time	Section 4.4.4
c^w	Demand-weighted average unit cost	dollars per unit	Section 4.4.4

A. Notation

Symbol	Meaning	Unit	Defined in
K^w	Purchase-weighted average ordering cost	dollars per order	Section 4.4.4
K	Major setup cost, incurred once whenever an order is placed at all	dollars per order	Section 4.5
k_j	Minor setup cost, incurred for each item included on that order	dollars per order	Section 4.5
T	Base period between order opportunities	time	Section 4.5.1
m_j	Number of base periods between replenishments of item j	dimensionless	Section 4.5.1
ℓ_j	Exponent in a power-of-two multiplier, $m_j = 2^{\ell_j}$; the subscript is dropped while following one item	dimensionless	Section 4.5.2

A. Notation

Symbol	Meaning	Unit	Defined in
$C(T_j)$	Relevant cost rate written as a function of the reorder interval	dollars per unit time	Section 4.5.3
g	Holding cost coefficient in the interval form, $h\lambda/2$	dollars per unit time squared	Section 4.5.3

A.8. Multi-Echelon Quantities

Stages in a serial system are numbered so that stage i supplies stage $i - 1$, with stage 1 facing the customer. In a distribution system the central warehouse is indexed 0.

Table A.8.: Quantities for systems in which one stock replenishes another.

Symbol	Meaning	Unit	Defined in
h_i	Installation holding cost rate at stage i , the total value accumulated through that stage	dollars per unit per unit time	Section 4.6.1

A. Notation

Symbol	Meaning	Unit	Defined in
h'_i	Echelon holding cost rate, the value added at stage i alone. In a serial chain $h'_i = h_i - h_{i+1}$; in a distribution system $h'_i = h_i - h_0$	dollars per unit per unit time	Section 4.6.1
k_i	Fixed ordering cost at stage i . There is no major setup in these sections, so this is the same k as Section 1.4.5	dollars per order	Section 4.6.2
g_i	The echelon holding coefficient at stage i : $\frac{1}{2}\lambda h'_i$ in a serial chain, where every stage sees the same rate, and $\frac{1}{2}\lambda_i h'_i$ in a distribution system	dollars per unit time squared	Section 4.6.2
T_i	Reorder interval at stage i	time	Section 4.6.2
$\bar{I}_i$	Average on-hand stock at stage i	units	Section 4.6.1

A. Notation

Symbol	Meaning	Unit	Defined in
$\bar{I}_i^e$	Average echelon stock at stage i	units	Section 4.6.1
G_r	The r th block of consecutive stages sharing one reorder interval		Section 4.6.2
M	Number of blocks in the partition	blocks	Section 4.6.2
$k(G_r), g(G_r)$	Block totals, $\sum_{i \in G_r} k_i$ and $\sum_{i \in G_r} g_i$		Section 4.6.2
$T(r)$	The interval shared by every stage in block G_r	time	Section 4.6.2
$\mathcal{C}^0$	The pinned set: the central warehouse together with the regions sharing its interval. Set in a calligraphic capital, to keep it apart from the cost rates written C		Section 4.6.3

A.9. Dynamic Lot Sizing Quantities

Chapter 5 plans one item over a finite horizon of N periods indexed $t = 1, \dots, N$. Every quantity below carries a period subscript, and the subscript is dropped in the many cases where the costs hold still. Notice that t is a period number here and a point in continuous time elsewhere in the book.

Table A.9.: Quantities that appear when the demand rate varies by period.

Symbol	Meaning	Unit	Defined in
N	Number of periods in the planning horizon	periods	Section 5.1.1
d_t	Requirement in period t . This is the discrete counterpart of the rate λ , and it replaces it for the whole of Chapter 5	units	Section 5.1.1
$d[t, u]$	Requirement of periods t through u inclusive, and zero when $u < t$	units	Section 5.1.1
$\bar{d}$	Average requirement per period over the horizon	units per period	Section 5.6.1

A. Notation

Symbol	Meaning	Unit	Defined in
k_t	Order cost in period t . The same k as Section 1.4.5, given a period subscript	dollars per order	Section 5.1.1
c_t	Unit purchase cost in period t	dollars per unit	Section 5.1.1
h_t	Cost of carrying one unit from the end of period t into period $t + 1$	dollars per unit per period	Section 5.1.1
Q_t	Order quantity received in period t	units	Section 5.2
I_t	Inventory at the end of period t , with $I_0 = 0$. Unlike the $I(t)$ of Section A.2 this is a period-end balance and not a continuous level	units	Section 5.2
y_t	One when an order is placed in period t , zero otherwise		Section 5.2

A. Notation

Symbol	Meaning	Unit	Defined in
$\hat{c}(t, u)$	Cost of one unit bought in period t and used in period u , which is c_t plus the holding rates of periods t through $u - 1$	dollars per unit	Section 5.3.2
$w(t, u)$	Window cost: the cost of a replenishment placed in period t that covers the requirements of periods t through u	dollars	Section 5.3.2
$V(u)$	Least cost of covering periods 1 through u and arriving at period $u + 1$ empty, with $V(0) = 0$	dollars	Section 5.4.2
$S(u)$	The period in which the last order of that least-cost plan starts	period	Section 5.4.2

A. Notation

Symbol	Meaning	Unit	Defined in
M	The large constant in the linking constraint of the mixed integer program. Not the block count of Table A.8	units	Section 5.4.3
VC	Variability coefficient, the variance of the requirements over the square of their mean		Section 5.6.1
W	Length of the planning window under a rolling horizon	periods	Section 5.7.1

Relevant cost is written TRC throughout Chapter 5 and means setup plus carrying, with the purchase term left out. Section 5.2.1 gives the one case in which it cannot be left out.

A.10. Requirements Planning Quantities

Chapter 6 plans every item of a product structure over a common horizon of periods. Three of its quantities are the same objects met earlier under other names, and the table says so where that is the case.

A. Notation

Table A.10.: Quantities that appear when requirements are computed rather than forecast.

Symbol	Meaning	Unit	Defined in
G_t	Gross requirement in period t : what the item is needed for, from a master schedule for an end item and from Equation 6.1 for everything else	units	Section 6.3
SR_t	Scheduled receipt in period t , an order already placed in a previous planning cycle. Not the $S(u)$ of Table A.9, which is a Wagner-Whitin pointer	units	Section 6.3

A. Notation

Symbol	Meaning	Unit	Defined in
I_t	Projected on hand at the end of period t . Deliberately the same symbol as Table A.9, because it is the same quantity: Equation 6.2 is Equation 5.1 with the scheduled receipts added	units	Section 6.3
NR_t	Net requirement in period t , floored at zero. Not the N of Table A.9, which is a horizon length	units	Section 6.3
POR_t	Planned order receipt in period t : what the lot sizing rule decides should arrive	units	Section 6.3

A. Notation

Symbol	Meaning	Unit	Defined in
Rel_t	Planned order release in period t , the receipt moved back by the lead time. This row is the record's output and the next level's input	units	Section 6.3
L	Lead time, in periods. The same L as Table A.1	periods	Section 6.3
SS	Safety stock, added to the gross requirement before netting	units	Section 6.3.2
q_{ij}	Quantity per: units of item j in one unit of item i . Equal to one throughout a distribution network		Section 6.2.2

The cumulative lead time is the sum of L down the longest path of a product structure. Section 6.3.3 shows why a horizon shorter than it cannot produce a plan.

A.11. Distribution and Loss Function Quantities

From Chapter 7 onward demand is a random variable, and the cost expressions need the expected amount by which it exceeds a stock level rather than its plain expectation. Section C.3 derives these functions and tabulates them for the distributions this book uses.

Table A.11.: Quantities that appear once demand is random.

Symbol	Meaning	Unit	Defined in
X	A random variable, usually a demand over a stated interval	units	Section C.3
$(X - b)^+$	$\max(X - b, 0)$, the amount by which X exceeds b	units	Section C.3
$g(x)$	Probability mass function when X is discrete, probability density function when it is continuous		Section C.3
$G(b)$	$P\{X \leq b\}$, the cumulative distribution function		Section C.3

A. Notation

Symbol	Meaning	Unit	Defined in
$G^0(b)$	$1 - G(b)$. For a discrete X this is $P\{X > b\}$ and not $P\{X \geq b\}$		Equation C.9
$G^1(b)$	First order loss function, $E[(X - b)^+]$: the expected shortage against a stock level of b	units	Equation C.9
$G^2(b)$	Second order loss function. Defined with a factor of $\frac{1}{2}$ and differently for discrete and continuous X	units squared	Equation C.10
$H^1(b), H^2(b)$	First and second order partial expectations, the integrals of $xg(x)$ and $x^2g(x)$ above b	units, units squared	Equation C.17
S	A random sum, $\sum_{i=1}^N X_i$, of a random number of random quantities	units	Equation B.1

A. Notation

Symbol	Meaning	Unit	Defined in
$N, N(t)$	The number of terms in a random sum. $N(t)$ counts demand occurrences in an interval of length t		Section B.5
$D(L)$	Demand over a lead time, the random sum $\sum_{i=1}^L D_i$. Written LTD in much of the literature	units	Equation B.5
$\phi(z), \Phi(z)$	Standard normal density and distribution function		Section C.3.4
$\Phi^0(z), \Phi^1(z), \Phi^2(z)$	The standard normal complement and loss functions, all dimensionless. Scaled by σ and σ^2 to give G^1 and G^2		Equation C.19

Notice the superscripts. They are orders of a single family and not powers: G^0 is a probability, G^1 is in units, and G^2 is in units squared, each obtained

A. Notation

by integrating or summing the one before it.

A.12. A Note on Other Sources

Inventory theory has an unfortunate amount of notational variation, and a reader moving between this book and the literature will meet the same quantity under several names. Two differences are worth knowing.

The fixed order cost is written k here and A in much of the classical literature; the unit cost is written c here and v in Silver et al. (2016) and the practitioner literature that follows it. And *Simulation Modeling using the KSL* writes $BO(t)$ for the quantity this book calls $B(t)$; the definition is identical.

B. Probability and Stochastic Processes Review

i Learning objectives

After reading this chapter you should be able to:

- review the probability results the text relies on
- work with the renewal and Poisson processes used in the inventory models
- compute the expectations of random sums that arise in lead-time demand

B.1. Random Variables and Expectation

TODO: write this section.

B.2. Sums of Random Variables

TODO: write this section.

B.3. The Poisson Process

TODO: write this section.

B.4. Renewal Processes

TODO: write this section.

B.5. Random Sums

A **random sum** adds up a random number of random quantities. Let N represent a non-negative integer random variable, let $X_1, X_2, \dots$ represent identically distributed random variables with the same distribution as some X , and let

$$S = \sum_{i=1}^N X_i, \quad S = 0 \text{ when } N = 0 \quad (\text{B.1})$$

Assume throughout that the X_i are independent of one another **and of** N . That last requirement is the one that fails in practice, and Section B.5.5 says what it costs.

Random sums appear twice in this book, and both times the quantity being summed is demand.

Demand over a random lead time. Chapter 8 needs the demand that occurs while a replenishment order is outstanding. Let D_i represent the demand in period i and let L represent the lead time measured in whole periods. Then the lead time demand is $D(L) = \sum_{i=1}^L D_i$, a random sum in which N is the lead time.

Demand that arrives in batches. A slow-moving item is not demanded every period. Let $N(t)$ represent the number of demand occurrences in an interval of length t and let D_i represent the size of the i th one. Then the demand over the interval is $\sum_{i=1}^{N(t)} D_i$, a random sum in which N counts occurrences rather than periods. Section C.4.3 uses this representation for intermittent demand.

B.5.1. The Mean

Condition on N . Given $N = n$ the sum has exactly n terms, so $E[S | N = n] = nE[X]$. Thus, $E[S | N] = NE[X]$, and taking the expectation of both sides,

$$E[S] = E[E[S | N]] = E[NE[X]] = E[N] E[X] \quad (\text{B.2})$$

Equation B.2 is Wald's equation, and it says the obvious thing: the expected total is the expected count times the expected size. Notice that it needs only the *means* of N and X , which is why it is the one result in this section that survives almost any weakening of the assumptions.

B.5.2. The Variance

The variance is where the interesting behavior is, and the obvious guess is wrong. Conditioning again, and using the law of total variance,

$$\text{Var}[S] = E[\text{Var}[S | N]] + \text{Var}[E[S | N]] \quad (\text{B.3})$$

Take the two terms in turn. Given $N = n$ the sum is of n independent terms, so $\text{Var}[S | N] = N \text{Var}[X]$ and the first term is $E[N] \text{Var}[X]$. And $E[S | N] = NE[X]$, so the second term is $\text{Var}[NE[X]] = (E[X])^2 \text{Var}[N]$. Thus,

B. Probability and Stochastic Processes Review

$$\text{Var}[S] = E[N] \text{Var}[X] + \text{Var}[N] (E[X])^2 \quad (\text{B.4})$$

The two terms are not symmetric, and that asymmetry is the point. The variability of the *sizes* enters multiplied by the expected count. The variability of the *count* enters multiplied by the square of the expected size. That is, uncertainty in how many contributes a term that grows quadratically in the typical size, while uncertainty in how much contributes only linearly in the typical count.

For example, an item demanded five units at a time is four times as exposed to uncertainty in the number of demands as an item demanded two and a half units at a time, all else equal. Recall from Section 4.6.3 that a supplier's reliability and a customer's predictability are different problems. Equation B.4 is why they are also different sizes of problem.

B.5.3. Lead Time Demand

Specializing Equation B.2 and Equation B.4 to $D(L) = \sum_{i=1}^L D_i$ gives the two results Section C.4.2 and Chapter 8 both use:

$$E[D(L)] = E[L] E[D], \quad \text{Var}[D(L)] = E[L] \text{Var}[D] + \text{Var}[L] (E[D])^2 \quad (\text{B.5})$$

Notice the units. $E[L]$ is in periods and $E[D]$ in units per period, so the product is in units. In the variance, $E[L] \text{Var}[D]$ is periods times units squared per period squared, and $\text{Var}[L] (E[D])^2$ is periods squared times units squared per period squared. Both come to units squared, which is the check that the two terms belong in the same sum. Notice also that L and D must be measured in the *same* period. A lead time in days and a demand per week is the most common error in applying Equation B.5, and it is invisible in the answer.

B. Probability and Stochastic Processes Review

For example, take an item averaging 5 units a week with a variance of 16, and a supplier whose lead time averages 10 weeks with a variance of 25. Then

$$E[D(L)] = 10(5) = 50 \text{ units}$$

$$Var[D(L)] = 10(16) + 25(5)^2 = 160 + 625 = 785 \text{ units}^2$$

so the standard deviation of lead time demand is $\sqrt{785} = 28.02$ units.

Read the two terms before moving on. The demand contributes 160 of the 785 and the lead time contributes 625, so about 80% of the variance the safety stock must cover comes from the supplier rather than from the customer. That split is typical, and it is the argument for spending on lead time reliability before spending on forecasting. You should compute both terms separately every time, because the total alone does not say where to act.

B.5.4. Batches, and the Compound Poisson

When N counts demand *occurrences* rather than periods, Equation B.4 applies unchanged with $N = N(t)$. What changes is that $E[N(t)]$ and $Var[N(t)]$ now have to come from somewhere, and Section B.4 is where.

One case is clean enough to state here. If demands occur according to a Poisson process with rate λ , then $N(t)$ is Poisson with $E[N(t)] = Var[N(t)] = \lambda t$, and the two terms of Equation B.4 collapse:

$$Var[S] = \lambda t (Var[X] + (E[X])^2) = \lambda t E[X^2] \quad (\text{B.6})$$

Thus, a compound Poisson demand needs only the *second moment* of the batch size, not its variance and mean separately. For example, at three

B. Probability and Stochastic Processes Review

demand occurrences on average, each averaging 5 units with a variance of 16, the total averages $3(5) = 15$ units with a variance of $3(16) + 3(25) = 123$, which Equation B.6 gives directly as $3(41) = 123$. The two routes agree, and that agreement is the arithmetic check for this section.

Notice what Equation B.6 implies. The variance to mean ratio of a compound Poisson demand is $E[X^2]/E[X]$, which exceeds one whenever the batch size is ever larger than one unit. That is the reason Section C.1 sends batched demand to the negative binomial rather than the Poisson, and it is why a Poisson *process* does not produce Poisson *period demand* unless every demand is for a single unit.

B.5.5. What This Does and Does Not Give You

Equation B.2 and Equation B.4 give two moments. They do not give a distribution, and the inventory models of Chapter 8 onward need one. Section C.4 is the bridge, and it is an assumption rather than a derivation: two numbers do not determine a distribution, so a family has to be chosen.

Two assumptions are worth stating plainly, because both fail in real data.

The count must not depend on the sizes. Suppose a large demand triggers an expedite that shortens the lead time. Then L and the D_i are dependent. Equation B.2 survives that under weaker conditions; Equation B.4 does not, and the true variance is usually smaller than it gives.

The sizes must not depend on one another. Demand that is auto-correlated, a run of heavy weeks followed by a run of light ones, breaks the first term of Equation B.4. Recall from Section 2.2.6 that burstiness is one of the things a demand history is characterized for. An item that tests as bursty is an item for which Equation B.4 understates the spread, and understating the spread understates the safety stock.

C. Common Probability Distributions

i Learning objectives

After reading this chapter you should be able to:

- identify the distributions used to model demand and lead time
- compute the loss functions that appear in inventory cost expressions

C.1. Discrete Distributions

Demand arrives in whole units, so a discrete distribution is the honest default and a continuous one is an approximation made for convenience. This section gives the four families the book uses, what each assumes, and the single diagnostic that usually decides between them. Section C.3 gives their loss functions.

The diagnostic is the variance to mean ratio. Let X represent a demand over some interval, and let

$$VMR = \frac{Var[X]}{E[X]} \quad (C.1)$$

C. Common Probability Distributions

Recall from Section 2.2.6 that a demand history is characterized before it is modeled. Equation C.1 is the characterization that selects among the families below, because each of them fixes the ratio or bounds it on one side. Notice that VMR is not the variability coefficient of Equation 5.10 or the squared coefficient of variation of Equation 2.3. Those are dimensionless; this one carries the units of X and is therefore a statement about a particular time bucket.

C.1.1. The Poisson Distribution

$$g(x) = \frac{\lambda^x e^{-\lambda}}{x!}, \quad x = 0, 1, 2, \dots, \quad E[X] = Var[X] = \lambda \quad (C.2)$$

The Poisson is what demand looks like when customers arrive independently at a constant average rate and each takes one unit. That is the standard model for a slow-moving part, and it is the distribution to reach for first when the item is demanded singly.

Notice what Equation C.2 fixes. The mean and the variance are the same parameter, so $VMR = 1$ exactly. Thus, the Poisson is not a family you fit to a variance; it is a family whose variance is decided the moment you estimate its mean. A demand history whose VMR is far from one is telling you that the Poisson is the wrong family, and no choice of λ will repair it.

C.1.2. The Geometric and Negative Binomial Distributions

The negative binomial is the family to reach for when $VMR > 1$, which is the usual case for real demand. Let $\beta = (1 - p)/p$. Then

C. Common Probability Distributions

$$g(x) = \binom{x+r-1}{r-1} p^r (1-p)^x, \quad x = 0, 1, 2, \dots, \quad E[X] = r\beta, \quad Var[X] = \frac{r\beta}{p} \quad (\text{C.3})$$

so that $VMR = 1/p > 1$ for every admissible p . The geometric is the special case $r = 1$, with $E[X] = \beta$ and $Var[X] = \beta/p$.

Two readings of Equation C.3 are useful and they are different. Counting failures before the r th success is the textbook reading and explains the binomial coefficient. The reading that matters here is that the negative binomial is a Poisson whose rate is itself random, drawn from a gamma distribution. That is, it is what you get when demand is Poisson but the rate varies from interval to interval, which is exactly the situation an inventory analyst meets. Thus, the negative binomial is the Poisson with the constant-rate assumption relaxed, and the extra variance is the price of relaxing it.

A warning about p . Two conventions are in circulation. Equation C.3 makes p the probability of *success*, so the mean is $r(1-p)/p$. Much of the fitting literature, including the selection rules of Section C.4, makes p the probability of *failure*, so the mean is $rp/(1-p)$. The two are exchanged by replacing p with $1-p$, and a fit that comes out inverted is almost always this and not an error in the data. You should check which convention a routine uses before passing it a parameter.

C.1.3. The Binomial Distribution

$$g(x) = \binom{n}{x} p^x (1-p)^{n-x}, \quad x = 0, 1, \dots, n, \quad E[X] = np, \quad Var[X] = np(1-p) \quad (\text{C.4})$$

Here $VMR = 1-p < 1$, so the binomial is the family for demand that is *less* variable than Poisson. That is rarer than the other direction, and

C. Common Probability Distributions

when it appears it usually has a structural cause: a fixed population of n machines each of which may or may not need the part this period, or a contract that caps the quantity. Notice that the support is bounded above, which is the strongest statement any of these families makes and should be believed only when something in the situation actually enforces it.

C.1.4. The Discrete Empirical Distribution

Let $S = \{x_1, \dots, x_k\}$ represent the distinct values observed and p_j their relative frequencies. The empirical distribution is the data used as its own model, with no family assumed.

It is the right choice more often than its simplicity suggests, and Chapter 7 uses it. Thus, when the history is long enough to populate the range and the decision is a single quantile, fitting a family buys nothing and risks a poor fit. Two costs come with it. The support is bounded by what happened to be observed, so a quantile beyond the largest observation cannot be produced at all, and a long tail is exactly where inventory decisions live. And the estimate inherits every irregularity of the sample, including the ones that are noise.

The rule of thumb. Use the empirical distribution when the question is inside the data and a fitted family when it is outside. A service level of 90% on two hundred observations is inside. A service level of 99% is not.

C.2. Continuous Distributions

A continuous distribution is an approximation to a demand that arrives in whole units, and it is made for two reasons. The arithmetic is easier, and once demand is aggregated over a lead time or a season the granularity stops mattering. This section gives the four families the book uses and, for each, the condition under which the approximation is safe. Section C.3 gives their loss functions.

C. Common Probability Distributions

The diagnostic here is the coefficient of variation. Let

$$c_X = \frac{\sqrt{\text{Var}[X]}}{E[X]}, \quad c_X^2 = \frac{\text{Var}[X]}{(E[X])^2} \quad (\text{C.5})$$

Unlike the variance to mean ratio of Equation C.1, c_X is dimensionless, so it compares across items and across time buckets. Each family below either fixes c_X or restricts it, and that is what selects among them.

C.2.1. The Exponential Distribution

$$g(x) = \lambda e^{-\lambda x}, \quad x \geq 0, \quad E[X] = \frac{1}{\lambda}, \quad \text{Var}[X] = \frac{1}{\lambda^2} \quad (\text{C.6})$$

Thus, $c_X = 1$ exactly. The exponential is the continuous counterpart of the Poisson: it is the time between Poisson arrivals, and it appears in this book as a lead time rather than as a demand. Like the Poisson it has no free parameter for variability, so it is a hypothesis to be tested rather than a family to be fitted to a spread.

Notice also that it is memoryless. A lead time modeled as exponential is one for which an order already three weeks late is no more likely to arrive tomorrow than one placed today. That is a strong claim about a supplier, and it is usually false.

C.2.2. The Gamma and Erlang Distributions

$$g(x) = \frac{x^{\alpha-1} e^{-x/\beta}}{\beta^\alpha \Gamma(\alpha)}, \quad x \geq 0, \quad E[X] = \alpha\beta, \quad \text{Var}[X] = \alpha\beta^2 \quad (\text{C.7})$$

with shape $\alpha > 0$ and scale $\beta > 0$. Then $c_X = 1/\sqrt{\alpha}$, so the shape parameter alone sets the variability and the scale sets the size. The gamma is the flexible workhorse of this appendix: it covers every $c_X > 0$, it lives

C. Common Probability Distributions

on the non-negative half line where demand lives, and it reduces to the exponential at $\alpha = 1$.

The **Erlang** is the gamma with integer α , which is the sum of α independent exponentials. That buys a closed form for the distribution function and nothing else, and Table C.4 shows the loss functions are the gamma's unchanged. Its importance here is structural rather than computational: because an Erlang is a sum of exponentials, mixtures of Erlangs can be built to match any mean and variance, which is the procedure of Section C.4.2.

Notice what Equation C.7 does *not* have. There is no closed form for $G(x)$, so a quantile requires a numerical routine. That is rarely an obstacle, and it is not an argument for the normal.

C.2.3. The Normal Distribution

The normal is the most used and the least defensible of the four. It is used because Section B.2 makes it the limit of a sum, so a demand aggregated over many periods or many customers tends toward it, and because its loss functions are the tabulated ones of Table C.2.

Its defect is the one every inventory text mentions and few quantify. The support is the whole real line, so a normal demand model assigns positive probability to negative demand, and the amount is governed entirely by c_X .

Table C.1.: The probability a normal model puts below zero, as a function of the coefficient of variation. Notice the shape of the failure. Below about $c_X = 0.3$ the defect is invisible, and past $c_X = 1$ a sixth of the distribution is nonsense.

c_X	$P\{X < 0\}$ under a normal model
0.2	0.0000
0.4	0.0062

C. Common Probability Distributions

c_X	$P\{X < 0\}$ under a normal model
0.6	0.0478
1.0	0.1587
1.5	0.2525

Table C.1 is the rule in practice. Thus, a normal model is safe for demand whose coefficient of variation is below roughly 0.3, questionable to about 0.5, and wrong above it. Recall from Example 2.4 that intermittent items routinely run past $c_X = 1$, which is precisely where the normal fails, and why Chapter 7 does not reach for it there.

C.2.4. The Lognormal Distribution

If $\ln X$ is normal with mean μ and variance σ^2 , then X is lognormal. It is the natural repair of the normal's defect: the support is strictly positive, the shape is right-skewed as demand data usually is, and it accommodates any c_X .

The parameterization is the trap. A fitted demand distribution reports the mean and variance of X , while the density is written in terms of the mean and variance of $\ln X$. The conversion is

$$\sigma^2 = \ln \left(\frac{\text{Var}[X]}{(E[X])^2} + 1 \right) = \ln(1 + c_X^2), \quad \mu = \ln(E[X]) - \frac{\sigma^2}{2} \quad (\text{C.8})$$

and Table C.4 is written in terms of $E[X]$ and $\text{Var}[X]$ for that reason. Notice that σ^2 depends on the data only through c_X , so the shape of a lognormal is set by the coefficient of variation exactly as the gamma's is by α .

Choosing between the gamma and the lognormal is usually not worth a long argument. Both live on the positive half line, both take any

C. Common Probability Distributions

c_X , and on demand data they rarely separate by enough to change a policy. The lognormal has the heavier right tail, so it gives the larger answer at a high service level, and that difference grows with c_X . Where the two disagree enough to matter, the disagreement is the finding, and Section C.4 says what to do about it.

C.3. Loss Functions

Two quantities appear in almost every stochastic inventory cost expression in this book, and neither is an ordinary expectation. How much demand goes unmet when b units are on hand, and how much stock is left over when b units were provided. Both are expectations of a *truncated* difference, and the functions that give them are called **loss functions**.

Let X represent a random variable, usually a demand over some interval. Let b represent a stock level the demand is compared against. The notation $(X - b)^+ = \max(X - b, 0)$ is used throughout, so $(X - b)^+$ is the amount by which demand exceeds b , and zero when it does not.

Four functions are needed. Let $G(b) = P\{X \leq b\}$ represent the cumulative distribution function of X . Then

$$G^0(b) = 1 - G(b), \quad G^1(b) = E[(X - b)^+] \quad (\text{C.9})$$

define the **complementary distribution function** and the **first order loss function**. The second order loss function is defined differently for the two cases, and the difference is not cosmetic:

$$G^2(b) = \begin{cases} \frac{1}{2}E[(X - b)^+(X - b - 1)^+] & X \text{ discrete} \\ \frac{1}{2}E[((X - b)^+)^2] & X \text{ continuous} \end{cases} \quad (\text{C.10})$$

C. Common Probability Distributions

Read G^0 carefully in the discrete case. Because $G(b) = P\{X \leq b\}$, the complement is $G^0(b) = P\{X > b\}$, which is $P\{X \geq b + 1\}$ and not $P\{X \geq b\}$. Every formula in this section depends on that convention, and using $P\{X \geq b\}$ instead produces answers that are correct only where the two happen to coincide. You should check which convention a source uses before taking a formula from it.

Why the second order function exists at all is worth a sentence, since the first is the one with the obvious meaning. Chapter 8 needs the expected backorder level of a policy that orders Q at a time, and that level is a *difference* of second order loss functions evaluated at r and $r + Q$. Chapter 7 needs only the first.

C.3.1. Discrete Random Variables

Let $g(x) = P\{X = x\}$ represent the probability mass function. Writing the expectation as a sum over the range gives

$$G^1(b) = \sum_{x>b} (x - b)g(x) = \sum_{x \geq b} G^0(x) \quad (\text{C.11})$$

$$G^2(b) = \frac{1}{2} \sum_{x>b} (x - b)(x - b - 1)g(x) = \sum_{x>b} G^1(x) \quad (\text{C.12})$$

The second form of each is the one to notice. A loss function is a tail sum of the function one order below it, which is what makes both computable by a single downward pass over the support.

For a non-negative X the pass can start from a moment. Notice that $G^1(0) = E[X]$ and that $G^2(0) = \frac{1}{2}E[X(X - 1)]$, the latter being the **second binomial moment**. Thus, for $b \geq 0$,

C. Common Probability Distributions

$$G^1(b) = E[X] - \sum_{0 \leq x < b} G^0(x), \quad G^2(b) = \frac{1}{2}E[X(X-1)] - \sum_{0 < x \leq b} G^1(x) \quad (\text{C.13})$$

Equation C.13 is how these are computed in practice. That is, one accumulating loop produces G^0 , G^1 and G^2 at every b from a single evaluation of the mass function at each point.

C.3.2. The Discrete Empirical Distribution

A demand history that is used directly, without fitting, is a discrete empirical distribution. Let $S = \{x_1, x_2, \dots, x_k\}$ represent its ordered set of distinct values and let $g(x_j) = p_j$ represent the mass at each. Inside the support the sums of Equation C.11 apply unchanged. Outside it they need care, because the support is finite in both directions.

$$G^1(b) = \begin{cases} E[X] - b & b \leq x_1 \\ \sum_{x > b, x \in S} (x - b)g(x) & x_1 < b < x_k \\ 0 & b \geq x_k \end{cases} \quad (\text{C.14})$$

Notice the first line. Below the smallest possible value every unit of demand exceeds b , so the truncation never binds and $G^1(b)$ is the plain expectation $E[X] - b$. It is *not* constant at $G^1(x_1)$, which is the value it takes only at $b = x_1$ itself. The same reasoning gives

$$G^2(b) = \frac{1}{2} (E[X^2] - (2b+1)E[X] + b(b+1)) \quad \text{for } b \leq x_1 \quad (\text{C.15})$$

and zero for $b \geq x_k$. For example, take $S = \{2, 5, 9, 14\}$ with masses 0.20, 0.30, 0.35 and 0.15, so that $E[X] = 7.15$ units. At $b = 0$ the first order loss function is $7.15 - 0 = 7.15$ units, not the $7.15 - 2 = 5.15$ that treating

C. Common Probability Distributions

it as constant below the support would give. The two agree at $b = 2$ and nowhere else.

C.3.3. Continuous Random Variables

Let $g(x)$ represent the probability density function. The sums become integrals and the tail relations survive:

$$G^1(b) = \int_b^\infty (x - b)g(x) dx = \int_b^\infty G^0(x) dx, \quad G^2(b) = \frac{1}{2} \int_b^\infty (x - b)^2 g(x) dx = \int_b^\infty G^1(x) dx \quad (\text{C.16})$$

For a non-negative X and $b \leq 0$ the truncation again never binds, so $G^1(b) = E[X] - b$ and $G^2(b) = \frac{1}{2} (E[X^2] - 2bE[X] + b^2)$. Thus, $G^1(0) = E[X]$ and $G^2(0) = \frac{1}{2} E[X^2]$.

The closed forms in Table C.4 are all obtained the same way, through the **partial expectation functions**. Let

$$H^1(b) = \int_b^\infty x g(x) dx, \quad H^2(b) = \int_b^\infty x^2 g(x) dx \quad (\text{C.17})$$

Expanding $(x - b)$ and $(x - b)^2$ inside the integrals of Equation C.16 and collecting terms gives

$$G^1(b) = H^1(b) - bG^0(b), \quad G^2(b) = \frac{1}{2} (H^2(b) - 2bH^1(b) + b^2G^0(b)) \quad (\text{C.18})$$

That is, a loss function for a new distribution follows from its partial expectations, and those are often the easier integrals. The lognormal entry of Table C.4 is derived exactly this way.

C.3.4. The Standard Normal Case

The normal distribution is used often enough that its loss functions carry their own notation. Let $\phi(z)$ and $\Phi(z)$ represent the standard normal density and distribution function, and let $\Phi^0(z) = 1 - \Phi(z)$. Then

$$\Phi^1(z) = \phi(z) - z\Phi^0(z), \quad \Phi^2(z) = \frac{1}{2} [(z^2 + 1)\Phi^0(z) - z\phi(z)] \quad (\text{C.19})$$

and for $X \sim N(\mu, \sigma^2)$ with $z = (x - \mu)/\sigma$,

$$G^0(x) = \Phi^0(z), \quad G^1(x) = \sigma\Phi^1(z), \quad G^2(x) = \sigma^2\Phi^2(z) \quad (\text{C.20})$$

Notice the units. Φ^1 and Φ^2 are dimensionless, and the scaling by σ and σ^2 is what puts G^1 in units and G^2 in units squared. A loss function computed without that scaling is one of the more common errors in this material, and the units are what catch it.

Table C.2 gives the three functions at the values of z that arise most often.

Table C.2.: The standard normal density, complementary distribution function, and first and second order loss functions. Notice that Φ^1 is strictly decreasing and stays positive, which is what makes the expected shortage fall as the stock level rises without ever reaching zero.

z	$\phi(z)$	$\Phi^0(z)$	$\Phi^1(z)$	$\Phi^2(z)$
-2.00	0.0540	0.9772	2.0085	2.4971
-1.00	0.2420	0.8413	1.0833	0.9623
0.00	0.3989	0.5000	0.3989	0.2500
0.50	0.3521	0.3085	0.1978	0.1048

C. Common Probability Distributions

z	$\phi(z)$	$\Phi^0(z)$	$\Phi^1(z)$	$\Phi^2(z)$
1.00	0.2420	0.1587	0.0833	0.0377
1.28	0.1758	0.1003	0.0475	0.0197
1.50	0.1295	0.0668	0.0293	0.0114
1.645	0.1031	0.0500	0.0209	0.0078
2.00	0.0540	0.0228	0.0085	0.0029
2.33	0.0264	0.0099	0.0034	0.0010
3.00	0.0044	0.0013	0.0004	0.0001

For example, take demand over a lead time as $N(50, 12^2)$ units and a stock level of 62 units. Then $z = (62 - 50)/12 = 1.00$, so from Table C.2 the probability of a shortage is $\Phi^0(1.00) = 0.1587$ and the expected shortage is $\sigma\Phi^1(1.00) = 12(0.0833) = 0.9996$ units, i.e. about one unit short on average.

That answer is worth checking a second way, and the partial expectation gives one. Reading $\phi(1.00) = 0.2420$ from the same table,

$$H^1(62) = \mu\Phi^0(z) + \sigma\phi(z) = 50(0.1587) + 12(0.2420) = 10.8390 \text{ units}$$

so from Equation C.18, $G^1(62) = 10.8390 - 62(0.1587) = 0.9996$ units. The two routes agree. Carried to full precision both give 0.999786 units, and the difference is the four decimal places Table C.2 is printed to.

That check is worth performing whenever a loss function is computed for the first time in a new setting, because it catches a missing σ immediately. Without the scaling the first route returns 0.0833 units while the second still returns 0.9996, and a disagreement by a factor of σ is unmistakable.

C.3.5. Loss Functions for Common Distributions

Table C.3 and Table C.4 collect the closed forms. In both tables g is the mass or density function and G^0 is the complementary distribution function, so that each loss function is expressed in terms of quantities the distribution already provides.

Table C.3.: First and second order loss functions for the discrete distributions used in this book. Notice that the Poisson and negative binomial forms have the same shape, a multiple of G^0 plus a multiple of g , which is what the downward recursion of Equation C.13 produces in closed form.

Distribution	Parameters and moments	Loss functions
Poisson	$g(x) = \frac{\lambda^x e^{-\lambda}}{x!},$ $x = 0, 1, 2, \dots$ $E[X] = Var[X] = \lambda$	$G^1(x) =$ $-(x - \lambda)G^0(x) + \lambda g(x)$ $G^2(x) =$ $\frac{1}{2} \{ [(x - \lambda)^2 + x] G^0(x) - \lambda(x - \lambda)g(x) \}$
Geometric, the number of failures before the first success	$g(x) = (1 - p)^x p,$ $x = 0, 1, 2, \dots$ $G(x) = 1 - (1 - p)^{x+1}$ $E[X] = \beta,$ $Var[X] = \beta/p,$ with $\beta = (1 - p)/p$	$G^1(x) = \beta(1 - p)^x$ $G^2(x) = \beta^2(1 - p)^x$
Negative binomial, the number of failures before the r th success	$g(x) =$ $\binom{x + r - 1}{r - 1} p^r (1 - p)^x,$ $x = 0, 1, 2, \dots$ $E[X] = r\beta,$ $Var[X] = r\beta/p,$ with $\beta = (1 - p)/p$	$G^1(x) = -(x - r\beta)G^0(x) + (x + r)\beta g(x)$ $G^2(x) =$ $\frac{1}{2} \{ [r(r + 1)\beta^2 - 2r\beta x + x(x + 1)] G^0(x) + [(r + 1)]$

C. Common Probability Distributions

Distribution	Parameters and moments	Loss functions
Binomial	$g(x) = \binom{n}{x} p^x (1-p)^{n-x},$ $x = 0, 1, \dots, n$ $E[X] = np,$ $Var[X] = np(1-p)$	No simpler form than Equation C.11. The support is finite, so the sums terminate and Equation C.13 computes both in one pass

Table C.4.: First and second order loss functions for the continuous distributions used in this book. The lognormal row is parameterized by the mean and variance of X itself rather than by those of $\ln X$, because that is what a fitted demand distribution reports.

Distribution	Parameters and moments	Loss functions
Exponential	rate $\lambda, x \geq 0$ $G^0(x) = e^{-\lambda x}$ $E[X] = 1/\lambda,$ $Var[X] = 1/\lambda^2$	$G^1(x) = \frac{1}{\lambda} e^{-\lambda x}$ $G^2(x) = \frac{1}{\lambda^2} e^{-\lambda x}$
Gamma	shape $\alpha > 0$, scale $\beta > 0$ $g(x) = \frac{x^{\alpha-1} e^{-x/\beta}}{\beta^\alpha \Gamma(\alpha)}$ $G(x)$ has no closed form $E[X] = \alpha\beta,$ $Var[X] = \alpha\beta^2$	$G^1(x) = \beta \left[\left(\alpha - \frac{x}{\beta} \right) G^0(x) + xg(x) \right]$ $G^2(x) = \frac{1}{2}\beta^2 \left\{ \left[\left(\alpha - \frac{x}{\beta} \right)^2 + \alpha \right] G^0(x) + \left(\alpha - \frac{x}{\beta} + 1 \right) xg(x) \right\}$

C. Common Probability Distributions

Distribution	Parameters and moments	Loss functions
Erlang	the gamma with integer α $G(x) = 1 - \sum_{j=0}^{\alpha-1} \frac{(x/\beta)^j e^{-x/\beta}}{j!}$	The gamma forms above, unchanged. The integer shape buys a closed form for $G(x)$ and nothing else
Normal	$X \sim N(\mu, \sigma^2)$, $z = (x - \mu)/\sigma$	Equation C.19 and Equation C.20
Lognormal, parameterized by its own mean and variance	$\sigma^2 = \ln \left(\frac{Var[X]}{(E[X])^2} + 1 \right)$ $\mu = \ln(E[X]) - \sigma^2/2$ $z = (\ln x - \mu)/\sigma$	$G^0(x) = \Phi^0(z)$ $G^1(x) = E[X] \Phi(\sigma - z) - x \Phi^0(z)$ $G^2(x) = \frac{1}{2} [(Var[X] + (E[X])^2) \Phi(2\sigma - z) - 2xE[X] \Phi(\sigma - z)]$

Two entries deserve a word. The lognormal expressions are Equation C.18 applied to the partial expectations $H^1(x) = E[X] \Phi(\sigma - z)$ and $H^2(x) = (Var[X] + (E[X])^2) \Phi(2\sigma - z)$. It is easy to quote one of those partial expectations as though it were the loss function itself, and the resulting figure is too large by roughly a factor of three at stock levels near the mean. Notice also that the gamma has no closed form for $G(x)$ while its loss functions *are* closed once G^0 is available numerically, which is the usual situation: a loss function is rarely harder to obtain than the distribution function it is built from.

C.3.6. Mixtures

Chapter 7 describes demand that is intermittent, and a mixture is one way to represent it. Let X be distributed as a mixture of X_1 and X_2 with weights p_1 and $p_2 = 1 - p_1$, so that

C. Common Probability Distributions

$$g_X(x) = p_1 g_{X_1}(x) + p_2 g_{X_2}(x), \quad G_X(x) = p_1 G_{X_1}(x) + p_2 G_{X_2}(x) \quad (\text{C.21})$$

Because the loss functions are expectations and the expectation operator is linear, they mix the same way:

$$G_X^1(x) = p_1 G_{X_1}^1(x) + p_2 G_{X_2}^1(x), \quad G_X^2(x) = p_1 G_{X_1}^2(x) + p_2 G_{X_2}^2(x) \quad (\text{C.22})$$

and so do the raw moments, $E[X^n] = p_1 E[X_1^n] + p_2 E[X_2^n]$. The variance does not, and the extra term is the reason mixtures are useful for intermittent demand:

$$\text{Var}[X] = p_1 \text{Var}[X_1] + p_2 \text{Var}[X_2] + p_1 p_2 (E[X_1] - E[X_2])^2 \quad (\text{C.23})$$

Notice that last term. Two components with identical variances still produce a mixture more variable than either, and the excess grows with the square of the distance between their means. Thus, a mixture can reach a variability the component families cannot, which is exactly what an intermittent demand series demands of a model.

What a mixture does not inherit is an inverse. There is no closed form for $G_X^{-1}(\cdot)$, which the newsvendor solution of Chapter 7 needs. However, the quantile is bracketed by the component quantiles. For any $p \in [0, 1]$, with $x_1 = G_{X_1}^{-1}(p)$ and $x_2 = G_{X_2}^{-1}(p)$,

$$\min\{x_1, x_2\} \leq G_X^{-1}(p) \leq \max\{x_1, x_2\} \quad (\text{C.24})$$

Thus, the search has a starting interval that is free. For a discrete mixture, step up from $\min\{x_1, x_2\}$; for a continuous one, use the bracket as the initial interval of a bisection search. Either way the bracket is what turns an unbounded search into a bounded one.

C.3.7. Computing Them

The KSL implements these functions directly, so nothing in Table C.3 or Table C.4 has to be coded by hand. A distribution that provides them implements `LossFunctionDistributionIfc`, which adds two methods:

```
fun firstOrderLossFunction(x: Double): Double
fun secondOrderLossFunction(x: Double): Double
```

Nine distributions implement it, and they are the ones tabulated above: Poisson, Geometric, NegativeBinomial, Binomial and DEmpiricalCDF for the discrete case, and Exponential, Gamma, Normal and Lognormal for the continuous case. `ShiftedLossFunctionDistribution` wraps any of them when the support has to be moved off the origin.

```
val ltd = Normal(mean = 50.0, variance = 144.0)

println(ltd.complementaryCDF(62.0))          // 0.158655, the
  ↪ shortage probability
println(ltd.firstOrderLossFunction(62.0))    // 0.999786, the
  ↪ expected shortage in units
println(ltd.secondOrderLossFunction(62.0))  // 5.424464
```

The Erlang is the Gamma with an integer shape parameter, and needs no separate class.

Check any implementation before trusting it, including this one. Two identities make that cheap, and both come from earlier in this section. For a non-negative random variable,

$$G^1(0) = E[X], \quad G^2(b) \geq 0 \text{ for every } b \quad (\text{C.25})$$

C. Common Probability Distributions

The first is Equation C.13 and its continuous counterpart read at $b = 0$. The second holds because G^2 is the expectation of a product of two non-negative quantities. Thus, an implementation whose $G^1(0)$ differs from the distribution's own mean has its parameters crossed somewhere, and one that returns a negative G^2 is wrong outright. That check is worth performing every time a loss function is used from a new source, because it is one line and it catches the two errors that actually happen: a rate used where a mean belongs, and a truncation applied at the wrong end.

C.4. Choosing a Demand Distribution

Every model from Chapter 7 onward takes a demand distribution as an input, and the distribution carries more of the answer than the policy formula built on top of it does. This section covers the two routes to one. **Fitting** uses a sample of demand. **Moment matching** uses only a mean and a variance, which is the situation whenever the quantity of interest was derived rather than observed.

C.4.1. Fitting to a Sample

The sequence is the same one *Simulation Modeling using the KSL* gives at length in its distribution modeling appendix, compressed here to what an inventory analyst needs.

1. **Characterize before hypothesizing.** Plot the series in time order, then as a histogram. Compute the mean, the variance, and the diagnostic of Equation C.1 or Equation C.5. Count the zeros.
2. **Hypothesize from the characterization, not from habit.** A variance to mean ratio near one points at the Poisson, above one at the negative binomial, below one at the binomial. A coefficient of variation past 0.5 rules out the normal by Table C.1.

C. Common Probability Distributions

3. **Estimate the parameters**, by maximum likelihood or by matching moments.
4. **Test the fit**, with a chi-squared test for a discrete family or a Kolmogorov-Smirnov test for a continuous one.
5. **Look at the fit**, as a density over a histogram and as a quantile plot. The tail is what an inventory decision uses, and a test statistic averages over the whole distribution.

The KSL does the last four steps. `PDFModeler` for continuous data and `PMFModeler` for discrete data estimate parameters across a set of families, score the candidates, and rank them:

```
val results = PDFModeler(data).estimateAndEvaluateScores()
```

Three cautions are worth more than the mechanics.

Failing to reject is not proof. A goodness-of-fit test asks whether the data are inconsistent with a family. With two hundred observations it will fail to reject several families at once, and with twenty thousand it will reject all of them. Thus, the test narrows the field and does not choose.

A sales history is not a demand history. Sales are censored at whatever was on the shelf, so fitting to sales understates both the mean and the variance, and understates them most in exactly the periods when the item mattered. Chapter 7 returns to this.

The reporting bucket changes the answer. Recall from Example 2.4 that the same transactions bucketed daily rather than monthly show more zeros and a longer interval between demands, which can move an item from one demand class to another. Thus, fit at the bucket the decision is made in, and state which one it was.

C.4.2. Matching the First Two Moments

Often there is no sample. The demand during a lead time, which Chapter 8 needs, is a random sum whose mean and variance follow from the moments of demand and of lead time by Section B.5, and no observations of it exist at all. What is available is $\hat{\mu}$ and $\hat{\sigma}^2$, and what is wanted is a distribution.

For example, take demand averaging 5 units a week with a variance of 16, and a lead time averaging 10 weeks with a variance of 25. Then

$$E[D(L)] = 10(5) = 50 \text{ units}, \quad \text{Var}[D(L)] = 10(16) + 25(5)^2 = 785 \quad (\text{C.26})$$

so $c^2 = 785/2500 = 0.3140$ and $VMR = 15.70$ units. Notice that the lead time variance enters multiplied by the *square* of the demand rate, which is why an unreliable supplier dominates a variable demand in almost every real instance.

The gamma match is the default. The gamma can match any mean and variance, and inverting Equation C.7 gives the parameters directly:

$$\hat{\beta} = \frac{\hat{\sigma}^2}{\hat{\mu}}, \quad \hat{\alpha} = \frac{\hat{\mu}^2}{\hat{\sigma}^2} \quad (\text{C.27})$$

On Equation C.26 that gives $\hat{\beta} = 785/50 = 15.70$ and $\hat{\alpha} = 2500/785 = 3.1847$. The check is immediate: $\alpha\beta = 50.0000$ and $\alpha\beta^2 = 785.0000$. That check is worth performing every time, because a matching procedure that does not reproduce its own targets is the most common failure in this material.

A mixture of two Erlangs is the alternative, and it is the one to use when the distribution must be discrete in spirit or when a phase-type representation is wanted. Algorithm C.1 returns $\text{Erlang}(k_1, \lambda_1, k_2, \lambda_2, p_1, p_2)$ matching $E[X]$ and $\text{Var}[X]$.

C. Common Probability Distributions

Algorithm C.1 The Erlang mixture procedure. Two branches, because below a coefficient of variation of one the mixture needs shapes above one to be tight enough, and at or above it two exponentials with different rates are already spread enough. Notice `disc`, which is exactly zero when c^2 is the reciprocal of an integer and can go slightly negative in floating point, so it is clamped rather than trusted. At that same point p_1 evaluates to one, and to a shade above one in floating point, so it is clamped as well. Neither clamp changes an answer; both stop the procedure returning something that is not a probability.

```
erlangMixture(E[X], Var[X]):
```

```
    c2 = Var[X] / (E[X] * E[X])

    if c2 < 1:                                // both components are Erlang
        k1 = floor(1 / c2)
        k2 = k1 + 1
        disc = max(0, k2*(1 + c2) - k2*k2*c2)    // exactly 0 when c2 = 1/k
        p1 = min(1, (k2*c2 - sqrt(disc)) / (1 + c2))
        lambda1 = (k2 - p1) / E[X]
        lambda2 = lambda1
    else:                                     // both components are exponential
        k1 = 1
        k2 = 1
        lambda1 = (2 / E[X]) * (1 + sqrt((c2 - 1/2) / (c2 + 1)))
        lambda2 = 4 / E[X] - lambda1
        p1 = lambda1 * (lambda2 * E[X] - 1) / (lambda2 - lambda1)

    p2 = 1 - p1
    return (k1, lambda1, k2, lambda2, p1, p2)
```

On Equation C.26, $c^2 = 0.3140$ falls in the first branch, giving $k_1 = 3$,

C. Common Probability Distributions

$k_2 = 4$, $p_1 = 0.5893$ and a common rate of $\lambda = 0.06821$ per unit. The mixture's mean is 50.0000 and its variance 785.0000, matching to six figures.

One implementation note. When c^2 is exactly $1/k$ for an integer k , the quantity under the square root in the first branch is exactly zero, and floating point arithmetic can make it very slightly negative. Thus, clamp it at zero rather than letting the square root fail.

Adan's rule selects a family rather than assuming one, and it is the procedure to use when the quantity being represented is known to be discrete. Let

$$a = c^2 - \frac{1}{\mu} = \frac{VMR - 1}{\mu} \quad (\text{C.28})$$

Notice that a is a rescaled statement about how the variance compares with the mean, so $a = 0$ is exactly the Poisson case. Table C.5 gives the rule, in the form of Adan et al. (1995) as extended by Rossetti and Ünlü (2011) to cover $a < -1$, where no discrete family can match.

Table C.5.: Adan's rule. Notice that the five cases partition the whole range of a , so the rule always returns something, and that the ordering is by increasing variability: bounded support, then equidispersion, then two degrees of overdispersion.

Condition	Family
$a < -1$	Gamma, by Equation C.27
$-1 \leq a < 0$	A mixture of $\text{Bin}(n, p)$ and $\text{Bin}(n + 1, p)$
$a = 0$	Poisson with mean μ
$0 < a < 1$	A mixture of $\text{NB}(r, p)$ and $\text{NB}(r + 1, p)$

C. Common Probability Distributions

Condition	Family
$a \geq 1$	A mixture of $\text{Geom}(p_1)$ and $\text{Geom}(p_2)$

In every mixture below, q is the weight on the first component.

Mixed binomial, for $-1 \leq a < 0$:

$$n = \left\lfloor \frac{-1}{a} \right\rfloor, \quad q = \frac{1 + a(n+1) + \sqrt{-an(n+1) - n}}{1 + a}, \quad p = \frac{\mu}{n+1-q} \quad (\text{C.29})$$

Mixed negative binomial, for $0 < a < 1$:

$$r = \left\lfloor \frac{1}{a} \right\rfloor, \quad q = \frac{a(r+1) - \sqrt{(r+1)(1-ar)}}{1 + a}, \quad p = \frac{\mu}{r+1-q+\mu} \quad (\text{C.30})$$

Mixed geometric, for $a \geq 1$:

$$q = \frac{1}{1 + a + \sqrt{a^2 - 1}}, \quad p_1 = \frac{\mu}{2q + \mu}, \quad p_2 = \frac{\mu}{2(1-q) + \mu} \quad (\text{C.31})$$

In Equation C.30 and Equation C.31, p is the probability of a failure, so that $\text{NB}(r, p)$ has mean $rp/(1-p)$ and $\text{Geom}(p)$ has mean $p/(1-p)$. That is the opposite of the convention in Equation C.3, and it is the single most likely thing to go wrong when these are implemented. The check of Equation C.27 applies here too and catches it at once: compute the mixture's mean and variance from its components and compare them with the targets.

C. Common Probability Distributions

On Equation C.26, $a = 0.3140 - 1/50 = 0.2940$, which falls in the mixed negative binomial branch. Then $r = \lfloor 1/0.2940 \rfloor = 3$, $q = 0.37788$ and $p = 0.93245$, and the resulting mixture has mean 50.0000 and variance 785.0000.

Notice that all three routes reproduce the same two moments and disagree about everything else. That is the honest summary of moment matching: two numbers do not determine a distribution, and the choice of family is an assumption that the procedure hides rather than removes. Where the answer matters, compute it under two of them and report the spread.

C.4.3. When the Demand Is Intermittent

A series in which most periods have no demand at all is not served well by any of the above, because a single family has to explain both how often demand occurs and how large it is when it does. Two responses are available.

The first is a **zero-modified** family, which mixes a point mass at zero with one of the families of Section C.1. Rossetti and Ünlü (2011) recommend a zero-modified form of Adan's rule when the probability of a zero period exceeds one half, and switching back to the ordinary rule below that.

The second is a **compound representation**, in which the time between demands and the size of a demand are modeled separately and the period demand is the sum of a random number of sizes. Section B.5 gives the moments. That representation is the more faithful one, and it is the one that connects to the demand classification of Section 2.2.6, where the interval between demands and the variability of the sizes are exactly the two axes.

Both are beyond what Chapter 7 needs, and both matter from Chapter 8 onward, where the quantity being modeled is demand over a lead time and the zeros no longer conveniently aggregate away.

D. Statistical Tables

D.1. Standard Normal Distribution

TODO: write this section.

D.2. Standard Normal Loss Function

TODO: write this section.

D.3. Cumulative Poisson Probabilities

TODO: write this section.

E. Kotlin and KSL Setup

i Learning objectives

After reading this appendix you should be able to:

- install the tools needed to run the simulation examples
- open the course code project and run an example
- add your own model to the project

E.1. What You Need

Two things, and nothing else:

- **A JDK, version 21 or later.** Temurin is a good default. Verify the installation with `java -version` at a terminal.
- **IntelliJ IDEA**, Community Edition. It has Kotlin support built in and understands Gradle projects without configuration.

You do not need to install Kotlin, Gradle, or the KSL separately. The course code project carries a Gradle wrapper that downloads the correct versions of Gradle and Kotlin on first use, and Gradle downloads the KSL from Maven Central.

E.2. The Course Code Project

The Kotlin examples live in a Gradle project whose structure mirrors the book:

```
code/
  build.gradle.kts      build configuration; the KSL dependency lives here
  settings.gradle.kts
  gradlew, gradle/      the Gradle wrapper
  src/main/kotlin/inventory/
    <topic>/            one package per subject
```

A package is named for its subject, so `inventory.dynamiclotsizing` holds the code of Chapter 5 and `inventory.newsvendor` holds the code of Chapter 7. Chapter numbers move when a book is revised and subjects do not, which is the same reason the section labels you see in cross-references are semantic rather than numbered.

E.3. Running Your First Model

From a terminal, in the `code` directory:

```
./gradlew run
```

The first run takes a minute while Gradle, Kotlin, and the KSL are downloaded; later runs start immediately. The default example evaluates a newsvendor order quantity by Monte Carlo simulation, using the item from the first exercise of Chapter 7, and prints expected profit across a grid of order quantities:

E. Kotlin and KSL Setup

Expected profit by order quantity (10,000 replications each)

Q	avg profit	half-width
240	5108.50	10.55
260	5421.87	14.11
280	5650.27	18.41
300	5789.95	23.41
320	5820.35	27.75
340	5811.68	31.38
360	5699.77	34.48

Best quantity on this grid: Q = 320, expected profit 5820.35

Two things are worth noticing about that output before you go further. The best quantity on the grid, 320, is close to the value the critical ratio gives analytically. That agreement is how you gain confidence that a simulation is built correctly. And the half-widths grow with the order quantity, because profit becomes more variable as more units are exposed to demand risk. A single replication would have told you neither.

To run it from IntelliJ instead, open the `code` directory as a project and use the gutter icon beside `fun main`.

E.4. Adding Your Own Model

TODO: write this section, covering a file in the right package, the `main` function convention, and pointing `application { mainClass }` at it.

E.5. The KSL Itself

The KSL is a large library, and this book uses a small part of it. For the library on its own terms, its packages, its process view, and its output analysis machinery, see *Simulation Modeling using the KSL*.

E. Kotlin and KSL Setup

The version this course pins is declared in `code/build.gradle.kts`:

```
implementation("io.github.rossetti:KSLCore:R1.6.2")
```

References

- Abaydulla, Anvar. 2016. “Grouping Techniques to Manage Large-Scale Multi-Item Multi-Echelon Inventory Systems.” PhD thesis, University of Arkansas.
- Adan, Ivo, Michel van Eenige, and Jacques Resing. 1995. “Fitting Discrete Distributions on the First Two Moments.” *Probability in the Engineering and Informational Sciences* 9 (4): 623–32.
- APICS. 1980. *Cycle Counting Training Guide*. American Production; Inventory Control Society.
- Axsäter, Sven. 2006. *Inventory Control*. 2nd ed. Kluwer Academic Publishers.
- Brooks, Roger B., and Larry W. Wilson. 1995. *Inventory Record Accuracy: Unleashing the Power of Cycle Counting*. John Wiley; Sons.
- Buck, James R., and Randall P. Sadowski. 1983. “Optimum Stratified Cycle Counting.” *IIE Transactions* 15 (2): 119–26.
- Croston, J. D. 1972. “Forecasting and Stock Control for Intermittent Demands.” *Operational Research Quarterly* 23 (3): 289–303.
- DeHoratius, Nicole, and Ananth Raman. 2004. “Inventory Record Inaccuracy: An Empirical Analysis.” *Management Science*.

References

- Ernst, Ricardo, and Morris A. Cohen. 1990. "Operations Related Groups (ORGs): A Clustering Procedure for Production/Inventory Systems." *Journal of Operations Management* 9 (4): 574–98.
- Federgruen, Awi, and Michal Tzur. 1991. "A Simple Forward Algorithm to Solve General Dynamic Lot Sizing Models with n Periods in $o(n \log n)$ or $o(n)$ Time." *Management Science* 37 (8): 909–25.
- Flores, Benito E., and D. Clay Whybark. 1986. "Multiple Criteria ABC Analysis." *International Journal of Operations and Production Management* 6 (3): 38–46.
- Iglehart, Donald L., and Richard C. Morey. 1971. "Inventory Systems with Imperfect Asset Information." *Management Science* 18 (8): B388–94.
- Johnston, F. R., and John E. Boylan. 1996. "Forecasting for Items with Intermittent Demand." *Journal of the Operational Research Society* 47: 113–21.
- Kampen, Tim J. Van, Renzo Akkerman, and Dirk Pieter van Donk. 2012. "SKU Classification: A Literature Review and Conceptual Framework." *International Journal of Operations and Production Management* 32 (7): 850–76.
- Kumar, Sameer, and Sant Arora. 1992. "Effects of Inventory Mismatch and Non-Inclusion of Lead Time Variability on Inventory Performance." *IIE Transactions* 24 (2): 96–103.
- Lenard, J. D., and B. Roy. 1995. "Multi-Item Inventory Control: A Multicriteria View." *European Journal of Operational Research* 87 (3): 685–92.
- Morey, Richard C. 1985. "Estimating Service Level Impacts from Changes in Cycle Count, Buffer Stock, or Corrective Action." *Journal of Opera-*

References

- tions Management* 5 (4): 411–18.
- Muckstadt, John A., and Amar Sapra. 2010. *Principles of Inventory Management: When You Are down to Four, Order More*. Springer Series in Operations Research and Financial Engineering. Springer.
- Nahmias, Steven, and Tava Lennon Olsen. 2015. *Production and Operations Analysis*. 7th ed. Waveland Press.
- Neeley, Paul. 1987. “A Framework for Cycle Counting.” *Production and Inventory Management* 24 (3): 64–69.
- Orozco, R. E. 1993. “Is ABC Cycle Counting a Wasteful Approach.” *American Production and Inventory Control Society International Conference Proceedings*, 536–37.
- Porteus, Evan L. 2008. “The Newsvendor Problem.” In *Building Intuition: Insights from Basic Operations Management Models and Principles*, edited by Dilip Chhajed and Timothy J. Lowe, vol. 115. International Series in Operations Research and Management Science. Springer.
- Ramanathan, Ramakrishnan. 2006. “ABC Inventory Classification with Multiple-Criteria Using Weighted Linear Optimization.” *Computers and Operations Research* 33 (3): 695–700.
- Rossetti, Manuel D. 2026. *Simulation Modeling using the Kotlin Simulation Library (KSL)*. Self-published. <https://rossetti.github.io/KSLBook/>.
- Rossetti, Manuel D., and A. V. Achlerkar. 2011. “Evaluation of Segmentation Techniques for Inventory Management in Large Scale Multi-Item Inventory Systems.” *International Journal of Logistics Systems and Management* 8 (4): 403–24.
- Rossetti, Manuel D., Nebil Buyurgan, Amit Bhonsle, Seda Gumrukcu, and

References

- Kiran Chittoori. 2008. “An Analysis of the Effect of Inventory Record Inaccuracy in a Two-Echelon Supply Chain.” *International Journal of Inventory Research*.
- Rossetti, Manuel D., Terry Collins, and Ravi Kurgund. 2001. “Inventory Cycle Counting: A Review.” *Proceedings of the 2001 Industrial Engineering Research Conference*.
- Rossetti, Manuel D., John English, Nebil Buyurgan, Ronald Walker, Seda Gumrukcu, and Long Yu. 2006. *Improving Inventory Record Accuracy Within Retail Store Operations*. {CELDi} Project Report. Center for Engineering Logistics; Distribution, University of Arkansas.
- Rossetti, Manuel D., and Yasin Ünlü. 2011. “Evaluating the Robustness of Lead Time Demand Models.” *International Journal of Production Economics* 134 (1): 159–76.
- Roundy, Robin. 1985. “98%-Effective Integer-Ratio Lot-Sizing for One-Warehouse Multi-Retailer Systems.” *Management Science* 31 (11): 1416–30.
- Sheppard, G. M., and K. A. Brown. 1993. “Predicting Inventory Record-Keeping Errors with Discriminant Analysis: A Field Experiment.” *International Journal of Production Economics* 32: 39–51.
- Silver, Edward A., David F. Pyke, and Douglas J. Thomas. 2016. *Inventory and Production Management in Supply Chains*. 4th ed. CRC Press.
- Stahl, Robert A. 1998. “Cycle Counting: A Quality Assurance Process.” *Hospital Material Management Quarterly* 20 (2): 22–28.
- Syntetos, Aris A., and John E. Boylan. 2005. “The Accuracy of Intermittent Demand Estimates.” *International Journal of Forecasting* 21 (2): 303–14.

References

- Syntetos, Aris A., John E. Boylan, and J. D. Croston. 2005. "On the Categorization of Demand Patterns." *Journal of the Operational Research Society* 56: 495–503.
- Tersine, Richard J. 1994. *Principles of Inventory and Materials Management*. 4th ed. Prentice Hall.
- Teunter, Ruud H., M. Zied Babai, and Aris A. Syntetos. 2010. "ABC Classification: Service Levels and Inventory Costs." *Production and Operations Management* 19 (3): 343–52.
- Varghese, Vijith, Manuel D. Rossetti, Edward Pohl, Server Apras, and Douglas Marek. 2012. "Applying Actual Usage Inventory Management Best Practice in a Health Care Supply Chain." *Working Paper, University of Arkansas*.
- Wagner, Harvey M., and Thomson M. Whitin. 1958. "Dynamic Version of the Economic Lot Size Model." *Management Science* 5 (1): 89–96.
- Young, S., and W. Nie. 1992. "A Cycle-Count Model Considering Inventory Policy and Record Variance." *Production and Inventory Management* 33 (1): 11–16.
- Zhang, R. Q., Wallace J. Hopp, and C. Supatgiat. 2001. "Spreadsheet Implementable Inventory Control for a Distribution Center." *Journal of Heuristics* 7 (2): 185–203.
- Zipkin, Paul H. 2000. *Foundations of Inventory Management*. McGraw-Hill.